



V93000 SOC

SmarTest Concepts

Version 8.6.0

Published on 2023-02-24

Notices

Legal Notices

All rights reserved. All text and figures included in this publication are the exclusive property of Advantest Corporation. Reproduction of this publication in any manner without the written permission of Advantest Corporation is prohibited. Information in this document is subject to change without notice.

Trademarks and Registered Trademarks

- ADVANTEST is a trademark of Advantest Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Warranty

The material contained in this document is provided "as is," and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Advantest disclaims all warranties, either express or implied, with regard to this material and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Advantest shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this material or of any information contained herein. Should Advantest and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology Licenses

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

Restricted Rights Legend

If software is for use in the performance of a U.S. Government prime contract or subcontract, Software is delivered and licensed as "Commercial computer software" as defined in DFAR 252.227-7014 (June 1995), or as a "commercial item" as defined in FAR 2.101(a) or as "Restricted computer software" as defined in FAR 52.227-19 (June 1987) or any equivalent agency regulation or contract clause. Use, duplication or disclosure of Software is subject to Advantest's standard commercial license terms for 93000 systems and non-DOD Departments and Agencies of the U.S. Government will receive no greater than Restricted Rights as defined in FAR 52.227-19(c)(1-2) (June 1987). U.S. Government users will receive no greater than Limited Rights as defined in FAR 52.227-14 (June 1987) or DFAR 252.227-7015 (b)(2) (November 1995), as applicable in any technical data.

Safety Notices

⚠ CAUTION A CAUTION notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a CAUTION notice until the indicated conditions are fully understood and met.

▲ Warning A WARNING notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in death or severe injury. Do not proceed beyond a WARNING notice until the indicated conditions are fully understood and met.

Eclipse IDE integration

SmarTest is based on the open-source Eclipse platform and offers all standard Eclipse functions.

In this part, you can read about the following concepts related to the Eclipse IDE:

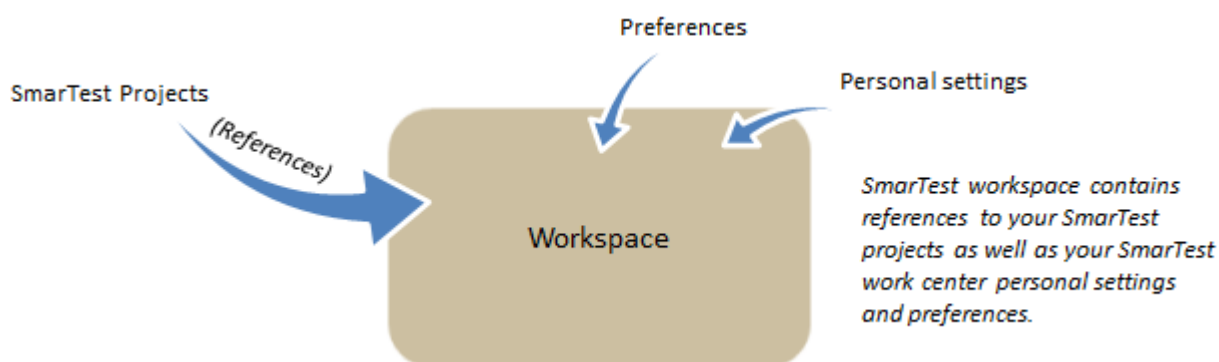
- SmarTest workspace
- SmarTest Work Center
- SmarTest project, its structure, and the model file.

If you want to learn more about Eclipse, you can:

- In SmarTest Work Center, choose **Help > Help Content**.
- Visit the *Eclipse website*.

SmarTest workspace

SmarTest workspace is a specialization of the standard Eclipse workspace - a specific, main working directory used to organize your work and preference settings - and shares its characteristics. It is a concept of grouping things together in a central hub for your SmarTest folders, files, and references to projects. However, there are some aspects that are specific to SmarTest. These aspects are mentioned below.



The Eclipse workspace is a collection of resources (project references, folders, files), organized in a tree structure. On top is the workspace root resource, with project references, folders and files underneath. For general details about workspaces, refer to the Eclipse documentation.

In addition, take note of the following SmarTest requirements and guidelines:

- Workspace directories must not be located within project directories.
Project directories can be located inside workspace directories, but we strongly recommend that you do not put them there. Instead, you should import project directories into workspaces by reference. Referencing projects prevents you from accidentally deleting projects when deleting a workspace.
- Workspaces cannot be shared between users. They also cannot be simultaneously used by different SmarTest instances.

To share data with other users, share the associated *projects*.

- Workspaces are specific to a SmarTest version.

While it may be possible to convert an older workspace for a new SmarTest release, it is often not possible to open a newer workspace with an older release. In particular, workspaces for SmarTest 8 cannot be opened with SmarTest 7.

A SmarTest workspace root directory contains the `project.map` file, and for locally cached remote projects, a `remote.map` file:

`projects.map`

The `projects.map` file is located inside the workspace. It contains the names and locations of all projects contributing to the workspace and possesses the following main characteristics:

Plain text (utf-8 encoded) format

- key-value pairs using the format: `key=value`
- One key-value pair per line. Line ends with `\n` (= `\x0a`).
- Spaces (' ' and `\t`) either side of the key are trimmed.
- Spaces (' ' and `\t`) before the value are trimmed.
- `\` can be used to escape characters. All characters after a `\` are taken literally. This can be used for leading or trailing spaces, or `=` within the project name.

Self-contained (except variables)

All relative paths are resolved against the current directory.

Variables

Variable entries are resolved against the current process environment.

Comments

Comments are optional and can only be placed at the beginning of a line and are started with the hash character:

```
# This is a comment
```

Comments are not written back.

Keys and values

- Keys represent project names.
- Values represent project paths.

Project path format

There are a number of project path formats:

Workspace external projects

Workspace external projects are physically located outside the workspace and are referenced by one of the following methods:

- Full absolute path - starting with a leading slash. For example:

```
GlobalProjectName1=/opt/libraries/project1
```

- Environment variables - can be used at the beginning of the path.

Environment relative paths start with the dollar sign followed by the environment variable enclosed with brackets. For example:

```
SharedProjectName=$(SRC_ROOT)/project3.
```

Workspace local projects

Workspace local projects are physically located inside the workspace and are referenced by a workspace relative path to the project root directory (the directory containing the `.project` file).

Local project entries have no leading or trailing slashes. For example:

```
WorkspaceLocalProjectName=someDir/project4.
```

A `projects.map` file not featuring a key-value pair is not regarded as an error, although this is typically a transient state.

The following is an example of a `projects.map` file:

```
# Absolute path outside workspace directory
GlobalProjectName1=/opt/libraries/project1

# Variable relative path outside workspace directory
SharedProjectName=$(SRC_ROOT)/project3

# Workspace local path. "project4" directory contains .project file
WorkspaceLocalProjectName=someDir/project4
```

remote.map

The `remote.map` file is used to store information about the remote addresses of a workspace when it originates from a remote location on a network file system. The address is specified as follows:

```
remote={ $PATH$ }
```

For more information about the working with locally cached remote workspaces, see Local Cache view in TDC 354110.

SmarTest automatically saves your personal settings when closing a workspace, either when switching to another workspace or when closing SmarTest.

Related concepts

SmarTest Work Center on page 8

SmarTest uses Eclipse as the integrated development environment (IDE) containing all the independent graphical user interfaces. This topic describes the different components that make up the SmarTest Work Center.

Related information

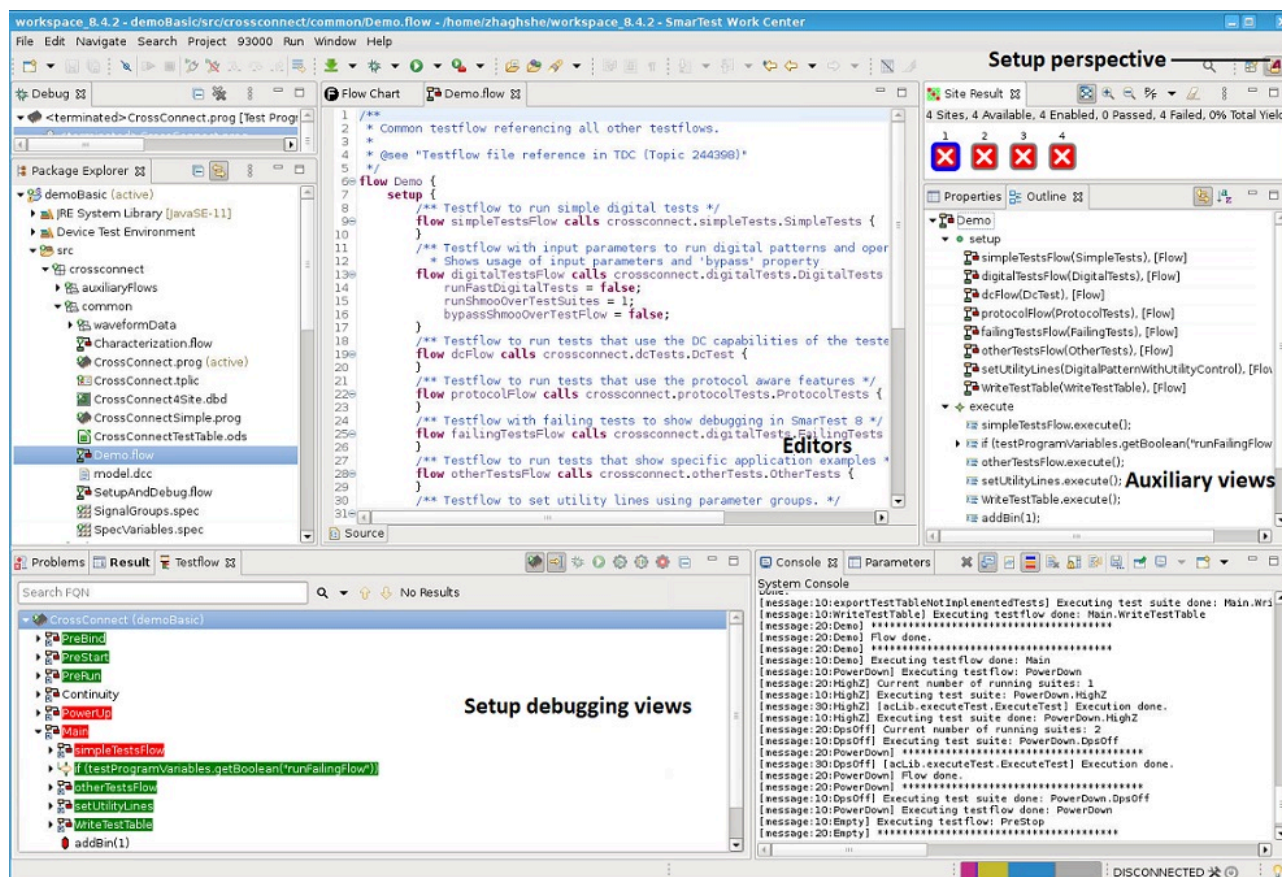
SmarTest projects on page 10

Test programs are organized in SmarTest projects. The content of each project is stored in a project folder in the file system. In addition, a project can also reference content from other projects. The folder structure within a project folder is partly mandatory and partly modifiable.

SmarTest Work Center

SmarTest uses Eclipse as the integrated development environment (IDE) containing all the independent graphical user interfaces. This topic describes the different components that make up the SmarTest Work Center.

The SmarTest Work Center is the main window which appears when SmarTest is started. The Work Center is organized as a *workspace* and contains several predefined "perspectives (Topic 244606)". These perspectives contain views and editors, and control what is displayed in the menus and toolbars.



- **Workspaces**

A workspace is a directory on the file system and can be seen as a *virtual* container of *references* to your projects (like setup projects or test method projects), and as a storage for the preferences and customization of your working environment. In SmarTest, workspaces are located by default in your \$HOME directory, but you can create them in another location as well.

- **perspectives (Topic 244606)**

A perspective consists of a specific configuration of editors and views. Perspectives are configured to support the main workflows, Setup, Device Debug, and Analysis. You can customize an existing perspective, or you can create your own perspectives to contain the tools and views required to support your usual tasks. Perspective configurations are associated with the *workspace* from which they are created.

- **Editors and views**

An editor is used to examine and edit objects such as configuration files or test methods. A view is used to examine and analyze data, or modify properties of selected elements within an editor or another view. Editors and views save modifications differently: Views save any

modification immediately and without requiring any further user input, modifications to a file opened in an editor require you to actively save the changes to the file.

The layout of editors and views on a page is controlled by their preferences, which are stored together with the settings of the active *workspace*.

- Preferences (Topic 244576)

A set of preferences allows you to keep a specific state and settings of views and editors. Preferences are stored within your *workspace*.

- *SmarTest projects* on page 10

Projects provide the basic directory structure and files for setting up, running, and debugging tests. When you start working with SmarTest, you first create a project. A project contains device-related information, for example, setup or test method data. For more information see *SmarTest projects* on page 10.

- Help view (Topic 244452)

The Help view is based on the TDC (Technical Documentation Center) viewer and can show context sensitive help (press **Shift+F1**) and the online help. To open the TDC Viewer on the menu click **Help > Help Content**.

SmarTest projects

Test programs are organized in SmarTest projects. The content of each project is stored in a project folder in the file system. In addition, a project can also reference content from other projects. The folder structure within a project folder is partly mandatory and partly modifiable.

A SmarTest project is a folder that includes one or more source folders as subfolders. For setting up a test, the project must also contain at least one *Test program* on page 93 file and the corresponding setup files, for example, *Testflow* on page 94 and *Pattern* on page 53 files.

Project folders may be stored within workspaces or outside of them. Note that workspaces cannot be shared. Especially if you plan to reference parts of projects in other projects (for personal reuse or collaborative development), it is therefore strongly recommend to store projects outside of any workspace. Projects and test setup files can *refer* to other projects and test setup files in other projects.

Contents of a SmarTest project

A SmarTest project contains:

- One or more *Test program* on page 93 files.
- One or more source folders.

The standard name of the default source folder is `src`, but you can choose a different name if desired. Additional source folders can be named as desired.

The subfolder structure within a source folder is not predefined. You can create any structure to fit your project. See below for some recommendations.

Test methods must be stored in subfolders of source folders, not directly in source folders.

- One or more output folders.

The default output folder is named `bin`. SmarTest manages the content of output folders automatically, and you should not modify it manually. Note that you can add more output folders that are only meant for test method code. SmarTest updates the output folders during the workspace build process. The build process can be full or incremental, depending on the test data change.

Additional contents of a SmarTest project

Besides the contents listed above, a SmarTest project contains the following additional hidden files:

.asourcepath

The `.asourcepath` file is derived from the standard Java project file `.classpath`.

It contains all relevant search locations for setups. These locations refer to source directories of projects inside the workspace.

At execution, the `.asourcepath` file of the project in which the executed `<testprogramName>.prog` file resides is used.

The file is auto-generated and should not be modified manually.

.aclasspath

The `.aclasspath` file is derived from the standard Java project file `.classpath`.

It specifies all relevant libraries and class folders used during execution.

The file is auto-generated and should not be modified manually.

.aproject

The `.aproject` file, using plain text (utf-8 encoded) format, specifies the SmarTest revision as key/value pairs used to build the project (auto-generated).

The file is created when a Smartest or a Smartest Bridge project is built and deleted when the project is cleaned.

The file is auto-generated and should not be modified manually.

.gitignore

`.gitignore` files are used to simplify version control of projects with Git.

It specifies the auto-generated SmarTest files and directories that are generated when you create a new SmarTest project.

For more information about the `.gitignore` file, see <https://git-scm.com/docs/gitignore>.

For more information about files that do not need to be under version control, see "SVN plugin (Topic 256873)".

.remote

The `.remote` file is used to store information about the remote addresses of projects when they originate from a remote location. The address is specified as follows:

```
remote=${$PATH$}/${$PROJECT$}
```

For more information about the working with locally cached remote workspaces, see Local Cache view in TDC 354110.

Note

All these dot-files are hidden files and you should not modify them. The internal format of these files is subject to change in every release.

Recommendations for source folder structures

We recommend using one of the following structures for the subfolders of a source folder:

- A structure based on the setup file type.
- A structure based on flows, with substructures for all corresponding setup elements.
- A structure based on the IP cores.

For more details and examples see *Recommended structure of a SmarTest project* and "Example project (Topic 254181)".

Naming projects and project folders

There are several rules on naming projects and project folders:

- The name of each project that is used or referenced in a workspace must be unique within that workspace.
- You cannot use a SmarTest keyword as the name of a file or a folder.
- The name of the project can be different from the name of the project folder.
- Project locations must be unique. That is, you cannot store different projects within the same location.
- All project folder names have to comply with the Java naming style. That is:
 1. The name cannot begin with a digit. Instead, the first character of a name can only be one of these characters: A-Z, a-z, _, \$. Also, it is recommended to use an upper-case letter as the first character of a file name and a small-case letter as the first character of a folder name.
 2. For the remaining part of the file or folder name, you can use any of the following characters: a-z, A-Z, 0-9, _, \$.
- SmarTest uses the project folder of the active test program as the root directory from which it refers to the source (typically called `src`) folder and all referenced files.

Sharing projects

In SmarTest you can share projects or test data files between SmarTest projects or workspaces by importing them into another project or workspace. There are two types of imports:

- Reference the project or test components from its original location.
- Copy the project or test components into your workspace, although we recommend against this approach.

For more details see *Test data reuse and sharing within and between SmarTest projects*.

For *collaborative development*, Advantest recommends to use either the referencing model or a branching concept as used by revision control systems.

Metadata

SmarTest projects include preference information as metadata, for example, individual path variables or filters on displayed files or folders. As a result, when you share a SmarTest project, you also share the related preferences.

Related concepts

SmarTest workspace on page 5

SmarTest workspace is a specialization of the standard Eclipse workspace - a specific, main working directory used to organize your work and preference settings - and shares its characteristics. It is a concept of grouping things together in a central hub for your SmarTest folders, files, and references to projects. However, there are some aspects that are specific to SmarTest. These aspects are mentioned below.

SmarTest Work Center on page 8

SmarTest uses Eclipse as the integrated development environment (IDE) containing all the independent graphical user interfaces. This topic describes the different components that make up the SmarTest Work Center.

Related reference

SmarTest project structure on page 12

Source files for a SmarTest project are collected in a project folder. These source files can be flexibly distributed in subfolders of this project folder, but there are some requirements and recommendations.

SmarTest project structure

Source files for a SmarTest project are collected in a project folder. These source files can be flexibly distributed in subfolders of this project folder, but there are some requirements and recommendations.

The following rules and guidelines apply to structuring SmarTest projects:

- For each SmarTest project, there must be one source folder (`src`). You can have more than one source folder. See the folders numbered # in the examples below.

You can recognize a source folder by the small hash symbol on its icon 

- The test program file must be located in a source folder.

- A source folder should contain only one subfolder to group all test method classes, testflows, and other test setup files that are related to the functionality you want to test. For examples, see the folders numbered # in the examples below.

Follow these guidelines in naming the subfolders:

- Folder names should reflect their content.
- Folder names should be unique to prevent naming conflicts in case the folders contain files with similar names.
- As a general recommended practice, a folder's name should begin with a lower case letter, whereas a file's name should begin with an upper case letter.

For additional information, see *Naming projects and project folders* on page 11.

Note	You cannot use SmarTest keywords or "SmarTest Setup Format keywords (Topic 343312)" for file or folder names.
-------------	---

- Within each subfolder, create new folders according to the categories of the tests you want to perform. These folders should contain the test method classes, testflows, and other test setup files that belong to the same test category.

The test method classes must always be part of a subfolder under the source folder and not be placed directly under the source folder.

In addition, you should create a "common" folder to group the test setup files that are used across multiple test categories. For instance, the DUT board file and the testflow file should always be kept in the `common` folder.

For examples, see the folders numbered # in the examples below.

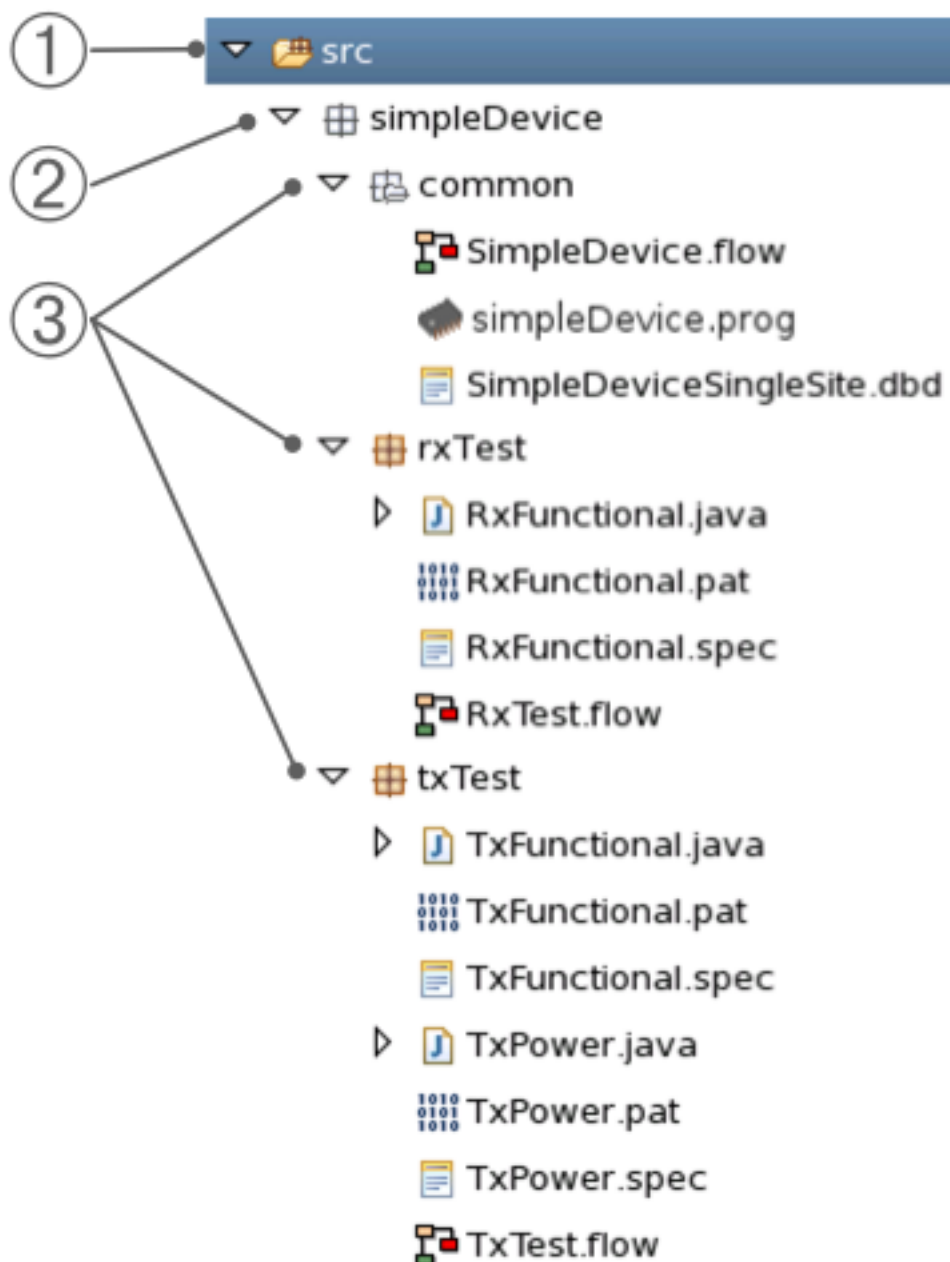
- If a test uses multiple setup files of the same type, you can introduce an additional folder and group those files in it.

For example, if you have several of pattern files for, say, a "Tx test", then create a `pattern` folder and group the pattern files in it. In the same manner, you can create `mainSpecs`, `level`, `timing`, `waveforms`, `actions`, `opSeq`, and `trSeq` folders to group those files together.

For examples, see the circles numbered # in the *More complex SmarTest project* on page 15 example below.

Note that all files in the source folder (`src`) are read during the binding process. Therefore, keeping non-setup files in the source folder slows down the binding process, and we recommend storing these files in folders outside the source folder.

Simple SmarTest project



Using subfolders to structure a project

This example shows the SmarTest project structure of a simple device that consists of a single IP core.

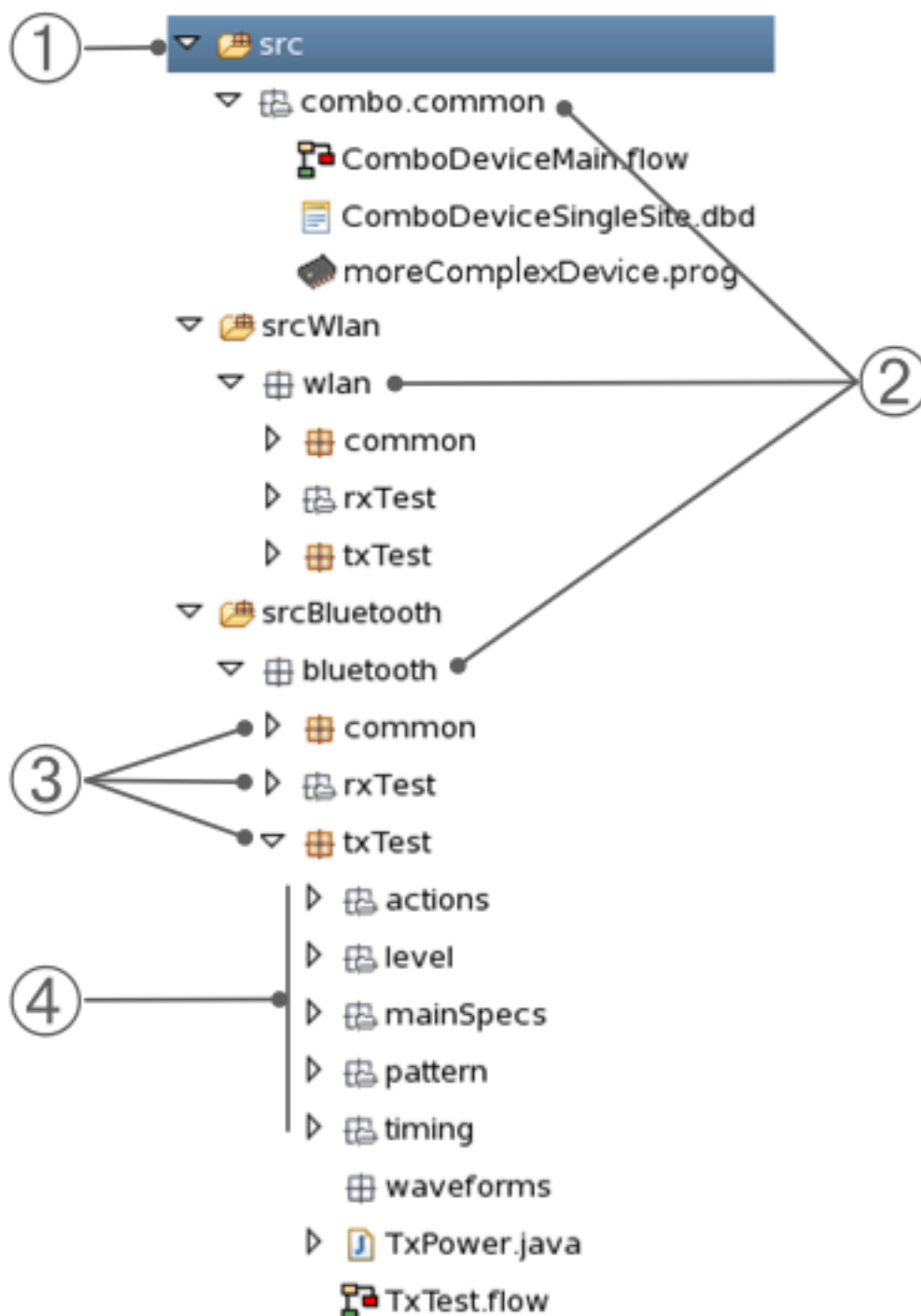
The SmarTest project for this device is structured according to the recommendations given above. That is, it has a single source folder `src` which, in turn, has a single main subfolder `simpleDevice` that contains all the test method classes, testflow files, and other test setup files.

There are two tests performed, an "Rx test" and a "Tx test". So, in the `simpleDevice` folder, two folders are created for these tests: `rxTest` and `txTest`. Each of these folders contains the test setup files, for example, the test method, the pattern, and the subflow file, that are used in the same test category.

There is also a third folder `common` which contains the test setup files that are common to multiple (or all) of the performed tests, for example the DUT board description and the testflow files.

More complex SmarTest project

To make a folder of a different SmarTest project become a source folder, right-click your current project and select: **SmarTest Build Path > New SmarTest Project Link**



Linking other SmarTest projects as additional source folders

Related information

SmarTest projects on page 10

Test programs are organized in SmarTest projects. The content of each project is stored in a project folder in the file system. In addition, a project can also reference content from other projects. The folder structure within a project folder is partly mandatory and partly modifiable.

Test data on page 21

A test is configured using a range of setup files, each of which describes a particular aspect of the overall setup.

Test programs, testflows, and test suites on page 92

SmarTest uses a hierarchy of structures to specify what happens during a test, from the test program at the top and down to test methods and instruments.

C++ support with SmarTest

SmarTest projects can use either the Java Native Interface (JNI) or the Simplified Wrapper and Interface Generator (SWIG) to support C++ compiled libraries.

The C++ support is realized by an additional project that contains either the JNI or SWIG code and the C++ libraries. This bridge project is referenced by your SmarTest project that contains the test program and other test data.

The classes and shared objects of the bridge project are loaded into SmarTest at start up only. Thus, if you modify the code in your bridge project, the changes will be visible only after you activate the test program again.

Note You have to maintain the make files that are used to compile the bridge project.

The C++ libraries (shared objects with the file extension `.so`) must reside in the `lib` directory of your bridge project. They are loaded with the Java native `System.loadLibrary()` from the Java code that also has to reside in the same bridge project.

If you cannot put the C++ library into the `lib` directory of your bridge project, you have to load it with the Java native `System.load()` method by specifying the absolute file path and filename.

Note Try to avoid using absolute file names, because this can cause errors if you relocate your library.

Java Native Interface (JNI)

The Java Native Interface (JNI) is a framework that enables test methods to call and be called by applications and libraries written in C++.

The JNI framework lets a C++ function use test method objects the same way that test method code uses these objects. A C++ function can create test method objects and then inspect and use these objects to perform its tasks. A C++ function can also inspect and use objects created by the test method code.

Simplified Wrapper and Interface Generator (SWIG)

The Simplified Wrapper and Interface Generator (SWIG) is an open-source software that enables you to connect applications or libraries written in C++ with Java.

SWIG generates JNI code to wrap C++ objects and is the recommended choice to add C++ support to your test methods conveniently. SWIG creates code that provides the glue between C++ and the test methods. It generates conversion code for functions with simple arguments. However, for complex types of arguments, you have to write the conversion code yourself. In some cases SWIG may add a performance overhead which you can avoid if you use JNI without wrappers.

Related information

<http://docs.oracle.com/javase/8/docs/technotes/guides/jni/>

Model file

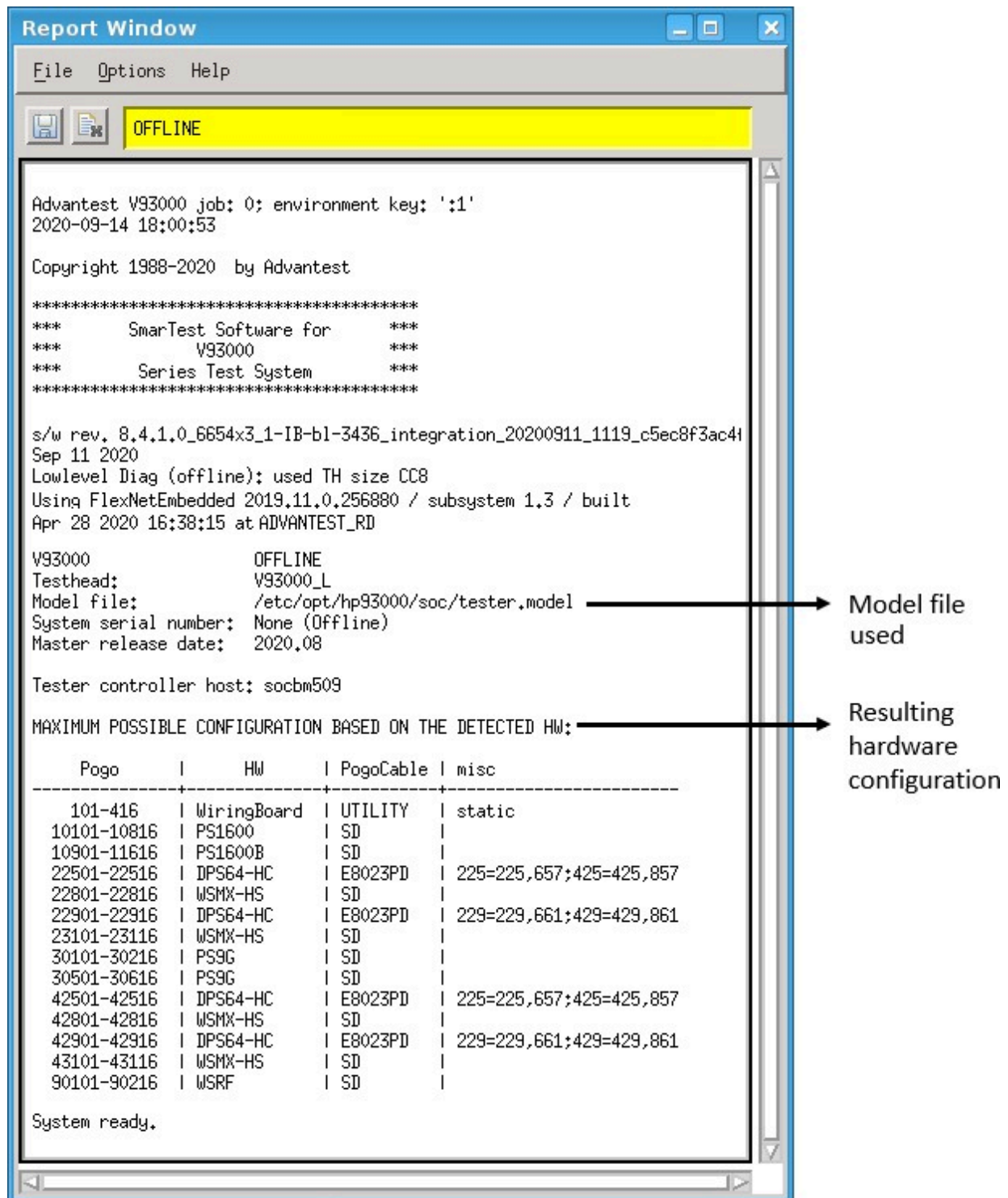
The model file contains the test head hardware configuration. It specifies what test head cards are installed, their features, and the mapping of these cards to pogo blocks on the DUT board interface. It also specifies the reference clock signal for the test system.

SmarTest needs this information in order to set the configuration and features of the test system resources for your addressed tests.

When SmarTest is started up in online mode (fiber optic connection between system controller and test head is operational), it reads the model file and reports the detected test system hardware in the **report window** in the **MAXIMUM POSSIBLE CONFIGURATION** table. In offline mode (fiber optic connection between system controller and test head is operational, or no test head is connected to the system controller) it shows the requested configuration defined by the model file. It also lists the referenced model file or a link pointing to it. The default location of the model file itself is at `/etc/opt/hp93000/soc/tester.model` (as shown in the figure below).

In typical installations, this is a link to `/etc/opt/hp93000/soc_common/tester.model`.

If a SmarTest version-specific file is required, `tester.model` file can also be a real file at the default location.



Test program data and structure

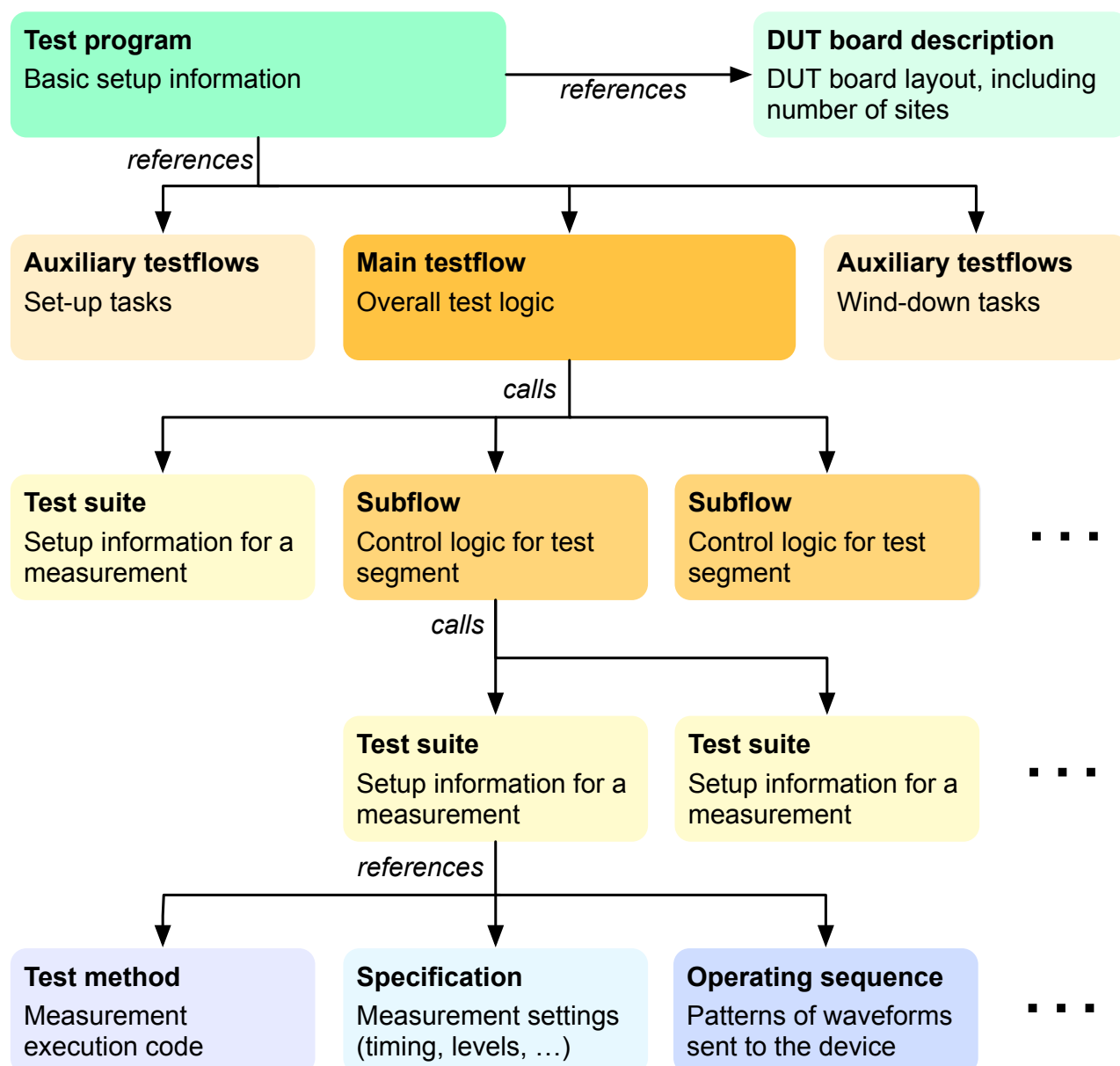
SmarTest simplifies your test program development. It decreases the time to market and reduces the cost of test.

There are several key concepts behind that such as:

- Device-test oriented workflows
- Unified way to set up the tester resources (instruments)
- Modular setups
- Easy assembly of test steps (pattern, actions, and transaction sequences) with the operating sequence
- Unified views across all test domains (DC, digital, protocol-aware, analog, mixed-signal, or RF)

In this section, you can learn about some of SmarTest's key concepts and the test program data and structure that are necessary to understand measurement setups.

The following graphic provides a high level overview of a SmarTest 8 test program and its building blocks. The structure is organized in a top-down approach with the test program file at the top and test methods at the bottom of the hierarchy.



1. The test program file
2. Main testflow: the central testflow in a test program
3. Subflows: a testflow is called from another testflow
4. Subflows: a testflow is called from another testflow
5. Description of how the DUT board connects pogo pins to DUT pins
6. Controls how the test method is executed
7. Controls how the test method is executed
8. Controls how the test method is executed
9. Setup files used by the Test programs, testflows and test suites
10. Performs setup tasks, automatically executed
11. Performs clean-up tasks, automatically executed

Test program building blocks in SmarTest

Test data

A test is configured using a range of setup files, each of which describes a particular aspect of the overall setup.

These setup files are used by *test programs*, *testflows*, and *test suites*, and together with them the setup files specify how a test is performed.

Test data is specified in the following test setup files:

- *DUT board description* on page 22: Describes how the DUT board is laid out and how it connects DUT pins to pogo pins.
- *Measurement specification* on page 24: Collects all parameters for a measurement, such as the *instruments* to use, the DUT signals associated to these instruments, the *level*, *timing*, and *wavetable* setups.
- *Pattern* on page 53: Contains the vectors of state characters that are used to perform the measurement.
- *Action* on page 57: Triggers an action by an instrument.
- *Operating sequence* on page 64: Describes the arrangement of patterns and actions for performing a measurement.
- *Test method* on page 87: Contains the detailed instructions for performing an individual measurement.

Some *general principles* apply to all these components.

Related information

Test programs, testflows, and test suites on page 92

SmarTest uses a hierarchy of structures to specify what happens during a test, from the test program at the top and down to test methods and instruments.

General principles for setups

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Unless noted otherwise, these principles apply to these types of setups:

- *DUT board description* on page 22
- *Measurement specification* on page 24
- *Operating sequence* on page 64
- *Pattern* on page 53

The similarities between setups of these types are:

- Setups can be specified in different locations:
 - *Test program* on page 93 (by referencing a setup file)
 - *Testflow* on page 94 (by referencing a setup file)
 - *Test suite* on page 100 (by referencing a setup file from the test suite specification in the testflow file)
 - *Test method* on page 87 (as part of the test method code)

Different parts of the same setup (for example, of the levels) can be specified in different locations. Where two parts do not conflict, both are valid. When the same parameter or setting is specified in multiple locations, the one in the lowest element from the list above is used.

For example, if a setting is specified both in a setup file referenced in the testflow and a file referenced in the test suite, the one from the test suite is used.

For specific types of tests, such as protocol-aware testing, additional types of setup files may be required, which are not described in this section.

- A setup file can import other setup files of the same type, but not of other types. This allows setups to be specified in a modular way, for example, to allow different engineers to work on different parts of a setup at the same time, or to create modules for different IP cores.
- Setup files are edited in the Setup editor and are written in the SmarTest Setup Format (SSF), except for pattern files.

The parts of a setup that are specified in the test method are edited in the code editor, as part of the test method code.

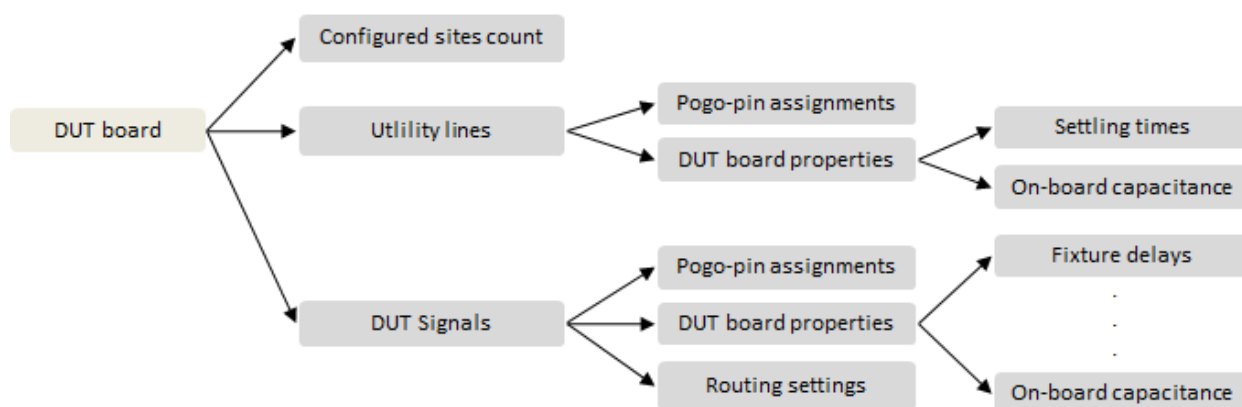
DUT board description

The DUT board description describes how the DUT board connects pogo pins to DUT pins (or, DUT *signals*) and which pogo pins belong to which sites.

SmarTest uses the DUT board description to determine which pogo pin (such as 10302) corresponds to which DUT signal (such as CLK). Because the DUT board description describes which DUT pins (DUT signals) in the different sites correspond to each other, it also enables SmarTest to automatically perform *Multisite testing*.

A DUT board description includes information on:

- Number of sites configured on the DUT board
- Connections between pogo pins and DUT signals
- Default properties for signals
- Routing controlled by utility lines
- DUT board properties and DUT properties



DUT board description

The DUT board file (.dbd) contains the DUT board description. That is, configured sites count, utility lines, their pogo pin mappings and DUT board properties. As well as, DUT signals, their pogo pin mappings per site, routing settings, and DUT properties.

A DUT board description is created for a DUT board and in principle changes only when the DUT board is modified. All *test programs* using the same DUT board should normally use the same DUT board description.

The DUT board description to use is specified in the test program. The entire test program uses the same DUT board description. A particular DUT board description may be used by several test programs.

A DUT board description file (.dbd) is shown below:

Related concepts

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Multisite testing in SmarTest on page 155

SmarTest offers built-in multisite testing without special requirements for test programs and test methods. The sites that are actually tested by a given measurement can be flexibly configured.

Default specification settings on page 122

Defaults for certain instrument settings can be specified in the DUT board description file using the `defaults.` prefix.

Pins and signals

SmarTest distinguishes between physical pins and logical signals. Only the latter are used in setting up measurements. Signals are mapped to pogo pins in the *DUT board description* on page 22, which implicitly maps them to DUT pins.

Pins

In SmarTest, only physical connections are called pins:

- DUT pins (often referred to as *pins*) are the physical connections of the DUT.
- Pogo pins provide the connections between the DUT board and the test head cards.

DUT pins are never addressed directly, and pogo pins are not addressed except in the DUT board description.

Signals

Signals can be considered logical pins: They have a particular function with respect to the DUT. By contrast, DUT pins have a particular location on the DUT.

With simple DUTs, there may be a one-to-one correspondence between signals and pins. But more complex DUTs multiplex multiple signals on one pin. At a given time, only one signal can be active on a DUT pin.

Tests in SmarTest always specify the signal to use, never the DUT pin.

Mapping signals to pins

Signals are defined in the *DUT board description* on page 22 and are mapped to DUT pins. The mapping is specified by assigning a pogo pin to each defined signal. Because the DUT board connects each pogo pin unambiguously with a DUT pin, this maps the signal to that DUT pin. A test can be adapted to a different DUT board by only changing the signal mapping in the DUT board description.

For multisite testing, a signal is defined with a list of pogo pins, one for each site on the DUT board.

Shared and ganged pogo pins are also supported. For details, see "DUT board description file reference (Topic 244395)".

In this example, two signals are defined for a DUT board with two sites:

- The first signal is CP. It is mapped to the DUT pins connected to pogo pin 11709 for site 1 and 11809 for site 2. Which DUT pins these are exactly depends on how the DUT board is physically laid out.
- The second signal, IO0, is mapped to the DUT pins connected to the pogo pins 11703 and 11803.

```
sites = 2;
signal CP {
  site 1 { pogo=11709; }
  site 2 { pogo=11809; }
```

```

}
signal IO0 {
  site 1 { pogo=11703; }
  site 2 { pogo=11803; }
}

```

Related concepts

DUT board description on page 22

The DUT board description describes how the DUT board connects pogo pins to DUT pins (or, DUT *signals*) and which pogo pins belong to which sites.

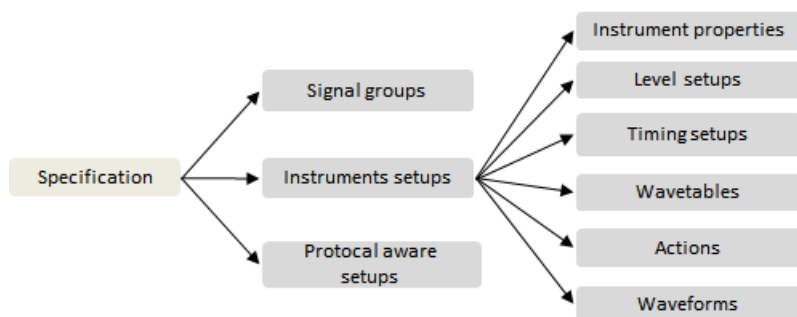
Measurement specification

The measurement specification (often referred to as "specification") describes which *instruments* a measurement uses and which settings are applied to them. A specification includes signal groups and aliases, timings, levels, wavetables, waveforms, and instrument setups, but does not include *patterns* or *operating sequences*. The measurement specification also includes the protocol and transaction sequence definitions for *protocol-aware testing*.

A measurement specification includes information on:

- Instruments to use
- DUT signals to use
- Timing specifications
- Level specifications
- Waveforms to use
- Settings for additional instrument parameters
- Protocol and transaction sequence definitions

These settings can be made directly by assigning a fixed value to a parameter or indirectly using variables and equations.



Measurement specification

The *specification file* (.spec) contains the measurement specification. That is, signal group definitions and instruments setups such as level sets, timing sets, wavetables, actions, waveforms, and protocol-aware test setups.

Measurement specifications are stored in measurement specification files. A specification file can contain the entire specification, or it can contain only a part of it and import additional specification files containing the other parts. This makes it possible to create *modular specifications* where, for example, one file sets all values through variables and equations and loads another file that contains the assignments of actual values to the variables.

The settings in a measurement specification file can be extended or overridden at runtime by the *test method*.

Measurement specifications are typically set at the *test suite* level or by the test method code. Measurement specifications are edited in the "setup editor (Topic 244574)".

Information about the DUT board and the sites are not contained in the measurement specification; they are found in the *DUT board description* on page 22. Because of this, the measurement specification does not need to be changed even if the test is run on more or fewer sites, or if the DUT board is exchanged.

A measurement specification file (`.spec`) is shown below:

Related concepts

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Modular setups on page 115

SmarTest provides modular setups - each setup file is self-contained. When you change a specific piece of information, only one file needs to be changed. SmarTest also provides a flexible setup file structure.

Protocol-aware testing on page 89

Protocol-aware testing is an integral feature of SmarTest that enables a direct communication to the DUT over a well-defined interface, the so-called protocol.

SmarTest Setup Format on page 134

SmarTest uses a domain specific language known as the SmarTest Setup Format (SSF) to specify the majority of the test data.

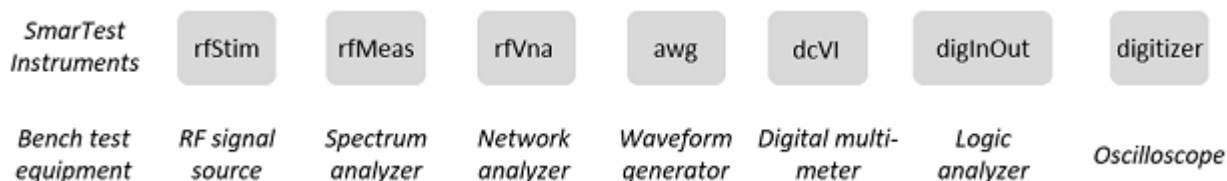
Fast adjustment on page 45

SmarTest provides fast adjustment to adjust the timing edges or level voltages based on a fast embedded search mechanism.

Instruments

Instruments provide you with a unified way to set up the tester hardware resources. They represent a layer between SmarTest test programs and the hardware resources of the test system. The instruments reflect the electronics of the hardware resources, and give access to the capabilities and features of the test head cards.

Every instrument uses the same elements, which describe the test system capabilities on the pogo pin level. This description of hardware resources and capabilities assures consistency and compatibility. It provides standard ways to set up particular measurements independent of the capabilities of a specific test head card.



Instruments

Instruments represent an abstraction of the common bench test equipment.

SmarTest provides three additional instruments: clock, loopback, and utility.



Instruments

clock instrument: provides a reference clock to the DUT.

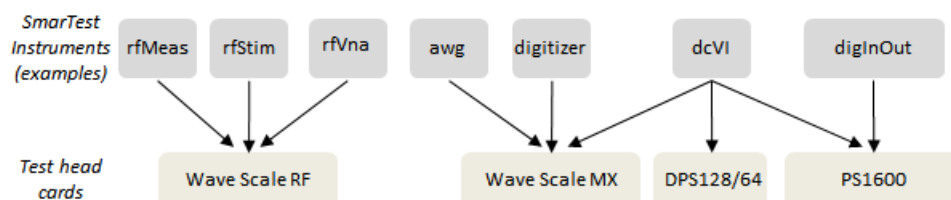
loopback instrument: receives data from one or more output pins and routes them to corresponding input pins, where the data is then driven back to the DUT.

utility instrument: allows control of the utility lines.

Instruments and test head cards

All measurement setups use instruments, as opposed to addressing the test head cards directly. To set up a measurement you specify the type of instrument to use and any specific settings. You also specify on which DUT signals the instrument will do the measurement.

Based on the DUT board description (which describes the connections from DUT signals to pogo pins), SmarTest assigns the actual test head card to be used for a measurement.



Test head cards and corresponding instruments

With instruments, SmarTest provides a unified way to set up the test head resources. For example, you set up a `dcVI` instrument the same way regardless of the underlying test head card.

Tests that have been developed with one combination of test head cards can directly be reused with any other combination, as long as the settings of your test program are within the specifications of the available test head cards. Instruments thus provide hardware abstraction for your tests.

Some test head cards may support wider ranges than others, or provide additional capabilities. These capabilities can be accessed from test methods if the appropriate extensions are present, but using them may limit the re-usability of the tests.

SmarTest instruments are an abstraction that will be translated to the settings for the specific test head cards during the bind phase. Some properties directly map to a setting of a test head card but in general they need to be translated to the corresponding programming of the specific hardware. During the translation to a specific hardware, additional derived properties are calculated.

Derived properties

Derived properties are properties that cannot be specified directly in the test program. Instead, they are the result of a calculation performed by a setup translation, possibly in a test head card specific way.

While there are many parameters that have to be calculated when programming the hardware, such values usually need not be available to the test program. However, in some cases, such values may be required by the test program to meaningfully do a data logging or to make a pass/fail decision.

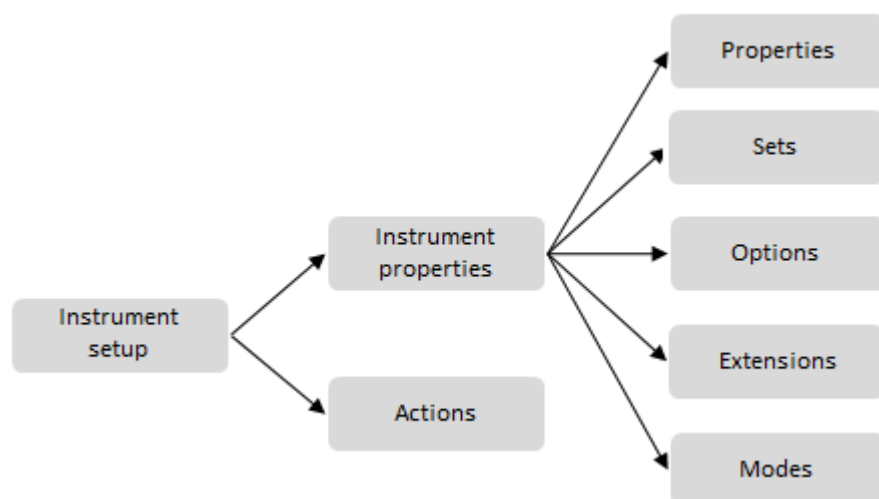
For example, consider the Wave Scale RF (WSRF) card as it drives a stimulus signal via its splitter. The exact attenuation varies for each path. The calibration can determine the deviations, but there is no way to compensate for it when routing the signals simultaneously to multiple DUT pins. In the best case, only one of the DUT pins can receive the exact power. Since the impact of the deviation is not relevant at the stimulated signal, but at the signal that is measured, the only way to achieve a correct pass/fail evaluation is in the test method code.

The derived properties can be queried by the test methods from the "device test APIs (Topic 244313)" to retrieve the instrument's setting for a given measurement, action, or a set.

Instrument configuration in SmarTest

The instruments to use for a measurement, their configuration, and the signals on which to use them are specified in the *Measurement specification* on page 24. The behavior of an instrument during a measurement is controlled by the *operating sequences*, *patterns*, and *wavetables* or waveforms applied to it.

Instrument setup elements



Instruments

*Instrument setup is part of the measurement specifications. It consists of two elements, **instrument properties** and **actions**. Instrument properties represent the initial settings of the instrument. Such as: basic properties, timing and level sets, options, extensions, and operation modes. They, except the sets, cannot be changed during the measurement execution. Actions control the instrument during the measurement execution. They are part of the setup but they are triggered for execution directly in an operating sequence, or a test method, or indirectly by a pattern that calls (anchors) the actions.*

All instruments are accessed in the same way. The main elements are:

- **Properties**
Properties specify the parameters of an instrument for a measurement. They are specified in a *Measurement specification* on page 24 file, or in a *Test method* on page 87. Some, the so-called instrument properties, remain unaltered during a measurement, except the timing and level sets. Other properties can be modified during a measurement.
- **Connections**
Each instrument has a set of defined connections that you can assign to your DUT signals. An instrument connection is represented by its pogo pin. Typically, instruments have `pos` (positive polarity) and `neg` (negative polarity) connections that enable you to connect the instrument to your DUT signals either single-ended or differently. For details, see "Setting up a differential connection (Topic 255511)".
- **Sets**
Sets group properties so that they can be changed together. The configuration of a digital instrument before a measurement is usually done with sets such as timing and level sets.
- **Options**
Options are instrument settings for declaring optional hardware resources or capabilities to be used in specific configurations. This enables SmarTest, for example, to derive optimization criteria for the available hardware.

- Extensions

Some instruments have multiple extensions. These provide access to feature sets that are not part of the base functionality of an instrument and may not be supported by all test head cards.

- Modes

Some instruments have multiple modes. A mode of an instrument is a specific way of using it. For example, an instrument may use one source in one mode and two sources in another, or an instrument may have a high-voltage mode that alters certain of its operating properties. Not all instruments have modes.

- Actions

Actions are parts of *operating sequences* which modify instrument properties at run-time, when they are triggered by a pattern, or executed as part of a sequential group in an operating sequence. Actions are also used to configure non-digital instruments.

To learn about how to set up the different instruments that SmarTest provides and use them in your measurements, refer to "Using instruments (Topic 342458)".

Instrument reference and test method library

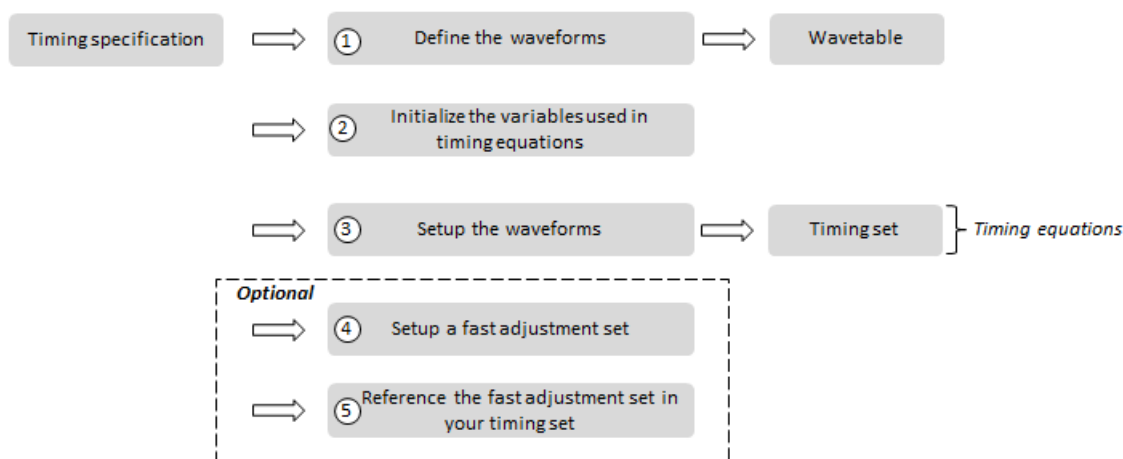
The example below illustrates assigning two instruments `dcVI` and `digInOut` to DUT signals within the specification file.

The `dcVI` instrument is assigned to the signals `D00`, `D02` and `D03`.

The `digInOut` instrument is assigned to the signals `D01` and `R01`.

Timing specification

The timing specification is part of the *measurement specifications* of digital instruments. It involves defining the digital waveforms in the *Wavetable* on page 34 and specifying the time points where the edges in the defined waveforms, for example: the drive or receive edges, exactly occur.

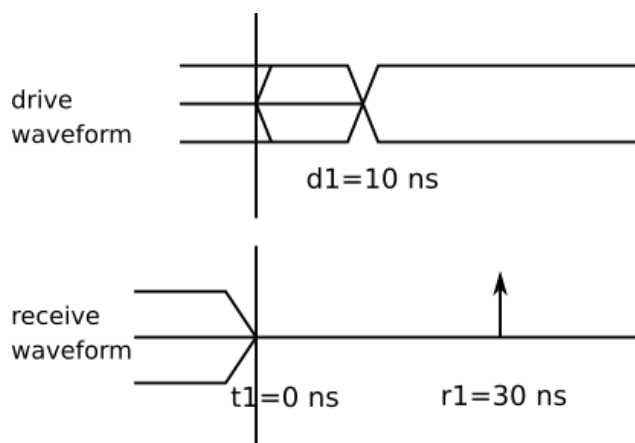


In the wavetable, you define the waveforms that are applied to or expected from your DUT. Here, each waveform is given an identifier called a "state character". Each state character is assigned one or more edges and edge actions to define the *structure* of the waveform. See, *Wavetable* on page 34 for details.

Timing setup of digital waveforms and period calculation

The timing setup of the waveforms that are defined in the wavetable is specified as a set of timing equations, or "timing set" for short. In a timing set, the timing equations define the points in time when the edges, for example drive or receive edges, occur. This defines the exact shape of a waveform that is applied to an input, or expected from an output DUT signal.

The edge delays for the drive, receive, and auxiliary edges are specified by the `digInOut` timing properties d , r , and t , respectively. When the auxiliary edge delay is not specified, then the drive edge delay is used by default, that is, $t = d$. See also, *digInOut sets and their properties*. This enables the possibility to have the auxiliary edge at a different timing than the drive edge by setting the t property to a different value. As soon as the driver is switched into or out of the auxiliary state, the t edge is used. This is important to know while using higher *x-modes* to utilize the waveform memory efficiently.



Drive edge and auxiliary edge

Period calculation

As stated above, timing sets can be defined using the timing group properties of the instrument. The data rates can be given either as period or as frequency. For each signal or a group of signals, you can define several timing sets to run your tests with. In that case, every timing set must have a period that can be expressed as an integer multiple of a common edge clock period.

A period given as an integer multiple of a common edge clock is also necessary when synchronizing instruments. In order to synchronize instruments, the system calculates the greatest common divisor of all participating periods of instruments.

When the participating periods do not satisfy the condition mentioned above,

- the common multiple for instrument synchronization may become very large resulting either in a considerable start overhead or in a bind error when it is impossible to find a common multiple or
- no common divisor can be found when multiple timing sets are used on one instrument, resulting in a bind error.

If you do not rely on exact ratios, you can specify the deviation that SmarTest can apply for choosing better fitting period in order to speed up the execution or to be able to bind the measurement at all. These deviations can be specified for the `digInOut` instrument using the following properties:

when data rate is given as period

- `minAllowedPeriod / maxAllowedPeriod`
- `periodDeviation`

when data rate is given as frequency

- `minAllowedFrequency / maxAllowedFrequency`
- `frequencyDeviation`

Note that when deviation or allowed ranges are specified, the system exclusively makes use of these settings and ignores the specified period or frequency.

Assigning values to timing sets and timing equations

Note

We recommend that you do not assign fractional numbers to values. If this cannot be avoided, always use floating point numbers.

Never use integer fractions such as $1/2$, because SmarTest drops the fractional part when dividing integers. For example, $1/2$ is calculated as 0 and $7/3$ is calculated as 2.

- Incorrect:
 - `d1=1/2ns`; This is calculated as $d1 = 1/(2 \text{ ns}) = 1/2 * 1/\text{ns} = 0 \text{ GHz}$, because the integer fraction $1/2$ is rounded to zero and $1/\text{ns}$ is 1 GHz.
 - `d1=1/2*1ns`; This is calculated as $d1 = 0 * 1 \text{ ns} = 0 \text{ ns}$ because the integer fraction is rounded to zero.
- Correct:
 - `d1 = 0.5ns`;
 - `d1 = 1.0/2.0 * 1ns`;
 - `d1 = 1.0ns/2.0`;

These equations will be evaluated as $d1 = 0.5 * 1\text{ns} = 0.5\text{ns}$ because the unit `s` is not in the denominator and the fractions are specified in terms of floating point numbers.

Do not evaluate a fraction, round its value, and specify it directly because that could lead to accuracy errors in your timing setup. For example, a period of "3.3333333 ns" (regardless of the number of digits) should always be specified in terms of timing equations as follows:

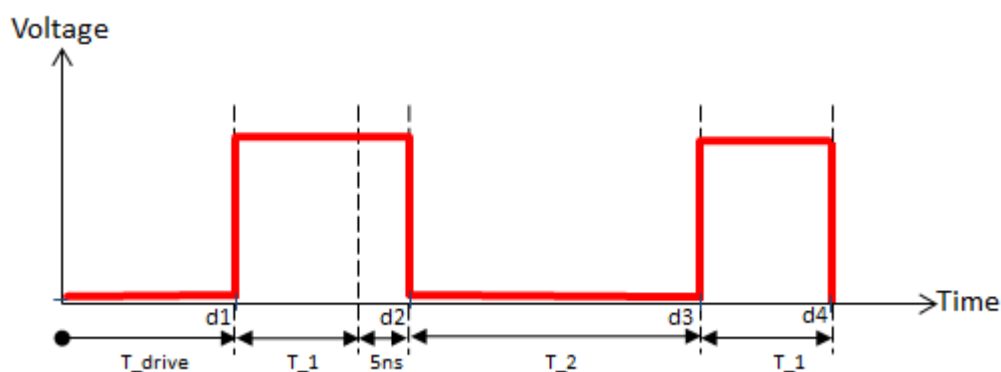
```
frequency= 300.0 MHz; // Define the frequency
period=1.0/frequency; // Specify the period in a timing equation as a fraction
// of floating point numbers (that is, 1.0/300.0). SmarTest will evaluate it
// with double arithmetic accuracy.
```

This way, you make sure that any fraction will always be evaluated consistently using SmarTest's double arithmetic accuracy. The generated waveform will not have accuracy errors.

Using fast adjustment to automatically search for the offsets of specific timing properties from your timing set

After you have setup your timing specification, you can use *fast adjustment* to perform a timing search. Note that using the fast adjustment is optional.

For all edges of the digital waveforms defined in the *Wavetable* on page 34, a timing equation in the timing set specifies where each edge does exactly occur. In the waveform A shown below, the edge "d1" occurs after the time period "T_drive" has elapsed. The timing equation that describes this is "d1=T_drive". Similarly for the other edges "d2", "d3", and "d4" in the figure below, the timing equations are: "d2=T_drive+T_1+5ns", "d3=T_drive+T_1+5ns+T_2", and "d4=T_drive+2*T_1+T_2+5ns", respectively.



```
spec timing_example {
// Initialize timing variables
var Time per=60ns;
var Time T_drive=per/4;
var Time T_1=8ns;
var Time T_2=25ns;
// Declaring the timing set name
set timingSet_1;
  setup digInOut DUT_Digital_signals {
// Wavetable definition
  wavetable wvt1 {
    xModes = 1 {
      A: d1:1 d2:0 d3:1 d4:0;
    }
  }
}
// Timing set
set timing timingSet_1 {
  period=per; // waveform period
  d1=T_drive;
  d2=T_drive+T_1+5ns;
  d3=T_drive+T_1+5ns+T_2;
  d4=T_drive+2*T_1+5ns+T_2;
}
}
```

Related concepts

Measurement specification on page 24

The measurement specification (often referred to as "specification") describes which *instruments* a measurement uses and which settings are applied to them. A specification includes signal groups and aliases, timings, levels, wavetables, waveforms, and instrument setups, but does not include *patterns* or *operating sequences*. The measurement specification also includes the protocol and transaction sequence definitions for *protocol-aware testing*.

Wavetable on page 34

The wavetable is part of the *timing specification* of digital instruments. It specifies what kind of digital waveform (NRZ, RZ,...) is applied to the DUT digital signals and what *x-mode* to use. The wavetable is defined in the measurement "specification file (Topic 244401)".

X-modes on page 40

An x-mode is a setup convention that fits multiple device cycles into a single test system cycle, or tester cycle, for short. The purpose of x-modes is to run tests with higher data rates, and to use memory more efficiently.

Fast adjustment on page 45

SmarTest provides fast adjustment to adjust the timing edges or level voltages based on a fast embedded search mechanism.

Level specification on page 43

The level specification is part of the *measurement specifications* of digital instruments. It involves setting up the power supply voltages for your DUT and the thresholds of the logic states for its digital signals.

Related information

Timing set (digInOut sets)

Waveform

Device cycle time and test system cycle time have to be distinguished.

Waveform in signal processing

In the processing of electrical signals, a waveform is a predefined variation of voltage or current over time. The visual representation of the shape of a signal processing waveform is a plot of this variation (y-axis) over time (x-axis).

In automated test equipment, signal processing waveforms serve as stimuli for a device test. Arbitrary signal processing waveforms are generated using digital synthesis techniques, and applied to a DUT.

An example of a digital signal processing waveform is shown below.



Digital signal processing waveform

Looking at a signal processing waveform from the DUT's perspective, a signal processing waveform is a sequence of edges and edge actions that is used to control and analyze a DUT.

A signal processing waveform as a time-continuous signal with the length of one period is specified by the following components:

- SmarTest waveform, or waveform in short, representing the shape, as defined in the *wavetable*, and represented by a state character
- timing specification, representing the placement of edges and edge actions, as done in the *wavetable*
- *level specification*.

Period of a waveform

The period of a waveform is the elapsed time from the begin of a waveform to its end, that is the time required for one complete cycle of the wave to occur. In case of a periodic waveform, the end is as well the starting point for the next instance of a periodic waveform. An example of a periodic digital waveform is shown below.



Periodic waveform

Device cycle

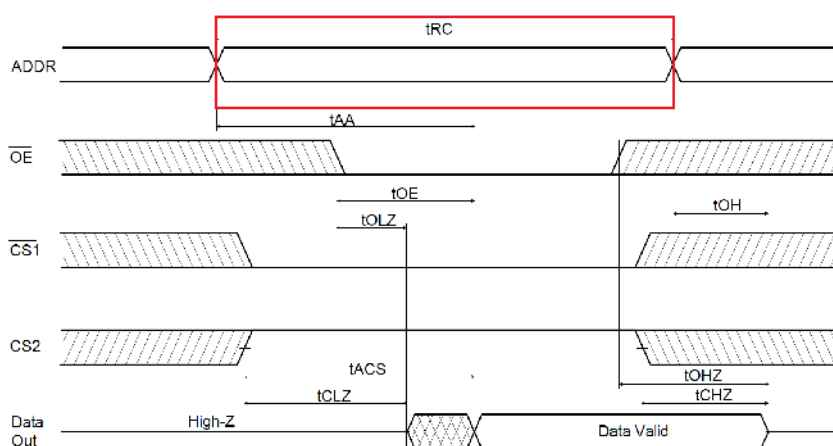
The device cycle is a sequence of (edge) actions that are applied to a DUT in the context of an operation on a DUT. It includes the definitions of waveforms for all DUT signals, or a subset of the DUT signals for this time interval.

Device cycle time

The device cycle time is the elapsed time between start of one operation on a DUT, for instance a memory read operation, and the earliest time of start of the next operation on the DUT as shown in the example below.

TIMING DIAGRAM

READ CYCLE



Device cycle time

As such, the device cycle time has a minimum characteristic. Device cycle time is ruled by latencies, internal processing times, and transfer times. Hence, the device cycle time does not necessarily correlate to integer multiples of the clock cycle time of the DUT.

Usually, the device cycle time is given in the datasheet of the DUT, per operation.

The basic test approach is to have the period of a device waveform and the device cycle time identical: a device waveform is a stimulus applied during a full device cycle.

DUT cycle time

The DUT cycle time is the time it takes to execute one device waveform on a DUT. Hence, it is identical with the device cycle time.

Test system cycle

The test system cycle, or tester cycle, is a sequence of one or more parallel tester waveforms that are applied to a DUT. It includes the definitions of tester waveforms for all DUT signals, or a subset of the DUT signals for this time interval.

In its simplest form, the tester waveform is identical with the device waveform. It is possible as well to place more than one device waveform and thus more than one device cycle into a test system cycle (tester waveform), using the *x-mode*. This allows for testing DUTs with device cycle times shorter than the shortest test system cycle time.

On the opposite side, it is possible to compose a device waveform by an integer sequence of up to 16 tester waveforms. The latter is called fractional *x-mode*, or padding mode. For instance, in X1/4-mode one device waveform is composed of four sequential tester waveforms.

Test system cycle time

The test system cycle time, or test system period, is the elapsed time between begin and end of processing a tester waveform. It is as well the time interval (sequencer period) between two subsequent sequencer instructions.

There is always a minimum figure which is determined by the digital card used, the available speed license, and the number of edge actions used in a tester waveform. Refer to "Minimum sequencer period (base rate) for digital cards (Topic 244169)" for more information, and the relationship of the base rate with the data rate.

Base rate

The "base rate (Topic 244169)" is a frequency defined by $1 / (\text{minimum test system cycle time})$.

Sequencer

This is a low-level execution engine inside a test processor. A sequencer triggers setup changes on the hardware level, controls the generation of waveforms, and drives the test execution per signal.

Edge

An edge is a transition of a digital signal from low to high (rising), or high to low (falling) at an instant on the time scale.

Edge position

The edge position, or edge delay, is a time indication which specifies the positioning of an edge relative to the beginning of the associated test system cycle.

Edge action

Refer to "edge action (Topic 252021)".

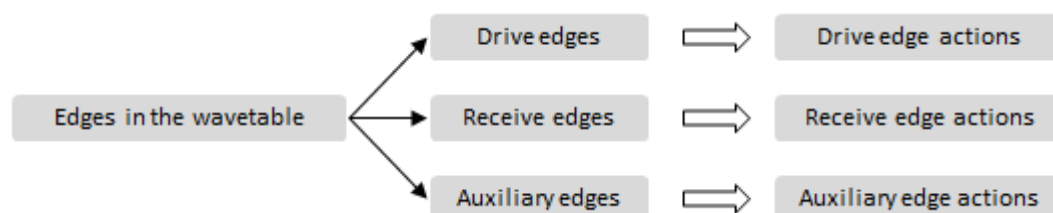
Wavetable

The wavetable is part of the *timing specification* of digital instruments. It specifies what kind of digital waveform (NRZ, RZ,...) is applied to the DUT digital signals and what *x-mode* to use. The wavetable is defined in the measurement "specification file (Topic 244401)".

For each digital signal that is part of the current measurement, the wavetable contains the definition of the digital waveforms that can be applied to it and the *x-mode* to use. In the wavetable, each defined waveform is identified with a unique state character and each state character is assigned one or more edges and edge actions.

State characters in the wavetable

A waveform is identified with a unique state character using any single character of the list "a-z,A-Z,0-9". The combination of edges and edge actions that are assigned to a state character is collectively referred to as a device cycle. A linear array of N state characters from the wavetable constitute a vector which you can use in the *pattern* file to apply the nth digital waveform to the nth digital signal at the same time, $1 \leq n \leq N = \text{number of digital signals}$. Note that you can define up to 256 waveforms in your wavetable. Also, you can set some state characters combinations as used or unused, or map them to another set of state characters.

Edges and edge actions in the wavetable

There are multiple types of edges and edge actions that you can specify for a waveform in the wavetable, for instance:

1. Drive edges and edge actions: a drive edge describes that the driver circuitry of the digital instrument connected to an input signal will drive an electrical level into the DUT signal. The drive edge action specifies what electrical level is driven.

2. Receive edges and edge actions: a receive edge describes that the receiver circuitry of the digital instrument connected to an output signal would receive an electrical DUT signal from it in order to compare it with an expected electrical level. The receive edge action specifies to what electrical level it should be compared.
3. Auxiliary edges and edge actions: an auxiliary edge is any edge other than drive or receive edges, for example, third and fourth-level edges. Auxiliary edge actions specify the action associated with the auxiliary edge. Auxiliary edges and edge actions are used in conjunction with drive edges and edge actions.

For a list of drive, auxiliary, and receive edge actions, see "list of drive, auxiliary, and receive edge actions (Topic 251208)".

The exact time points where the edges occur and the exact electrical levels of the edge actions are defined in the *timing* and the *level* sets of the digital instrument, and not in the wavetable. The edge delays for the drive, receive, and auxiliary edges are specified by the timing properties *d*, *r*, and *t*, respectively. When the auxiliary edge delay is not specified, then the drive edge delay is used by default, that is, $t = d$. See also *digInOut sets and properties*.

Note

Drive edges that have the same timing (that is, occur at the same place in a waveform) cannot be used together in the same waveform. Using them together would make the measurement execution undefined and unpredictable.

⚠ CAUTION Incorrect stimulus signals

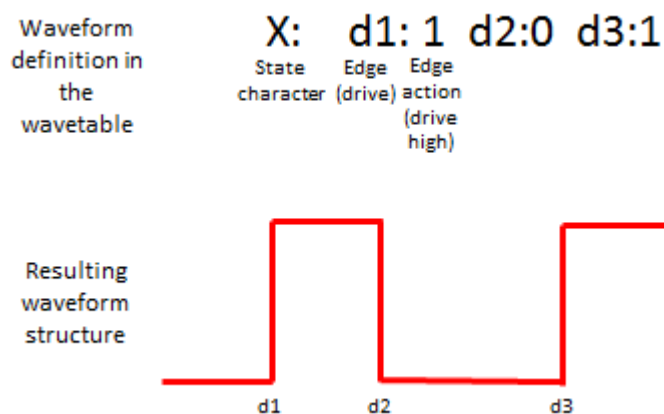
Using two drive edges with the same timing in one waveform makes the measurement execution unpredictable and may lead to test escapes.

➔ Do not specify waveforms that contain more than one drive edge with the same timing.

Setting up a wavetable for x-modes up to X8 mode

The waveform "X" having the shape shown in the figure below is defined in the wavetable as shown in this example. The waveform is given a state character to identify it (x in this example). Then, the required edges (*d1*, *d2*, and *d3*) and edge actions (1, and 0) are assigned to this state character.

```
// Wavetable is defined in the specification file
spec timing_example {
// Wavetable can be setup for the digInOut instrument
  setup digInOut Digital_signal {
    // Wavetable definition
    wavetable wvt1 {
      // Specifying the x-mode to use
      xModes = 1 {
        // The waveform shape is defined: It is identified by the state character "X".
        // Then, the edges d1, d2, and d3 and the corresponding edge actions 0 and 1
are
        // assigned to it.
        X: d1:1 d2:0 d3:1;
      }
    }
  }
}
```



You can use this example to apply higher x-modes than X1. As the waveform x contains 3 drive actions, and the total number of drive actions is limited to 8 in one tester cycle, only the x-mode X2 could be applied here.

Setting up a wavetable for x-modes higher than X8 mode and reducing the number of waveforms generated from it

X10, X12, X14, and X16 modes are supported when the `digInOut` instrument on the Pin Scale SL (PSSL) card is operating in the precision timed mode (that is, `precisionTimed=true;`). When these x-modes are used, you have to make sure that the number of generated waveforms is within the limit of 256 waveforms per wavetable. You can achieve this by defining which state character combinations (waveforms) are used or unused in your wavetable and by mapping sets of state character combination to different ones. Also, you can only assign one edge and edge action per state character when the x-mode is greater than X8.

The following example illustrates a wavetable setup for X16 mode. The wavetable defines three state characters: L, H, and X. Each state character has exactly one edge and one edge action assigned to it. The number of possible state character combinations that can be generated from this wavetable is 3^{16} . To keep the wavetable within the 256-waveform limit, define which state character combinations should be used or unused (using the `used` and `unused` elements) and map sets of the state character combinations to different ones (using `map`).

When using the `used`, `unused`, or `map` elements with X10 mode and above, you need to split the statement into two parts:

1. The element (for example: `unused`) is specified followed by the first x-mode/2 (for instance, eight for X16 mode) state characters of the required state character combination, each separated by a `*` symbol. An additional `*` is specified right after the last state character to indicate that this is the first part. Note that any required auxiliary edge actions (if any) must be assigned only to the state characters that are in the first part.
2. In the next line, the element, in this example: `unused`, is specified again followed by the remaining x-mode/2 (for instance, eight for X16 mode) state characters of the required state character combination, each separated by a `*` symbol. An additional `*` is specified right before the first state character to indicate that this is the second part. Note that the auxiliary edge actions (if any) that are assigned to state characters located in the second part will be ignored.

For the `map` element, this syntax applies to both, the original as well as the target state character combination.

In the code example below, the `unused` element sets the state character combination `LLLL LLLL LLLL LLLL` as unused. Then, the `map` element maps the state character combinations `[H X]HH HHHH HHHH HH[H X]` to `XXXX XXXX XXXX XXXX`:

```
spec demo_x16mode_wavetable {
```

```

setup digInout IO_1 {
...
//Wavetable definition
wavetable wvt1 {
  // Specifying X16 mode (16 device cycles (state character) per tester cycle)
  xModes=16;
  // Defining state characters (waveforms)
  L: r1: L;
  H: r1: H;
  X: r1: X;
  // Reducing the possible number of state character combinations
  // Removing the combination LLLL LLLL LLLL LLLL
  // First, the first 8 (x-mode/2) state characters namely: LLLL LLLL are specified
  // Notice that an additional "*" symbol is used after the last state character.
  unused L * L * L * L * L * L * L * L *;
  // Next, the last 8 (x-mode/2) state characters namely: LLLL LLLL are specified
  // Notice that an additional "*" symbol is used before the first state character.
  unused * L * L * L * L * L * L * L * L *;
  // Mapping a set of state character combinations:
  // [H X]HH HHHH HHHH HH[H X] into XXXX XXXX XXXX XXXX
  // First, the first 8 (x-mode/2) state characters namely: [H X]HH HHHH are
  specified
  // Notice that an additional "*" symbol is used after the last state character on
  both sides.
  map [H X] * H * H * H * H * H * H * H * => X * X * X * X * X * X * X * X *;
  // Second, the last 8 (that is, X-mode/2) state characters namely: HHHH HH[X H] are
  specified
  // Notice that an additional "*" symbol is used before the first state character on
  both sides.
  map * H * H * H * H * H * H * H * [H X] => * X * X * X * X * X * X * X * X *; //
  defines mapping for wavetable B
}
}
}

```

Setting up a wavetable on differential connections

To define the wavetable for your differential connections, you have two options based on the way how the drive edge actions for the negative connection are defined. Namely, using a standard wavetable setup as in the example above or an extended wavetable setup:

1. Using a standard wavetable setup:

In this method, you define your wavetable as in the single-ended connection. You only have to define the waveforms for the positive connection. Upon binding, SmarTest will automatically create the waveforms for the negative connection according to the following table:

Drive edge action	Positive connection (user assigned)	Negative connection (SmarTest assigned)
F00	F00	F10
F10	F10	F00
F0Z	F0Z	F1Z
F1Z	F1Z	F0Z

For a list of edge actions in the wavetable, refer to "list of drive, auxiliary, and receive edge actions in the wavetable (Topic 251208)".

The code snippet below shows an example where the wavetable for the positive connection IO_1 is defined. The drive edge actions for the negative connection are automatically

assigned upon binding according to the table above. That is, for the waveform "0" the drive edge action for the negative connection is F10, for waveform "1" it will be F00, and so on.

```
// Wavetable is defined in the specification file
spec differential_standard_wavetable {
    // The wavetable is set up only for the positive connection of the differential
    pair.
    setup digInOut IO_1 {
        wavetable myStandardWavetable {
            // Specifying the x-mode to use:
            xModes = 1 {
                // The waveforms and their shapes are defined.
                // Note: The edge actions for the negative connection are obtained from the
                table.
                // Waveform "0" is assigned the edge "d1" and the edge action "F00".
                // "F10" will be used for the negative connection by default.
                0: d1:F00;
                // Waveform "1" is assigned the edge "d1" and the edge action "F10".
                // "F00" will be used for the negative connection by default.
                1: d1:F10;
                // Waveform "T" is assigned the edge "d1" and the edge action "F1Z".
                // "F0Z" will be used for the negative connection by default.
                T: d1:F1Z;
            }
        }
    }
}
```

2. Using an extended wavetable setup:

This is similar to the standard wavetable setup, you define the wavetable as if you are using single-ended connection. The difference is that in this method you specify the drive edge action for the negative connection as well. This method can be used when you want to assign a particular drive edge action to the negative connection and not use the value that SmarTest automatically assigns as indicated in the table above. The following code snippet shows this method. For instance, for the waveform "T" the drive edge action for the positive connection is F1Z and for the negative connection it is also F1Z (that is, not the drive edge action F0Z that SmarTest normally assigns to the negative connection (see the table above) when you use a standard wavetable setup):

```
spec differential_extended_wavetable {
    // The wavetable is set up only for the positive connection of the differential
    pair.
    setup digInOut IO_1 {
        wavetable myExtendedWavetable {
            // Specifying the x-mode to use:
            xModes = 1 {
                // Defining the waveforms "0", "1", and "T" and specifying their shapes.
                // Note: The second edge actions are meant for the negative connection.
                0: d1:F00,F10;
                1: d1:F10,F00;
                T: d1:F1Z,F1Z;
            }
        }
    }
}
```

Reducing the number of potential waveforms for higher X-modes and selecting the appropriate type of drive edge

Consider the following code for a JTAG TCK pogo pin.

```
setup digInOut myTCK_Port {
    set timing myTCK_Timing {
        period = 100.0 ns;
        d1 = 0.0 ns;
        d2 = 25.0 ns;
```

```

    }
}

setup digInOut myTCK_Port {
    wavetable myTCK_Wavetable {
        xModes=5;
        0 : d2:0; // drive edge no. 2 F00
        1 : d2:1; // drive edge no. 2 F10
        Z : d1:Z; // drive edge no. 1 FNZ
        used 0 0 0 0 0;

        ...
        used 1 1 1 1 0;
        used 1 1 1 1 1;
        used Z Z Z Z Z;
    }
}

```

This code above makes use of the fact that the digital data waveforms based on "0" and "1" are used on this DUT mutually exclusive with the one of going to the high-Z status (tri-state) of the driver (there are no test vectors required such as 0 1 0 Z 0). This allows you to restrict the number of waveform combinations from 243 down to $32 + 1 = 33 \leq 256$ as shown above.

However, there are two drive edges defined, one at d2 for data, and one at d1 for the tri-state action. To have two drive edges placed in a tester cycle is not possible in x-mode X5, as being outlined in *X-modes* on page 40.

The solution is to have the first edge for entering the high-Z status redefined as a pure auxiliary edge.

The following line of code

```
Z : d1:Z; // drive edge no. 1 FNZ
```

must be replaced by

```
Z : t1:Z; // auxiliary edge no. 1 FNZ
```

As the value of t1 of the auxiliary edge is not specified, it will be identical with the one of d1. Refer to "List of drive, auxiliary, and receive edge actions in the wavetable" (topic 251208).

Related concepts

Timing specification on page 28

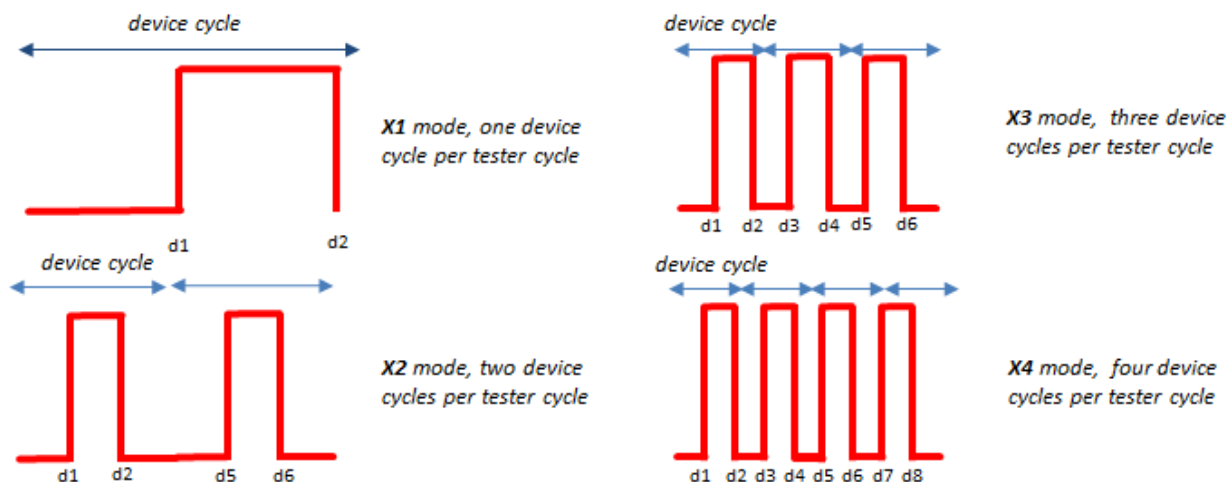
The timing specification is part of the *measurement specifications* of digital instruments. It involves defining the digital waveforms in the *Wavetable* on page 34 and specifying the time points where the edges in the defined waveforms, for example: the drive or receive edges, exactly occur.

Measurement specification on page 24

The measurement specification (often referred to as "specification") describes which *instruments* a measurement uses and which settings are applied to them. A specification includes signal groups and aliases, timings, levels, wavetables, waveforms, and instrument setups, but does not include *patterns* or *operating sequences*. The measurement specification also includes the protocol and transaction sequence definitions for *protocol-aware testing*.

X-modes

An x-mode is a setup convention that fits multiple device cycles into a single test system cycle, or tester cycle, for short. The purpose of x-modes is to run tests with higher data rates, and to use memory more efficiently.



The figure above depicts sequences of device cycles with identical device waveforms. This is not a necessary condition for using x-modes. Rather, SmarTest allocates all combinations of device waveforms packed into one tester cycle. For instance, the following code snippet results in the generation of the 256 device waveforms 00000000, 00000001, 00000010, ..., 11111100, 11111101, 11111110, 11111111 within one tester cycle.

```
xModes = 8; {
  0: d1:F00;
  1: d1:F10;
}
```

Higher data rates with x-modes

The "X" implies multiplication, for example, X2 mode means that there are two device cycles found in one tester cycle. This multiplying effect is accomplished by fitting more drive, receive, or auxiliary edges in each tester cycle. For example, a device cycle containing one pulse (represented by two drive edges) could occur four times within a tester cycle. However, the maximum number of edges in a tester cycle is limited

- maximum 8 drive edges, 8 receive edges, and 8 auxiliary edges per tester cycle for the cards AVI64, PS1600, PS5000, PS9G, PSSL.
- maximum 16 drive edges, 16 receive edges, and 16 auxiliary edges per tester cycle for the card PSSL card operating in the "Precision timed mode, PSSL card" (topic 343961).

SmarTest supports the X1, X2, X3, X4, X5, X6, X7, and X8 modes.

Note

SmarTest will support X10, X12, X14, and X16 modes if the `digInOut` instrument on the Pin Scale SL (PSSL) card is operating in the "Precision timed mode, PSSL card" (topic 343961), that is, `precisionTimed=true;`). With those higher X-modes, the PSSL card can provide data rates of up to 16 Gbit/s.

Long periods with higher x-modes

Some cards have limitations on the device cycle edge placements when using very big tester periods (in the order of more than 10 μ s), where edges may be restricted to the beginning of the tester cycle. Occurrence of an edge outside this limit will lead to a bind error. For X1 mode, where device cycle and tester cycle are identical, the edge timing error message reports the specific limits. For X2 and higher modes, an additional message indicating an invalid user period/x-mode combination is served, which also reports the restriction limit.

Memory utilization with x-modes

SmarTest always allocates memory for 256 waveforms. Using a higher x-mode increases the waveform memory utilization. For example, in X1 mode, only one state character (waveform) is applied per tester cycle. In this case, you will make full use of the allocated waveform memory only if you define the 256 waveforms manually. However, in a higher x-mode, such as X4 mode, four state characters are sequentially applied to the DUT per tester cycle. You still define your wavetable as for the X1 mode. However, SmarTest will automatically generate the combinations of four state characters (for up to a maximum of 256 waveforms) and thus more waveform memory will be utilized.

Since the x-mode specifies how many state characters are sequentially grouped together in one tester cycle, you can specify the following number of drive and receive edges. The table is based on the maximum number of edges that can be placed into one tester cycle.

x-mode	Maximum number of drive edges and receive edges per state character (waveform)	Edges that can be defined in one state character (waveform)
X10, X12, X14, X16 of PSSSL in precision timed mode	1	d1 r1
X8	1	d1 r1
X7	1	d1 r1
X6	1	d1 r1
X5	1	d1 r1
X4	2	d1, d2 r1, r2
X3	2	d1, d2 r1, r2
X2	4	d1, d2, d3, d4 r1, r2, r3, r4
X1	8	d1, d2, d3, d4, d5, d6, d7, d8 r1, r2, r3, r4, r5, r6, r7, r8

You can reduce the number of generated state character combinations (waveform sequences in one tester cycle) to keep them within the maximum limit of 256 waveforms by declaring some of the generated combinations as unused or mapping them to other combinations. .

Note

Reducing the number of waveforms to the maximum of 256 waveforms per wavetable is necessary in particular when higher x-modes are used, such as the precision timed mode on the Pin Scale SL (PSSL) card. For x-modes higher than X8 mode, you specify the used, unused, or mapped state character combinations in two steps per state character combination. See the example "Setting up a wavetable for x-modes higher than X8 mode" in *Wavetable* on page 34.

X-mode transparency in SmarTest

In general, moving from X1 to higher x-modes, you will continue to define your timing specification (timing set and wavetable) as if you work in X1 mode. Upon execution, SmarTest automatically generates the required state character combinations (waveforms) from your wavetable and reflects the required changes on your timing sets for the higher x-mode. See the example section.

Note

For X-modes higher than X8, you specify the used, unused, or mapped state character combinations in two steps per state character combination. See the example, "Setting up a wavetable for x-modes higher than X8 mode" in *Wavetable* on page 34.

X-mode constraints

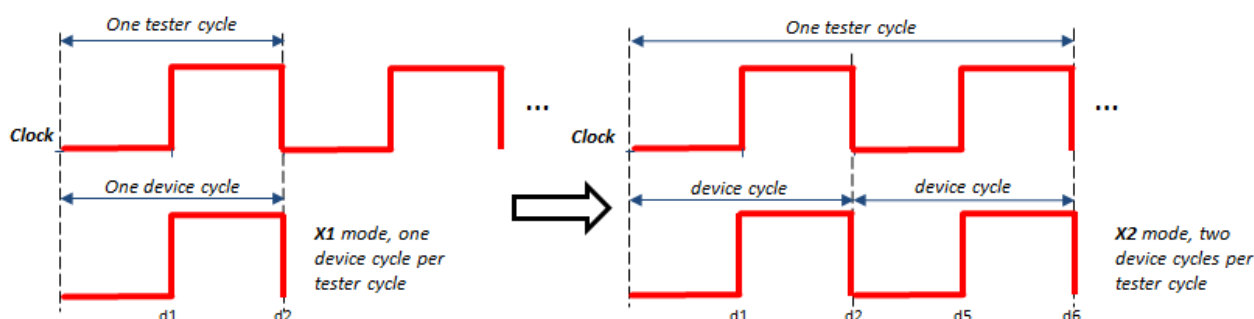
There are some constraints related to the X-mode in use:

- The number of vectors in the *Pattern* on page 53 must be an integer multiple of the x-mode. For example, if you use X8 mode, the pattern must consist of at least 8 vectors or an integer multiple of this number. A pattern that consists of say, 701 vectors is not allowed in X8 mode, as this would leave the vector positions 702 to 720 in the tester cycle undefined.
- The number of vectors between sequencer instructions must be an integer multiple of the x-mode. For example, in this sample sequence of sequencer instructions, the number of vectors generated with the "genVec (Topic 246397)" instruction must be an integer multiple of the used x-mode for the reason explained above:

```
changeTiming
genVec
changeTiming
```

- For x-modes X5 and higher, only one drive, receive and auxiliary edge, and corresponding edge action can be specified per state character. The same restriction is applicable to x-modes higher than X8 running on a PSSS card in precision timed mode.

The example below shows how you specify your timing specifications for X2 mode. The figure on the left shows the setup in X1 mode. Moving to the X2 mode (right) requires no additional effort from you. That is, you define the waveforms in the wavetable and specify the timing equations for the edges d1 and d2 as if you work in X1 mode. Upon execution, SmarTest automatically generates the state character combinations for the X2-mode as well as the timing equations for the additional edges d5 and d6 and modifies the period of the tester cycle accordingly. The waveform applied to your DUT signals is the one on the right (X2 mode). Note that the Pattern editor and the Timing Debug view are unaffected by the choice of x-mode. They display the vectors or waveforms that are applied to your DUT signals in terms of device cycles not tester cycles.



Related concepts

Wavetable on page 34

The wavetable is part of the *timing specification* of digital instruments. It specifies what kind of digital waveform (NRZ, RZ,...) is applied to the DUT digital signals and what *x-mode* to use. The wavetable is defined in the measurement "specification file (Topic 244401)".

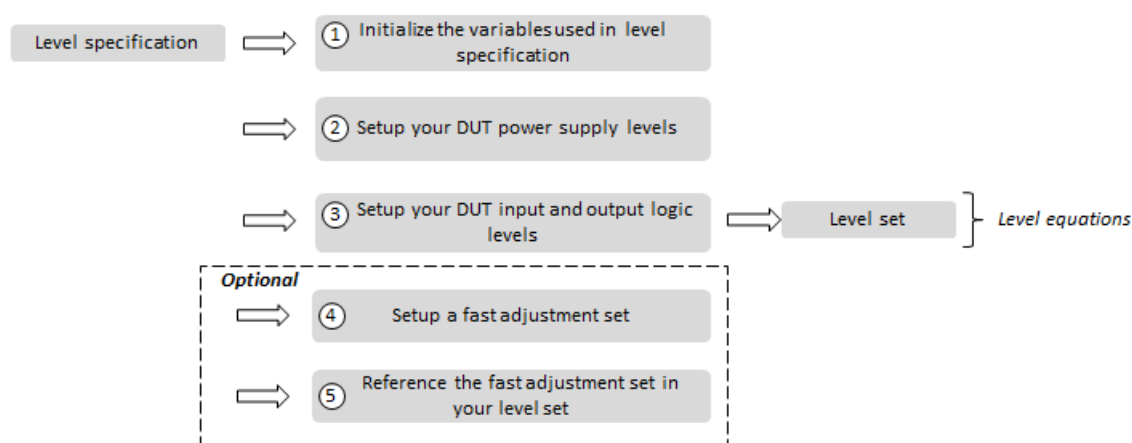
Timing specification on page 28

The timing specification is part of the *measurement specifications* of digital instruments. It involves defining the digital waveforms in the *Wavetable* on page 34 and specifying the time points where the edges in the defined waveforms, for example: the drive or receive edges, exactly occur.

Level specification

The level specification is part of the *measurement specifications* of digital instruments. It involves setting up the power supply voltages for your DUT and the thresholds of the logic states for its digital signals.

Level specification in SmarTest



Power supply level setup, dcVI instrument

The DC power supply voltage level to power up your DUT (typically, vcc, vdd, or vss) is setup via the instrument level properties, in particular, the `level.vforce` property of the `dcVI` instrument. The example below shows how the power supply level is specified.

Note

The `digInOut` instrument can also be used to power supply your DUT. However in this case, the power supply voltage level would be specified as for digital signals as shown below.

Digital signals level setup, level sets and level equations

The level specification for the digital signals is defined as a set of level equations, or "level set" for short. In a level set, the level equations specify the level properties, for instance, the voltage thresholds of the logic states that are applied to the input or expected from the output DUT signals, namely `vil` and `vih`, and `vol` and `voh`. These voltage thresholds are part of the level set properties of the `digInOut` instrument (`vil` and `vih` are also supported by the `clock` instrument). Upon execution, they are used to reflect the exact values to assign to the drive and receive edge actions of the waveforms defined in your wavetable. See the example below.

Note

You can only assign values of the expected data type to specification variables, constants, or instrument properties.

For example, the property `level.linear.steps` expects a value of integer (long) type, both `vdd` and `OV_swing` are of type long:

- Incorrect:

```
level.linear.steps = (vdd - OV_swing)/0.005000;
```

A division always returns a floating point value, even if the result is a whole number (for example, 1.0). As `level.linear.steps` expects an integer, this will result in an error.

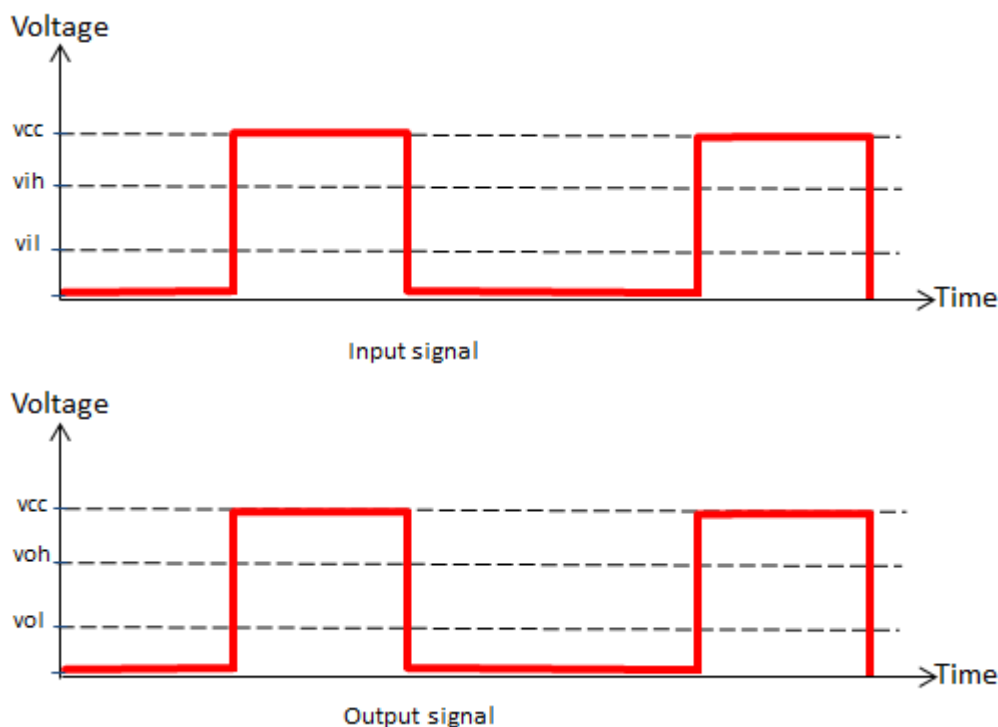
- Correct:

```
level.linear.steps = (vdd - OV_swing) * 200;
```

Using fast adjustment to automatically search for the offsets of specific level properties from your level set

After you have setup your level specification, you can use *fast adjustment* to perform a level search. Note that using the fast adjustment is optional.

In the level set, the level equations can be used to assign the voltage levels to be applied on the DUT inputs, or the levels that are expected from its outputs. For example, the voltage thresholds `vil` and `vih` (logic levels low and high, respectively) applied to the input signals or `vol` and `voh` (logic levels low and high, respectively) expected from the output signals:



An example of level specification specified within a measurement specification file is shown below:

Related concepts

Measurement specification on page 24

The measurement specification (often referred to as "specification") describes which *instruments* a measurement uses and which settings are applied to them. A specification

includes signal groups and aliases, timings, levels, wavetables, waveforms, and instrument setups, but does not include *patterns* or *operating sequences*. The measurement specification also includes the protocol and transaction sequence definitions for *protocol-aware testing*.

Timing specification on page 28

The timing specification is part of the *measurement specifications* of digital instruments. It involves defining the digital waveforms in the *Wavetable* on page 34 and specifying the time points where the edges in the defined waveforms, for example: the drive or receive edges, exactly occur.

Wavetable on page 34

The wavetable is part of the *timing specification* of digital instruments. It specifies what kind of digital waveform (NRZ, RZ,...) is applied to the DUT digital signals and what *x-mode* to use. The wavetable is defined in the measurement "specification file (Topic 244401)".

Fast adjustment on page 45

SmarTest provides fast adjustment to adjust the timing edges or level voltages based on a fast embedded search mechanism.

Related information

Level set and Timing (digInOut sets)

Fast adjustment

SmarTest provides fast adjustment to adjust the timing edges or level voltages based on a fast embedded search mechanism.

With fast adjustment, you can perform an embedded search over a specific property (for example, a timing edge or a level voltage) from the timing or level set. Three use cases for executing a fast adjustment search are:

- To search for the offset of a timing or level property that is at the left boundary of a pass-fail (PF) transition.
- To search for the offset of a timing or level property that is at the right boundary of a fail-pass (FP) transition.
- To search for the offset of a timing or level property that is at the middle of a fail-pass-fail (FPF) transition eye, that is the average value of the two offset at the FP transition and PF transition respectively.

Fast adjustment is a method to search for the boundaries or middle of the eye for level and timing in order to find the best strobe or threshold position.

The search result (offset) is automatically added to the original value of the searched property after the search is done. The result is applied automatically to all test suites that refer to the executed fast adjustment set by specifying in the corresponding timing or level sets. Therefore, you do not need to specify additionally to apply the fast adjustment result.

Note

There is no way to reset the fast adjustment offset. If you do not want the offset to be applied, you have to use a timing or level set without specifying the `fastAdjustment`.

You can manually set up a fast adjustment search following the procedure in "Using fast adjustment (Topic 256889)", you can also programmatically set up a fast adjustment search using the Device Setup API.

You can retrieve the fast adjustment result in the test method via the result accessor *IFastAdjustmentResults* if needed. The fast adjustment result handling follows "General guidelines for preserving and retrieving results (Topic 347927)".

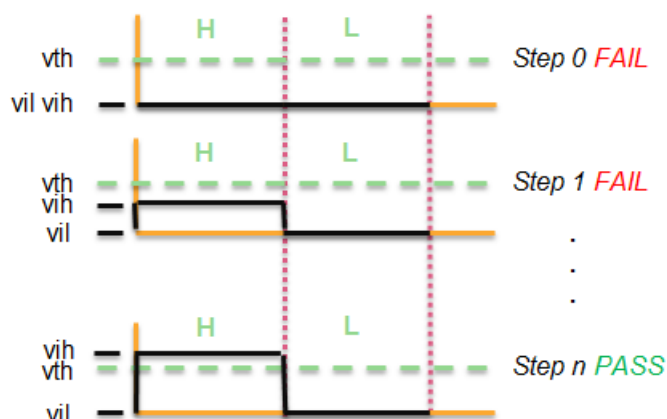
The following figures illustrate the concept of fast adjustment:

Level search

You can perform a fast adjustment search on the voltage properties from the level set, such as `vil`, `vih` or `vth`(see *level.resource*). The search finds the offset according to the search result type. This offset is added to the original value of the voltage property.

For details on the level set properties, see *digInOut level set properties* in *digInOut sets and their properties*.

Level Search (increments or decrements voltages)



This figure illustrates how a fast adjustment search works for adjusting the property v_{ih} with the FP offset. The search is performed by incrementing the v_{ih} voltage in a specified number of steps. The search finds the offset of v_{ih} that results in a FP transition at step n . This offset is added to the original v_{ih} value in the level set. All level sets that reference this fast adjustment set will also make use of the search results.

For more details, see the example "Performing a fast adjustment search on the high voltage drive level (Topic 256889)".

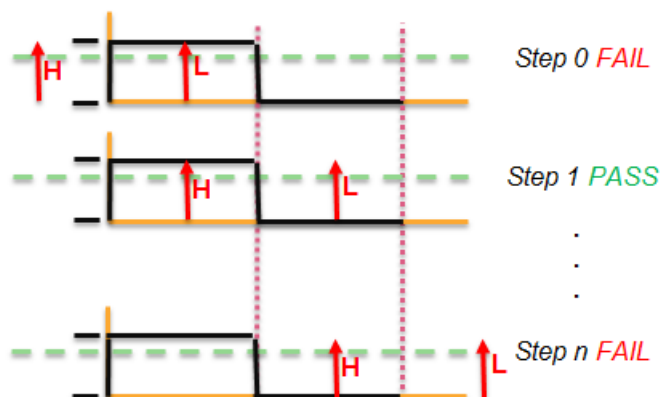
Timing search

You can perform a fast adjustment search on the timing properties from the timing set, such as *drive* or *receive* edges (see *timing.resource*).

The search finds the offset according to the search result type. This offset is added to the original values of the timing set properties.

For more details on these edge types see, *Wavetable* on page 34 and *digInOut timing set properties* in *digInOut sets and their properties*.

Timing Search (shifts edges)



This figure illustrates how a fast adjustment search works for adjusting the receive edges with the FPF offset. The search is performed by shifting the *receive* edges within a range in a specified number of steps. The search finds the FPF offset that is at the middle of the fail-pass-fail transition eye. This offset is added to the original values of the receive edges. All timing sets that reference this fast adjustment set will also make use of the search results.

For more details, see the example "Performing a fast adjustment search on the receive edges (Topic 256889)".

Search algorithm

The embedded search is a combination of a linear search, a binary search and a histogram search. It is mandatory to specify the linear search. Binary search and histogram search are optional. They are used to refine the search result after the linear search.

- Linear search

The linear search scans the entire specified search range for closed FPF eyes and chooses the largest one for extracting the offset according to the search result type (FP, PF or FPF). If there are no closed FPF eyes, it also considers open eyes.

For example, a 10-step linear search is shown as follows:

```
F-P-P-F-F-P-P-P-F-F-F
```

The second closed FPF eye (F-P-P-P-F) is the largest eye for extracting the offset according to the specified search result type. If neither the binary search or the histogram search is specified, the search is ended and the offset is added to the original value of the search property automatically:

- PF: return the offset that is correspond to the P at the PF transition.
- FP: return the offset that is correspond to the P at the FP transition.
- FPF: return the offset that is correspond to the middle of the FPF eye.

Note

If the `fastStop` mode is enabled, the linear search stops after the first FPF eye is found. For the example shown above, the first closed eye (F-P-P-F) is used for extracting the offset.

- Binary search

The binary search continues after the linear search if it is specified. The binary search range is the F-P or P-F transition, or both. It stops when the transition ranges are reduced to be smaller than the binary resolution. If the histogram search is not specified, the search is ended and the offset is added to the original value of the search property automatically:

- PF: return the offset that is correspond to the P at the PF transition.
- FP: return the offset that is correspond to the P at the PF transition.
- FPF: return the offset that is correspond to the middle FPF eye.

To exclude any fail results that are present at the start (end) of the putative pass region, you can move the start (end) of the pass area, as determined by the binary search, to the right (left) with n consecutive steps with pass results, where n is definable. The step width equals the binary search resolution.

- Histogram search

Histogram search starts after the binary search if it is specified. It adjusts the P boundaries of the closed FPF transition eye more accurately. The number of steps and the search resolution are definable.

Elements of setting up a fast adjustment search set

Element	Sub-element	Required	Description
searchType		Yes	The three types of transition to find the offset for: • PF - pass to fail • FP - fail to pass • FPF - fail to pass to fail For example: <pre>searchType = FPF;</pre>
timing Either level or timing must be specified. They are mutually exclusive.	resource	Yes	The level property to be searched, which is one of the following properties: • receive - receive edges • drive - drive edges contain both diveOnly and tristateOnly • diveOnly - fast drive edges • tristateOnly - slow drive edges For example: <pre>timing.resource = receive;</pre>
	linear.start	Yes	The offset to start the linear search from. For example: <pre>timing.linear.start = 100 ns;</pre>
	linear.steps	Yes	The number of steps to perform the linear search (excluding the start step). For example: <pre>timing.linear.steps = 10;</pre>
	linear.stepWidth	Yes	The step size to perform the linear search.
	binary.resolution	No	Binary search resolution, which the searched distance between a pass and fail step is less than.
	histogram.resolution	No	The step size of the histogram search.
	histogram.minStableErrorCount	No	Specifies the number of search steps where the error count must be stable. The histogram search runs after binary search until the number reaches minStableErrorCount.

Element	Sub-element	Requ	Description
level Either level or timing must be specified. They are mutually exclusive.	resource	Yes	The level property to be searched, which is one of the following voltage properties: • vil - high driver levels • vih - low driver levels • viOffset - the search of the offsets for vih and vil at the same time • viSwing - driver swings • vol - low receiver levels • voh - high receiver levels • voOffset - the search of the offsets for voh and vol at the same time • voSwing - receiver swings • vth - receiver thresholds • vt - termination voltages For example: <pre>level.resource = vih;</pre>
	linear.start	Yes	The offset to start the linear search from. For example: <pre>level.linear.start = 1V;</pre>
	linear.steps	Yes	The number of steps to perform the linear search (excluding the start step). For example: <pre>level.linear.steps = 10;</pre>
	linear.stepWidth	Yes	The step size to perform the linear search. For example: <pre>level.linear.stepWidth = 0.1V;</pre>
	binary.resolution	No	Binary search resolution, which the searched distance between a pass and fail step is less than.
	histogram.resolution	No	The step size of the histogram search.
	histogram.minStableErrorCount	No	Specifies the number of search steps where the error count must be stable. The histogram search runs after binary search until the number reaches minStableErrorCount.
fastStop		No	If set to true, the fast adjustment search stops as soon as the first valid eye or a transition is found. If set to false, adjustment will search for the biggest eye in search range.
errorCountLimit		No	Specifies the minimum number of errors. If there are fewer errors than this limit, the current search step is considered as pass.
minPassCount		No	Specifies the minimum number of consecutive passes for a transition that must be seen in order to stop the binary search.
resultSignals		No	The DUT signal(s) for which the search is defined. The search result depends on pass/fail result of these signal(s). The default setting is that the result is taken from the signals where the fast adjustment set is defined.
tracing		No	Enables to trace the results for each search step performed. The search results (offsets) are printed to the console view.

Waveform specification

SmarTest enables you to define analog or digital waveforms. This topic provides information on setting up analog waveforms.

Analog waveforms

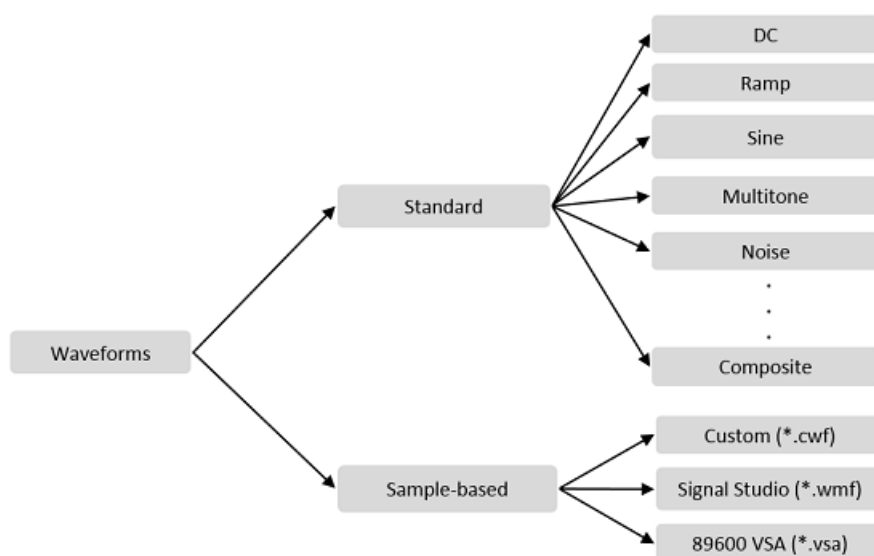
Digital waveforms can be set up, for example, using the wavetable in the timing specification of digital instruments. Analog waveform data used for the waveform definitions in `awg`, `rfStim` and `dcVI` instruments are contained in the waveform specification. They contain the complete signal description including the timing information (sample rate).

You can define analog waveforms using the `waveform` element in the "measurement specification file (Topic 244401)". The required waveform is assigned to an instrument, for example, `awg`, `dcVI`, or the `rfStim` instrument by referencing the waveform in the action definition of the instrument's setup in the specification file. The waveform description comes in two different flavors:

- "standard waveform (Topic 250941)" based on a shape (sine, multi-tone, etc.)
- "sample-based waveform (Topic 250943)" based on a list of individual, equidistant samples

Note

For `rfStim` and `awg` instruments, the amplitude of a waveform is normalized upon binding to the range -1.0 to 1.0. Also, the waveform should consist of at least 32 samples. The total number of samples must be an integer multiple of two.



Waveform types

SmarTest provides standard waveforms, for example: *DC*, *Ramp*, and *Noise*, and supports sample-based waveforms that are either custom formatted or are from external software such as *Signal Studio*, or *89600 VSA*. These waveforms are defined and setup using the waveform element in the specification file (`.spec`).

Standard Waveforms

"Standard waveform (Topic 250941)" descriptions are the preferred way to describe analog waveforms. They are easy to set up, easy to understand, and it is easy to modify them.

The signal quality can differ significantly between setups using standard and sample-based waveforms. When standard waveforms are used in the setup, SmarTest optimizes the instrument setup for maximum performance (sample rate, filter selection, compensation, and pre-distortion).

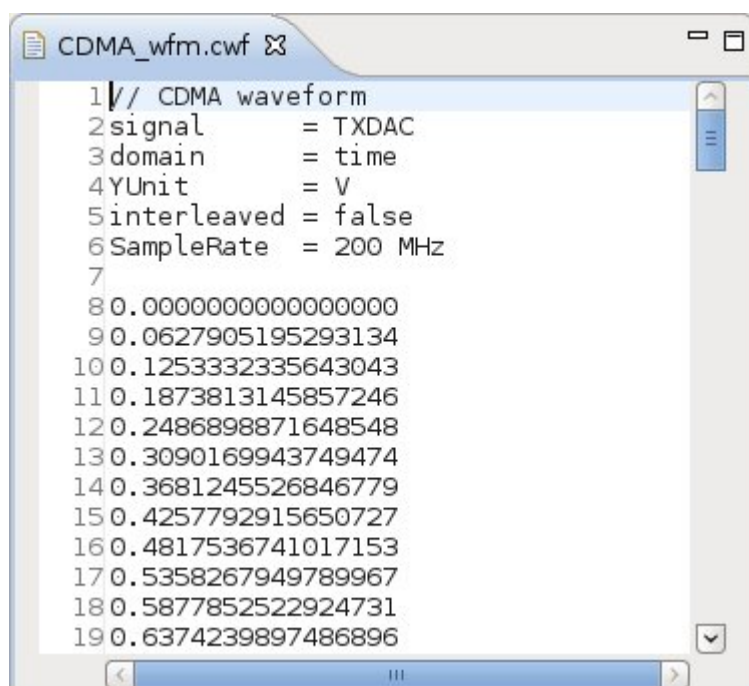


Standard Waveforms

Sample-based waveforms

"Sample-based waveforms (Topic 250943)" are contained in external files of different formats. The supported file formats are plain ASCII (.cwf), Signal Studio (.wfm), and 89600 (.vsa). Plain ASCII files consist of an optional header section with metadata, and a data section with the actual sample values. The file format is backwards compatible with the custom waveform file format used in SmarTest 7.

The sample rate is either part of the waveform specification or the waveform file header. If both values are set, the value from the specification is taken; resampling is not performed. The format of the numbers within a waveform file is arbitrary.



Custom waveform file

Note

SmarTest removes the DC content of all waveforms sourced by AWGs, and rescales the AC content to the amplitude set by the property `amplitude`. The offset DACs (digital to analog converters) exclusively drive the DC content. They can be set up using the properties `offset` and `offsetNeg`.

Duration of analog waveforms

Sample size, rate, and the repeat count define the duration of sample-based waveforms. The duration of standard waveforms can be determined by the property `duration`, and that of sine waves also by the property `periods`.

Hardware specific remarks**PS1600 AWG**

The PS1600 AWG supports only standard waveforms with sample rates from 1 Hz to 2 MHz.

WSMX HR

WSMX HR supports AWG sample rates from 10 ksps (10 sps for ramps) to 200 ksps.

WSMX HS

WSMX HS supports AWG sample rates from 1 Msps to 500 Msps.

The example below shows two sample-based waveforms and one standard waveform (`sine`) are defined within in the measurement specification file.

Related concepts

Wavetable on page 34

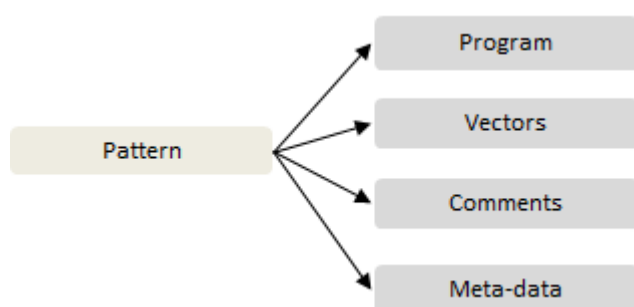
The wavetable is part of the *timing specification* of digital instruments. It specifies what kind of digital waveform (NRZ, RZ,...) is applied to the DUT digital signals and what *x-mode* to use. The wavetable is defined in the measurement "specification file (Topic 244401)".

Pattern

A pattern is a series of vectors and sequencer instructions. During a measurement, the pattern is applied to a set of the DUT signals. It can also be used to control the execution of actions of the instruments connected to these DUT signals.

Patterns are structured as:

- Sequencer instructions
- Series of vectors
- Comments (optional)
- Metadata (optional)



Pattern

The pattern file (.pat) consists of four entries: program, vectors, comments, and metadata. Each entry is contained in a separate file. The program entry contains sequencer instructions. For example, anchors to actions. The vector entry contains the vectors to be applied to your DUT signals. The comment and the metadata entries are optional. The files that make up the pattern are combined in a single .zip archive

Sequencer instructions

"Sequencer instructions (Topic 244374)" control your test execution. For example, they allow you to do a loop or branch the test execution, or call an action and anchor it to a specific vector, or repeat a certain vector or set of vectors.

Vectors

A vector is a linear array of state characters (waveform DC/DC power units). The state characters encode waveforms. The active *wavetable* lists which state character represents which waveform.

Each state character is applied to its corresponding DUT signal, at a given time stamp. Each vector must contain the same number of state characters, one for each DUT signal to which the pattern applies.

Using patterns in SmarTest

Patterns can be specified directly in *test suites*. Alternatively, they can be grouped in an *operating sequence*, which in turn is specified in a test suite.

The following example shows a pattern specified in a test suite, and an operating sequence specified in another test suite that calls three patterns.

Another pattern file that is opened in the pattern editor is shown below.

The screenshot shows the **PatternEditor_EditorMode_InstructionAndAction** window. It contains a table with columns for Vector#, Instruction, Comment, and Actions. The Actions column is further divided into DUT signals (D00(SC) to R03(SC)) and State characters. Annotations include:

- X-mode**: Points to the 'Signal X-Mode: x1' header.
- Site memory sharing status**: Points to 'Site Memory Sharing: OFF'.
- Comments (vector specific)**: Points to the 'Comment' column.
- Sequencer instructions**: Points to the 'Instruction' column.
- Vector numbers**: Points to the 'Vector#' column.
- Annotations**: Points to the 'Annotated' checkbox.
- Tabs**: Points to the 'Pattern' and 'Global Comments' tabs at the bottom.
- DUT signals**: Points to the columns D00(SC) through R03(SC).
- Name of anchored action**: Points to the 'vforce_1' action.
- State characters**: Points to the columns H, L, and H in the Actions section.

Vector#	Instruction	Comment	Actions
0			D00(SC) D01(SC) D02(SC) D03(SC) R00(SC) R01(SC) R02(SC) R03(SC)
1	RPTV 1, 10	Repeat vector	0 0 0 0 L L L L
2			1 0 0 0 H L L L
3			0 1 0 0 L H L L
4			1 1 0 0 H H L L
5	LOOP 20	Repeat loop	0 0 1 0 L L H L
6		Repeat loop	1 0 1 0 H L H L
7	LOOPEND		0 1 1 0 L H H L
8		An anchored action	1 1 1 0 H H H L
9			0 0 0 1 L L L H
10			1 0 0 1 H L L H
11			1 1 0 1 H L H H
12			0 0 1 1 L L H H
13			1 0 1 1 H L H H
14			0 1 1 1 L H H H
15			1 1 1 1 H H H H
16			0 0 0 0 L L L L
17			1 0 0 0 H L L L
18			0 1 0 0 L H L L
19			1 1 0 0 H H L L
20			0 0 1 0 L L H L
21			1 0 1 0 H L H L
22			0 1 1 0 L H H L
23			1 1 1 0 H H H L
24			0 0 0 1 L L L H
25			1 0 0 1 H L L H
26			0 1 0 1 L H L H
27			1 1 0 1 H H L H
28			0 0 1 1 L L H H
29			1 0 1 1 H L H H
30			0 1 1 1 L H H H

PatternEditor_EditorMode_InstructionAndAction

Memory pooling and memory sharing

Memory pooling and memory sharing are two techniques for using the memory of a test head card more efficiently.

Both memory pooling and memory sharing allow channels to utilize the memory of other channels within the same pool of channels.

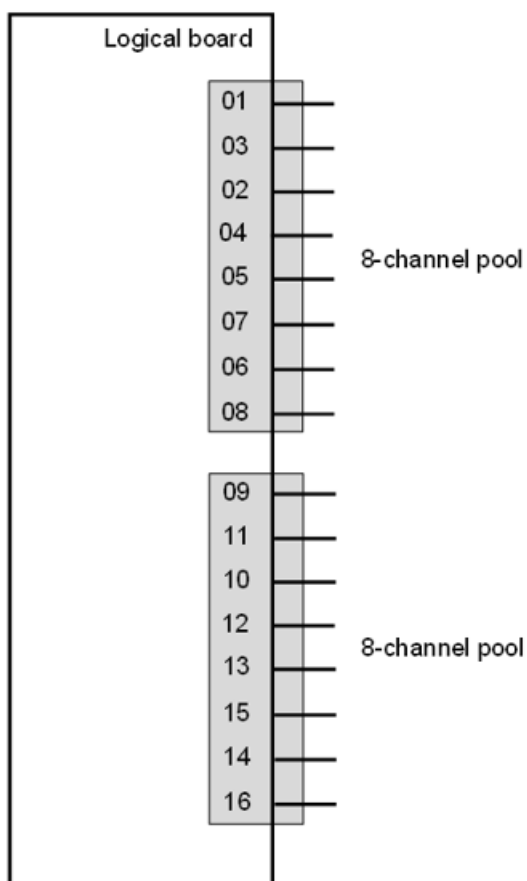
Memory pooling does not require adaptations to test programs, test methods, or patterns. Memory pooling operates transparently. Memory pooling is always active on test head cards that support it.

For memory sharing, you must explicitly specify the pattern to be stored in the shared memory as described in "Changing the Site Memory Sharing setting of a pattern file (Topic 349012)".

Memory pooling

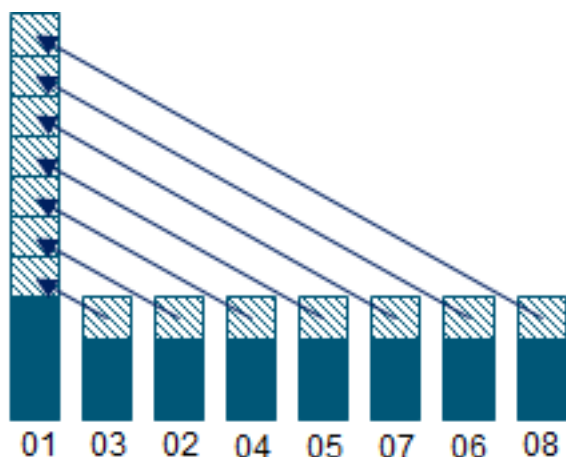
Channels that belong to the same 4-channel pool (PS9G, PSSSL), or to the same 8-channel pool (PS1600, PS5000) use the same physical memory. This feature helps in memory pooling.

A pool of channels is defined along the logical board the channels belong to. Refer to the situation of the 8-channel pools depicted below.



8-channel pools of memory pooling

Memory pooling allows you to distribute the vector memory within a 4-channel pool, or an 8-channel pool flexibly among the channels of the pool as depicted in the figure below.



Memory pooling

It is the capability of one channel to use the memory of other channels on a test processor which are licensed but not used. For example, if both pogo pin 10101 and pogo pin 10102 have 64 MV of licensed memory, and pin 10102 only uses 20 MV, up to 108 MV of memory is available for pin 10101: 64 MV of its own memory plus 44 MV of unused memory from 10102.

A pogo pin can only get access to pool memory if its license is greater or equal than the one which is donating. That is, memory cannot be transferred to pogo pins with less licensed memory.

For example, assume this licensing configuration:

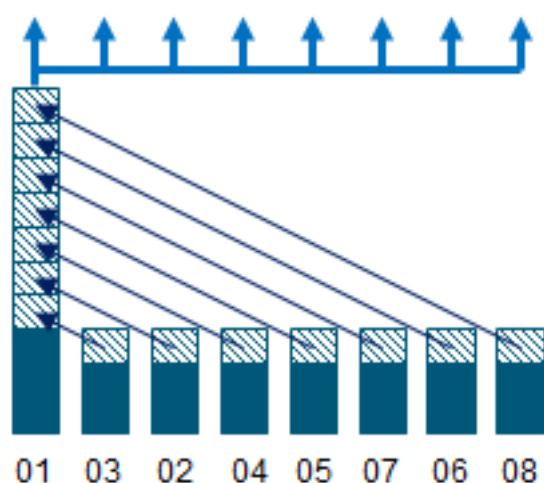
Pogo pin	Licensed memory
10101	112 MV
10102	64 MV
10103	16 MV
10104	16 MV

In this case, unused memory from pogo pin 10102 could be made available to pin 10101, but not to pogo pins 10103 or 10104. Unused memory from pogo pin 10103 could be made available to any pogo pin, including 10104.

Memory pooling is always active; there is no need to activate or configure it. The details of its operation are likewise automatic.

Memory sharing

In multisite testing, memory sharing allows several pogo pins that belong to the same memory pool and are assigned to the same signal (for different sites) to access the same instance of pattern data held by the first pogo pin. If a shared pattern is executed, the sequencers for the shared pogo pins read the data from the same shared memory block. Without memory sharing, the pattern data for each pogo pin is stored in its own memory.



Memory pooling and memory sharing

For example, in the following four-site configuration, where sites 1 and 2 for signal DP0 are configured to share the same memory of the 10101 pogo pin, during execution the pattern data is sent to pogo pins 10101 and 10102:

Pogo pin	Signal	Site
10101	DP0	1
10103	DN0	1
10102	DP0	2
10104	DN0	2
10105	DP0	3

Pogo pin	Signal	Site
10107	DN0	3
10106	DP0	4
10108	DN0	4

Although pogo pins 10105 and 10106 also belong to the DP0 signal, they are not configured as part of the same memory pool, so they cannot share the memory of pogo pin 10101. Instead, their pattern can be stored in the memory of pogo pin 10105 and sent from there to pins 10105 and 10106, or their pattern can be stored independently.

Memory sharing is a property of a pattern. You can specify whether a pattern is to use memory sharing when you create it, and you can later change this setting in various ways, as described in "Changing the Site Memory Sharing setting of a pattern file (Topic 349012)".

If memory sharing is active for a pattern, test methods cannot make site-specific modifications to the pattern - all sites must always execute the same pattern.

Action

Actions control the functions that an instrument can perform. For example, they can be used to stimulate the DUT and measure its response.

Actions are defined in the "measurement specification (Topic 244401)".

Calling an action

Actions can be triggered for execution from the following test program components:

- a *test method*
- a *pattern*
- an "operating sequence (topic 244261)".

Available actions

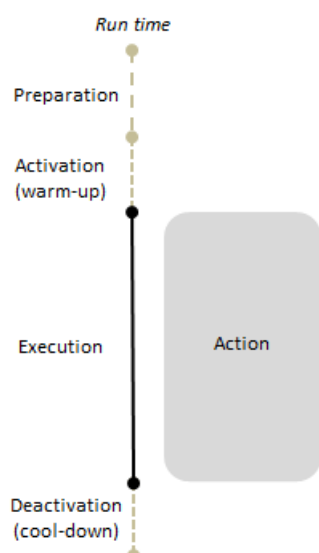
For a catalogue of available actions refer to the "list of instrument actions (topic 248884)".

Lifecycle of an action

To know the lifecycle of an action is relevant for the programming of repeated and synchronized actions.

The lifecycle of an action consists of the following phases:

- 1. Preparation:** The instrument might require some time to prepare itself (for example, to configure its hardware) before it can be activated to execute the action. This is known as the preparation phase.
- 2. Activation (warm-up):** The time period between the end of the instrument preparation phase and the beginning of the action execution. The activation (warm-up) phase is used, when required, to assure an exact time alignment of the action execution start.
- 3. Execution:** After the activation phase, the action starts executing. It is now in the execution phase.
- 4. Deactivation (cool-down):** The time period from the end of the execution phase until the instrument becomes unblocked. That is, it is available for other actions.



Action lifecycle

The following figure shows an example that illustrates how this information is shown in the "Timing Debug (Topic 246069)" view:



Action lifecycle as shown in the Timing Debug view

The settings of the respective instrument cannot be modified until the action has finished executing, or it is terminated.

In general, an action is terminated in one of the following cases:

- The execution of the action regularly ends, for instance a value has been captured, or an instrument status parameter has been changed, and the control is returned to the calling instance.
- The instrument it controls gets disconnected.
- The end of the operating sequence has been reached. For actions that continue to execute after the end of the operating sequence has been reached, refer to "Infinitely repeated execution of an action in a keep-alive context (Topic 357268)".
- The DUT signals they are applied to are used by a next element in the operating sequence, such as another action.
- You have explicitly set the action to terminate.

For an example, see *Repeating a sourceWaveform action of an awg instrument for a specific number of times* on page 63.

For an action with indefinitely repeated execution, that is an action running in an endless loop, the preparation, activation, and deactivation phases occur only once.

Calling an action

You can call an action from a test method, or from a vector in a pattern, or from an operating sequence to control an instrument.

Calling an action from a test method

The actions being available for test method development can be found in the Device Test API instrument package `"xoc.dta.instrument"`, grouped per interface.

Actions must have been defined in the *instrument setup*.

You use the "Device Setup API" (topic 252686) to create an instrument setup.

In a test method, actions are executed in the `execute()` section. Refer to "Test method structure" (topic 244308). An action in a test method is executed within an *operating sequence*, embedded in a sequential group, or anchored to a vector in a pattern.

Calling an action from a pattern

Actions can be triggered for execution from a *pattern* by synchronizing them with one of the vectors that the pattern contains. An action is synchronized to a specific vector in the pattern by an anchor. This anchor indicates that an action is to be executed when reaching the vector (identified by the vector number) with the anchor.

An anchor is represented by the red arrow in the figure below.

See "Vector element in the pattern file (topic 244371)" and the "Action call instruction" (topic 245222) instruction for more details.

Interruptive and non-interruptive actions called from a pattern

There can be three types of actions in a pattern:

Internal interruptive actions

Internal actions are performed on an instrument which is part of the pattern. The actions are applied to some or all of the DUT signals to which the pattern is applied. Executing an internal action stops the execution of the pattern temporarily. When the action has finished, the execution of the pattern is resumed.

Internal actions are always interruptive because the execution of the pattern has to wait for the completion of the action as both pattern execution and action execution potentially act on the same signals.

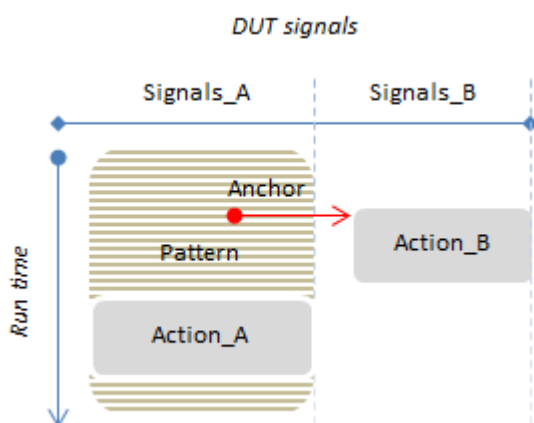
External interruptive actions

External actions are performed on an instrument which is not part of the pattern. In this case, the pattern execution can continue without a pause. However, you can explicitly make the action interruptive, that is, pause the pattern for the duration of the action completion by adding the statement `anchoring="interruptive"` as shown in the examples below.

External non-interruptive actions

External actions are performed on an instrument which is not part of the pattern: They are applied to DUT signals to which the pattern is not applied. Executing a non-interruptive action is performed without stopping the pattern.

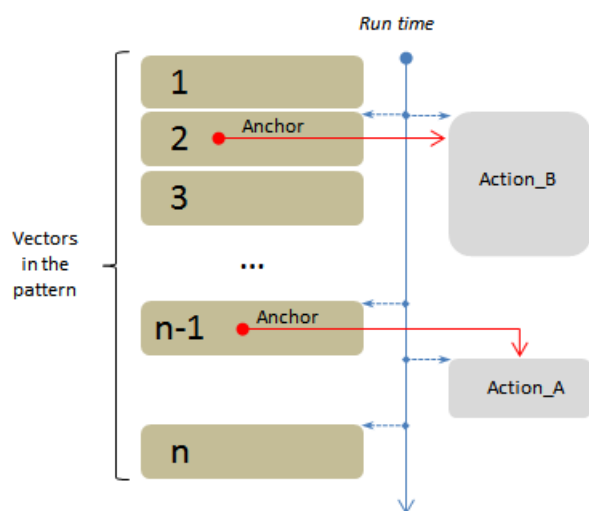
An interruptive action starts executing after the vector to which it is anchored has been executed. By contrast, a non-interruptive action starts executing at the same time as the vector to which it is anchored.



Interruptive and non-interruptive actions

Action_B in the figure above is non-interruptive. It applies to a different set of signals than those which the pattern applies to.

Action_A in the figure above is interruptive. It applies to the same set of signals that the pattern applies to. Note that an interruptive action can also apply to a subset of those signals.



Executing actions anchored in a pattern

Action_B in the figure above is a non-interruptive action. It is anchored to vector 2. When this vector is reached, both the action and the vector start executing at the same time.

Action_A in the figure above is an interruptive action. It is anchored to vector n-1. This action starts executing after the vector n-1 to which it is anchored has been executed. Vector n will start executing after the Action_A has been executed.

Single action anchored to a pattern

An example of an XML encoded anchor with a single action is given below.

```
<Program>
  <Anchor vector="191" anchoring="interruptive">
    <Instrument id="CP,DS0,DS7,IO0,IO1,IO2,IO3,IO4,IO5,IO6,IO7,Q0,Q7,S0,S1,MR_">
      <Instruction id="actionCall" value="ppmuIfVm1"/>
    </Instrument>
  </Anchor>
</Program>
```

General hints on data in pattern

Note

Entries in pattern files are related to each other, and modifying one of them with an external tool can easily break these relationships and render the whole pattern corrupt. We strongly recommend to work with pattern files using the Pattern editor (topic 249143). For more details, see working with pattern files (topic 247437).

Test methods can use the Device Test API (topic 244313) to modify patterns. This only modifies the patterns in memory at runtime, and the pattern files remain unchanged. The Device Test API cannot modify patterns that are not used in the test program.

Multiple actions in parallel called from a pattern

You can put more than one action into an anchor. If such an anchor is reached during pattern execution, the execution of all actions referred to in the anchor starts simultaneously.

The prerequisite to use this feature is that two parallel actions do not work on the same signal.

An anchor cannot contain parallel actions for instruments both inside and outside the pattern.

Multiple actions in parallel anchored to a pattern

An example of an XML encoded anchor with multiple actions in parallel is given below.

```
<Anchor vector="191" anchoring="interruptive">
  <Instrument id='pin01'>
    <Instruction id="actionCall" value="action01"/>
  </Instrument>
</Anchor>
<Anchor vector="191" anchoring="interruptive">
  <Instrument id='pin01'>
    <Instruction id="actionCall" value="action02"/>
  </Instrument>
</Anchor>
```

Sequence of actions called from a pattern

Instead of anchoring one or multiple parallel actions you can put a sequence of actions into an anchor. The test system sequentially executes the actions listed in the sequence.

This feature should help in the Transition of combined actions to single actions (topic 354095). If you do not make use of an anchored sequence of actions, the split of combined actions might imply the insertion of dummy vectors in a pattern between the single actions.

To add an action sequence to a pattern, use the method `addActionSequence` of the Device Setup API. In this linked section you will also find a code snippet to illustrate its usage.

Some restrictions apply to sequences of actions:

- A sequence of actions cannot be combined with multiple parallel actions.
A sequence of actions cannot be anchored to the same spot as other actions that would start executing simultaneously. If you need such a combination, consider using an operating sequence instead.
- All actions in the sequence must have the same type.
Either all actions in the sequence must be interruptive, or non can be interruptive.
- All actions in the sequence must either be internal or all must be external.
The instruments of the actions in the sequence must either be all part of the pattern or must all be outside of the pattern.

To insert a sequence of actions, you use the same procedures of pattern setup as you do for plain actions: Pattern Editor, Device Setup API, SmarTest Setup Format API.

Sequence of actions anchored to a pattern

An example of an XML encoded anchor with a sequence of actions is given below.

```
<Anchor vector="191" anchoring="interruptive">
  <Instrument id='pin01'>
    <Instruction id="actionCall1 in sequence" value="forceVoltage"/>
    <Instruction id="actionCall2 in sequence" value="measureCurrent"/>
  </Instrument>
</Anchor>
```

Calling an action from an operating sequence

An "Operating sequence (topic 244261)" is used to execute a list of patterns or actions, or a mix of both. For example, an operating sequence can be used to source and capture waveforms applied to your DUT with different sampling rates in a single measurement, or to apply an RF stimulus and measure the resulting signal at the same time.

Actions can be listed directly in the operating sequence. In this case, they will be executed in the order they are listed. The action can also be anchored to a pattern that is listed in the operating sequence. In this case, it will be executed when the pattern is executed, or, more precisely, when the vector it is anchored to in the pattern will have been reached.

An example of specifying actions in an operating sequence is shown below:

For details on actions and the operating sequence, see *Operating sequence* on page 64 and "Operating sequence file" (topic 244408).

When calling an action from an operating sequence, it can be narrowed, that is, it can be restricted to be executed on just a subset of the signals for which it was defined. See the section below, and "actionCall" (in topic 244408)" for the syntax.

Note that actions in operating sequences are normally arranged according to the start of their *execution phases*. This means that the preparation and activation of an action takes place before the point in time where the action is placed by the operating sequence. For example, when two actions follow each other, the second action is prepared and activated while the first one is still executing.

This kind of optimization with this time saving overlap is not always possible. In particular, when the preparation or activation can interfere with previous patterns or actions, the execution of the subsequent action may have to be delayed until those other elements have been completed.

If an action with this kind of interference comes after a "wait" (topic 244408) element, this time saving overlap of action also can be prevented. `wait` keeps up signal states from previous elements. For this reason, using `wait` to introduce a specific delay to the execution of an action may require additional measures, as described in "Synchronizing actions" (topic 354415).

Narrowing actions

As a default, an action potentially acts on all signals of an instrument as being specified in the instrument setup.

By default, a called action acts on all signals that are specified in the instrument setup. However, you can specify that it should be applied to only some of those signals, which is called "narrowing".

For example, to apply the `myAction` action to only the `VCC1` and `VCC2` signals (or, in other words, to narrow it to `VCC1` and `VCC2`):

```
actionCall myAction VCC1,VCC2;
```

Note that narrowing requires that at least one enabled signal is specified. If all specified signals are disabled in the *DUT board description* (using `@Disabled`), no narrowing is performed: The action is applied to all signals from the instrument setup.

Synchronizing actions

It is frequently necessary to start multiple actions synchronously, that is, at precisely the same time. In some cases, it may also be required to start actions with a precise offset between them, for example, to create signals with a particular phase.

SmarTest offers different means to coordinate actions in these ways.

Refer to "Synchronizing actions" (topic 354415) how to use this feature.

Repeated execution of an action

You can specify how often the execution of an action should be repeated.

Normally, actions are executed only once.

Many actions can be executed repeatedly.

Finite number of execution repetitions

This is achieved by specifying the `repeatCount` property for an action in the specification file (`*.spec`) and assigning a repetition value to it:

```
// Repeating an action for 10 times
repeatCount=10;
```

Repeating a sourceWaveform action of an awg instrument for a specific number of times

You can use the `repeatCount` property in the setup of the `sourceWaveform` action of the `awg` instrument to repeat it for a specific number of times:

Infinitely repeated execution of an action

There are actions that can be defined to be infinitely executed, that is the action is executed in an endless loop. These types of actions are useful, for example, in case you want to source a waveform into your DUT continuously, or to apply a constant DC voltage to it.

The infinitely repeated execution of an action will come to an end if one of the following conditions is met:

- The action is forced to *terminate*.
- The instrument gets disconnected.
- The next parallel group of the operating sequence has been reached that involves this instrument.
- The end of the measurement (operating sequence) has been reached. This case will not be applicable if the action is executed in a keep-alive context. For an ongoing execution of the action beyond this border, refer to the description of the keep-alive context below.

If you make use of infinitely repeated execution of an action you will have to observe certain restrictions. Refer to "Infinitely repeated execution of an action (Topic 358672)" for the details.

Before an action is terminated, you cannot modify the settings of the instrument it uses.

Infinitely repeated execution of an action in a keep-alive context

You can put actions with infinitely repeated execution, and only these, in a keep-alive context of the instrument to have them running also after the end of a measurement execution.

If a keep-alive context is active, the subject actions with infinitely repeated execution will continue to be executed after the end of the measurement, that is after the end of the operating sequence.

For a termination of a keep-alive action, the mechanisms to terminate an action with infinitely repeated execution are applicable, refer to the section above, and also to *Action* on page 57.

The keep-alive context is an instrument property. If keep-alive is active it will be applicable to all actions of this instrument with infinitely repeated execution.

For information about the keep-alive context specific per instrument, refer to "Infinitely repeated execution of an action in a keep-alive context (Topic 357268)".

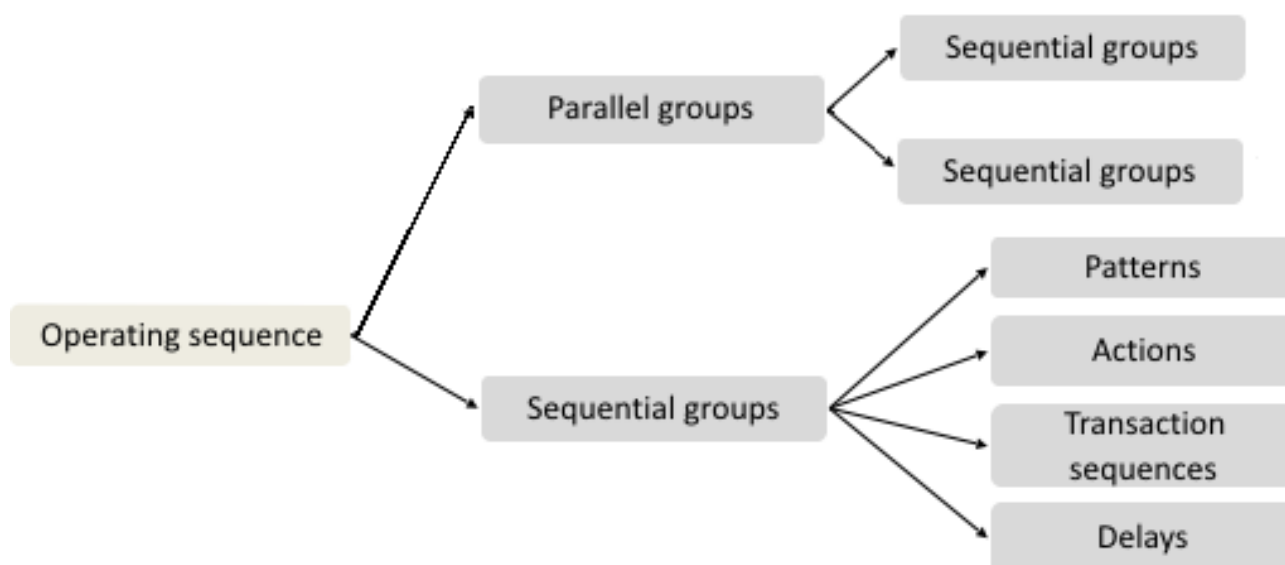
Operating sequence

An operating sequence specifies the order in which a set of patterns, actions, and transaction sequences are executed. Elements can be executed in series and in parallel.

An operating sequence provides a hierarchical structure for the execution of patterns and related elements on the tester. These elements are contained in parallel and sequential groups.

Sequential groups execute the elements they contain one after the other - the second element begins executing after the first one has finished. They can contain any combination of *patterns*, *actions*, *transaction sequences*, delays (wait times), and parallel groups.

Parallel groups start executing all elements at the same time. They contain only sequential groups.



Operating sequence

For a full description of the structure of an operating sequence, see Operating sequence setup reference (topic 244108).

Nesting elements in an operating sequence

Parallel and sequential groups can be nested inside each other: A parallel group can contain sequential groups, and a sequential group can contain parallel groups. This nesting is not limited in depth - a sequential group can contain a parallel group, which can contain a sequential group, which can contain a parallel group, and so on.

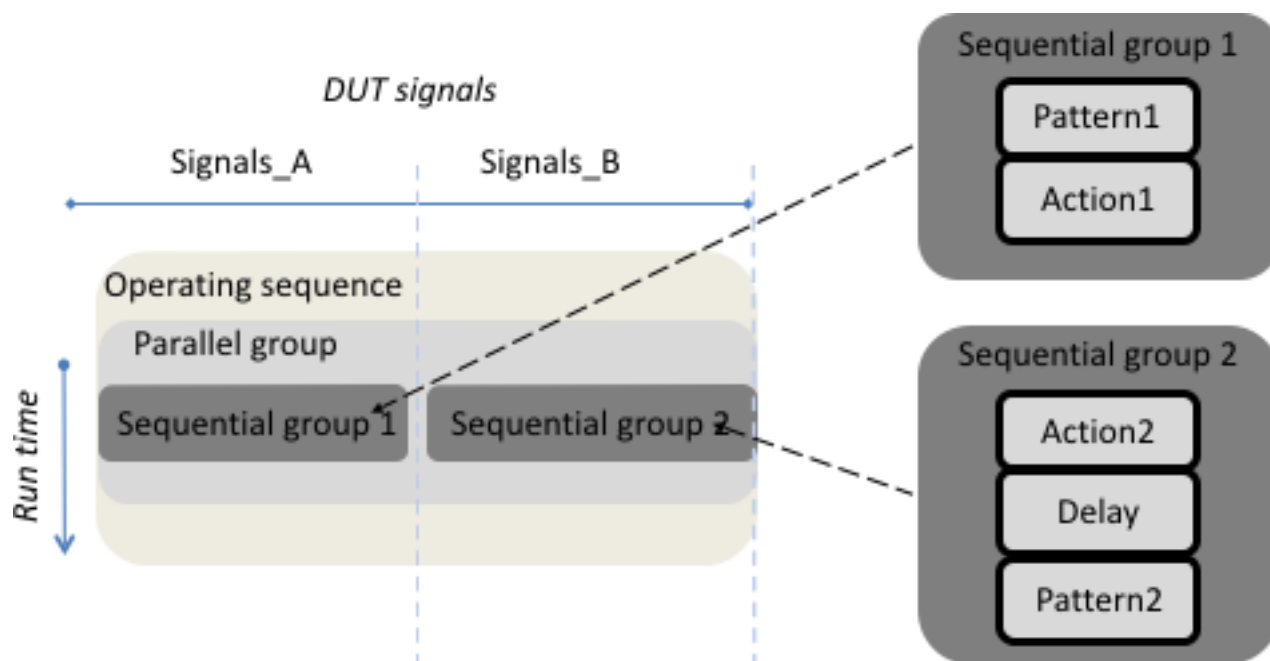
However, parallel and sequential groups cannot nest within themselves. That is, a sequential group cannot directly contain another sequential group (but it can contain a parallel group that contains a sequential group) and a parallel group cannot directly contain another parallel group (but a sequential group that contains a parallel group).

Patterns, actions, and transaction sequences cannot be nested, they can only be contained in sequential groups.

Synchronizing elements of an operating sequence

Three methods exist for synchronizing different parts of an operating sequence: parallel groups, anchored actions, and wait elements:

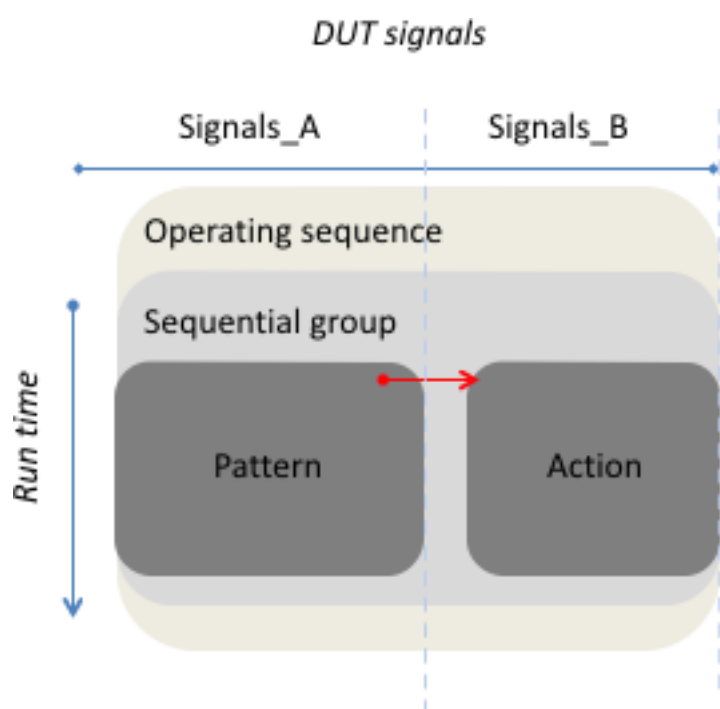
- Parallel groups can be used to start any number of different parts at the same time: Each sequential group contained in the same parallel group is started at the same instance, so making sure that two or more different parts of the operating sequence are synchronized can be achieved by putting each of them in a separate sequential group and encapsulating those in a parallel groups.



Note that the system may need to add padding to ensure that all elements are indeed started at the same time, as described below.

When the previous parallel group contained patterns or transaction sequences with a non-deterministic duration, the system resynchronizes the timing (topic 256797) before starting the next parallel group.

- Anchored *actions* are performed when the pattern vector to which they are anchored is executed. This can be used, for example, to reconfigure an instrument at a precisely determined instance.

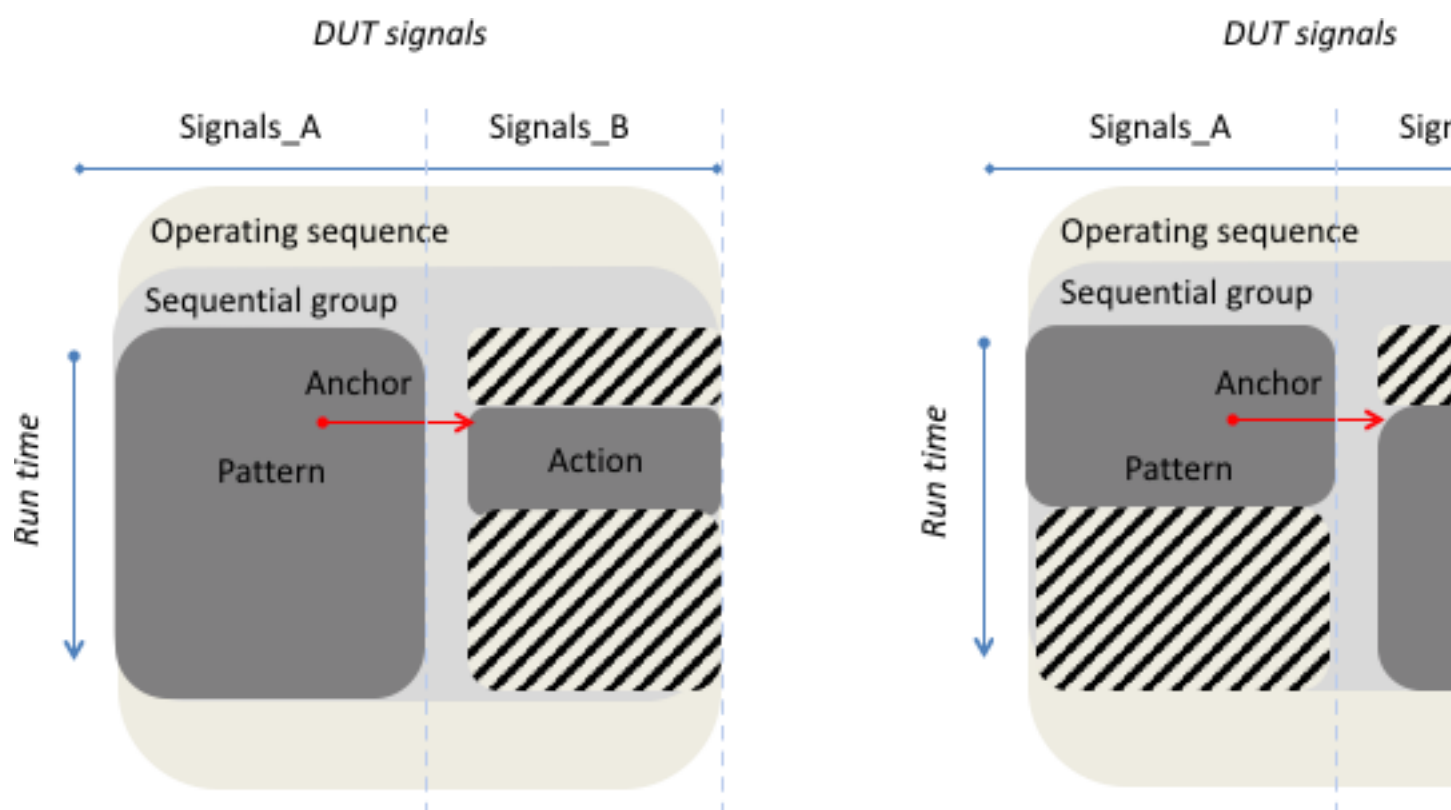


- wait elements (in topic 244108) can be used to delay the execution of the following elements in a sequential group for a fixed time, for example, to ensure that the DUT reaches a stable condition. While this can also be used for simple synchronization purposes, using parallel groups and anchored actions is usually more robust.

Padding

Parallel execution, and synchronizing in particular, may require padding, that is, the insertion of break vectors or idle sequences. Such padding is applied automatically by the system; manual padding is available through wait elements (in topic 244108).

Padding can be applied before an element, for example, to ensure that all sequential groups in a parallel group start simultaneously. It can also be applied after an element. For example, when two patterns execute in parallel and one finishes before the other, padding is applied on the finished pattern's signals until the other pattern finishes executing.



In the graphic above, padding is applied on the signal group Signals_B both before and after the patterns or actions.

Padding is not applied after elements that do not terminate by themselves, such as keep-alive actions and infinite loops. Keep-alive actions (topic 357268) continue to run even between measurements until they are explicitly terminated. Infinite loops are automatically terminated by the start of the next parallel group (or the end of the measurement).

Padding usually consists of break vectors. However, when transaction sequences are used, padding can also be the idle sequence (topic 252690) defined for the protocol.

Padding can have performance implications, as described in *Minimizing padding in parallel groups* on page 85 for details.

Conditional execution and loops

The flow of execution within a sequential group can be controlled using conditional gotos: A condition is evaluated, and if it is true, the execution continues not with the next instruction in the sequence but at a named location elsewhere within it.

This can be used to create loops (when the named location is earlier in the sequence) or to omit parts of the operating sequence (when the named location is later).

Conditional execution cannot be used to go to a different sequential group.

For more information, see *Operating sequence conditional goto jumps* on page 70.

Types of operating sequences

In addition to the standard operating sequences described here, two special types exist:

- Burst operating sequences just call other operating sequences; they do not possess the structure described below. They often can be executed much faster. For details, see "SmartBurst" (topic 353398).

- Burstable operating sequences are operating sequences that can be called from burst operating sequences. They are standard operating sequences, but must observe certain limitations as described in "SmartBurst" (topic 353398).

Performance considerations

Operating sequences can, in general, be constructed without taking performance into consideration. However, in cases where you are prepared to tweak the code to maximize performance, there are areas you can consider.

Padding that is used to synchronize elements can introduce delays in execution. Reducing such padding is a complex issue that is discussed in *Minimizing padding in parallel groups* on page 85.

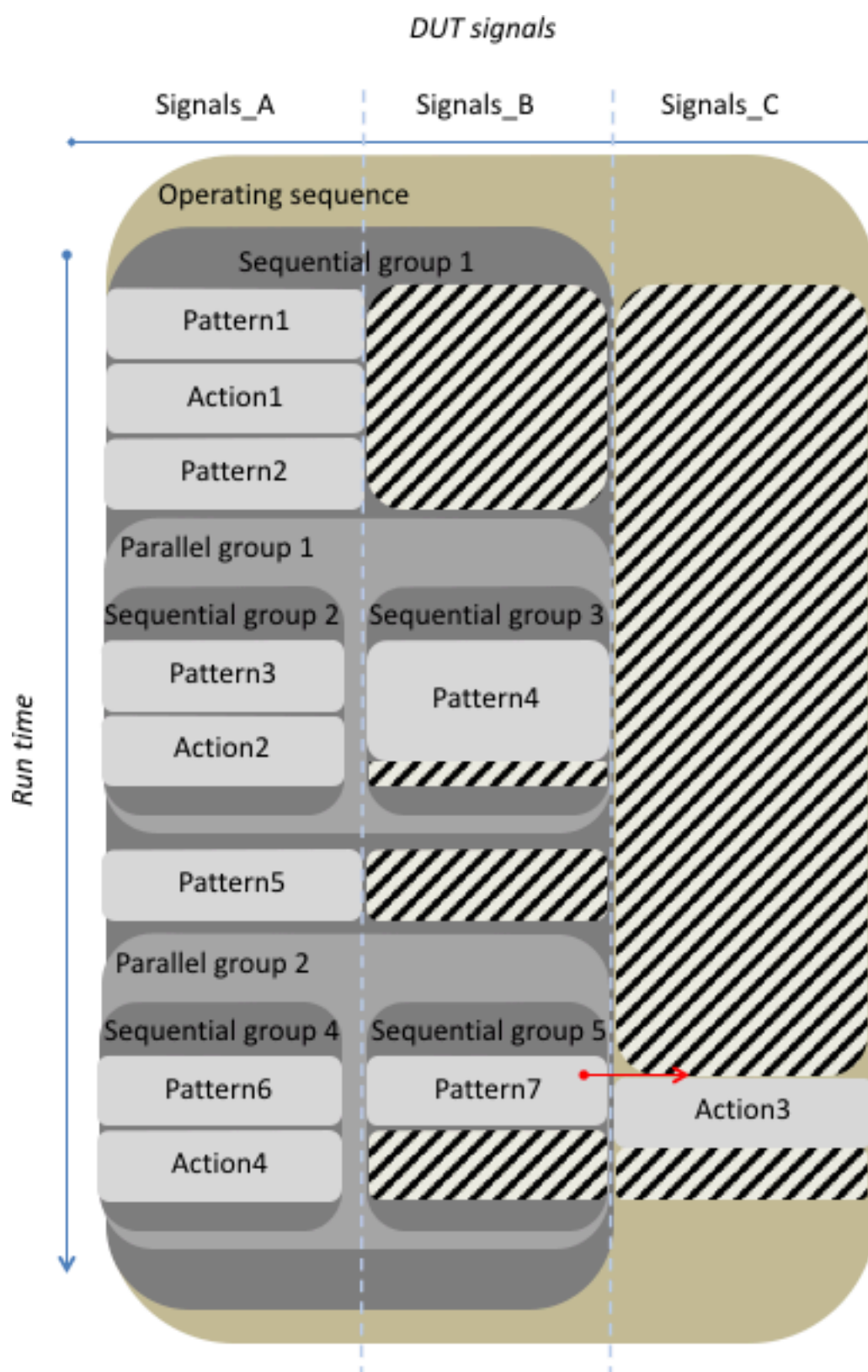
Operating sequences that produce a large amount of data may require a long time to upload that data. If such uploads cannot be performed in the background, they may delay subsequent measurements. In such rare cases, it may improve performance to split the big operating sequence into multiple smaller ones.

Operating sequence execution walkthrough

This is a schematic example of an operating sequence that contains a sequential group and several elements within it.

Parallel groups can directly contain only sequential groups (not action or pattern calls), action and pattern calls in a parallel group need to be enclosed in sequential groups, even if there is no actual sequential execution.

The parallel groups start their sequential groups synchronously. Padding with break vectors is automatically added (groups with dashed lines in the figure).



Executing this operating sequence results in these activities:

1. Pattern 1 is executed.
2. Action 1 is executed.
3. Pattern 2 is executed.

4. Parallel group 1 is executed. This means that sequential groups 2 and 3 start executing at the same time, so:
 - a. Pattern 3 and pattern 4 are executed simultaneously.
 - b. When pattern 3 has finished executing, action 2 is executed. It does not matter whether pattern 4 is still executing or has finished.
5. When both action 2 and pattern 4 have finished executing, pattern 5 is executed.
6. Parallel group 2 is executed, starting the execution of sequential groups 4 and 5 at the same moment. This means:
 - a. Pattern 6 and pattern 7 begin executing.
 - b. When the anchor in pattern 7 is reached, action 3 begins executing.
 Note that action 3 does not belong to the operating sequence. Actions anchored to a vector in a pattern are never considered part of the sequential group containing the pattern, even though their execution is tied to the execution of the group.
 - c. Action 4 is executed. It does not matter whether patterns 6 and 7 and action 3 are still executing.

More a more detailed description of the execution flow, see Execution model (topic 347810).

Sample operating sequence setup

Related concepts

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Pattern on page 53

A pattern is a series of vectors and sequencer instructions. During a measurement, the pattern is applied to a set of the DUT signals. It can also be used to control the execution of actions of the instruments connected to these DUT signals.

Action on page 57

Actions control the functions that an instrument can perform. For example, they can be used to stimulate the DUT and measure its response.

Operating sequence conditional goto jumps

Conditional goto enables you to skip or repeat parts of an operating sequence by jumping backwards or forwards within an operating sequence.

Jumps are achieved by introducing conditions and target labels into operating sequences. When a condition is checked, the pass/fail status of all involved signals - the condition pass/fail result - is considered to determine whether a jump is made or not. The condition can be configured to trigger a jump on a pass or a fail, depending on the needs of the application.

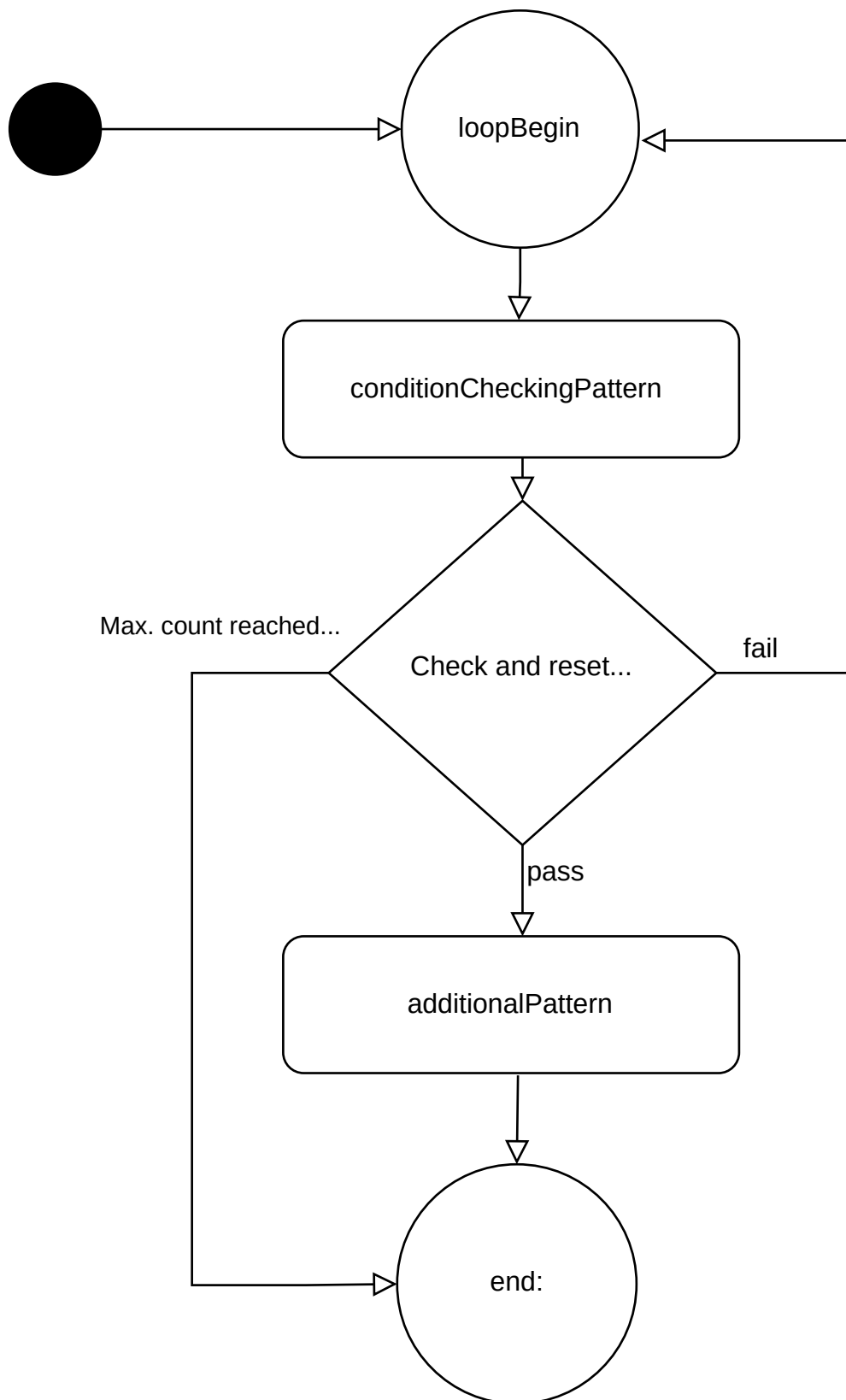
As conditions are evaluated per site, each site can execute different conditional goto jumps, executing different patterns and actions in parallel. In addition, multiple parallel-running sequential groups can execute conditional gotos independently from each other.

Note that conditional goto lets you reset the signal pass/fail value in the conditional check. This way you can directly influence the conditional jumps either in the same or in a following sequential group.

Conditional goto elements are added to operating sequences either manually in setup files or programmatically by using the "Device Setup API (Topic 252686)" in the test method.

SmarTest tools such as the **Operating sequence** view and the **Timing Debug** view support conditional goto and show all executed patterns and actions.

The following example illustrates the main components of a conditional jump implementing a loop and how it operates:



loopBegin:..

Viewer does not support full SVG 1.1

As the pattern `conditionCheckingPattern` in the operating sequence is executed, and all involved signals pass, the operating sequence continues with the execution of the pattern `additionalPattern`.

If any involved signals fail, the operating sequence loops back to the `goto` address `loopBegin`, the loop is repeated until a pass result is achieved and the operating sequence continues with the execution of the pattern `additionalPattern`.

The method `jumpCondition.hasFailedAndReset` is used to reset the pass/fail flag to `pass`, so that subsequent results are reevaluated. Without this reset, a fail will always trigger the loop.

So that loops cannot continue indefinitely, a loop limit value `maxCount` is specified. This set to 5 in the example above. After five loops without a pass, the global signal pass/fail flag is set to `fail`, and the `if` block is exited with a jump to the label `end:`, skipping the pattern `additionalPattern`.

Alternatively, you can also change the condition to `jumpCondition.hasPassedAndReset` and the logic is reversed.

Execution information

Due to the nature of conditional jumps, the actual executed route through the operating sequence is not always obvious. To help visualize this route, you can use the `IEExecutionInfo` method of the Device Test API to generate a list of the executed jump target labels:

```
getExecutedTargetLabels(String signal)
```

Returns all the executed target labels in execution order.

```
MultiSiteStringArray getLastExecutedTargetLabels(String signal, int
numberOfLabels)
```

Returns the last *numberOfLabels* executed target labels in execution order.

For more details, see *IEExecutionInfo*.

Overhead

Executing an `if` block requires several additional blocks of execution time:

- Waiting for each channel's output data queue to be flushed, which is necessary to determine all individual signals' pass/fail results. (The amount of time depends on the period.)
- Propagating the signal pass/fail results. (The amount of time depends on the period and the channel distribution across cards.)
- Embedded call to record the jump. (5 μ s.)
- With faster periods (> 200 ns), 1.5- μ s break vectors are inserted after the jump to provide time to load the data queue.

During all of these wait times, break vectors are generated, if necessary, in a similar way to "Wait instruction (Topic 246404)".

To display the actual time taken by conditional `goto` setups for a particular device being tested, including any additional break vectors, use the "Timing Debug view (Topic 246069)". Conditional `goto` delays are device- and setup-specific and cannot be accurately estimated from other devices and test programs.

Supported cards and actions

Conditional `goto` is supported only on the cards listed below. A conditional group (actions, pattern and `if` block contributing to a jump) can only contain the listed action types:

- PS5000: Supports `iforce`, `vforce`, `imeas`, and `vmeas` actions.
- PS1600: Supports `iforce`, `vforce`, `imeas`, and `vmeas` actions.
- DPS128: Supports `iforce` and `vforce` actions.
- XPS256: Supports `iforce` and `vforce` actions.
- AVI64: Supports `iforce` and `vforce` actions.

Limitations

- Conditional goto is not supported on all cards. For more information, see *Supported cards and actions* on page 72 above.
 - PS5000 cannot be used simultaneously with PS1600 or DPS128 cards.
 - XPS256 and PS1600 are not simultaneously in the same conditional goto group.
- DC or PMU action pass/fails do not contribute to jump decisions.
- Nested loops are not supported by conditional goto. That is, the code between a backward goto statement and the label to which it jumps cannot contain other backward goto statements. (It can, however, contain forward goto statements.)
- imeas or vmeas actions must be preceded within the same label by vforce or iforce, respectively.
- The **Timing Debug** and the **Operating Sequence** views display the executed patterns and actions. Conditional goto jumps are not shown, and are replaced by break vector padding instead.
- During debugging, stop points in operating sequences cannot be used in conjunction with conditional goto.
- In the **Timing Debug** view, signals inside conditional goto blocks are automatically excluded from scope measurements for the whole operating sequence. Other signals not involved in conditional goto can be scoped in the Timing Debug View.

If all visible signals in the layout are inside conditional goto blocks, no scoping is possible and an information dialog is displayed.

- Conditional goto loops result in propagation delays, adding some associated break cycles.
- In offline mode (emulated test program execution), an operating sequence containing conditional goto jumps can only be visualized up to the first executed goto instruction. Any elements logically executed after that are not displayed by the **Timing Debug** view and the **Operating Sequence** view.

Test programs executed in online mode can be fully visualized.

Example:

```
{
    sequential _seq_1
    {
        patternCall Pattern.SetRegX_1;
        patternCall Pattern.CheckADCResult;
        if (jumpCondition.hasPassedAndReset()) // "if" statement for conditional
goto
        {
            goto(STOP);
        }
        patternCall Pattern.SetRegX_2;

        STOP:
        patternCall Pattern.ReadADC;
    }
}
```

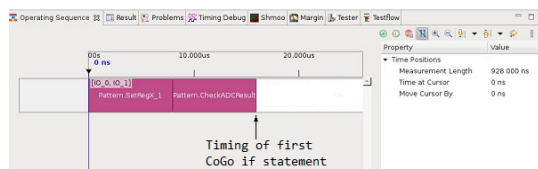
```
}

```

When the patterns return a `pass` result, the following pattern should be executed:

```
patternCall Pattern.SetRegX_1;
patternCall Pattern.CheckADCResult ;
patternCall Pattern.ReadADC ;
```

However, the Operating Sequence view does not display operating sequence elements after the first executed `goto` instruction:



Conditional goto elements

Labels and conditions are used in operating sequences to specify conditional jumps. Conditions elements are contained in "if" blocks.

Labels

Label are used to identify points in an operating sequence that can be the target of a `goto` jump.

Labels are specified in sequential groups and are only valid within the sequential group to which they belong. The same label name can be reused in more than one sequential group.

In the following example, "jumpTargetLabel:" is the name of the first label. The label name is case-sensitive and closed with a colon (:).

```
sequence Start {
    parallel {
        sequential {
            jumpTargetLabel:           // Label used to identify a CoGo jump target.
            actionCall V1;
            actionCall measI;
            ...
        }
    }
}
```

if blocks

`if` blocks are specified in sequential groups and used to determine the behaviour of the jump. The two essential components are:

Condition

Determines whether the jump is executed on a pass or a fail. It also specifies whether the current signal's pass/fail flag is reset.

Body

Used to specify:

- Jump target label
- In addition for backward jumps:
 - Maximum number of times that a backward jump can be made.
 - Exit target label used in case the maximum number of jumps are executed.

The following is an example of an operating sequence `if` block:

```
if ( jumpCondition.hasFailedAndReset() ) {
    // Backward jump on FAIL to label "Start"
```

```
// If maxCount is exceeded, fail the test, ...
// ... and continue at end:
goto(START, 5, end);
```

if blocks cannot be nested.

Evaluating if blocks requires time. For details see *Overhead* on page 72.

Condition

Each if block contains exactly one condition, which determines whether the current pass/fail result of the associated measurement matches.

The supported conditions are:

jumpCondition.hasPassed()

A pass result triggers a jump to the label specified in the goto statement.

jumpCondition.hasFailed()

A fail result triggers a jump to the label specified in the goto statement.

jumpCondition.hasPassedAndReset()

A pass result triggers a jump to the label specified in the goto statement and resets the result to pass for all participating signals independently of the condition evaluation.

jumpCondition.hasFailedAndReset()

A fail result triggers a jump to the label specified in the goto statement and resets the result to pass for all participating signals independently of the condition evaluation.

Body

The body can contain the following elements:

global.setPathFailed()

Only valid within the body of if blocks.

Sets the operating sequence global pass/fail flag to fail and is merged to the global per-site pass/fail result.

The operating sequence's global fail result is required when a measurement should fail even when the signal pass/fail results are reset during the condition checks.

```
actionCall V4;
actionCall measI;
patternCall basic.pattern.compareUpperThresholdCross;
if ( jumpCondition.hasFailedAndReset() ) {
    global.setPathFailed();
    goto(end);          // When upper threshold still not reached -> end failed
}
```

The value of the operating sequence's global pass/fail flag can be accessed using the *IDigInOutPassFailResults* interface:

```
xoc.dta.resultaccess.IDigInOutResults.passFail()
```

goto(...)

Each if block body can contain one goto call, which must always be the last instruction in the block.

The following forward or backward goto call variants can be used:

goto(label)

Syntax for forward jumps (skipping code).

Specifies the target label of the conditional jump.

Valid within both the body of `if` blocks and directly in sequential groups.

The jump target *label* must be located after the `goto` call in the sequential group.

This form of `goto` statements can also be used outside of `if` blocks in sequential groups. For example:

```
endFail:           // Label
actionCall V05;    // Action
goto(end);         // goto call specifying a jump to end;
```

```
goto(targetLabel, maxCount, maxCountLabel);
```

Syntax for backward jumps (loops).

Specifies the target label and associated behaviour (number of backward jumps and exit route) of the conditional jump.

Valid only within the body of an `if` block.

targetLabel

Backward jump target. Must be located before the `goto` call in the sequential group.

maxCount

To avoid infinite loops, the number of possible loop body executions (elements between the label and the `goto`) is limited using the *maxCount* parameter. When the specified limit is reached, the execution jumps forward to the provided *maxCountLabel*. This means that a maximum of *maxCount*-1 backward jumps can be performed.

maxCount must be a positive integer.

If the *maxCount* value is reached, the operating sequence's global pass/fail flag is set to `fail`.

maxCountLabel

Forward jump target used to exit after the maximum specified number of backward jumps have been executed. Must be located after the `goto` call in the sequential group.

Device Test API

Conditional jumps can be set up by using the Device Setup API in a test method. For details of the relevant methods, see:

- *InterfaceISetupOpSeq*
- *ISetupOpSeqIfBlock.JumpCondition*
- *COGOFunctionType*
- *COGOFunctionType.Global*
- *Interface IGoto*

The number of executed cycles can be retrieved using `getExecutedCycleCount()` from the *IResult* interface.

Individual pattern pass/fail information can be retrieved using `getPatternPassFail()` from the *IDigInOutCallPassFailResults* interface. Using this method, you can retrieve the loop count if a specific pattern is only used within a certain conditional `goto` loop.

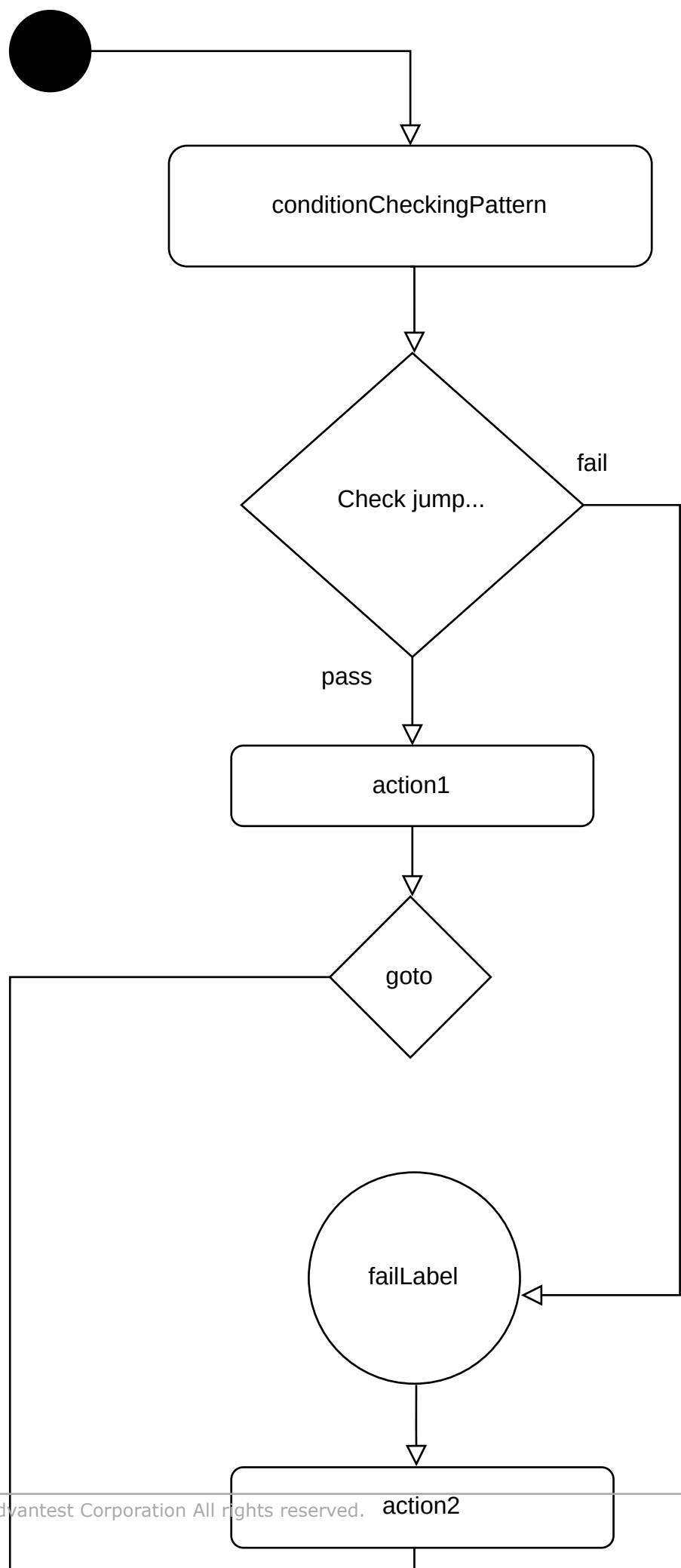
Conditional goto example: jumping forward on fail

The following code example illustrates how to use a pattern fail result to skip operating sequence elements not relevant to the failed state.

If a `fail` result is obtained from the execution of a pattern (in this example, `conditionCheckingPattern`), the `jumpCondition.hasFailed()` condition is met. The fail triggers a jump forward to the jump target `failLabel` specified in `goto(failLabel)`. The action `action1` and the unconditional `goto(endLabel)` are skipped.

Only the pattern result, and not the action's pass/fail result, is evaluated for the CoGo jump.

If a `pass` result is obtained, execution of the operating sequence continues after the `if` block. The action `action1` is executed and the operating sequence jumps forward to the label `endLabel` when execution reaches the unconditional `goto (endLabel)`.



sequence jump

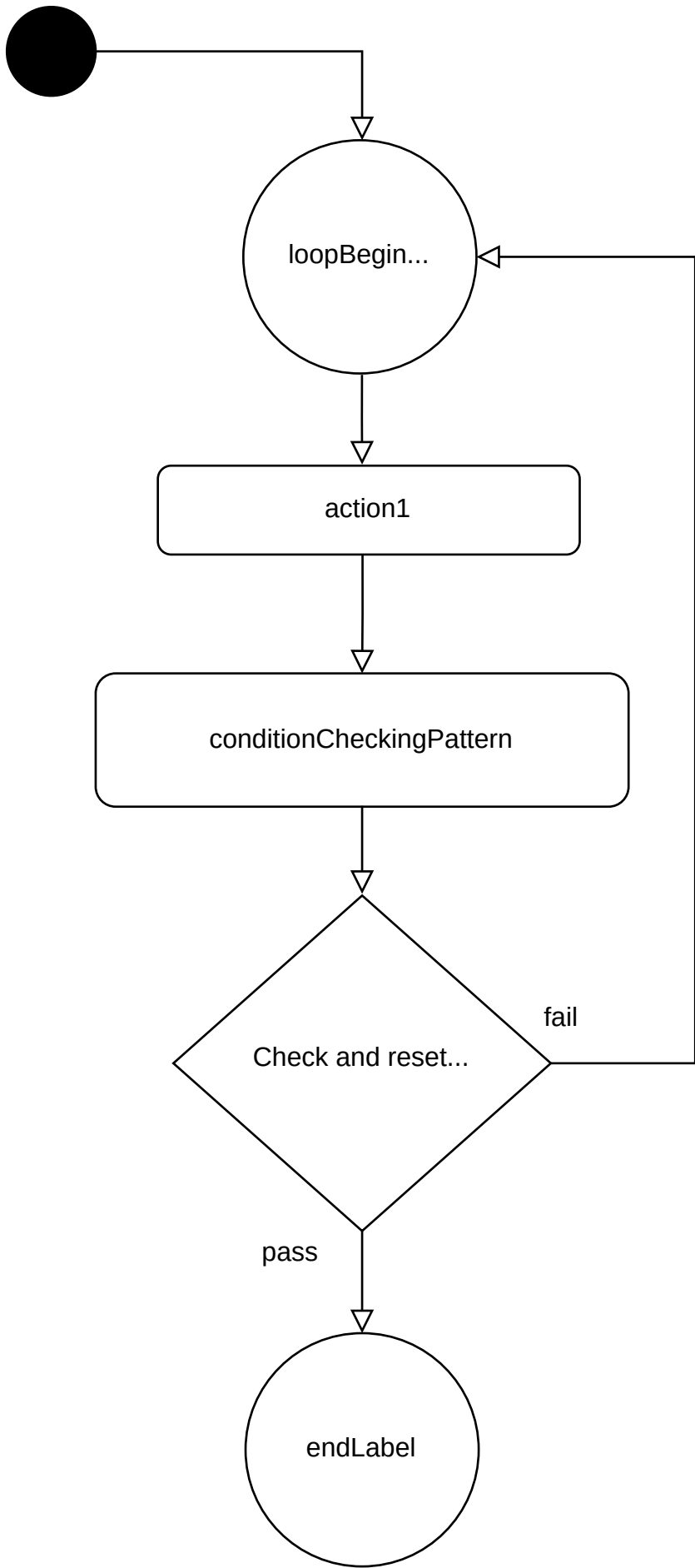
```
sequence jumpForward {  
  parallel {  
    sequential {  
      patternCall conditionCheckingPattern;  
      if ( jumpCondition.hasFailed() ) {  
        goto(failLabel);          // On fail, go to label 1, failLabel.  
      }  
  
      actionCall action1;  
      goto(endLabel);             // Go to label 2, endLabel.  
  
      failLabel:                   // Label 1  
        actionCall action2;  
  
      endLabel:                   // Label 2  
    }  
  }  
}
```

Conditional goto example: jumping back on fail

The following code example illustrates how to use a pattern fail result to repeat a measurement until a pass result is received, at which point the operating sequence can continue.

If a `fail` result is obtained from the execution of a pattern (in the example, `conditionCheckingPattern`), the condition `jumpCondition.hasFailedAndReset()` is met. The fail triggers a jump to the target `loopBeginLabel` specified in `goto(loopBeginLabel, 5, endLabel)`.

Only the pattern result, and not the action pass/fail result, is evaluated for the jump.



sequence Loop {...

In addition to matching on a fail, the condition `jumpCondition.hasFailedAndReset()` also resets the fail flag to `pass`. Without this reset, the loop condition cannot be reevaluated, remains in the failed state and therefore cannot be exited.

For backward jumps, which would create an endless loop if the condition does not change, a repeat limit is set in the `goto(targetLabel, maxCount, maxCountLabel)` statement. On failing `maxCount` times (five times in the example), the loop is exited with a jump to the `maxCountLabel` target (`endLabel` in the example).

If `maxCount` is triggered, the global flag is set to `fail`.

```
sequence Loop {
  parallel {
    sequential {
      loopBeginLabel: // This loop illustrates a jump backward on fail.
      // Label.
      actionCall action1;
      patternCall conditionCheckingPattern;
      if ( jumpCondition.hasFailedAndReset() ) {
        // Backward jump on fail to label "loopBeginLabel"
        // If maxCount (5) repeats is exceeded, fail the test, ...
        // ... and continue at endLabel:
        goto(loopBeginLabel, 5, endLabel);
      }
      endLabel:
    }
  }
}
```

Conditional goto code examples

This section includes an overview of code examples for using conditional goto.

The following examples can be used with the Crossconnect board:

- *Conditional goto example: linear fail-pass* on page 82
- *Conditional goto example: linear pass-fail* on page 82
- *Conditional goto example using the Device Setup API: binary search* on page 83

The following action examples can be used with the Incorruptus training board:

- *Conditional goto example: linear action* on page 83
- *Conditional goto example: hysteresis action* on page 84
- *Conditional goto example: binary action* on page 85

Conditional goto loop example

An example showing the how to use the Device Setup API to create an operating sequence loop with conditional goto followed by a pattern call.

Introduction

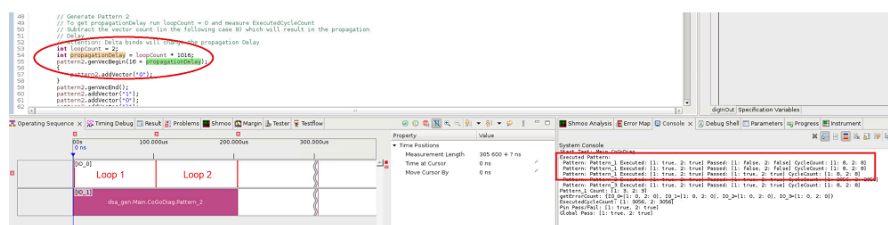
The example code uses a simple loop to check the functionality and propagation delays introduced with conditional goto.

`Pattern_2` delays the driving signal (repeat loop in the pattern), enabling the simulation of different loop counts.

The propagation delay is hardware dependent and the propagation delay value (line #54 in the screenshot) must be adjusted to match the hardware you are using.

For example:

```
int propagationDelay = loopCount = 1016;
```



Code example

Conditional goto example: linear fail-pass

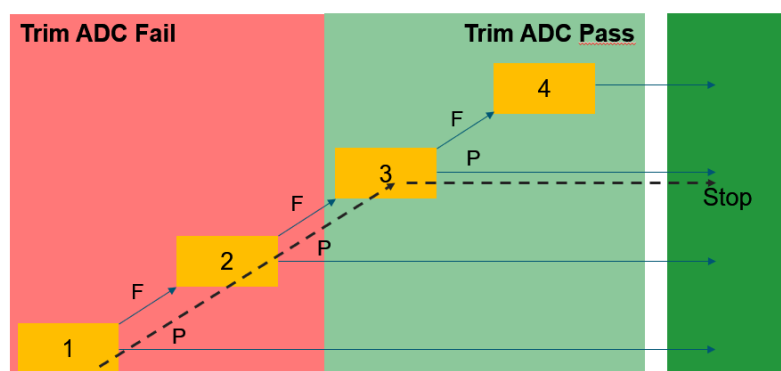
Linear search using conditional jumps based on the device response.

Introduction

The following implementation requires a specific device functionality, because the measurement and logic to compare against a specified limit is done within the device. Different trim register values are programmed through pattern (or protocol) instructions and an internal programmable ADC/Comparator reports the measured result.

Linear trim search:

1. Set trim register value in the pattern and increase the register value for each step.
2. Check the device result from internal ADC conversion.
3. If device fails, continue to the next step.
4. If the pattern passes, and the correct trim value is found, stop the execution of the operating sequence by jumping forward to label `STOP`.



Code example

Conditional goto example: linear pass-fail

Linear search using conditional jumps based on the device response using the pass to fail transition.

Introduction

The following implementation requires a specific device functionality because the measurement and logic to compare against a specified limit is done within the device.

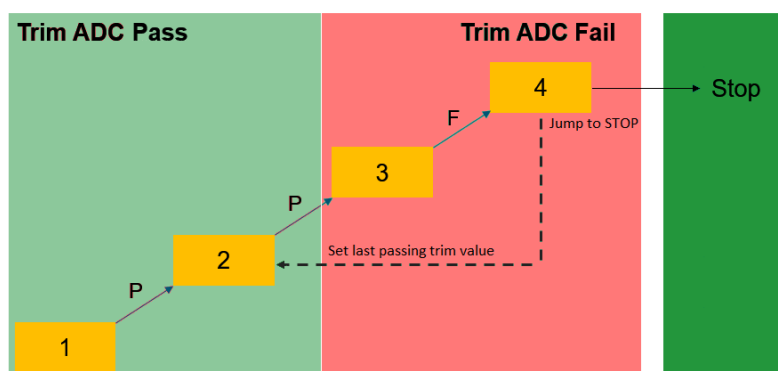
Different trim register values are programmed through pattern (or protocol) instructions and an internal programmable ADC/Comparator reports the measured result.

The best trim is the last passing value, therefore we step back one value after we find the pass to fail transition.

Linear trim search:

1. Set trim register value in the pattern and increase the register value for each step.
2. Check the device result from internal ADC conversion.

3. If device passes, continue to the next step.
4. If the pattern fails, and the correct trim value is found, stop the execution of the operating sequence by jumping back to the last-known pass value.



Code example

Conditional goto example using the Device Setup API: binary search

Binary search implementation using the Device Setup API for the linear search examples.

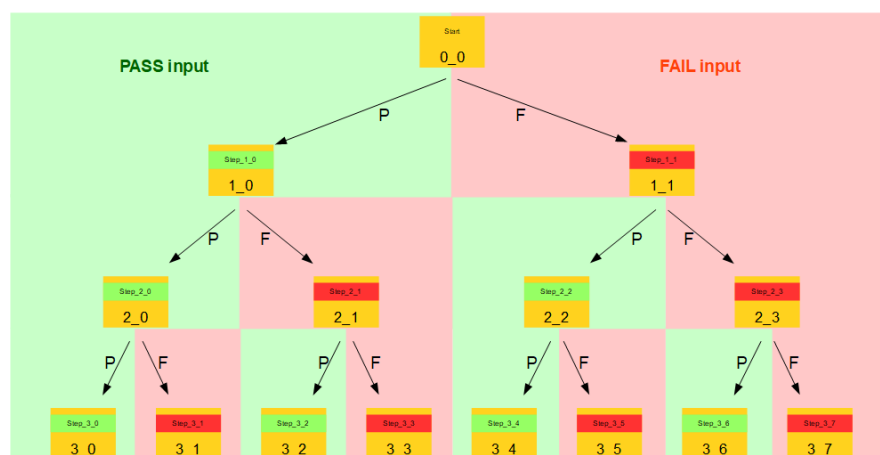
Introduction

An example illustrating how to implement a binary search for the following linear examples:

- *Conditional goto example: linear fail-pass* on page 82
- *Conditional goto example: linear pass-fail* on page 82

Error checking mechanism is implemented by checking if the capture results are available. Checking if a specific pattern was executed cannot be applied because `CallPassFail` results cannot be used with `Capture` results, as these are mutually exclusive.

Alternatively, you can run a dummy action and use that as a flag for the conditional execution path but this requires more processing and takes longer.



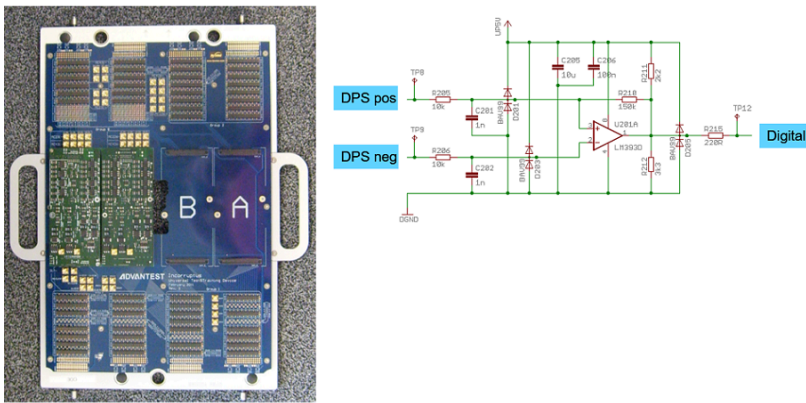
Code example

Conditional goto example: linear action

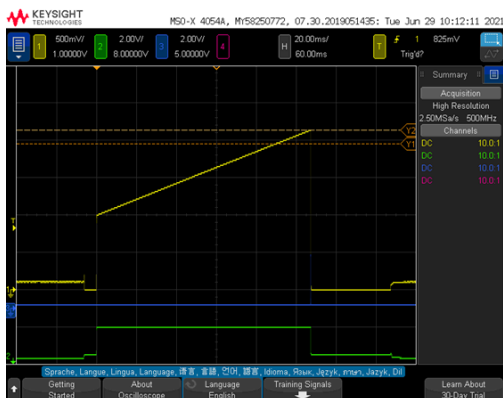
A conditional goto example using a comparator circuit on the Incompact training board illustrating a linear action.

Introduction

This example uses the LM393 comparator circuit on the Incompact training board.



The voltage on the input side of the comparator is increased step by step until the digital output trips.



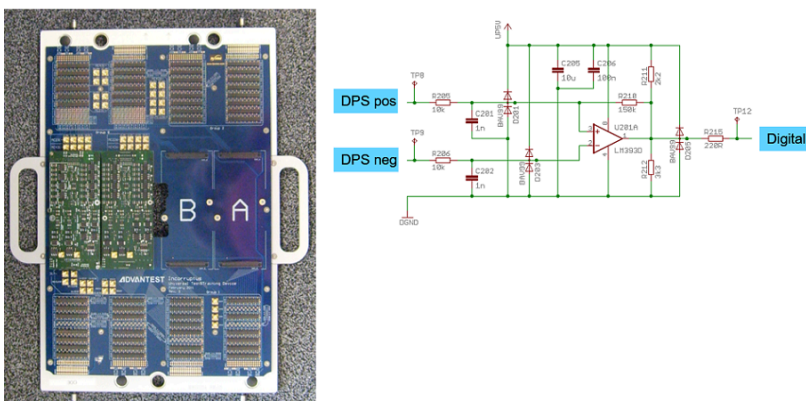
Code example

Conditional goto example: hysteresis action

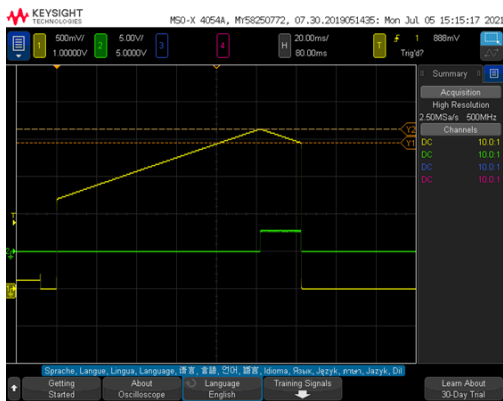
A conditional goto example using a comparator circuit on the Incorruptus training board illustrating a hysteresis action.

Introduction

This example uses the LM393 comparator circuit on the Incorruptus training board.



Run a hysteresis on the same setup as before.



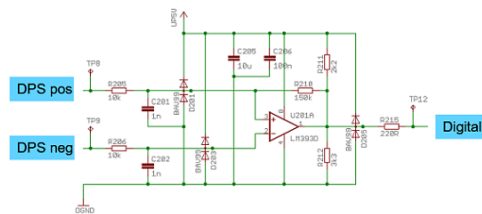
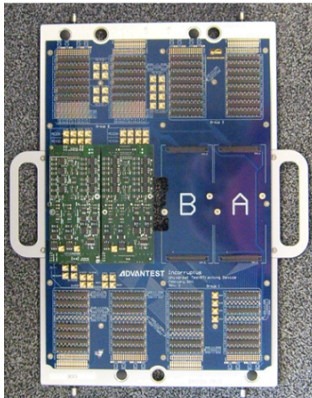
Code example

Conditional goto example: binary action

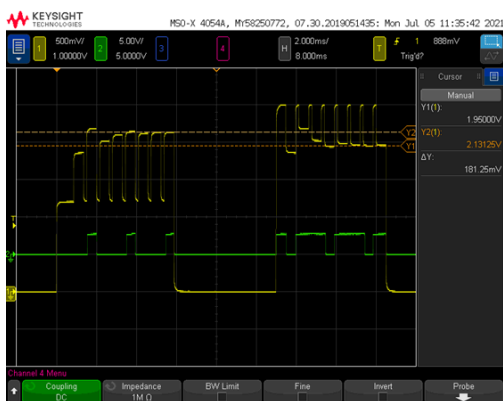
A conditional goto example using a comparator circuit on the Incorruptus training board illustrating a binary action.

Introduction

This example uses the LM393 comparator circuit on the Incorruptus training board.



Using a binary search algorithm.



Code example

Minimizing padding in parallel groups

Parallel groups start all sequential groups they contain synchronously, within configurable tolerances. Doing so often requires the system to insert padding, which can have performance

implications. In time-critical parts of the test program, it may be worthwhile to take steps to minimize this padding.

Padding can take the form of break vectors, user vectors, or user-defined idle sequences:

- Break vectors are automatically generated by the system. If break vectors are used, for technical reasons there must be at least eight vectors. (However, a multiple of eight is not required, so that, for example, nine break vectors can be inserted.)
- User vectors are specified using the padding (topic 244408) keyword in the operating sequence setup and do not have this minimum requirement.
- Idle sequences (topic 252690) are defined in protocol-aware testing and also can be shorter than eight vectors.

Reasons for padding

Padding is used in connection with parallel groups to ensure that all contained sequential groups start simultaneously. There are two relevant conditions:

- The tester cycle count for each sequential group must be a multiple of eight.
- The maximum difference between the starting times for the next cycle in each sequence must not be greater than the synchronization accuracy (see topic 244408). (That is, when the first sequence starts its next cycle, the next cycles in all other sequences must start within the time specified by the synchronization accuracy.)

Consequently, there are factors that can increase the required padding:

- The more different periods the different sequential groups use, the more padding is usually needed.

The amount of padding is determined by the least common multiple of all periods, so especially periods with small greatest common divisors can result in additional padding.

Note that instruments without explicitly set periods, such as RF instruments, may still internally use periods. Such implicit periods are not externally readable.

- The greater the synchronization accuracy (that is, the smaller the value set for it), the more difficult it becomes to synchronize all sequences and the more padding may be needed to reach a point where all cycles are multiples of eight within the specified interval.

Not all factors that influence padding can be controlled, and it is often effectively impossible to calculate the precise amount of padding that will be applied.

Measures to reduce padding

You can do several things to reduce padding. Which measures makes sense depends on the test program, and it is possible that none of them are useful for a given test program.

- Reduce the synchronization accuracy by specifying a larger value for syncAccuracy (see topic 244408).

A looser synchronization accuracy provides for a wider zone that is considered "simultaneous". However, accuracy that is too loose may create invalid measurements, so you need to specify a value that is still small enough to ensure proper operation of the DUT.

- Choose more compatible periods, that is, periods with a larger greatest common divisor.

If the periods for some sequential groups can be varied to a certain degree, it is better, for example, to choose periods of 300 ns and 360 ns (greatest common divisor is 60) than 300 ns and 350 ns (50) or 370 ns (10).

- Avoid bypassable parallel groups.

A bypassable parallel group requires SmarTest to apply padding both before and after it. The padding must always be calculated, whether the group is bypassed or not.

You should, therefore, remove all bypassable attributes (see topic 244408) after debugging the test program unless they are actually needed in production testing.

- Convert patterns to transaction sequences.

The advantage of this approach is that, as part of the protocol, you can define an idle sequence (topic 252690) that is used for padding instead of break vectors and does not have the eight-vectors minimum requirement.

However, using large transaction sequences can increase binding times and so carries its own performance implications. Which approach is better must be evaluated for each test program.

Test method

A test method is a Java class that controls the measurement execution and retrieves, evaluates and logs the results. The test method to use is specified in the test suites; testing a DUT involves executing these test suites and thus the associated test methods.

Structure of test methods

A test method can consist of four Java methods:

```
testMethod {  
    initialize() // optional  
    setup()      // optional  
    update()     // optional  
    execute()    // required  
}
```

The actual measurements are performed in the `execute()` method; the other three methods are intended for various phases of setting up the measurement:

initialize()

The `initialize()` method is executed early in the transition from the "loaded" to the "bound" execution state, before the *PreBind testflow* is executed.

This lets you specify values for the data objects that have been created when the test program was loaded. For example, you can set test descriptor properties to default values that can be overwritten from the "test table (Topic 249089)" in the *PreBind testflow*.

setup()

The `setup()` method is executed during the bind process, after the *PreBind testflow* has been executed.

It lets you programmatically create setup data, same as are specified in setup files, using the "Device Setup API (Topic 254754)". Setup data specified here overwrites data from the setup files and from the test table.

update()

The `update()` method is executed in the transition from the "bound" to the "testflow ready" execution state, before the *PreStart testflow* is executed.

It lets you use the "Device Test API (Topic 255624)" to modify the test data derived from the setup in memory, that is, without affecting any of the setup sources (setup files, test table, or `setup()` method).

execute()

The `execute()` method is executed during the "testflow running" state of the test program.

It lets you run measurements and retrieve, evaluate, and log the results using the Device Setup API.

All these methods are defined in the `TestMethod` class. Any test method class that uses them overwrites them in their code.

APIs for test methods

SmarTest comes with a test method library that provides ready-to-use test methods for many typical testing tasks.

In addition, you can create your own user test methods to meet specific requirements, either from the beginning or by modifying the source code of standard test methods. Both standard and user test methods are written in Java, are edited in the code editor, and access the test system through the Device Test API and other APIs.

Test methods can modify or create many parts of the *setup* on the fly, such as timings, levels, or *patterns*. However, some setups cannot be modified, for example, the *DUT board descriptions*. Test methods can specify parameters - similar to function arguments - that are passed to each instance of the test method when it is called in a *test suite*.

These APIs are available for test methods:

Device Setup API

The Device Setup API is used in the `initialize()` and `setup()` methods of test methods to programmatically specify test data. It uses the SSF API (see below) to write the specified test setup files into your project.

It provides methods to define the name and the content of test setups, for example, a measurement specification or operating sequence.

See "Device Setup API reference (Topic 252686)" for details.

Device Test API

The Device Test API is used in the `update()` and `execute()` methods of test methods.

In the `update()` method, it can be used to modify the test data that is used by the measurement. In the `execute()` method, it is used to execute the tests. This includes configuring and executing measurements on the tester hardware, evaluating and logging measurement results, and binning.

See "Device Test API reference (Topic 244313)" for details.

SSF API

The SSF API is used by the Device Setup API (see above) to dynamically generate and modify SmarTest setup files. Its methods are not normally called directly by test methods.

The SSF API can also be used to convert test data generated by external tools into SmarTest setup files, for example, in custom conversion tools.

See "SmarTest Setup Format API reference (Topic 252687)" for details.

RF Demodulation API

The RF demodulation API provides methods for demodulating waveforms based on one of the supported modulation standards.

See "RF demodulation API reference (Topic 252076)" for details.

The example code below provides an overview of a test method:

Related information

Test data on page 21

A test is configured using a range of setup files, each of which describes a particular aspect of the overall setup.

Protocol-aware testing

Protocol-aware testing is an integral feature of SmarTest that enables a direct communication to the DUT over a well-defined interface, the so-called protocol.

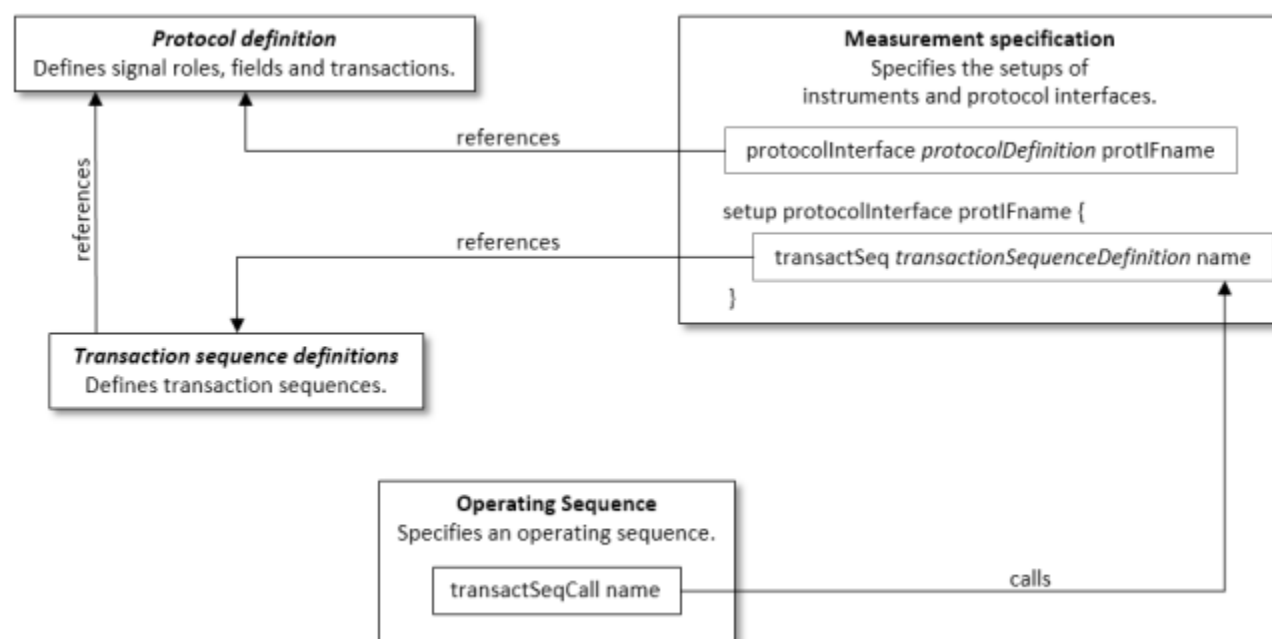
A protocol is the language that SmarTest uses to send data to the DUT and to understand the responses from the DUT. SmarTest maintains the connection to the DUT during these transmissions, which always follow the communication rules defined in the corresponding protocol definition file.

Protocol-aware testing with SmarTest enables you to test and verify standard protocols, like JTAG, or complex protocols, like PCIe. As an integral feature of SmarTest, it lets you set up and debug the communication with the DUT in the same way as functional or parametric tests.

All required setup data is specified in the SmarTest Setup Format (SSF). Setup data of protocol-aware tests is defined and specified in these files:

- Protocol file
- Transaction sequence definitions file
- Measurement specification file

This figure provides an overview of the relationships:



Graphical overview of the relationship of the protocol-aware setup files

Protocol file

The protocol file contains the description of a transaction protocol. The description includes the definition of signal roles, fields and transactions, for example, `read(addr, value)`, `write(addr, value)` and `reset()`. A protocol can be viewed as a user-defined instrument, similar to an instrument provided by SmarTest, for example `dcVI` or `digInOut`. The signal roles, fields and transactions of a protocol are similar to the properties of a system-defined instrument.

Transactions are defined in the protocol file. A transaction is a defined sequence of fields used to exchange information between the DUT and SmarTest. A simple transaction is, for example, writing a specified value to a specified register. Transactions can be defined to be static or parameterized. Parameterized transactions are, for example, `read(addr, value)`

and `write(addr, value)`. Static transactions are, for example, `reset()` and `write(0xFF, 0x1F)`. The values of transaction parameters must be specified in the transaction sequence, the measurement specification, or in a test method.

A signal role is the name for an actual signal used by the protocol interface, for example, `MDIO` or `MCKL`. Signal roles are mapped to the corresponding actual signals in the measurement specification. They are used by the protocol to communicate with the DUT via transactions.

Protocol-aware testing supports signal roles that are shared between multiple sites and are assigned to receive or drive pogo pins. In this case, you can use only transaction sequence parameter values that are the same for all enabled sites.

A field is a part of a transaction that contains data of a specific type, for example a register or a physical address. A field can be defined to be static or parametrized. The values of field parameters must be specified in the measurement specification or a test method.

Automatic method generation

When the workspace is built, SmarTest automatically scans all folders in it for protocol files and converts them to methods you can directly use. For more information, see "Setting up protocol-aware tests (Topic 347924)".

This also creates the transaction sequence and measurement specification files described below. While it is also possible to create these files manually, there is normally no need for doing so. We recommend that you use the automatically generated interface unless there are specific reasons for not doing so.

Transaction sequence definitions file

The transaction sequence definitions file contains multiple transaction sequence definitions for the referenced protocol. A transaction sequence definition specifies one transaction sequence that consists of transactions of the referenced protocol.

Referencing the protocol is done by importing the protocol into the transaction definitions file.

A transaction sequence can be viewed as a user-defined action, similar to an action of a system-defined instrument, for example, `vforce`.

Measurement specification file

The measurement specification file needs three entries to specify protocol-aware tests.

1. A declaration of the transaction sequence, for example:

```
transactSeq transactionSequenceName;
```

2. A definition of the protocol interface, for example:

```
protocolInterface protocolName protocolInterfaceName {
    signalRole1 = signal1;
    signalRole2 = signal2;
    ...
}
```

3. A definition of the transaction sequences within the protocol interface setup, for example:

```
setup protocolInterface protocolInterfaceName {
    transactSeq transactionDefinitions transactionSequenceName {
        ...
    }
    ...
}
```

The first entry declares a transaction sequence.

The second entry specifies the protocol interface by referencing a protocol (`protocolName`) and assigning an interface name and signal roles to the corresponding signals. As the protocol interface can be seen like a user-defined instrument, you basically specify an instrument of type `protocolInterface` that can be called by a name, for example, JTAG. In addition, by assigning the signals to the signal roles you specify the signals to be used by this protocol interface for a measurement.

The third entry is the setup of this interface. As a transaction sequence can be seen like an instrument action, you basically specify the setup of the protocol interface specified with the second entry. The name of the protocol interface (`protocolInterfaceName`) is used to tell SmarTest on which signals the transaction sequence is performed. The `transactSeq` references a transaction sequence in the transaction sequence definitions and assigns a name to that "action" and specifies parameters, if required. These parameters are again similar to parameters of an instrument action. The transaction sequence specified in the setup section is called from an operating sequence or pattern with the keyword `transactSeqCall`.

Note You can have multiple protocol interfaces specified in one specification file. However, each must have a different name and can specify separate setups.

The graphic below compares a transaction sequence to an instrument action:

DC Action	Transaction Sequence
<pre>setup digInOut JTAGx + sig12 { action vforce ppmu { forceValue = 0.0 V; } }</pre>	<pre>setup protocolInterface JTAG { transactSeq transaction_seq.DemoSequences2.TSeq3 bit_stream128 { reg = 0xaf; value = <0xaaaaf0f0aaaaf0f0aaaaf0f0aaaaf0f0>; len = 128; } }</pre>

A transaction sequence is like a custom instrument

A protocol-aware file (.spec) is shown below:

Test programs, testflows, and test suites

SmarTest uses a hierarchy of structures to specify what happens during a test, from the test program at the top and down to test methods and instruments.

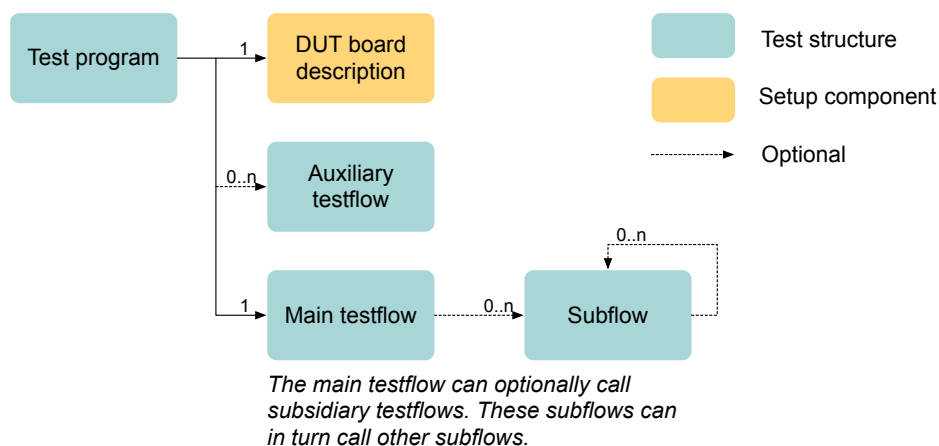
Each of these structures plays a specific role in the execution of a test:

- *Test programs* determine the overall extent of the test.
- *Testflows* specify the flow of execution at the top level and determine which measurements are performed.
- *Test suites* combine test methods with specific measurement settings.

These structures access and specify a range of setup elements that are used to determine the specific execution of a test. More information on this setup data is available in *Test data* on page 21.

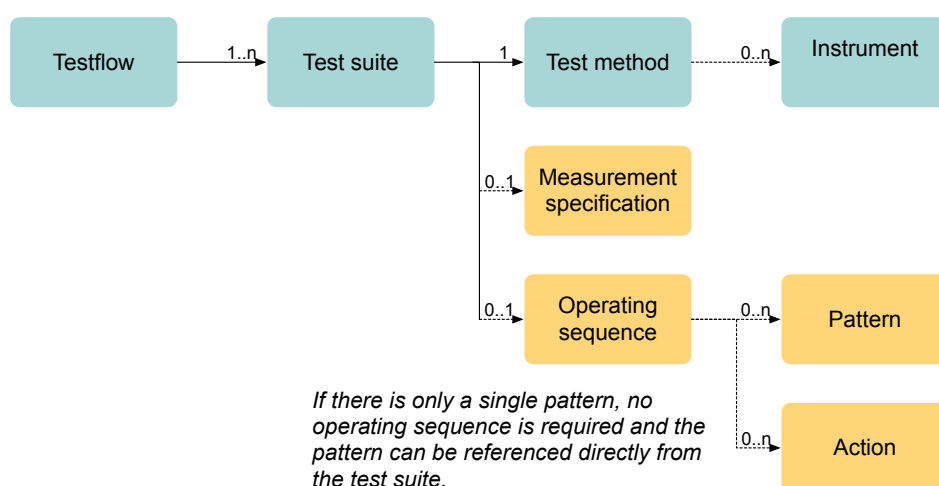
The graphic below provides a simplified overview of how these structures and the setup data are related. (Note that it describes the logical relationships, not necessarily the syntactical setup.)

Test program structure



Testflow structure

This structure applies to all testflows – main testflow, subflows, and auxiliary testflows.



1. The collection of all setup and programmatic information required to perform a device test
2. Description of how the DUT board connects pogo pins to DUT pins

3. *Testflows that are specified in the test program but are not part of the execution hierarchy of the main testflow*
4. *Main testflow: the central testflow in a test program*
5. *Subflows: a testflow is called from another testflow*
6. *A structure for performing the individual measurements in a device test*
7. *Test suite: controls how the test method is executed*
8. *Contains the detailed instructions for performing an individual measurement*
9. *Collects all parameters for a measurement*
10. *Describes the arrangement of patterns and actions for performing a measurement.*
11. *Contains the vectors of state characters that are used to perform the measurement*
12. *Triggers an action by an instrument*
13. *Instruments represent a layer between SmarTest test programs and the hardware resources*

Size limits for SmarTest files

The following table specifies the maximum supported SmarTest file sizes:

File type	Extension	Supported size limit
DUT board file	.dbd	5000 lines
Operating sequence file	.seq	5000 elements (see the "file description (Topic 244408)" for more information)
Pattern file	.pat	No practical limit
Protocol file	.prot	No practical limit
Specification file	.spec	20000 lines
Testflow file	.flow	5000 lines 1500 items
Test program file	.prog	5000 lines
Transaction sequence definitions file	.trseq	No practical limit

Note that larger files than specified above may still work in some cases. However, such larger files exceed the limits SmarTest is built to handle. They may reduce performance and reliability and should not be used.

A test program file (.prog) is shown below:

Related concepts

Test suite on page 100

A test suite is an element of a *testflow* that specifies the test method to use and associated settings. It controls how the *test method* is executed at this point in the testflow.

Related information

Test data on page 21

A test is configured using a range of setup files, each of which describes a particular aspect of the overall setup.

Test program

A test program is the collection of all setup and programmatic information required to perform a device test. Its main components are the main testflow, auxiliary testflows, and information about the testing environment.

A test program specifies:

- The *DUT board description* on page 22 to use.
- The main *testflow* that specifies how the DUT will be tested.

Optionally, it can also specify:

- *Auxiliary testflows* on page 96, for example, for connecting and disconnecting the DUT.
- Ignored sites, that is, configured sites that are not to be tested in *multisite testing*.
- The "test program licensing requirements file (Topic 252940)", which contains a list of the test program features that require licenses.
- One or more *variables* that are available globally in testflows and measurements in the test program.
- The number of utility lines per pogo block.

A test program does not itself contain all these elements, it just references files that contain them. This makes it easy to reuse elements between multiple test programs.

Test programs, workspaces, and projects

Workspaces and projects are Eclipse concepts that are available in SmarTest as well. Test programs are connected to both but do not directly correspond to either:

- A workspace usually collects projects related to a particular device or family of devices. It is usually owned by a user and can be used only by a single user at a time.

Projects can be physically stored inside the workspace or be stored outside of it and imported into the workspace. We strongly recommend storing projects outside of workspaces.

- A project groups all elements needed for a particular device test setup. These elements are referenced by test programs as noted above. A particular project can be used by several test programs, and a test program can use several projects.
- A test program is part of a project, but may reference additional projects, for example to reuse elements from these projects. It references everything that is needed to execute a device test.

In principle, neither test programs nor projects require a particular folder structure. However, you should arrange your files in folders and subfolders to best meet your needs as described in *Recommended structure of a SmarTest project*.

Related concepts

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Related reference

SmarTest project structure on page 12

Source files for a SmarTest project are collected in a project folder. These source files can be flexibly distributed in subfolders of this project folder, but there are some requirements and recommendations.

Testflow

A testflow is a structure for performing the individual measurements in a device test. It consists of a number of *test suites*, bins, and control logic such as loops and branches.

A testflow is contained within a *test program*. It can call other testflows; testflows called in this manner are termed "subflow".

The "Testflow (Topic 255242)" displays the selected testflow and all of its subflows in a hierarchical structure.

A testflow specifies:

- *Test suites* to execute, their order, and the loops and branches that control their execution

- Subflows (other testflows) to execute.
- Bins to which DUTs are assigned according to their test results.
- Settings for testflow parameters.
- Values for testflow variables (if the testflow contains such variables).

All of these settings are optional, although a testflow that does not specify at least one test suite or subflow will not do anything when executed.

Testflows can define variables, the values of which can be set in the testflow editor or by program code in a test method. Variables can be used in the control logic (for example, to enable or disable a branch of the test based on the value of a variable). See *Variables and parameters in SmarTest* on page 102 for details.

Testflows can also have parameters. The value of a parameter is set when the execution of a testflow is started by the entity that calls the testflow. Most commonly, parameters are used when a testflow calls a subflow (see *Types of testflows*, below). See *Variables and parameters in SmarTest* on page 102 for details.

A testflow always has two sections:

1. A `setup` section, which defines the elements - *test suites* and subflows - that are used in the testflow.
2. An `execute` section, which specifies the order of executing the defined elements, including order of execution, branching, binning, and dynamic variable assignments.

Testflows are stored in testflow files. These files can contain complete testflows, or they can contain part of a testflow and import other testflow files. This enables testflows to be set up in a modular way, so that subflows can be reused in multiple test programs.

Types of testflows

The structure of testflows is always the same, but they can be distinguished by where and how they are integrated into a test program:

- **Main testflow:** This is the central testflow in a test program. Each test program must have exactly one main testflow. The main testflow is executed automatically when the test program is executed.
- **Auxiliary testflows:** *Auxiliary testflows* on page 96 are testflows that are specified in the test program but are not part of the execution hierarchy of the main testflow. SmarTest executes some of them automatically if they are present, triggered by specific conditions. For example, the `PreRun` testflow is executed before the main testflow.
- **Subflows:** Each testflow can call other testflows, in the same way that a programming function can call other functions. When a testflow is called from another testflow, it is a subflow.

Subflows can in turn call other subflows, but not recursively. (That is, a subflow cannot call itself nor any testflow from which it has been called directly or indirectly.)

Testflows are executed serially, that is, the execution of one testflow is finished before the execution of the next testflow at the same level starts. This does not apply to subflows, which are executed as part of the calling testflow. (But two subflows called from the same testflow are executed serially.)

A main testflow file (`.flow`) `LabMain` is shown below:

A subflow `FunctionalTests` called by the main testflow `LabMain` is shown below:

Related concepts

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Variables and parameters in SmarTest on page 102

SmarTest variables and parameters are similar to variables and parameters of any programming language. They are used with test programs and their components like, testflows, test suites, test methods, measurement specifications, protocol-aware testing setups, and the APIs such as the Device Test API. They are also used with external components such as the Test Cell API and recipes.

Auxiliary testflows

Auxiliary testflows are testflows that are specified by name in the test program file and run outside the main testflow. Some of them, with reserved names, are executed automatically by SmarTest at defined points during the execution of a test program or in response to particular events. They implement administrative tasks that are related to executing a test but are not part of the test itself, such as loading limits from a file or disconnecting the DUT.

Other than not being part of the execution hierarchy of the main testflow, auxiliary testflows are just like normal testflows.

All auxiliary testflows are optional. If you create a testflow with one of the reserved names and specify it in the test program, it gets executed at the appropriate time. If no such testflow exists, the normal test execution continues. For example, if a test program contains no `PreBind` testflow, no user-specified actions are executed before the test program is bound, but the binding of the test program is performed normally.

Test methods can determine the type of testflow in which they are being executed by calling the `getFlowTypeName()` method.

General auxiliary testflows

These auxiliary testflows are executed automatically when they are present:

PreBind

Is executed on the "available sites (Topic 250860)" before the test program is *bound*. It is used, for example, to load a test table from disk.

When the `PreBind` testflow is executed, testflows and test suites have been initialized.

PreStart

Is executed on the "available sites (Topic 250860)" after the test program is *bound*, once before the lot is started. It can be used, for example, to power up the active DUT board circuit.

It is recommended that utility power supplies are turned on in the `PreStart` testflow.

PreRun

Is executed on the "enabled sites (Topic 250860)" before the main testflow is executed. It can be used, for example, to implement adaptive testing by bypassing test suites.

PowerUp

Is executed on the "active sites (Topic 250860)" at the start of the main testflow. It can be used, for example, to test continuity and establish power supply for the device.

PowerDown

Is executed at the end of the main testflow or when the test program is manually stopped by a user; it is executed on all sites on which `PowerUp` had been executed. It can be used, for example, to power down the device, perform custom binning, or do necessary clean-up.

PostRun

Is executed after the main testflow and (if PowerDown is specified) after PowerDown has been executed on all sites. PostRun is executed exactly once on all sites on which PreRun has been executed. It can be used, for example, to run a custom binning algorithm.

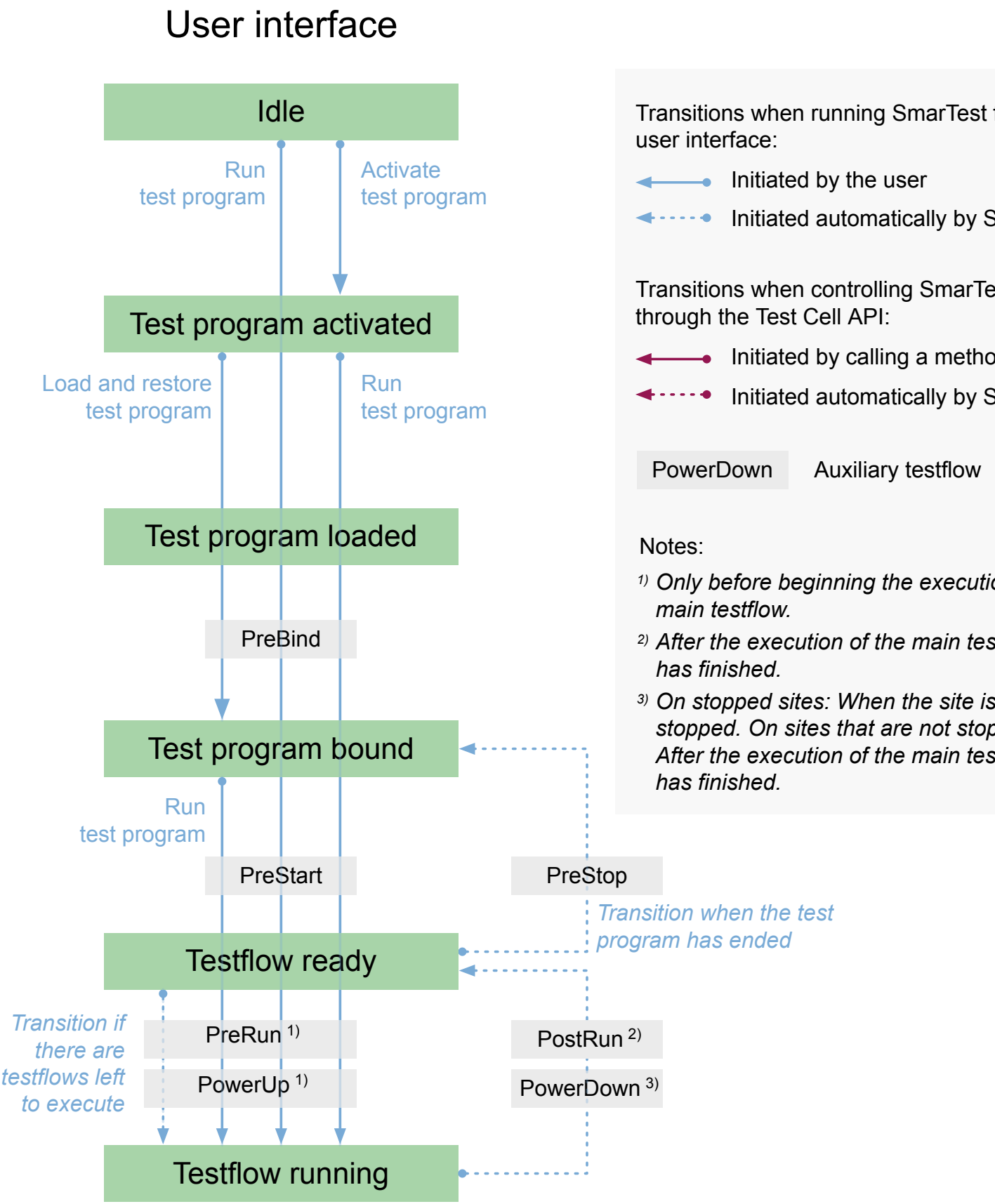
PreStop

Is executed after the test program, once before the lot is stopped; it is executed on all sites on which PreStart had been executed. It can be used, for example, to power down the DUT board.

We recommend using `PowerUp` and `PowerDown` auxiliary testflows in every test program. Use `PowerUp` to establish the power supply for the DUT and to perform any required contact tests, like a continuity test. Use `PowerDown` to power down the device safely and for any complex binning operations.

This graphic shows when the various general auxiliary testflows are executed:

Overview of states and transitions



To see the full execution sequence of a test program refer to *Test program execution* on page 141 and for a detailed overview of what happens during the execution of a test program, see *Detailed test program execution sequence* on page 144.

Test-cell-related auxiliary testflows

This auxiliary testflow is available for test programs running in test cell settings:

Continuity

Is executed by test cell controllers when a continuity check for a proper connection between DUT and tester is required. It should contain all tests needed for checking the connection and produce `pass` as the final result when a connection exists, `fail` otherwise.

If the "Test Cell Control Tool (Topic 248026)" is used, the `Continuity` testflow is executed when the **Contact Check** button is clicked. Outside of test cell environments, it can be called from the main testflow before the test starts.

We recommend adding a Continuity testflow to all test programs.

See "Test-cell-related auxiliary testflows (Topic 343180)" for more information.

Other auxiliary testflows

Certain auxiliary testflows for special purposes are supported as well:

Bench

Is executed by the SmartShell server to provide interactive access to SmarTest. (The testflow is not needed for SmartShell to simply execute a test program in SmarTest.)

Except in connection with SmartShell, this testflow is ignored.

For more information, see "Bench auxiliary testflow (Topic 350389)".

Custom auxiliary testflows

In addition to the auxiliary testflows listed above, all of which have specific predefined roles during the execution of SmarTest, you can also add auxiliary testflows for purposes of your own, so-called custom auxiliary workflows. Any testflow defined outside the main testflow that does not have one of the predefined names listed above is a custom auxiliary testflow.

Custom auxiliary testflows specified before the main testflow in the test program file are executed after

PreBind testflow

Sample `PreBind` testflow:

PowerUp testflow

Sample `PowerUp` testflow:

PowerDown testflow

Sample `PowerDown` testflow:

Continuity testflow

Sample `Continuity` testflow:

Related concepts

Test program execution on page 141

Executing a test program with SmarTest is the process of preparing all corresponding test data, converting them into an executable test, and executing the test.

Related reference

Detailed test program execution sequence on page 144

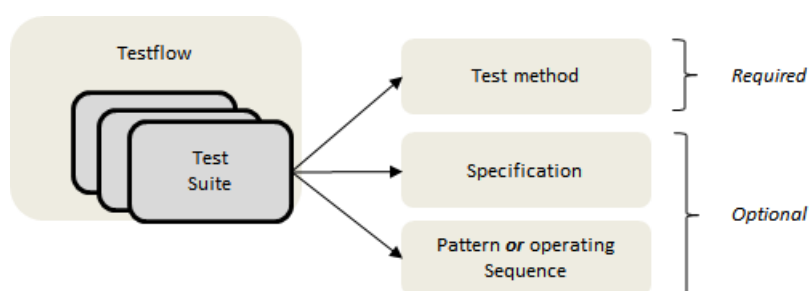
The basic sequence of execution for test programs is similar for engineering and production environments. It involves passing through the different execution states, with optional auxiliary testflows being executed automatically at several stages if they are present.

Test suite

A test suite is an element of a *testflow* that specifies the test method to use and associated settings. It controls how the *test method* is executed at this point in the testflow.

A test suite specifies:

- The *test method* to use (required)
- Any test method parameters required by the test method
- Optionally, a *measurement specification*
- Optionally, an *operating sequence* or *pattern*



1. A testflow consists of multiple test suites, sub flows , or a mix of both
2. A Java class that controls the measurement execution and retrieves, evaluates and logs the results
3. Describes which instruments a measurement uses and which settings are applied to them
4. A series of vectors and sequencer instructions
5. Specifies the order in which a set of patterns, actions, and transaction sequences are executed.
6. Specifies the order in which a set of patterns, actions, and transaction sequences are executed.

Test suites are defined in testflow files. In general, a test suite is used to combine a test method (Java code) with appropriate settings so that it can be executed in a testflow. Note that there are also special test suites, like shmoo test suites, which enable you to perform a shmoo test in flow, and margin test suites that enable you to perform a margin test in flow .

Test suites are edited in the "Setup editor (Topic 244574)".

Test methods and test suites, measurement objects

In test methods, you can declare one or more measurement objects using either the `IMeasurement` interface or the `ILocalMeasurement` interface. This object stores the test setup data that is required to execute a certain measurement. In your testflow, every test suite calls a test method. Thus, every test suite that performs a measurement has its own measurement object. This object references the test setup data that is required to execute the measurement.

The values stored in a measurement object are derived from the test setup data, for example, the specification file you associate with it in the test table, the `setup()` or `update()` sections of its test method, or from the test suite that calls the test method. For example:

1. In a test method, you define a measurement object:

```
public IMeasurement myMeasurementObj;
```

or

```
public ILocalMeasurement myMeasurementObj;
```

You can define more than one measurement object in a test method.

2. A test suite of a testflow can call this test method and use its measurement objects (in this example, `myMeasurementObj`) to assign the setup data and other parameters that are required for the measurement execution. For example, you can assign the operating sequence file:

```
myMeasurementObj.operatingSequence =  
    setupRef(myDevice.myTest.myOpSeqFiles.myOperatingSequence);
```

You can also assign the setup data and set the test method parameters from within the test method itself. For details, see

Test methods can find out their test suite by calling the `getTestSuiteName()` method.

The example below shows that five test suites are specified within a testflow:

Related concepts

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Testflow on page 94

A testflow is a structure for performing the individual measurements in a device test. It consists of a number of *test suites*, bins, and control logic such as loops and branches.

Test method on page 87

A test method is a Java class that controls the measurement execution and retrieves, evaluates and logs the results. The test method to use is specified in the test suites; testing a DUT involves executing these test suites and thus the associated test methods.

Variables and parameters in SmarTest on page 102

SmarTest variables and parameters are similar to variables and parameters of any programming language. They are used with test programs and their components like, testflows, test suites, test methods, measurement specifications, protocol-aware testing setups, and the APIs such as the Device Test API. They are also used with external components such as the Test Cell API and recipes.

Variables and parameters in SmarTest

SmarTest variables and parameters are similar to variables and parameters of any programming language. They are used with test programs and their components like, testflows, test suites, test methods, measurement specifications, protocol-aware testing setups, and the APIs such as the Device Test API. They are also used with external components such as the Test Cell API and recipes.

Note that in test methods and other modules that are *created in Java* for SmarTest, the standard variables of this programming language are available as usual, but they are not described in detail below. The following description mainly covers SmarTest specifics of variables and parameters.

Variables and parameters in the context of SmarTest

Term	Description
Predefined objects	These objects are inherited from a superclass (provided by SmarTest). These are used in test methods and you cannot assign a different name to them. For example, the mostly used predefined object is <i>context</i> that is defined in <code>TestMethod</code> .
Test program variables	Test program variables are values that are set in the test program file using the "var (Topic 244389)" keyword and can be read in all testflows and test methods. By default, they can be modified only in the <i>general auxiliary testflows</i> (PreBind, PreStart, PreRun, PowerUp, PowerDown, PostRun, PreStop) and in the <code>initialize()</code> methods of test methods. If they are declared as "mutable (Topic 244389)", they can also be modified elsewhere. See <i>Test program variables</i> on page 107 for more information. SmarTest provides a number of predefined variables (topic 253245).
Testflow variables	Variables of a testflow that are declared by the keyword <code>var</code> in the testflow file.
Testflow parameters	Variables of a testflow that are declared by the keywords <code>in</code> and <code>out</code> in the testflow file.
Test method variables	Variables of a test method that are not declared <code>public</code> , for example, fields that are declared <code>private</code> .
Test method parameters	Variables of a test method that are declared <code>public</code> and are accessible outside the test method. For details on accessing these fields, see "Accessible test method fields (Topic 253646)". (Note that there is no relation to test suite parameters (topic 343645), which control the way in which a test suite is executed and logged.)
Specification variables	Variables of a measurement specification that are declared by the keyword <code>var</code> in the specification file.
Operating sequence parameters	Predefined variables that can be used to configure the operation of burstable operating sequences.

All testflow and test method parameters of an active test program can be accessed through `context.testProgram().selectParameters()`. All test program variables of an active test program can be accessed through `context.testProgram().variables()`.

Usage of variables and parameters

Variables and parameters are used in the setup data, for example, to pass values to test methods or between testflows, or to define measurement specification variables (typically called spec variables), such as frequency, voltage, or current thresholds. Variables are containers for values that may change, for example, during the execution of a test program. Typically, variables are used as counters in loops or as flags that are evaluated for branching or enabling a specific data log mode.

Parameters are like variables, they store values. Typically, parameters are used to pass values, for example, from a test suite to a test method or between testflows. However unlike most variables, parameters can also be accessed from the outside. For example, test method parameters can be accessed and modified from the test suites that call them and testflow parameters can be accessed and modified by the main testflow that calls them. An instrument property is a parameter as well. With instrument properties the initial settings of an instrument are specified.

In addition to the standard Java data types, SmarTest provides "multisite data types (Topic 244329)" that enable you to define test method variables that automatically handle data for multiple sites.

Variables and parameters of test programs and testflows must be initialized because SmarTest implicitly declares them with the corresponding multisite data type.

Variables and parameters: declaration, scope, and lifetime

Variables can be defined in the test program, in testflows, in measurement specifications, in protocol definitions, in test methods and with APIs like the Device Test API (Java) or the Test Cell API (C++):

- Test program variables, including system and STDF variables, are global. They can be read throughout the test program components, for example, from the testflows and also from external components such as the Test Cell API and recipes.

For more information, see *Test program variables* on page 107.

- Testflow variables are local to the `execute` section of a testflow. They can be defined, read, and modified only in the `execute` section of the testflow. They are not available to the test program, other testflows (unless passed as a parameter value), or the Test Cell API. The lifetime of a testflow variable is the time of execution of the testflow, that is, as long the `execute` section of a testflow is running.

```
var myVariable = 10;
```

- Specification variables are local to the specification file. However, they can be read by other specification files in the import hierarchy and also by test methods. They are not available to the test program, testflows or the Test Cell API. In contrast to test program and testflow variables they do not need to be initialized when they are declared, because their data type is specified. The lifetime of a specification variable begins when the test program is bound and lasts until there is a rebinding (which may also be caused implicitly, for example, when the content of a specification file is modified).

```
var Frequency freq = 10MHz;
```

- Test Cell API variables are local to the test cell client. They can only be read and modified from within the client in which they are created and are unavailable in any test program. Note that in contrast to SmarTest APIs, the Test Cell API is written in C++.

The lifetime of a test cell variable is the time of the execution of the client in which it is defined.

Note that, in some *SmarTest execution states*, the modification of variables is restricted. For example, while the main testflow is running, test program variables (unless they have been declared to be always mutable) cannot be modified.

Also note that most variables are not reset automatically between touchdowns; testflow variables are among the exceptions. If you want to reset them, you must do so explicitly.

Parameters have different scopes and lifetimes compared to variables. A major difference is that they can be accessed from outside the element in which they are defined:

- Testflow parameters are defined locally in a testflow. However, they can be accessed from other testflows. The data type is determined and declared automatically.

```
// This will be declared as MultiSiteDouble
in inputParameter = 21.0;
// This will be declared as MultiSiteLong
out outputParameter = 42;
```

- Test method parameters are defined as public member variables. They can be accessed from the test suites that call them, and indirectly from other test methods, such as through testflows, see "Accessible test method fields (Topic 253646)".

For test methods and all APIs (apart from the Test Cell API and EDL API), declaring variables and parameters must be done explicitly and follow the rules of the Java programming language. When you declare a variable or parameter as `public`, a public field is created but it remains `null` unless it gets initialized explicitly:

```
public IPattern myPatternObject;
```

Note

Normally, declared measurements and test descriptors get initialized by SmarTest automatically. However, if the automatic initialization is not needed for a measurement, the annotation `@NoAutoInitialization` can be used with the declaration. In this case, the declared measurement remains `null` until it gets initialized explicitly. For more information, see "Declaring and initializing measurements with `NoAutoInitialization` (Topic 354547)".

- Operating sequence parameters are valid in the operating sequence. Their lifetime is the execution of the operating sequence.

Summary of variables and parameters

Overview of variables and parameters

Type	Declaration	Scope	Lifetime	Modified in ...
Test program variable	Test program file	Test program	Test program activation	Standard: Test Cell API, or some auxiliary testflows Mutable: As standard, and in test method code
Testflow variable	<code>execute</code> section of testflow	<code>execute</code> section of testflow	<code>execute</code> section of testflow	<code>execute</code> section of testflow
Testflow parameter	Testflow file	Testflow file	Test program activation	<code>execute</code> section of testflow
Test Cell API variable	Test cell client	Test cell client	Execution of the test cell client	Test cell client
Test method variable	Test method	Test method	Test program activation ^[1]	Test method code

^[1] Member variables are initialized when the test program is activated and live until the next reactivation. Other types of variables, for example, standard variables created in the `execute()` section of a test method, may have shorter lifetimes.

Type	Declaration	Scope	Lifetime	Modified in ...
Test method parameter	Test method	Test method, testflow containing the test suite	Test program activation	Test method, testflow
Specification variable	Specification file	All specification files, test methods	Test program binding (until rebinding)	Test method, equation calculation in specification
Operating sequence parameter	Burstable operating sequence setup	Operating sequence	Execution of the operating sequence	Not modifiable

Availability of variable and parameter types by execution state

Execution state	Test program	Testflow	Test suite	Measurement objects
<i>Test program activated</i>	Read, write	n/a	n/a	n/a
<i>Test program loaded</i>	Read, write	Read, write	Read, write	Read, write
<i>Test program bound</i>	Read, write	Read, write	Read, write	Read, write
<i>Testflow ready</i>	Read, write	Read	Read, write	Read
<i>Testflow running</i>	Read	Read	Read	Read

Namespaces and types of test program variables

Namespaces are used to distinguish between different types of test program variables, such as system variables which are relevant to the system behavior, and STDF variables, which are relevant for data logging. The namespace of a variable is a prefix with a period, for example:

```
STDF.LOT_ID;
```

The following namespaces are supported:

- **STDF** for STDF variables.
- **SYS** for system variables.
- **ENV** for "environment variables (Topic 252893)": This namespace is not used within test programs but may be needed for access through the Test Cell API or a recipe. It is also needed for access to the variables outside a test program, such as for "dynamic file names in run configurations (Topic 349273)".
- **TP** for test program variables. This namespace is only required when accessing test program variables from the Test Cell API or recipes.
- Test cell variables do not have a separate namespace. They can be used only in the test cell software.

This table shows where the different types of variables are available. A shaded cell indicates that a namespace prefix is required; where a cell is not shaded (for example, test program variables in test programs), no prefix must be used.

	Variables				
	STDF	System	Environment	Test program	Test cell
Prefix	STDF	SYS	ENV	TP	-
Test program	Yes	Yes		Yes	
Test cell	Yes	Yes	Yes	Yes	Yes
EDL data log	Yes	Yes	Yes	Yes	Yes

	Variables				
	STDF	System	Environment	Test program	Test cell
Run configuration (see topic 349273)			Yes	Yes	

For information on defining variables in test tables, see "Specifying values for test program variables in a test table" (topic 343955). For accessing variables from test methods, see *ITestProgramVariables*.

Measurement objects in test methods: declaration, scope and lifetime

In test methods, you can declare one or more measurement objects using either the `IMeasurement` interface or the `ILocalMeasurement` interface. This object stores the test setup data that is required to execute a certain measurement. In your testflow, every test suite calls a test method. Thus, every test suite that performs a measurement has its own measurement object. This object references the test setup data that is required to execute the measurement.

The values stored in a measurement object are derived from the test setup data, for example, the specification file you associate with it in the test table, the `setup()` or `update()` sections of its test method, or from the test suite that calls the test method. For example:

1. In a test method, you define a measurement object:

```
public IMeasurement myMeasurementObj;
```

or

```
public ILocalMeasurement myMeasurementObj;
```

You can define more than one measurement object in a test method.

2. A test suite of a testflow can call this test method and use its measurement objects (in this example, `myMeasurementObj`) to assign the setup data and other parameters that are required for the measurement execution. For example, you can assign the operating sequence file:

```
myMeasurementObj.operatingSequence =  
    setupRef(myDevice.myTest.myOpSeqFiles.myOperatingSequence);
```

You can also assign the setup data and set the test method parameters from within the test method itself. For details, see

Modifications on a measurement object can have local or global scopes:

1. Local: Local modifications to the test data of a measurement affect this particular measurement only. Declare the measurement object with the `ILocalMeasurement` interface and use the measurement object to access and modify the test data.
2. Global: Global modifications to the test data of a measurement object affect all measurement objects that reference the changed test data. Declare the measurement object with the `IMeasurement` interface if no local modifications are required. Otherwise, declare it with the `ILocalMeasurement` interface.

For more details of declaring measurement objects, see "Guidelines on using `IMeasurement` or `ILocalMeasurement` (Topic 348358)".

The values referenced by a measurement object are not reset until the test program is re-executed.

Related concepts

SmarTest execution states and states transitions on page 149

During the preparation and execution of a test program, SmarTest has several distinct execution states.

Related information

Test programs, testflows, and test suites on page 92

SmarTest uses a hierarchy of structures to specify what happens during a test, from the test program at the top and down to test methods and instruments.

Test program variables

Test program variables can be accessed globally throughout a test program, making their content available to all testflows and test methods and providing a way for them to exchange information.

Test program variables are declared in the "test program file (topic 244389)". By default, they can be modified only in certain limited phases of the test program execution, outside the main execution phase. They are often called "non-mutable".

If variables are declared as always mutable, they can be modified anywhere in the test program. However, accessing them in many cases requires locking and unlocking them which makes working with them more complicated. These variables are sometimes simply called "mutable".

Defining test program variables

Test program variables are defined in the test program file using the "var (topic 244389)" keyword. A variable can be defined either in the main test program file or in an *imported file*.

The definition includes the initial value of the variable and the type (which can be defined either implicitly or explicitly, as described *below*).

Optionally, the variable can be declared to be *modifiable* throughout the test program by adding the "@Options (topic 244389)" annotation with the `mutable = Anytime` setting; for example:

```
@Options(mutable = Anytime) var Long TestGlobalVar = 2;
```

Scope and lifetime

A test program variable is available in all testflows and test methods of the test program.

It is defined at the start of the test program and exists until the end of the test program.

Data type

Every test program variable has a fixed data type that is set when the variable is created. Available data types include `Boolean`, `Double`, `Long`, and `String`. Test program variables are automatically "multisite (topic 244329)" variables of the appropriate type.

We recommend to declare the data type explicitly, for example:

```
var Long myVar = 3;
```

When you are defining variables that are mutable anytime, you must declare the type.

Background and foreground access

For always mutable test program variables, it makes a difference whether the testflow or test method is running in the foreground or the background, as described in the following sections. (For non-mutable variables, this is not relevant.)

A test method is running in the foreground from the start of its execution until it calls either `releaseTester()` or `continueFlow()` or until its execution ends.

It is running in the background after it calls *releaseTester()* or *continueFlow()* until its execution ends.

A testflow is running in the foreground when none of the test methods it has started are running in the foreground, and it is running in the background otherwise.

Reading test program variables

Test program variables can be read in any testflow and test method.

Reading non-mutable variables and reading mutable variables in the foreground does not require additional steps. Reading mutable variables in the background requires locking them first. See *Read locks and write locks* on page 109 for details.

To read a test program variable, use the appropriate getter method of the *ITestProgramVariables* interface - *getBoolean()*, *getDouble()*, *getLong()*, or *getString()*. To read a variable in a testflow:

```
testProgramVariables.getDouble("myDoubleVar");
```

To read it in a test method:

```
context.testProgram().variables().getDouble("myDoubleVar");
```

If a testflow cannot be certain that a given test program variable exists, it can check that using the *isDefined()* method:

```
testProgramVariables.isDefined("myDoubleVar");
```

The same check in a test method:

```
context.testProgram().variables().isDefined("myDoubleVar");
```

As a convenience, the query for the existence of a variable can be combined with the read access and a default value can be specified that is returned when the variable does not exist. This is done using the *getBooleanOrElse()*, *getDoubleOrElse()*, *getLongOrElse()*, or *getStringOrElse()* method. In a testflow:

```
testProgramVariables.getDoubleOrElse("myDoubleVar", 0.5);
```

In a test method:

```
context.testProgram().variables().getDoubleOrElse("myDoubleVar", new  
MultiSiteDouble(0.5));
```

The value of *myDoubleVar* is returned if that variable exists and 0.5 if it does not. (The variable is not created if it does not exist.) The alternate value - 0.5 in the above example - can also be another variable or expression that evaluates to a value with a multisite data type. While the conversion to a multisite value happens automatically in the testflow, the test method must perform it explicitly.

For test methods, reading variables with a *Get...OrElse* method has the same locking requirements as reading it with a standard *Get* method. Testflows do not need to lock variables before reading them.

Modifying test program variables

Always mutable test program variables can be modified in any testflow and test method. Non-mutable variables can be modified only in the *general auxiliary testflows* (*PreBind*, *PreStart*, *PreRun*, *PowerUp*, *PowerDown*, *PostRun*, *PreStop*) and in the *initialize()* methods of test methods (topic 346129).

Modifying variables in the foreground does not require additional steps. Modifying them in the background requires locking them first. See *Read locks and write locks* on page 109 for details.

Test program variables are modified using the `set()` method of the *ITestProgramVariables* interface. To modify a variable in a testflow:

```
testProgramVariables.set("myDoubleVar", 3.0);
```

To modify it in a test method:

```
// Define and populate a MultiSiteDouble variable named (for example) "myValues"  
context.testProgram().variables().set("myDoubleVar", myValues);
```

Read locks and write locks

Multiple test methods can be executed at the same time, and therefore multiple test methods can try to access the same test program variable at the same time. This is not an issue for non-mutable variables but could lead to conflicts such as data races for mutable variables.

To avoid such conflicts, a test method must in many cases acquire a read lock or a write lock before it can access a mutable variable:

- A read lock prevents all other testflows and test methods from modifying the locked variable. They can still read it. Any number of testflows or test methods can have a read lock for the same variable at the same time. Write-locked variables cannot be read-locked.

Read locks ensure that the value of the variable does not unexpectedly change. The locking test method can rely on the value staying the same until the lock is released.

A mutable variable is locked for reading using the `acquireReadLock()` method. When it is read while running in the foreground, it is automatically read-locked, even if `acquireReadLock()` was not called.

- A write lock prevents all other testflows and test methods from reading or modifying the locked variable. Only one testflow or test method can have a write lock for the same variable at the same time. Read-locked variables cannot be write-locked.

Write locks ensure that no testflow or test method reads the variable while it has an outdated value.

A mutable variable is locked for writing using the `acquireWriteLock()` method. When it is modified while running in the foreground, it is automatically write-locked, even if `acquireWriteLock()` was not called..

A testflow or test method can acquire a lock only while running in the *foreground*. To access mutable variables in the *background*, it must acquire the appropriate lock while still running in the foreground.

If a test method cannot be certain that a given test program variable exists, we recommend to check for the existence of the variable first, using the `isDefined()` method. Trying to acquire a lock for an undefined variable throws an exception.

A read or write lock is automatically released when the execution of the locking testflow or test method ends. It is not possible to release a lock before that.

When locks are needed

Locks are never needed for non-mutable test program variables.

Read locks are needed to read an always mutable variable while running in the *background*. Write locks are needed to modify a mutable variable while running in the background.

Reading or modifying mutable variables while running in the *foreground* does not require an explicit lock: Any foreground read access automatically read-locks the variable; any

modification write-locks it. (Note that this lock remains until the execution of the accessing testflow or test methods ends.)

However, even when no explicit locking is needed, it may make sense to lock a variable. For example, if it is known or suspected that a test method may need to modify a variable, it can be helpful to write-lock it as early as possible so that other test methods do not read its outdated value.

Accessing locked variables

Any access by a testflow or test program that would violate the lock is deferred: The execution of the testflow or test method is paused until all blocking locks have been released.

A testflow or test method is paused when it tries to:

- Read a write-locked variable.
- Modify a read-locked or write-locked variable.
- Acquire a read lock for a write-locked variable.
- Acquire a write lock for a read-locked or write-locked variable.

Locks that a testflow or test method has acquired itself never cause it to be paused, only locks that other testflows or test methods have acquired.

Programming SmarTest

SmarTest supports extending its functionality by programming scripts and add-on modules in a number of ways. The most powerful of these ways are *test methods*, which let you fully control the tester hardware to create custom tests.

While *patterns* and *testflows* also are a form of programming, they are not described here. Despite the name, *test programs* themselves are not programs in the strict sense of the word and are therefore not described here either.

Types of code modules

SmarTest supports several types of code modules for different purposes:

Test methods

SmarTest comes with a range of "predefined test methods (Topic 255583)" for common testing requirements, and you can create your own test methods to meet requirements that the included test methods do not cover.

Test methods are programmed in Java, and you can use all Java-related tools that Eclipse provides. This includes not only editors, but also *Debugging* on page 185 tools.

Access to the test system for test methods is provided by several APIs, as described in the *Programming support* on page 111 section, below.

Further information of test methods is available in *Test method* on page 87.

Programs

Custom tools for controlling test program execution in production applications, usually involving accessing and controlling test cell equipment, are written as C++ programs using the "Test Cell API (Topic 247407)".

External tools

External tools can be programmed for a variety of purposes, as your workflows require. In particular, SmarTest includes the "Data logging reference (Topic 252002)" for accessing both the live data log stream and stored data log files for custom monitoring and analysis purposes and the "SmarTest Setup Format API (Topic 252687)" for converting external test description to and from SmarTest test programs and setup files.

Programming support

Tools

All standard Eclipse tools are available for programming inside SmarTest. In addition, SmarTest includes a range of specialized tools for creating test methods and other ATE-specific code. These tools are concentrated in the "Device Debug perspective (Topic 244556)", which supports you in finding errors in custom test methods, both in the code itself and in the test logic.

APIs

These are the main APIs for setting up and executing tests:

"Device Test API (Topic 244313)"

Used for controlling the test hardware during tests, for making dynamic adjustments to test settings during a test, and for retrieving test results.

"Device Setup API (Topic 252686)"

Used to programmatically generate *test setup files*, particularly when large numbers of setup files with only minor differences are needed.

"SmarTest Setup Format API (SSF API) (Topic 252687)"

Used to create and modify static setup files, for example, to convert test specifications to or from external sources. Often, this API is not used directly but called by other APIs, such as the Device Setup API.

The SSF API cannot be used directly in test methods, although the Device Setup API can.

"Test Cell API (Topic 247407)"

Used to access and control test cell equipment and the execution of test programs on production equipment. This is a C++ API.

All APIs are Java APIs, unless noted otherwise.

An overview of where each API is typically used is provided in "API and TML reference (Topic 247400)".

Other ways to program SmarTest

In addition to full-fledged programs for running test programs in test cell environments, you can also use script-like "recipes (Topic 248947)" for this purpose. They are simpler than programs and quicker to create, but more restricted in their capabilities. You can run them from the Test Cell Control Tool or "under program control (Topic 249774)".

IP protection

SmarTest helps you to protect your IP by letting you encrypt test programs before they leave a secure environment and by encrypting the data logs created by these test programs.

Test programs reflect the devices they are meant to test and, as such, may contain proprietary information that you do not want to reveal to external persons and entities.

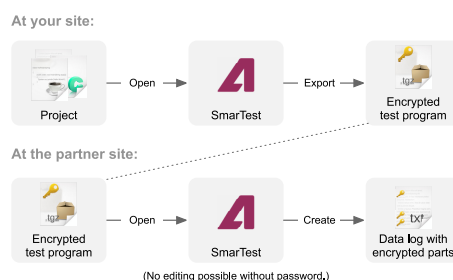
To support you in protecting the confidentiality of this information even when sharing test programs with external partners such as foundries, SmarTest can encrypt both sensitive sections of test programs and the data logs created by these test program sections.

Overview

Test programs are encrypted by exporting them in an encrypted format. Encryption is granular at the testflow level, which means that you can choose to encrypt testflows that contain sensitive information but leave others freely accessible.

Running these exported test programs does not require a password, but accessing the files in the encrypted portions does. Commonly, the exported test programs are provided to external partners but the passwords are not, ensuring that these partners can run the tests but not view any confidential information.

Running these test programs produces data logs in which the information created from the encrypted testflows is likewise encrypted. The unencrypted parts of the data log can be accessed normally, but evaluating or converting the encrypted parts requires a specific password (which can be different from the test program password).



Setting up test programs

Which parts of a test program are encrypted can be configured in the project properties and further fine-tuned by editing the `ARCHIVES.mip` file.

Test programs can be encrypted in part or completely. Often, only the sensitive parts are encrypted to allow external partners access to nonsensitive parts, for example, for fine-tuning settings for production.

When only parts of test programs are encrypted, all setup files accessed by encrypted testflows are encrypted as well by default. If you need to share setup files between encrypted and unencrypted testflows, special considerations apply:

- All shared test methods are automatically added to the protected archive by SmarTest; you do not need to do anything. These test methods can still be accessed by test suites in both encrypted and by unencrypted testflows.
- All specification files must remain unencrypted. You must manually add them to the `ARCHIVES.mip` file by adding `*.spec` to the `excludedFiles` section. For example:

```
excludedFiles {
  *.spec
```

```
}
```

Note that all specification files must be excluded, even those that are not shared between encrypted and unencrypted parts of the test program.

Encrypting test programs

To encrypt a test program after it has been set up, it is exported. This generates a run-only copy of the test program, which can be executed but not modified. To modify this test program, it must be imported into SmarTest, which requires the specified password. (Often, it is easier to keep the original, unencrypted version, and modify that when required.)

The exporting process, including the prior setup, is described in "Exporting an encrypted test program (Topic 256740)".

Modifying encrypted test programs

To modify an exported test program, it must be converted into an engineering workspace, which requires the password that was specified during the exporting process. (Often, it is easier to keep the original, unencrypted version, and modify that when required.)

The importing process is described in "Exporting a workspace to an engineering workspace (Decrypting an encrypted test project) (Topic 256964)". The overall debugging workflow is described in "Debugging an encrypted test program (Topic 343689)".

Encrypted data logs

Data log encryption is configured in the same way as the test program encryption. It is available only for encrypted test programs.

To specify which parts of the data log to encrypt, the corresponding testflows are specified in the ARCHIVES.mip file. Relevant events produced by these testflows are encrypted in the resulting EDL data logs. To enable the data log encryption, specify a separate data log password in the ARCHIVES.mip file.

When a test program with activated data log encryption is executed, relevant events generated by the encrypted testflows are automatically encrypted. These encrypted events cannot be viewed or converted to STDF without specifying the password. When an encrypted data log is accessed with any tool without specifying the password, the encrypted parts are simply skipped.

For more information on working with encrypted data logs, see "Data log encryption (Topic 349543)".

Note that STDF does not support encryption, so any encrypted data log that is converted to STDF (when the password is provided) is fully readable. However, any STDF records created from encrypted EDL events are marked as sensitive, as described in "Sensitive data in STDF records (Topic 349940)".

Modular setups

SmarTest provides modular setups - each setup file is self-contained. When you change a specific piece of information, only one file needs to be changed. SmarTest also provides a flexible setup file structure.

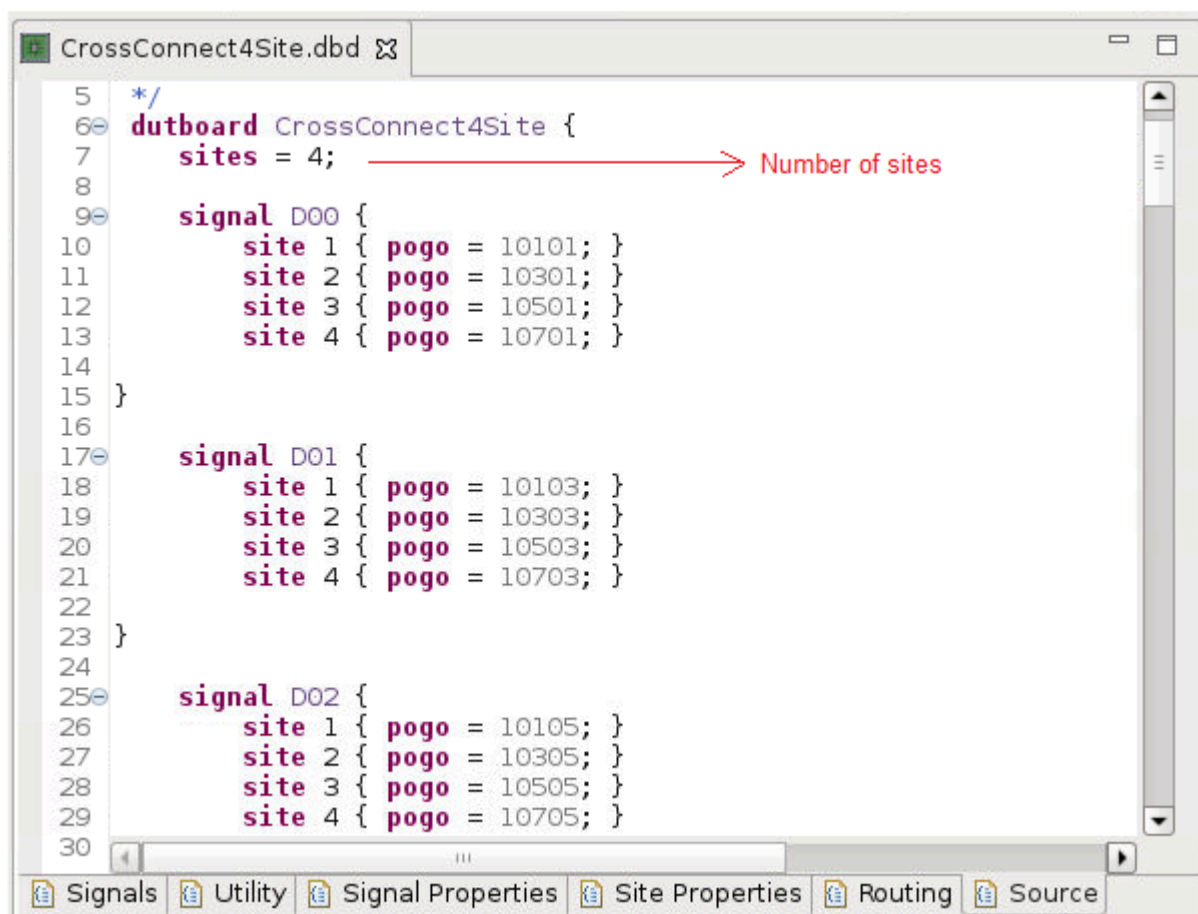
You can distribute the test method, testflow, and other test setup files over various folders and split the measurement specifications (level setup, timing setup, wavetables, actions, and signal groups) into separate specification files. For example, you can specify the signal group or the signal alias definitions for each IP in your DUT in separate specification files and then import them into your main specification file. You can also define the testflow for each IP in separate testflows (subflows) and then call them in your main testflow.

Test program files and DUT board description files can also be modularized, although to a limited degree:

- Test program files can contain other test program files that contain typed variable declarations, but no other content. For details, see "Modular test program files (Topic 244389)".
- DUT board description files can import other DUT board description files, but they can only contain DUT board properties, no site counts or signal definitions. For details, see "Modular DUT board descriptions (Topic 345539)".

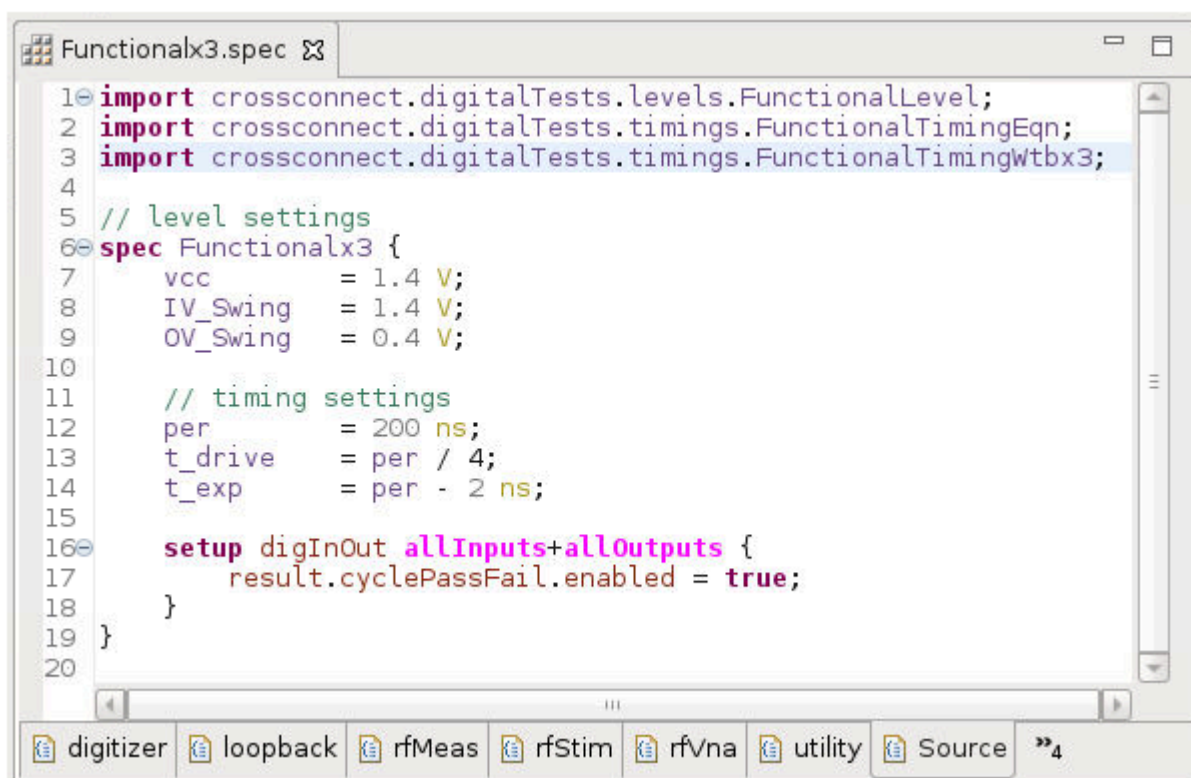
With this modularity and flexibility, it is easier for you to reuse setups, maintain future extensibility, perform collaborative development, debug and maintain your test programs, and use revision control systems efficiently.

The site count information is part of the DUT board description. When it needs to be changed, only the DUT board description file needs to be modified (see arrow). Other files remain unaffected.



SmarTest provides a flexible file setup structure. For example, you can define the timing and the level setups and wavetable definitions for each IP in your DUT in separate specification files and then import them to your main specification file.

In the following example, the main specification file imports the timing and the level setups and the wavetable definition from three separate specification files.

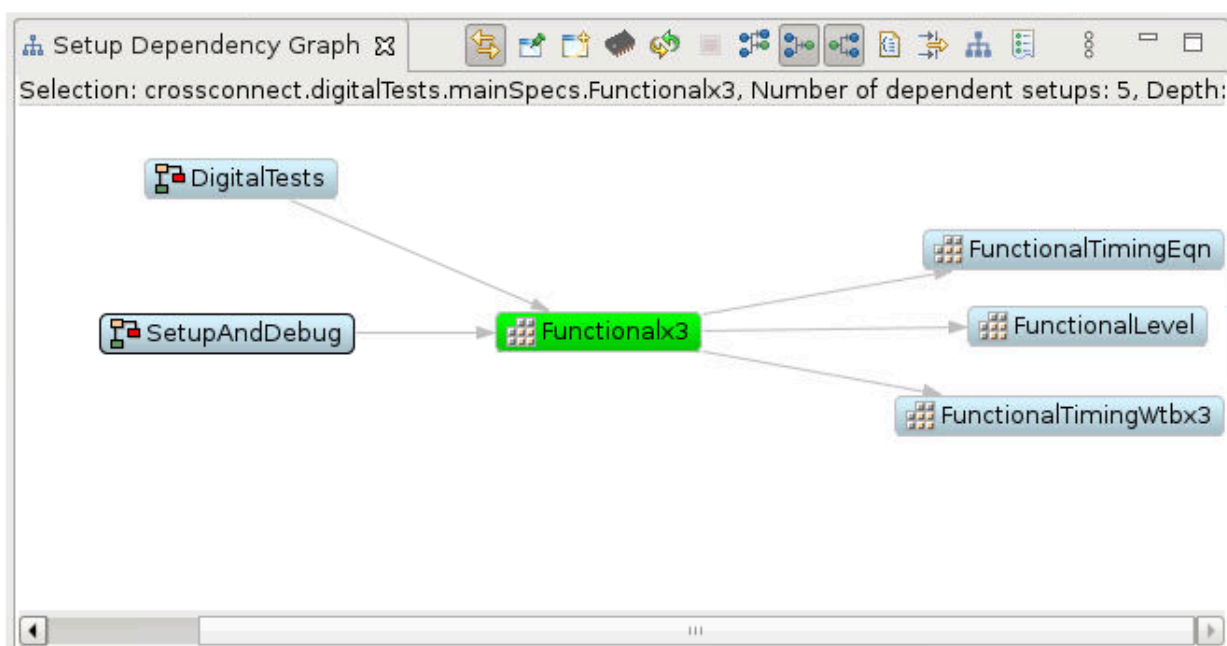


```

1 import crossconnect.digitalTests.levels.FunctionalLevel;
2 import crossconnect.digitalTests.timings.FunctionalTimingEqn;
3 import crossconnect.digitalTests.timings.FunctionalTimingWtbx3;
4
5 // level settings
6 spec Functionalx3 {
7     vcc      = 1.4 V;
8     IV_Swing = 1.4 V;
9     OV_Swing = 0.4 V;
10
11     // timing settings
12     per      = 200 ns;
13     t_drive  = per / 4;
14     t_exp    = per - 2 ns;
15
16     setup digInOut allInputs+allOutputs {
17         result.cyclePassFail.enabled = true;
18     }
19 }
20

```

In the example below, the **Setup Dependency Graph** view shows the files that the current file is importing (an outward arrow) as well as the files that reference it (inward arrows)



Related concepts

Test data reuse and sharing on page 118

You can share setup files or whole SmarTest projects between different projects or workspaces by copying them or importing them (collaborative development). Furthermore, you can reuse the test data inside your SmarTest projects by referencing setup files of the same type.

Collaborative development with SmarTest on page 122

SmarTest supports collaborative (also called concurrent or distributed) development. This covers the full range of complex production projects and test programs, functional blocks down to single test setup components.

Fully qualified names on page 131

SmarTest uses the concept of fully qualified names in many locations to unambiguously identify components. The details of these names depend on the type of component.

Related reference

SmarTest project structure on page 12

Source files for a SmarTest project are collected in a project folder. These source files can be flexibly distributed in subfolders of this project folder, but there are some requirements and recommendations.

Test data reuse and sharing

You can share setup files or whole SmarTest projects between different projects or workspaces by copying them or importing them (collaborative development). Furthermore, you can reuse the test data inside your SmarTest projects by referencing setup files of the same type.

Sharing test data

To share a test data file between multiple projects or workspaces, you can either copy it from the source project or reference it in place. If you copy the file, any changes made to one copy do not affect any other copies or projects. If you reference it, any changes affect all projects in which it is shared.

You can modify referenced files only if you have write access to them and when nobody else has opened them for editing. For the *collaborative (also called concurrent or distributed) development* of projects or test data, SmarTest supports revision control systems (also called version control systems), for example, CVS, SVN and Git.

For details on how to import a project or files, see "Importing a SmarTest project into your workspace (Topic 245939)" or "Importing project files into your project (Topic 250250)".

Reusing test data

One set of test data can be used in multiple places of a project by referencing the file that contains it from other files. If you change referenced files before the binding process, the changes affect all locations where they are referenced. If you change imported data after the binding process, the changes affect only a temporary copy in memory, not the file on disk, and therefore have no effect on other locations that reference the same file.

You can reuse test data files only within files of the same type - specification files in specification files, pattern files in pattern files, and so on. You can import files by adding appropriate statements to the files into which they are imported, for example, an `import` statement in a specification file, a `Call` (`patternCall` or `actionCall`) statement in a pattern file, or a `flow` statement in a testflow file.

To get an overview of the whole file import and reference relationships and the mechanism behind the process, refer to the topic *File import and reference mechanism in SmarTest*.

Note

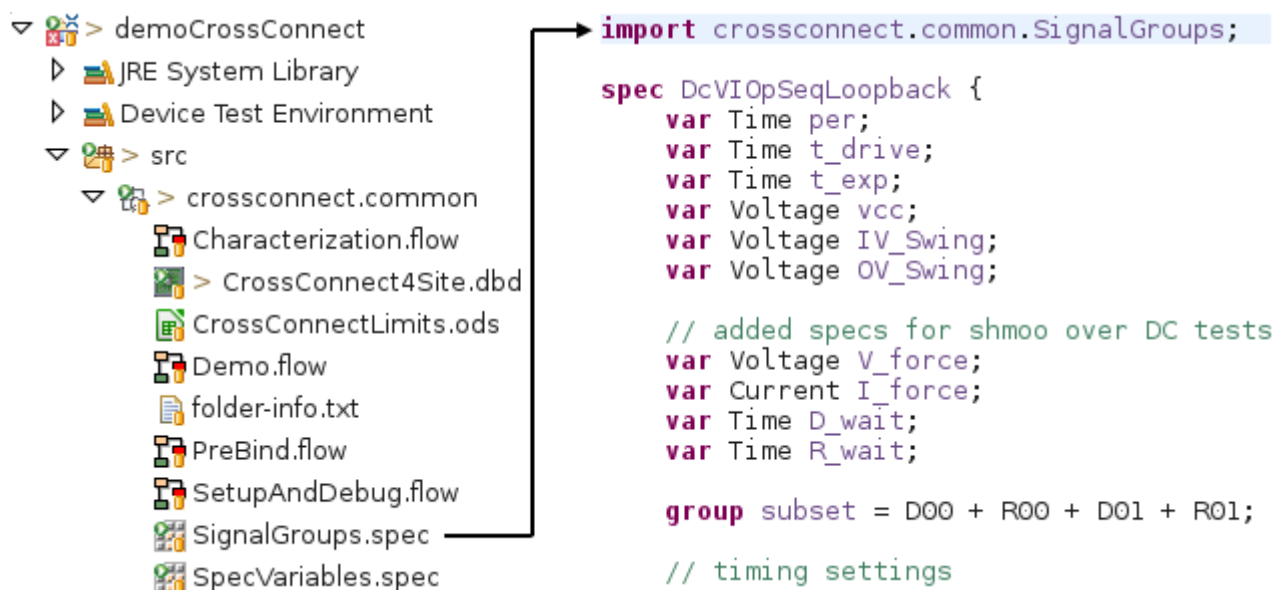
Make sure that you only reuse setup data that is never modified during the debug so that debugging a test does not affect other tests.

Reusing specification files

You can reuse specification files that contain signal group definitions, or timing and level setups by importing them into the main specification file. In the figure below, `SignalGroups.spec` contains the signal group definitions and is located in the path `src/crossconnect/common/`

SignalGroups.spec. This file can be imported into the main specification file using the import statement as follows:

```
import crossconnect.common.SignalGroups;
```



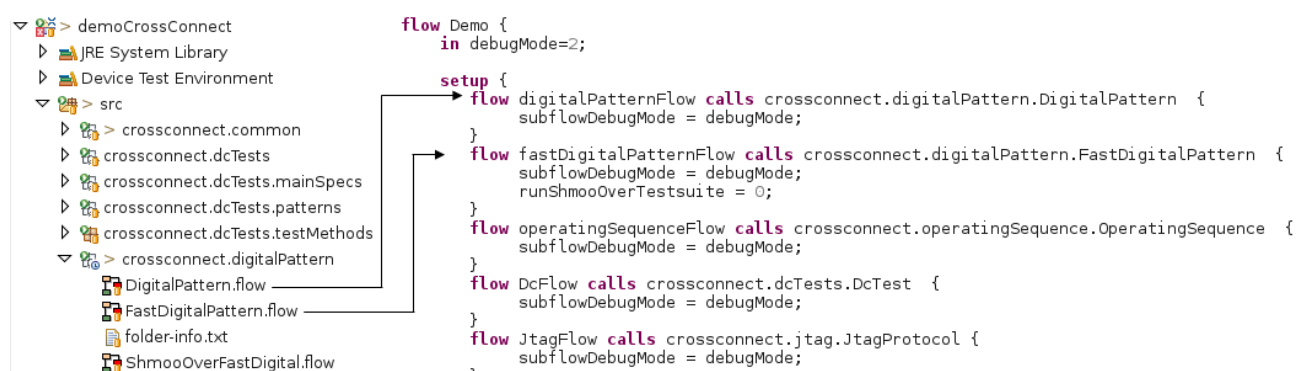
Reuse of setup data

Reusing testflow files

You can reuse setup files by referencing them in another setup file of the same type. For instance, a testflow can be referenced in another testflow file and a pattern can reference another pattern.

In the example figure below, the testflow files `digitalPattern.flow` and `fastDigitalPattern.flow` located in the path `src/crossconnect/digitalPattern` are referenced in the setup section of another testflow file using the `flow` statement as follows:

```
flow digitalPatternFlow calls crossconnect.digitalPattern.digitalPattern;
flow fastDigitalPatternFlow calls crossconnect.digitalPattern.FastDigitalPattern;
```



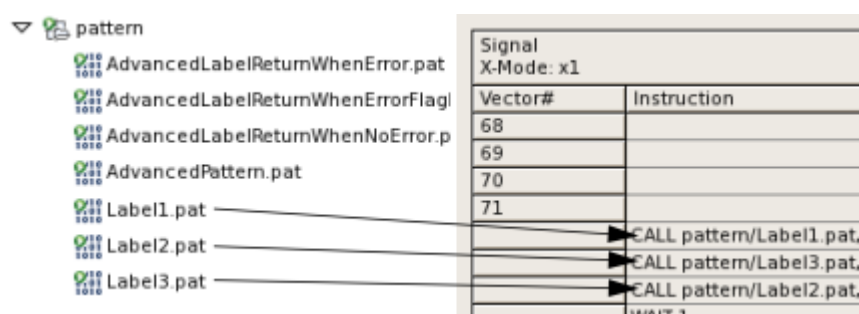
Reuse of setup data

Reusing pattern files

In the example below, the pattern files `Label1.pat`, `Label2.pat`, and `Label3.pat` are referenced in another pattern file using the `call` statement. It is assumed that the files are

located in the file path `pattern/Label1.pat`, `pattern/Label2.pat`, and `pattern/Label3.pat`, respectively. They are referenced in the pattern file using the `Call` statement as follows:

```
<Instruction id="Call" value="pattern/Label1.pat"/>
<Instruction id="Call" value="pattern/Label2.pat"/>
<Instruction id="Call" value="pattern/Label3.pat"/>
```



Reuse of setup data

Notice that you can also reference pattern files from an operating sequence file. For example, the pattern files `Label1.pat`, `Label2.pat`, and `Label3.pat` from the example above would be referenced in an operating sequence using the `patternCall` statement as follows:

```
patternCall "pattern/Label1.pat"
patternCall "pattern/Label2.pat"
patternCall "pattern/Label3.pat"
```

Related concepts

Collaborative development with SmarTest on page 122

SmarTest supports collaborative (also called concurrent or distributed) development. This covers the full range of complex production projects and test programs, functional blocks down to single test setup components.

General principles for setups on page 21

SmarTest uses different types of setups for different purposes. However, certain principles apply to all of them, as described below.

Related information

SmarTest projects on page 10

Test programs are organized in SmarTest projects. The content of each project is stored in a project folder in the file system. In addition, a project can also reference content from other projects. The folder structure within a project folder is partly mandatory and partly modifiable.

Combining and splitting setup files

In your test program, you can use a single specification file that has all the measurement specifications or split the information into dedicated files. Similarly, you can have a single DUT board file containing the complete description or separate files for each DUT board properties.

About this task

You can have a single specification file or separate files for some or each of the following:

- Signal group definitions
- Timing setup
- Level setups
- Wavetables
- Actions definitions

Similarly, you can use a single DUT board file or distribute the DUT board description among multiple files. In this case, the main DUT board file should contain the site count specification

as well as the definitions of all DUT signals. The additional files can contain information on the "DUT board properties (Topic 253659)" such as deembedding, settling time and fixture delays. This approach is very useful when you use different DUT boards, each with its characteristic fixture delay or other DUT board properties, during the test program development. For detailed description and remarks on using split DUT board files and use case examples, refer to the topic "DUT board file reference (Topic 244395)" and the related information.

Note

You can have "multiple DUT board files (Topic 343710)" in your SmarTest project. However, you can use only one file in your test program at a time, for instance, the one with the applicable DUT board properties.

The choice of using a single file or split files is dependent on the complexity of your SmarTest project and your development requirements. That is, for a simple testing scenario where the signal groups are used only locally within a single test and where the timing and level setups are only setting the specification variables, you might go for the single file approach. On the other hand, when you want to reuse the specification information among multiple tests or SmarTest projects, and to facilitate future extensibility, collaborative development, and meet the requirements for revision control, the split files approach could be a better option.

Note

During debugging, keep in mind that modifying setup data which is reused among multiple tests will affect other tests as well. You can modify the test data used by a test suite *locally* in order not to affect the original test data used by other test suites. You can also use the **Measurement view** to make modifications during debugging without affecting your original test data. This means, your modifications will be reset once the test program or the testflow has finished execution.

In this task, you will learn how to split your main specification file into separate specification files. Then, how to combine those files again into the main specification file with the use of the `import` mechanism.

Procedure

1. Create a dedicated specification file for each of the measurement specifications you would like to split, for example, a file for the signal group definitions, another one for the timing setup, and a third one for the level setup.
2. Cut and paste the respective information from the main specification file to the file you have created for it.
3. You can combine the measurement specifications that you have now in dedicated files into your main specification file again using the `import` statement for each of those files.

For example, assuming that the signal group definitions is in a file called `signalGroup.spec` and the file is located in the path `src/txTest/signalGroup.spec`, the import statement in your main specification file would be specified as:

```
import txTest.signalGroup.spec
```

In general, when a specification file uses a measurement specification, say the signal group definitions, which is not defined in the file itself but in another specification file, then you have to import it into this file and into all the files that use it. In this example, the specification files containing the timing and level setups that you have created in step one must also import the `signalGroup.spec` file using the import statement shown above.

See the section "Guidelines for importing setup files" in *test data reuse and sharing within and between SmarTest projects* for guidelines on importing files.

Results

You have split your measurement specifications into dedicated files and then combined them again into the main specification file using the `import` statement.

What to do next

When other measurements can also use the dedicated specification files that you have created, you can reuse them by using the import statements as explained in step three above.

Related concepts

Test data reuse and sharing on page 118

You can share setup files or whole SmarTest projects between different projects or workspaces by copying them or importing them (collaborative development). Furthermore, you can reuse the test data inside your SmarTest projects by referencing setup files of the same type.

Collaborative development with SmarTest on page 122

SmarTest supports collaborative (also called concurrent or distributed) development. This covers the full range of complex production projects and test programs, functional blocks down to single test setup components.

Related reference

SmarTest project structure on page 12

Source files for a SmarTest project are collected in a project folder. These source files can be flexibly distributed in subfolders of this project folder, but there are some requirements and recommendations.

Collaborative development with SmarTest

SmarTest supports collaborative (also called concurrent or distributed) development. This covers the full range of complex production projects and test programs, functional blocks down to single test setup components.

Depending on your objectives, you may want to reuse as much as possible or reuse only setup data that is never touched when debugging. In most cases you can identify different categories of setup data, for example, timing or level specifications or testflows, which shall be developed concurrently by different team members.

The *SmarTest project* and *workspace* concept supports different development strategies. Advantest recommends to identify the test data categories and organize them in different files and folders. *SmarTest project structure* on page 12 shows, how you can organize SmarTest projects in an efficient structure.

To support the full functionality of revision control systems (also called version control systems), like CVS, SVN or Git, the SmarTest *test data* is stored in text files. The files can use different syntax, for example, the SmarTest Setup Format or the Java programming language.

SmarTest is shipped with third-party plug-ins for CVS and Git, see **Help > About SmarTest Work Center**, click **Installation Details**, tab **Features**.

Additionally, SmarTest plug-ins technology allows you to add, for example, administration, communication, and control tools.

For more information about Eclipse and its concepts, see **Help > Help Content > Workbench User Guide** or visit the *Eclipse web site*, where you can find detailed information and tutorials for its basic concepts, which are also used in SmarTest.

Default specification settings

Defaults for certain instrument settings can be specified in the DUT board description file using the `defaults.` prefix.

You can assign default values for certain instrument and action properties to signals defined in the *DUT board description*. These defaults apply to those signals in all tests unless other values are assigned to the same properties in a *specification*, through a "test table (Topic 249089)", or in *test method* code.

To specify a default, add it to the signal definition in the DUT board description file. For example, to specify a default `vrange` for the VCC signal:

```
signal VCC {
    defaults.level.vrange = 10V;
    ...
}
```

Note that expressions are not supported in the DUT board description file. It is not possible to specify a default as a formula or based on another setting or a variable. Defaults can only be specified using literal values. You can use all units and prefixes listed in "SmarTest units and prefixes (Topic 244378)".

Settings that require derived units (that is, units that are a combination of two or more SI base units) must be specified using units in bracket notation. You can do so in two ways:

- By specifying the value as a literal and adding the derived unit in square brackets.

For example, to specify a current slew rate, you use `[A/s]` as the unit:

```
regulation.islewrate = 0.4[A/s]
```

It is not possible to use units with prefixes in bracket notation. For example, you cannot specify `[mA/s]`.

- By defining a variable with the derived unit in square brackets and assigning it a suitable equation.

For example:

```
var [A/s] myIslewrate = 400mA/1s;
var [V/s] myVslewrate = 40uV/1s;
```

It is not possible to use units with prefixes in bracket notation. For example, you cannot specify a variable with `[mV/s]`.

⚠ CAUTION Make sure that all specified defaults are valid values.

Some default settings in the DUT board description file cannot be checked for validity when the file is edited because relevant information (such as whether ganging is applied) is not available.

If invalid defaults are specified, they are applied during testing and can lead to invalid measurements and other unexpected behavior.

Available default settings

You can specify defaults for these settings:

- `alarm.currentClampTimeToAlarm`
- `alarm.continueReporting`
- `connectMode`
- `disconnectMode`
- `level.irange`
- `level.precisionVoltageRange`
- `level.vrange`
- `protection.property`

You can specify any property from the *protection* group.

- `regulation.property`

You can specify any property from the *regulation* group.

- `safeConnect`

Developing and using functional (IP) blocks

Functional blocks (also known as intellectual property or IP blocks) are developed in individual SmarTest projects, the testbed projects. They provide the required framework to develop, execute and debug a functional block before it is used in SmarTest projects that are executed in production.

SmarTest projects usually contain multiple functional blocks that are developed collaboratively in distributed teams and implemented to the final SmarTest project. Testbeds enable you to develop functional blocks in teams independently of the implementing SmarTest project.

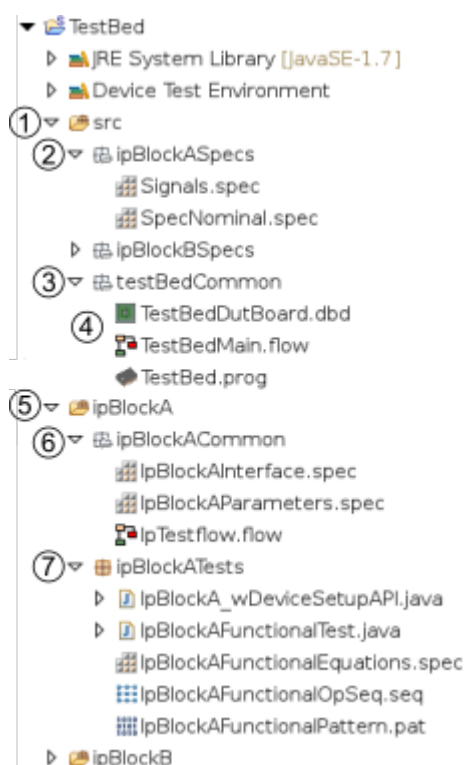
Once a functional block reaches a certain level of maturity, it can easily be reused and implemented to multiple SmarTest projects. This is realized by using the same structure and file names in both the testbed and production project. The SmarTest Work Center automatically uses the corresponding files. That means, if you develop in the testbed project, the files specified in the testbed are used. If you linked the functional block to a production project, the production files are used accordingly.

Functional blocks are implemented to SmarTest projects that are designed for high volume manufacturing, or failure analysis.

SmarTest provides a protection mechanism to help you *protect your IP* against information disclosure.

Structure of a testbed project

A testbed project is structured similar to other *SmarTest projects*. However, there are some aspects that you should observe to ensure that the functional block you are developing can be properly integrated into the production project.



The structure of a testbed project

1 - Standard source folder

Contains all packages and subfolders required to develop one or more functional blocks within a testbed project. This is part of the structure of each SmarTest project.

We recommend that you develop only a single functional block within one testbed project (or a set of very similar blocks).

2 - Functional block specification package

Contains the specification files of one functional block that will be re-defined in the implementing project, for example, the signal specification.

The implementing project must define a package with the same name. If a testbed contains multiple functional blocks, you must define a separate package for each functional block. In the example to the left, these are IpBlockASpecs and IpBlockBSpecs.

The focus of a testbed project is on developing the functional block. Thus the values in the measurement specifications are usually not real or very relaxed.

3 - Testbed common package

Contains files that are required by and are specific to the testbed framework. This is identical to a common package of any SmarTest project.

4 - Testbed common files

Files that are specific to the testbed, for example, the DUT board description and a main testflow for the testbed test program.

5 - Functional block-specific sub-folder

Contains all packages and files required to test this functional block.

6 - Functional block-specific common package

Contains files that are required by and are specific to the functional block. For example, parameter and interface specifications.

7 - Functional block-specific test package

Contains the files required to test one or more features of this functional block. This can be segmented further, for example, to separate digital from analog tests.

Developing functional blocks

To develop your functional block and make it reusable by other SmarTest projects, you need to follow the structure outlined in the figure above. Developing is always done in the testbed project, which enables you to develop, execute and debug your functional block independently from the production projects that will implement it. The following files are mandatory to be defined in your testbed project:

Note You can choose your own file names.

Parameter specification

It defines the specification variables that is used by the implementing SmarTest project set their values during execution of a testflow. This enables you to use different values while

developing the functional block and in production. This file must reside in the functional block-specific common package (6).

Signal specification

This assigns the DUT signals to the functional block signals during the development of the functional block in the testbed. The same file name is used in the implementing SmarTest project. There it contains the signal definitions that match the production project. Thus this file must reside in the functional block specification package (2).

Interface specification

Ties together the parameter and signal specification by importing their content. As this is functional block-specific, it must reside in the functional block-specific common package (6).

The functional block-specific test data is developed and stored in an individual functional block-specific test package (7). It contains all files specific to your functional block, for example, pattern, operating sequences, test methods and additional measurement specifications.

The testbed test program is identical to any other SmarTest test program. It calls at least a Main testflow, that can call other testflows or execute test suites. You can also use auxiliary testflows to condition your DUT during engineering to execute the functional block tests.

Note

Use version management tools, like git, or SVN, if you develop SmarTest projects in distributed teams.

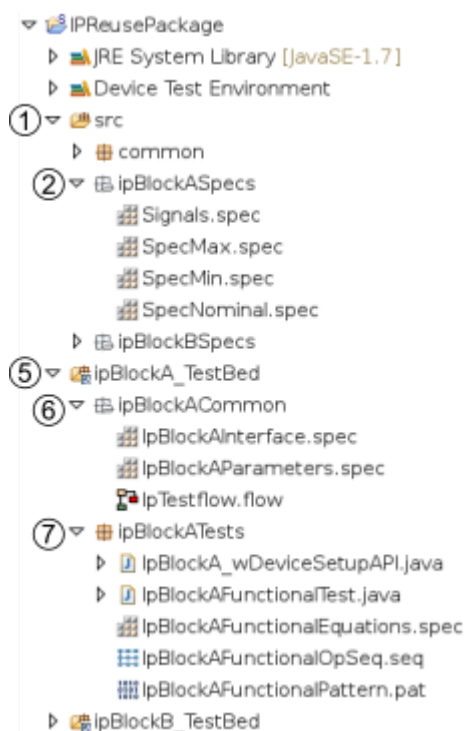
Using functional blocks

Once a functional block reaches a certain level of maturity, you can implement it to a production project. Both projects use the same functional block-specific structure and set of functional block-specific files. The files only differ in their content. This enables you to use one set of values while developing a functional block in the testbed project and another set of values with the production project, even after it was implemented to a production project.

The focus of a production project is on device testing, thus the values in the

To use a functional block with a production project, you need to:

- Have a signal specification file in your implementing SmarTest project. This must reside in the same folder and package and must have the same file name as the one used in the testbed project. Refer to (2) in the figure above and below.
- Link the functional block-specific source folder of the testbed project to the implementing project using a **SmarTest Project link**. Refer to (5) in the figure below.
- Call the functional block-specific testflow from a testflow, for example the Main testflow, of the implementing project.



Using functional blocks in a SmarTest project

1 - Standard source folder

This is part of the structure of each SmarTest project.

2 - Functional block specification package

Contains the specification files of one functional block that hold the values of the implementing project, for example, the signal specification. This must contain at least the files used within the testbed project. Additional specifications files can be added, for example, to execute the test with different settings, for example, min or max settings. If you have multiple functional blocks in your testbed, you have to define at least the package of the functional block that you implement in this production project. Also refer to (2) in the figure above.

Note The focus of a production project is on device testing. Thus the values in the measurement specifications are usually real values from design or empiric values.

5 - Project link to functional block-specific subfolders

This is the functional block-specific subfolder of the testbed. It physically still resides in the testbed but is linked to the implementing SmarTest project.

6, 7 - Functional block-specific subfolders

Because the testbed folder (5) is linked, not copied, its structure is identical to the one in the testbed (see the figure above).

Related concepts

Collaborative development with SmarTest on page 122

SmarTest supports collaborative (also called concurrent or distributed) development. This covers the full range of complex production projects and test programs, functional blocks down to single test setup components.

Test data reuse and sharing on page 118

You can share setup files or whole SmarTest projects between different projects or workspaces by copying them or importing them (collaborative development). Furthermore, you can reuse the test data inside your SmarTest projects by referencing setup files of the same type.

IP protection on page 113

SmarTest helps you to protect your IP by letting you encrypt test programs before they leave a secure environment and by encrypting the data logs created by these test programs.

Related information

SmarTest projects on page 10

Test programs are organized in SmarTest projects. The content of each project is stored in a project folder in the file system. In addition, a project can also reference content from other projects. The folder structure within a project folder is partly mandatory and partly modifiable.

File import and reference mechanism in SmarTest

SmarTest allows you to reuse setup files. This topic describes the mechanism behind, gives an overview of the whole file import and reference relationships.

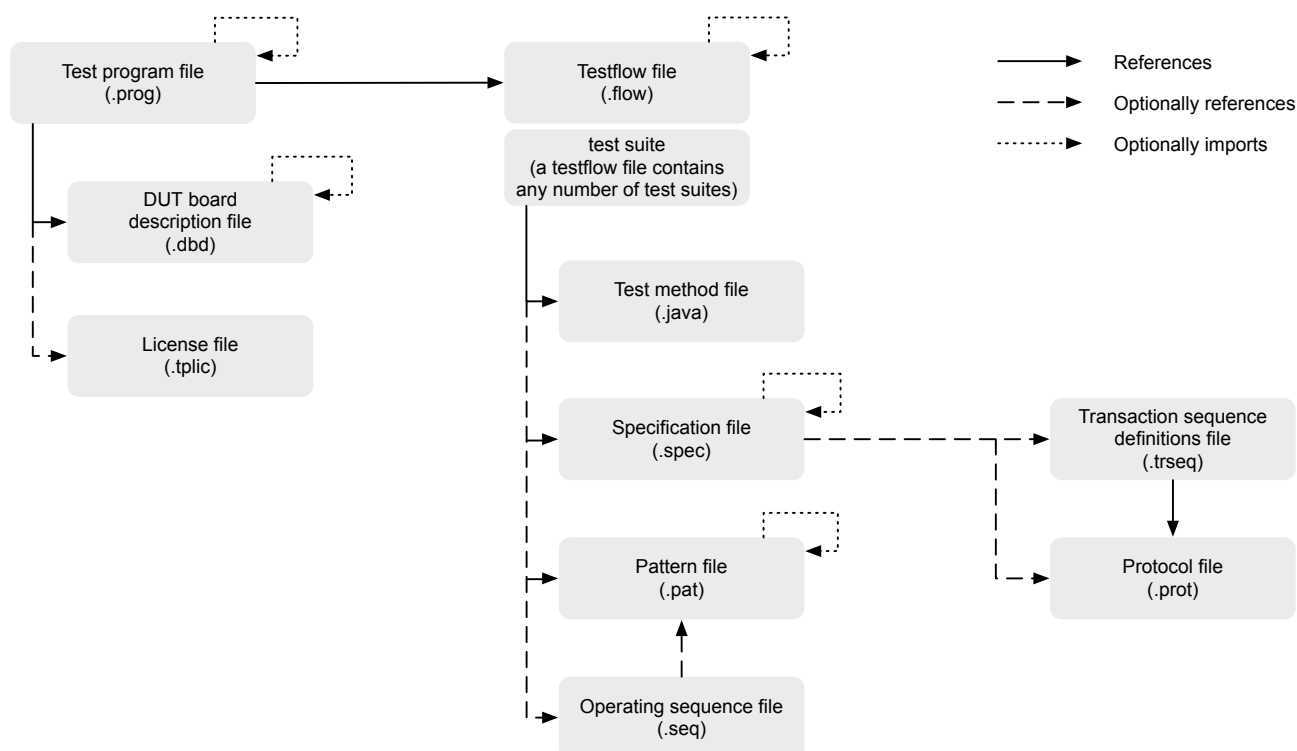
Files import and reference mechanism in SmarTest, overview

You can import and reference external files in various places, for example, in test suite setups, testflow setups, specification files, pattern files, and test program files. To import or reference

a test method or a setup file in another setup file, there is a set of guidelines that you must observe:

- You always use the relative file path (relative to the defined source folder, typically `src`).
- The relative file path is always specified as a fully qualified name. Using partially qualified names or unqualified names is not supported.
- The file name extension is always omitted.
- Objects, such as signals or instruments, in different setup files may have the same names, which leads to conflicts if one of these files is imported into another one. In order to avoid these naming conflicts, we recommend that you set up naming guidelines, especially for distributed development teams, that ensure that the same name is not given to multiple different objects.

The following diagram gives an overview of the setup files and their import and reference relationships. For details on a specific file, see the descriptions below.



Files import and reference mechanism in SmarTest

Referencing setup files in your test program

In your test program, you can reference the following setup files:

- DUT board description file
- Testflow file
- License file

For example, when referencing the DUT board file `myDutboard.dbd` located in `src/a/b/` `myDutboard.dbd`, you can specify it using its relative file path (relative to the source folder `src`) specified as a fully qualified name:

```
a.b.myDutboard
```

In addition to referencing these files, you can also import other test program files. These imported files can only contain variables, and all of those variables must be typed.

See the description of the `dutboard`, `flow`, `licensing`, and `import` elements in the "test program file reference (Topic 244389)" for more details on how to reference and import, respectively, those files in your test program file.

Referencing testflow files in your main testflow

In your main testflow, you can reference (call) other testflow files (subflows). For example, the file path of the subflow `mySubflow.flow` located in `src/a/b/mySubflow.flow` can be specified using its relative file path (relative to the source folder `src`) specified as a fully qualified name:

```
a.b.mySubflow
```

See the description of the element `flow` in the "testflow file reference (Topic 244398)" for more details on how you can call subflows in your main testflow file.

Referencing test methods and setup files in your test suites

In a test suite, you can reference:

- Test method file
- Specification file
- Pattern file
- Operating sequence file

For example, the test method `functional.java` located in `src/a/b/functional.java` can be specified using its relative file path (relative to the source folder `src`) specified as a fully qualified name:

```
a.b.functional
```

See the description of the element `suite` in the "testflow file reference (Topic 244398)" for details on how to reference the test method and the setup files in your test suites.

Importing and referencing setup files in your DUT board file

In a DUT board file, you can import other DUT board files that contain the definitions of DUT board properties such as `fixtureDelay` or `deembedding`.

For example, the DUT board file `DUT_properties.dut` located in `src/a/b/DUT_properties.dut` can be specified using its relative file path (relative to the source folder `src`) as a fully qualified name:

```
a.b.DUT_properties
```

You can reference external files such as `*.citi` and `*.s2p` files that contain the data for the DUT board properties.

Importing and referencing setup files in your main specification file

In a specification file, you can import or reference:

- Specification file
- Protocol definition file
- Transaction sequence definitions file.

For example, you can import specification files that contain the signal group definitions, timing sets, level sets, actions, or wavetables. Note that for the imported specification file, all the files in its import hierarchy are also implicitly available in the importing specification file.

For example, the specification file `signalGroups.spec` located in `src/a/b/signalGroups.spec` can be specified using its relative file path (relative to the source folder `src`) as a fully qualified name:

```
a.b.signalGroups
```

You can reference external files that contain waveform sample data such *.wfm, *.vsa, and *.cwf files.

You can also reference protocol definitions file and the transaction sequence definitions files for protocol-aware testing.

See the description of the element `import` in the "Specification file reference (Topic 244401)" for details on how to import other specification files into your main specification file. Also, see the description of the elements `waveform`, `protocolInterface` and `transactSeq` for details on how to reference external files namely: referencing external files that contain a waveform's sample data and your protocol definition file and transaction sequence definitions file, respectively.

Importing protocol definition file in your transactions sequence definitions file

In your transaction sequence definitions file, you can import the protocol definitions file. For example, For example, the protocol definitions file `Jtag32.prot` located in `src/a/b/Jtag32.prot` can be specified using its relative file path (relative to the source folder `src`) specified as a fully qualified name:

```
a.b.Jtag32
```

See the description of the element `import` in the "transaction sequence definitions file reference (Topic 252691)" for details on how to import protocol definition files into your transaction sequence definitions file.

Referencing pattern files in your main pattern file

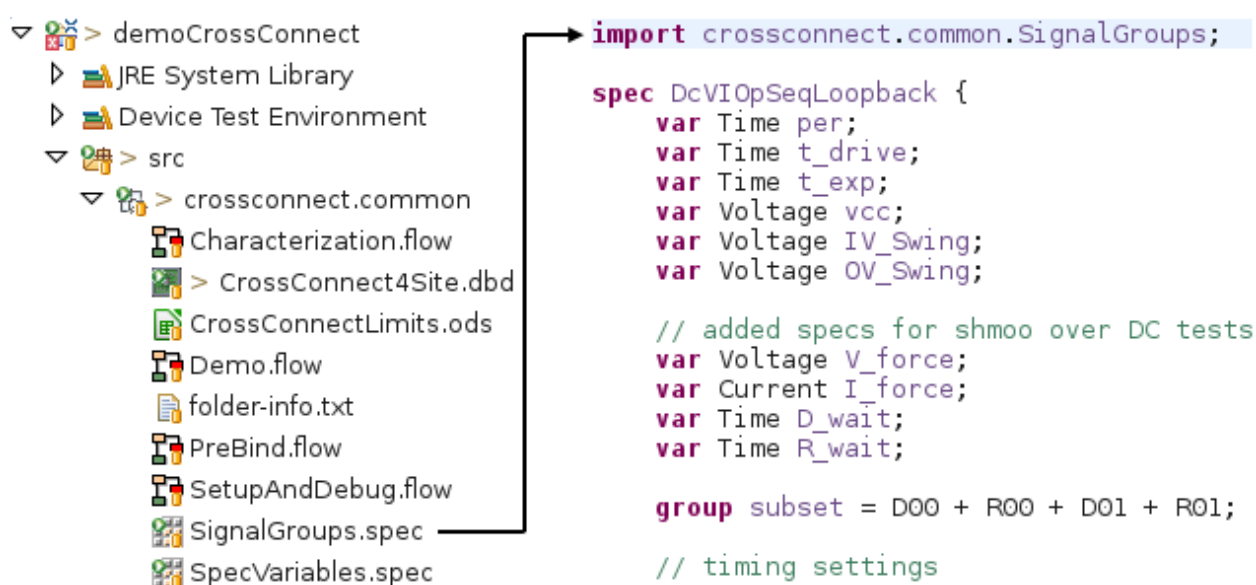
In a pattern file, you can reference (call) other pattern files. When calling a pattern file, you can specify its path as described above. For example, to call the pattern file `PatA.pat` located in `src/a/b/PatA.pat`, you can specify it using its relative file path (relative to the source folder `src`) specified as a fully qualified name:

```
a.b.PatA
```

See, "patternCall (Topic 244923)" instruction reference (Topic 244923)" for details on how to call other patterns in your pattern file.

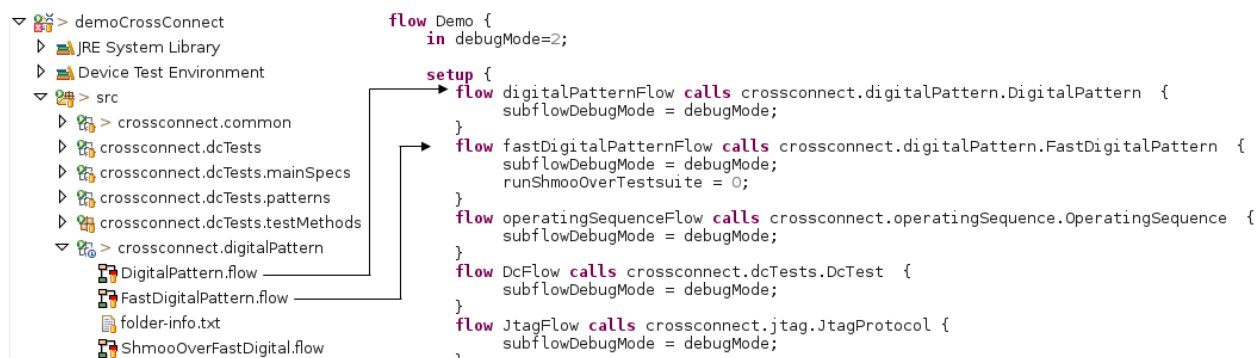
The following examples show how this looks in the editors for some of the setup files:

- Specification file



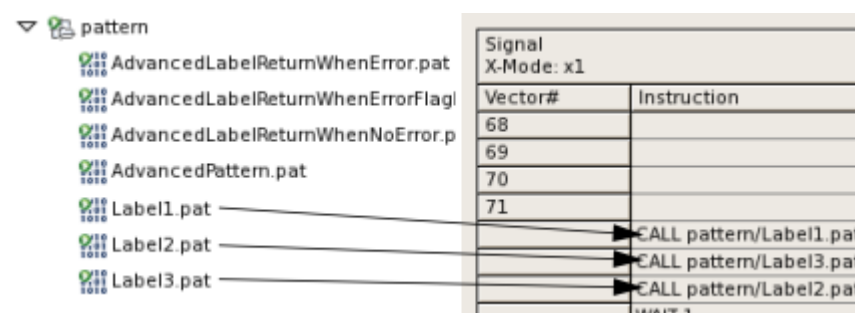
Reuse of setup data

• Testflow file



Reuse of setup data

• Pattern file



Reuse of setup data

Fully qualified names

SmarTest uses the concept of fully qualified names in many locations to unambiguously identify components. The details of these names depend on the type of component.

Fully qualified names are a common concept in IT for unambiguously specifying objects. SmarTest requires fully qualified names whenever the simple name of an object would not be enough to identify it.

Fully qualified names for executable components are unique within a test program and allow components with the same name in different testflows to be distinguished from each other. Test program variables do not have fully qualified names because they are global (which means that their names already must be unique).

How fully qualified names are specified depends on the nature of the named object:

- Setup files are identified by their location in the file system. Their fully qualified names are basically relative paths starting in the source folder.
- Components that are being executed (such as test suites or test descriptors) are identified by the hierarchy of calling testflows and test suites.
- API packages, interfaces, classes, and methods are identified by their package hierarchy.

In some cases, in particular regarding testflows, you need to use different qualified names depending on whether you want to address the setup file or the element being executed. For example, if you want to specify the testflow setup file, you must use the location in the file system. If you want to specify the testflow as it is being executed, you must use the calling hierarchy.

Elements in fully qualified names are separated by dots.

Note

When referencing setup files with their fully qualified names, the corresponding filename extensions, for example, .flow or .spec, are omitted.

Fully qualified names of specific components

Setup files

All kinds of setup files are specified by their location in the file system relative to the project folder.

For example, a testflow file that is located in `<project folder>/src/common/basic_flows` and is named `test2.flow` has the fully qualified name `common.basic_flows.test2`.

The specification file `DpsOn.spec` in the folder `<project folder>/src/common/mainSpecs/` has the fully qualified name `common.mainSpecs.DpsOn`.

Testflows

Testflows being executed are identified by their name. If they have been called from another testflow, the fully qualified name of that testflow must be prefixed to their name.

For example, the fully qualified name of the main testflow is (usually) `Main`.

If the main testflow calls the testflow `tf3`, and that testflow calls another one named `tf1`, the fully qualified name of `tf1` is `Main.tf3.tf1`.

Test suites

For *test suites*, the fully qualified name is the name of the test suite, prefixed by the fully qualified name of the testflow that contains it.

For example, if the testflow `tf1` mentioned above contains the test suite `setupSuite`, its fully qualified name is `Main.tf3.tf1.setupSuite`.

Testflow variables

For testflow variables, the fully qualified name is the name of the variable, prefixed by the fully qualified name of the testflow in which it is defined.

For example, if the `myTestflow` testflow mentioned above contains the variable `waitTime`, its fully qualified name is `Main.tf3.tf1.waitTime`.

Test methods

For *test methods*, the fully qualified name is created from the relative file path of the test method, beginning in the source folder.

For example, the fully qualified name of the test method `src/IP1/testMethods/myTestMethod.java` (`src` is the source folder) is `IP1.testMethods.myTestMethod`.

Test descriptors

For test descriptors, the fully qualified name is the name of the test suite which calls the test method, followed by the test descriptor field in the test method.

For example, the fully qualified name of the test descriptor represented by the `IFunctionalTestDescriptor` object `myFTD` in a test method called from the `setupSuite` test suite described above is `Main.tf3.tf1.setupSuite.myFTD`.

Test program configuration settings

Each setting in a "test program configuration file (Topic 352872)" has a fixed fully qualified name. These names must always be used as noted; they cannot be modified, and you cannot create fully qualified names of your own for settings.

Fully qualified names for executable components are unique within a test program and allow components with the same name in different testflows to be distinguished from each other. Test program variables do not have fully qualified names because they are global (which means that their names already must be unique).

Note

You must not assign the same fully qualified name to a test method and a testflow within a SmarTest program. Otherwise, a duplicate error occurs.

Fully qualified names and wildcards

In some locations where fully qualified names are specified - for example, in test tables - you can use wildcard characters. This is done using a modified glob syntax, which is described in *Glob syntax* on page 133.

Related information

Fully qualified name

Glob syntax

SmarTest's glob syntax lets you use patterns for specifying strings, in particular, for fully qualified names. The syntax comprises two wildcard tokens (`*` and `**`) that match zero or more characters and zero or more path segments, respectively, as well as full access to Java regular expressions.

The syntax allows for placeholders that are path-segment aware, and for full regular expressions:

Any number of characters within one file path segment. (That is, `*` matches any character except a period.)

For example, `"*_io"` matches `"abc_io"` and `"test_io"` but not `"meas.abc_io"`.

`"meas.*_io"` matches `"meas.abc_io"` but not `"abc_io"`.

Any number of full or partial path segments (including leading and trailing periods), including zero segments.

For example, `"meas**abc_io"` matches `"meas.abc_io"` and `"meas.volt.input.abc_io"`.

`"**"` (without any preceding or following characters) matches any string, including any qualified name or file path.

regex:regular-expression

A regular expression that follows the syntax of Java regular expressions.

For example, `"regex:ab[cde]"` matches `"abc"` and `"abe"` but not `"ab"` or `"abs"`.

For a full list of supported syntax options, see the official *Java documentation*.

Note that the `regex` expression is not available in all places in SmarTest where the glob syntax is used. For example, it is not supported in test tables.

Related information

Java regular expression syntax

SmarTest Setup Format

SmarTest uses a domain specific language known as the SmarTest Setup Format (SSF) to specify the majority of the test data.

The SSF provides:

- **Simplicity:** Navigation options and Content Assist functionality to simplify your work and provide assisted setups
- **Readability:** the syntax is close to its semantic and uses terms from the testing domain which results in your setups being intuitive and human readable
- **Usability:** you get instant feedback before binding on any errors, for example, typos, type and unit mismatch, unresolved references, syntax errors, as well as suggestions for quick fixes
- **Loadability:** your setup files are easy to access, a file containing corrupted content can still be opened and fixed

The following setup files are based on the SSF:

1. Test program files (.prog)
2. DUT board files (.dbd)
3. Specification files (.spec)
4. Flow files (.flow)
5. Operating sequence files (.seq)
6. Transaction sequence files (.trseq)
7. Protocol files (.prot)

You can edit your setup files using the setup editor. This image shows the DUT board file. The source tab of the setup editor provides a text editor with content assist while other tabs (Signals, Utility, Signal Properties, Site Properties and Routing) provide tabular editors.

The screenshot displays the SmarTest Setup Editor interface. On the left, the 'Source' tab shows the SSF code for 'CrossConnect4Site.dbd'. The code defines three signals (D00, D01, D02) and their connections to four sites. On the right, the 'Site Properties' tab shows a table with 8 rows and 7 columns. The columns are 'Signal', 'disabled', and 'Pogo' (with sub-columns for Site 1, Site 2, Site 3, and Site 4). The table lists connections for signals D00 through D07.

	Signal	disabled	Pogo			
			Site 1	Site 2	Site 3	Site 4
1	D00	false	10101	10301	10501	10701
2	D01	false	10103	10303	10503	10703
3	D02	false	10105	10305	10505	10705
4	D03	false	10107	10307	10507	10707
5	D04	false	10109	10309	10509	10709
6	D05	false	10111	10311	10511	10711
7	D06	false	10113	10313	10513	10713
8	D07	false	10115	10315	10515	10715

Related concepts

Protocol-aware testing on page 89

Protocol-aware testing is an integral feature of SmarTest that enables a direct communication to the DUT over a well-defined interface, the so-called protocol.

Fully qualified names on page 131

SmarTest uses the concept of fully qualified names in many locations to unambiguously identify components. The details of these names depend on the type of component.

Related information

Test data on page 21

A test is configured using a range of setup files, each of which describes a particular aspect of the overall setup.

Assisted setups and navigation

SmarTest is built on the Eclipse IDE and makes use of its navigation options and content assist features to facilitate your work.

Navigation options in SmarTest

You can efficiently navigate in SmarTest to point of interest when developing or debugging test setups. SmarTest provides the following navigation options.

- **F3** function key opens the predefined source of a certain value or expression in the assigned editor according to the current focus (active selection).

Examples:

- When, on the Test Program Explorer, a testflow is selected F3 opens it with the default editor for this testflows (flow sequence or flow data editor).
- When the focus is on a pattern in the pattern explorer F3 opens the pattern editor for this pattern.
- **Goto Definition, Open Declaration** from the shortcut menu highlights the source of a certain value or expression, for example:
 - From a pattern file reference in a testflow file displayed in the Setup editor to the Pattern editor.
 - From an import declaration in a specification file to the specification file specifying the declaration.

Note

If NOT in debug mode AND the optional `uses` statement is NOT applied, navigation using F3 to an action is inhibited when the action name is ambiguous (not unique).

The following is an example of the `uses` statement:

```
sequence DcLoopback uses
  crossconnect.dcTests.mainSpecs.MixedInstrumentDc {

    parallel {
      sequential {
        actionCall dcViIFVM1;
      }
    }
  }
...
```

Where the action name `dcViIFVM1` is not unique.

The action naming has no limitations so that operating sequences can be reused, and therefore can lead to ambiguity.

If the action being called has a name that is not unique, **F3** navigation fails if not in debug mode.

F3 navigation is always possible in debug mode because the specification being used is known (no ambiguous elements) and it is possible to navigate to the action.

- **Show in** from the shortcut menu highlights the selected element in an alternative debug tool, for example from:
 - Operating Sequence view to the Error Map view, Measurement view, and Timing Debug view.
 - Results view to the Site Result view and the Test Table view..
 - Problems view to Setup Dependency Graph view and other Eclipse views.

Double-click an entry to open and navigate to the potential source of the problem.

 - Timing Debug view to the Error Map view and the Measurement view.
 - Shmoo view to the Signal Analyzer view.
 - Tester View to the Instrument view, Measurement view, and Testflow view
 - Testflow View to Parameters, Flow Chart view, and the Results view.
 - Specification files to Setup Dependency graph view and other Eclipse views.
 - Pattern Editor to the to the Error Map view, Measurement view, and Timing Debug view.
 - Measurement view to the Instrument view, Timing Debug view, and Tester view.
 - Testflow or test method file in a Setup Editor to the Setup Dependency graph view and other Eclipse views.
 - Site Result view to the Results view.
 - Error Map view to the Measurement view and Timing Debug view.
- **Open ...** and **Open With** from the shortcut menu highlights the selected element in an alternative debug tool, for example:
 - Operating Sequence view: Open the operating sequence in the Operating Sequence editor and Pattern editor.
 - Results view: Open with the source of the error in the Statistics Editor, Testflow Editor, and Alarm Editor.
 - Timing Debug view: Open the executed pattern in the pattern editor.
 - Testflow View: Open the Implementation, execution, or the declaration.
 - Specification files: Open declaration, call and type hierarchies, open in an editor, and so on.
 - Testflow files: Open Testflow files in an editor.
 - Measurement view: Open test table or specification file in an editor.
 - Variable view: Open declared type, Open declaration, call and type hierarchies, open in an editor, and so on.
 - Error Map view: Open the associated pattern in the pattern Editor.
 - Parameters view: Open the associated pattern in the pattern Editor.
- **Linking** or **Pinning** views: Automatically updates the views based on the current selection in the opened editor and any changes therein, for example, the **Package Explorer** view is updated once you open a file in the editor and shows the location of that file in your SmarTest project structure.

Navigation to set up details or alternative views

From editors, views and wizards, you can directly navigate to the next level of a setup definition or view the test data in a alternative view.

Navigate to the next level of a setup definition

You can press **F3** to jump to the source of an entry or getting possible valid entries for a field.

View the test data in a different view

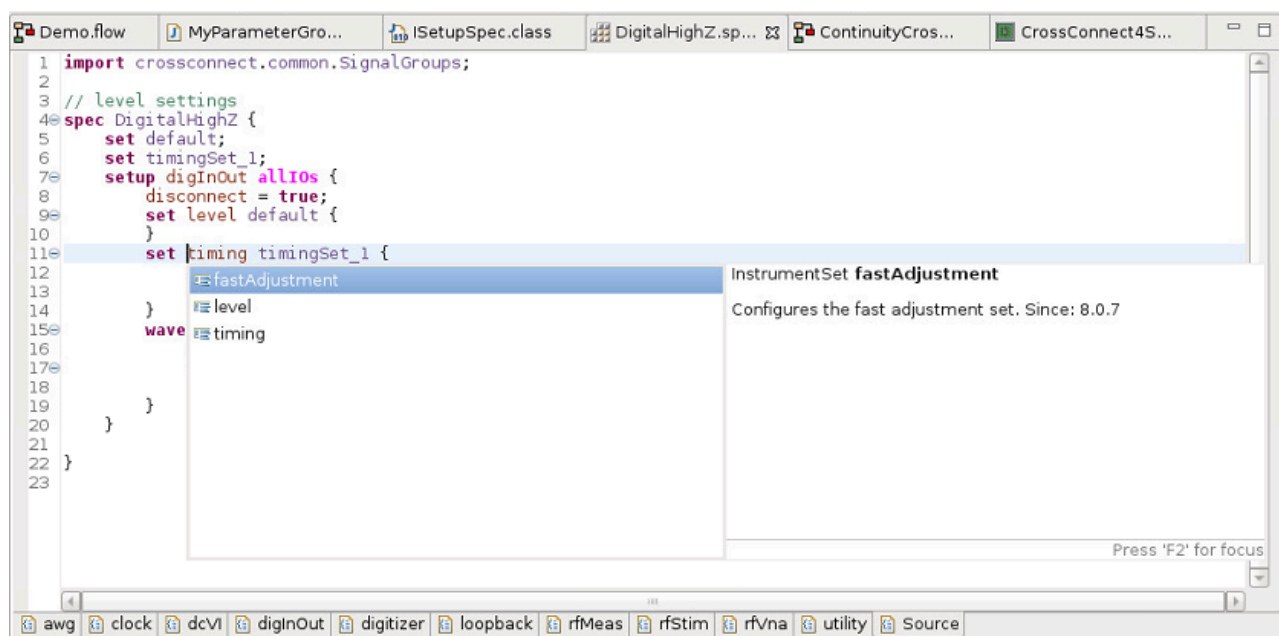
1. Right-click the test data of interest.
2. From the shortcut menu, click **Show in**.

Content assist and dynamic help for editing setup files

When editing your setup files in the SSF (SmarTest Setup Format) source editors, the Content Assist functionality provides the following features:

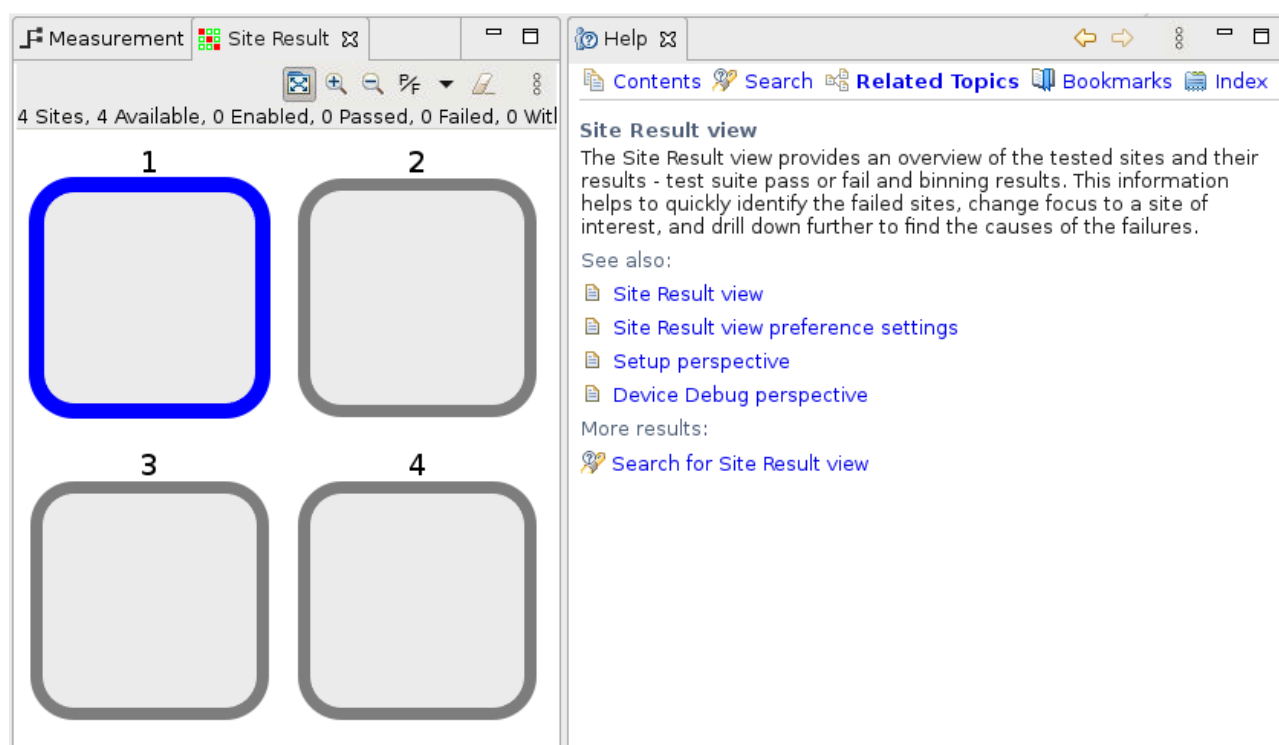
Context sensitive auto-completion list (content assist)

Start typing and press **Ctrl+Space** to open the Content assist while you type, for example in the **source** tab of the setup editor. SmarTest lists, for example, all the elements and properties that are supported at this line and shows the context sensitive help.



Context-sensitive help

Where ever you are: **Shift+F1** provides context-sensitive help:



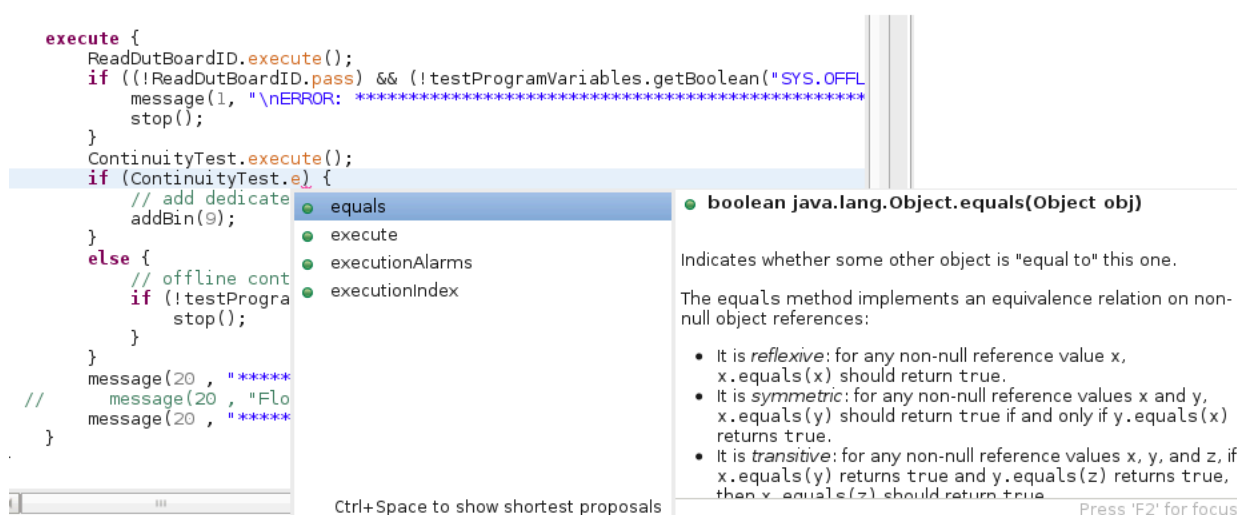
Content assist and dynamic help for editing test methods

The SmarTest Work Center provides various context-sensitive features to assist you with typing and getting help.

Context-sensitive auto-completion list (content assist)

1. In the Editor view, start typing and press **Ctrl+Space** to open Content Assist.

In the example below, the methods of the object measurement are listed and the corresponding documentation of the currently selected method in the list is shown.



Test Method Editor: Content assist

2. Use the mouse or the arrow key to select a method. The window updates accordingly providing you the documentation of the currently selected method.

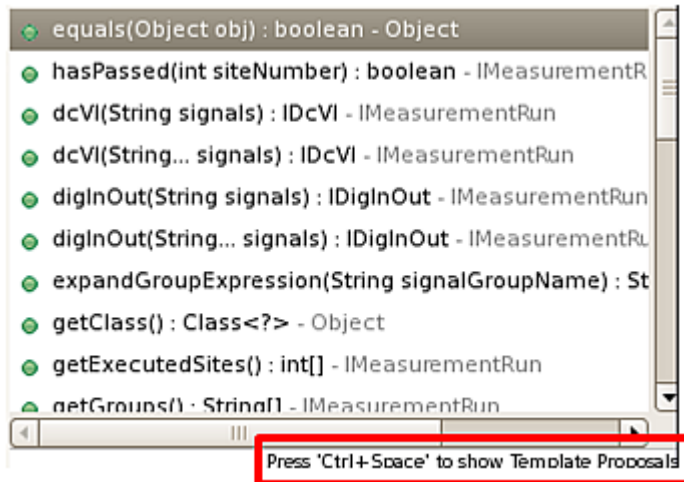
Note

When entering code in the test method editor, the context-sensitive assistance is also displayed if you type a . (dot) after a field or method.

Context-sensitive selection of code snippets

1. In the Editor view, start typing and press **Ctrl+Space** to open Content Assist.
2. Press **Ctrl+Space** again to switch to the next smart filter, called `Proposal`, for the displayed methods.

With each following **Ctrl+Space** you switch to the next smart filter.



Code snippets with content assist

Context assist proposals

Eclipse also includes default content assist proposal types for C/C++, Java and XML editors such as:

- Parsing-based Proposals
- Template Proposals
- Java Type Proposals
- Java Non-Type Proposals
- Word Proposals
- XML Tag Proposals
- XML Template Proposals

These can be enabled under **Windows > Preferences > <editor name> > Editor > Content Assist > Advanced**.

For example, enabling **Template Proposals** under **Windows > Preferences > C/C++ > Editor > Content Assist > Advanced** includes templates defined in **Preferences > C/C++ > Editor > Templates** in the content assist proposal list.

Test program execution

Executing a test program with SmarTest is the process of preparing all corresponding test data, converting them into an executable test, and executing the test.

Test programs are used in engineering and production environments:

- In engineering, test programs are usually executed manually from the SmarTest user interface. Typical goals in this environment include developing and debugging test programs as well as debugging and characterizing DUTs.
- In production, test programs are usually executed through the Test Cell API and related interfaces. The typical goal is to test and sort mass-produced devices that are intended for sale to customers.

In both cases, the same internal SmarTest modules and processes are used, with the main difference being the finer-grained access to *execution states and transitions* offered in the Test Cell API.

Execution hierarchy

The execution of a test program is a hierarchical process:

- A test program defines what testflows are used when preparing, binding, starting, stopping, and executing the main testflow. The different testflows in the test program are referred to as "*auxiliary testflows*" (see below).
- A testflow is an arrangement of test suites and subflows that are executed in the order specified in the testflow. The order can include conditional execution (for example, a test suite is executed only when a variable has a particular value) and loops. A testflow can be used as one of the auxiliary testflows in the test program file, or as a subflow in another testflow.
- Test suites call test methods, which consist of executable Java code.

Running a test program thus means executing the code in the test method instances in the order in which the test suites that call them are arranged in testflows.

Execution order

The different auxiliary testflows in the test program are executed automatically at certain points during a test program execution. Their purpose is to set up the test environment before a test begins or perform clean-up tasks after it finishes.

Auxiliary testflows (that is, all testflows listed below except for the main testflow) are optional. If an auxiliary testflow is not present, execution continues with the next step. Furthermore, test methods define certain methods that are also automatically executed by the system.

The full execution sequence:

1. `initialize()` methods

The `initialize()` methods of all test methods specified in the test program are executed first. They can be used to specify default values, which may later be overridden, for example, from the test table.

Note

Although in the current version of SmarTest, the `initialize()` methods of the different test methods are executed in a deterministic order when executing the whole test program, this is not the case when debugging the test program and when using **Load and Restore** on a single testflow or test suite. Furthermore, the order may change in future versions of SmarTest. Therefore, the methods should not rely on being called in any particular order.

2. PreBind testflow.

The `PreBind` testflow is executed once during the *"load test program" transition*. It is typically used for reading the test table from disk, for example, to configure limits or bins. In production, this typically happens once before testing a number of lots with the same test program.

3. `setup()` methods

The `setup()` methods of all test methods specified in the test program are executed first. They can be used to programmatically calculate setup data, overriding the settings from files and test table when necessary.

Note

Although in the current version of SmarTest, the `setup()` methods of the different test methods are executed in a deterministic order when executing the whole test program, this is not the case when debugging the test program and when using **Load and Restore** on a single testflow or test suite. Furthermore, the order may change in future versions of SmarTest. Therefore, the methods should not rely on being called in any particular order.

4. `update()` methods

The `update()` methods of all test methods specified in the test program are executed first. They can be used to modify the setup data in memory, without affecting any of the file-based setup sources (manually created setup files), programmatically created setup files, test table.

Note

Although in the current version of SmarTest, the `update()` methods of the different test methods are executed in a deterministic order when executing the whole test program, this is not the case when debugging the test program and when using **Load and Restore** on a single testflow or test suite. Furthermore, the order may change in future versions of SmarTest. Therefore, the methods should not rely on being called in any particular order.

5. PreStart testflow

This testflow is executed during the transition from the "test program bound" state to "testflow ready". It is typically used to turn on the power supply to the DUT board.

6. PreRun testflow

The `PreRun` testflow is executed once for each DUT during the transition from the "test program bound" state to "testflow ready" of the main testflow. This testflow can be used to modify test program variables, test limits, and other settings.

7. PowerUp testflow

The `PowerUp` testflow is executed for each DUT during the transition from the "test program bound" state to "testflow ready". It is typically used to perform tests like continuity and to power up the device by connecting the required instruments to the pogo pins. If this testflow is not specified, SmarTest powers up the device according to the instrument setup when the main testflow is executed. (See "Connecting and disconnecting instruments (Topic 254151)" for more information.)

8. Main testflow

The `Main` testflow is executed for each DUT in the "testflow running" state. If it contains any subflows, they are also executed for each DUT.

In the `Main` testflow, all `execute()` methods of the specified test methods are executed.

9. PowerDown testflow

The `PowerDown` testflow is executed for each DUT during the "stop test program" transition. It is typically used to power down the device by disconnecting the instruments, but can also be used for complex binning algorithms or for clean-up procedures. If this testflow is not defined, SmarTest powers down the device automatically when the measurement

is finished. (See "Connecting and disconnecting instruments (Topic 254151)" for more information.)

10.PostRun testflow

The `PostRun` testflow is executed once for each DUT during the "stop test program" transition. It is executed on all sites on which the `PreRun` testflow was executed. It can be used, for example, for complex binning algorithms or for clean-up procedures.

11.PreStop testflow

The `PreStop` testflow is executed during the transition from the "testflow ready" state to "test program bound". It is typically used to cut the power supply to the DUT board.

For a detailed overview of what happens during the execution of a test program, see *Detailed test program execution sequence* on page 144.

When an exception occurs or if `stop()` is called on some sites, the execution sequence is changed and some testflows might be executed out of order. See *Exception handling and stopping a site* on page 145 for detailed information.

Memory consumption

How much tester memory a test program requires can depend on the way it is bound and executed.

During the binding process, SmarTest tries to use the tester memory as efficiently as possible. The best results are achieved when all test program components are bound at the same time, which happens when the entire test program is bound after having been activated.

On the other hand, if only some parts are bound first - for example, because an individual testflow is executed before the entire test program is bound - the tester memory may not be utilized as efficiently.

That means that a test program may experience out-of-memory errors during development - where it is common to execute subflows in isolation - that would not occur in production settings, where test programs are typically started, bound, and executed as a whole.

To test for this, you can activate the test program again and immediately bind it. If the memory overflow disappears, it was an artifact of the development process.

More information about SmarTest memory issues is available in "SmarTest memory usage" (topic 354489).

Detailed test program execution sequence

The basic sequence of execution for test programs is similar for engineering and production environments. It involves passing through the different execution states, with optional auxiliary testflows being executed automatically at several stages if they are present.

For descriptions of the states and transitions mentioned below, see *SmarTest execution states and state transitions*.

1. Activating the test program:

- a. SmarTest reads the test program file.
- b. The source path information is set. This enables access to the setup files, for example, measurement specification files and user test methods.
- c. User variables defined in the test program file are initialized.

2. Loading the test program:

- a. The `main` testflow, all subflows called from the main testflow, and all *auxiliary testflows* are instantiated. This includes executing the testflow's `setup` part.
- b. The `initialize()` methods of all test methods called by the test suites defined in the testflows are executed.
- c. The `PreBind` testflow is executed.
- d. All test methods called by the test suites defined in the testflows are instantiated. This includes executing the `setup()` method of all test methods.
- e. All measurements and data structures for test results contained in the test methods are instantiated.

During instantiation, SmarTest reads the data structure into memory and replaces all references to set up data, variables, and so on with the actual data in memory.

On subsequent executions, some of this data may be cached. See "Changes to test program execution in SmarTest 8.2.0 (Topic 346189)" for details.

3. Binding setup data:

- a. All setup equations are resolved.
- b. The setup data is downloaded into the tester hardware.
- c. The `update()` method of all test methods is executed.

4. Running the test program:

- a. The `PreStart` testflow is executed.
- b. A test program start event is written to the data log.
- c. All test program variables are written to the data log.
- d. The `PreRun` testflow is executed.

This is the last option to modify test program variables. If the `PreRun` testflow modifies setup data that has already been bound, the modified data (but not other data) is bound again.

- e. The `PowerUp` testflow is executed.
- f. The `main` testflow is executed.

This executes all testflows and test methods that are directly or indirectly part of the main testflow in the order specified by the main testflow.

See *Running a testflow on page 147*, below, for information of what happens during the execution of each individual testflow.

Note

On repeated executions of a test program, any manually or programmatically changed setups are kept in memory as long as the setup files on disk have not changed.

Repeat executions may, therefore, be performed with different setup data than prior executions. If execution with the same setup data is desired, any modified data must be restored from the context in the `update()` method of the test method (to start a lot with unaltered setups) or the `execute()` method (to start each individual test with fresh setups).

5. Stopping the test program

- a. A test program end event is written to the data log.
- b. The `PowerDown` testflow is executed.
- c. The `PostRun` testflow is executed.
- d. The `PreStop` testflow is executed.

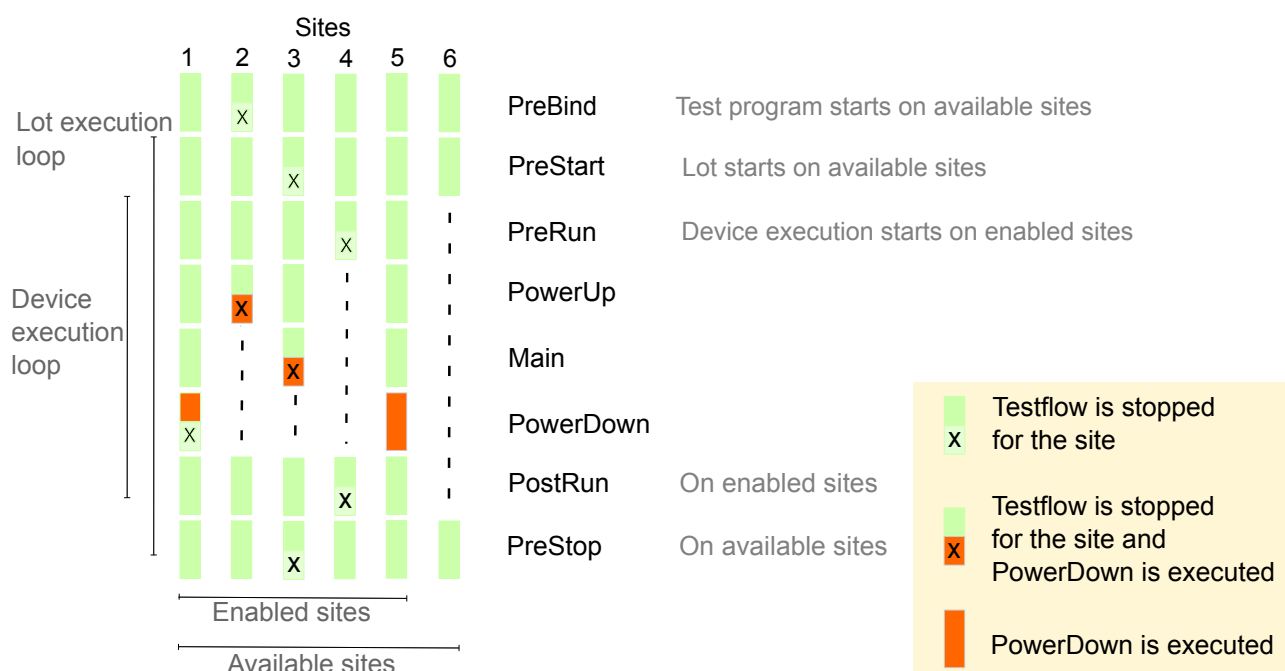
Exception handling and stopping a site

When an exception occurs in a testflow, execution of the testflow is automatically stopped for all active and inactive sites (that is, all enabled sites). Already executing test suites are finished, but no new test suites are started (except those contained in the `PowerDown` and `PostRun` testflows). An exception may therefore change the execution sequence.

Calling `stop()` stops the execution of the testflow for all specified sites if called in a test method or for all active sites if called in a testflow, but may not do so immediately. The stop happens at the next possible time, which is often the end of a test suite execution.

Stopping a testflow for a site does not stop the test program; in some cases, execution continues with the next testflow; in other cases, testflows are executed out of order.

The following diagram depicts an execution of testflows of a setup with six available sites.



Sites in testflows

The execution sequence:

1. PreBind testflow

The `PreBind` testflow is executed with all "available sites (Topic 250860)" (here sites 1 to 6) being active. If a site is stopped (here site 2), the `PreBind` testflow does not continue on this site. The next testflow to be executed is the `PreStart` testflow.

If an exception occurs, the testflow is interrupted on all sites, and no further testflows are executed.

2. PreStart testflow

The `PreStart` testflow is executed with all "available sites (Topic 250860)" being active. If a site is stopped (here site 3), the `PreStart` testflow does not continue on this site. The next testflow to be executed is the `PreRun` testflow.

If an exception occurs, the testflow is interrupted on all sites, and no further testflows are executed.

3. PreRun testflow

The `PreRun` testflow is executed with all "enabled sites (Topic 250860)" (here sites 1 to 5) being active. If a site is stopped (here site 4), the `PreRun` testflow does not continue on this site. The next testflow to be executed is the `PostRun` testflow, skipping the `PowerUp`, `Main` and `PowerDown` testflows.

If an exception occurs, the testflow is interrupted on all sites, and no further testflows are executed.

4. PowerUp testflow

The `PowerUp` testflow is executed on sites that are still "active (Topic 250860)" after completion of the `PreRun` testflow (here sites 1, 2, 3, and 5). If a site is stopped (here site 2), the `PowerUp` testflow does not continue on this site, and the `PowerDown` testflow is executed out of order. The next testflow to be executed on this site is the `PostRun` testflow.

If an exception occurs, the testflow is interrupted on all sites, the `PowerDown` testflow is executed for all sites, and no further testflows are executed.

5. Main testflow

The `Main` testflow is executed on sites that are still "active (Topic 250860)" after completion of the `PowerUp` testflow (here sites 1, 3, and 5). If a site is stopped (here site 3), the `Main`

testflow does not continue on this site and the `PowerDown` testflow is executed out of order. The next testflow to be executed on this site is the `PostRun` testflow.

If an exception occurs, the testflow is interrupted on all sites, the `PowerDown` testflow is executed for all sites, and no further testflows are executed.

6. `PowerDown` testflow

The `PowerDown` testflow is executed with sites that are still "active (Topic 250860)" after completion of the `Main` testflow (here sites 1 and 5). If a site is stopped (here site 1), the `PowerDown` testflow does not continue on this site.

If an exception occurs, the testflow is interrupted on all sites, and no further testflows are executed.

7. `PostRun` testflow

The `PostRun` testflow is executed on all sites that were "active (Topic 250860)" for the `PreRun` testflow (the "enabled sites (Topic 250860)", here sites 1 to 5). If a site is stopped (here site 4), the `PostRun` testflow does not continue on this site. The next testflow to be executed on this site is the `PreStop` testflow.

If an exception occurs, the testflow is interrupted on all sites, and no further testflows are executed.

8. `PreStop` testflow

The `PreStop` testflow is executed on the sites that were "active (Topic 250860)" for the `PreStart` testflow (the "available sites (Topic 250860)", here sites 1 to 6). If a site is stopped (here site 3), the `PreStop` testflow does not continue on this site.

If an exception occurs, the testflow is interrupted on all sites.

Running a testflow

When SmarTest executes a testflow, it performs the steps below. These steps apply to the main testflow and all testflows it calls directly or indirectly, but not to auxiliary testflows, such as `PreBind` or `PowerDown`.

1. A testflow start event is written to the data log.
2. Each test suite in the execution flow is executed. The order in which the execution happens is determined by the arrangement of test suites in the testflow.

Executing a test suite includes:

- a. A test suite start event is written to the data log.
 - b. The measurement specified in the test suite is performed in the tester.
 - c. The results are retrieved.
 - d. The results are evaluated as specified by the test method.
 - e. The test results are written to the data log as specified in the test method.
 - f. A test suite end event is written to the data log.
3. A testflow end event is written to the data log.

Additional functions for engineering applications

To enable efficient development and debugging of test programs, SmarTest offers additional functions for partial or experimental execution:

- Executing arbitrary testflows of the active test program, including auxiliary testflows, as the main testflow.
- Executing a testflow as a standalone testflow. Auxiliary testflows are not executed in addition to the specified testflow, but any testflow explicitly called from the specified testflow are executed.
- Setting breakpoints in a testflow or test method for debugging purposes.

If you set a breakpoint during debugging, the execution sequence changes:

1. The execution is stopped at the breakpoint.

The debugging views (for example, Measurement view and Timing Debug view) display the bound data and results of the most recent measurement.

2. When the last measurement is re-executed, this sequence is used:

- a.** If either bound or unbound setup data was modified, the setup data is rebound and downloaded into the hardware.
- b.** The measurement is performed with the latest setup data.
- c.** The results are retrieved.
- d.** The debugging views are refreshed to display the bound data and results of the last measurement.
- e.** The execution remains stopped at the breakpoint.

3. After resuming the execution at the current breakpoint, the testflow or test method will be executed as described in *Running a testflow on page 147*.

Related concepts

Test program execution on page 141

Executing a test program with SmarTest is the process of preparing all corresponding test data, converting them into an executable test, and executing the test.

SmarTest execution states and states transitions on page 149

During the preparation and execution of a test program, SmarTest has several distinct execution states.

Testflow on page 94

A testflow is a structure for performing the individual measurements in a device test. It consists of a number of *test suites*, bins, and control logic such as loops and branches.

SmarTest execution states and states transitions

During the preparation and execution of a test program, SmarTest has several distinct execution states.

Each of these states determines which parts and settings of the test program - such as testflow or test program variables, test limits, setups, or instrument specifications - can be modified. SmarTest also takes specific actions based on the current execution state, such as checking out licenses or starting data logging.

SmarTest enters a different state in response to certain actions you perform in the SmarTest user interface or commands received through the Test Cell API. In some cases, it also changes states automatically based on the execution progress.

The sequence of states required to execute a test program, as well as the available transitions between the states, are always the same. However, when the SmarTest user interface is used, some transitions are automatically combined so that the intermediate states are passed through automatically. For example, running a test program automatically also loads and binds it - these transitions are not individually selectable. By contrast, the Test Cell API offers functions to execute any transition individually.

The different states and the available transitions between them are described below.

Execution states

The various states:

- *Idle*: No test program has yet been selected.
- *Test program activated*: The test program to work with has been chosen and parsed on disk.
- *Test program loaded*: The test program has been loaded into memory and the execution order has been determined.
- *Test program bound*: The test data have been calculated and loaded into the tester hardware.
- *Testflow ready*: The execution of the test program has begun, but a testflow is not currently being executed.
- *Testflow running*: A testflow is being executed.

State transitions

Transitions between states can happen for three reasons:

- **User interaction**: You perform an action in the user interface (such as choosing a menu command) that triggers a state transition.

This kind of transition trigger is typical in engineering applications.

- **Test Cell API command**: When SmarTest is controlled through the "Test Cell API (Topic 247407)", certain commands trigger state transitions.

This kind of transition trigger is typical in production applications.

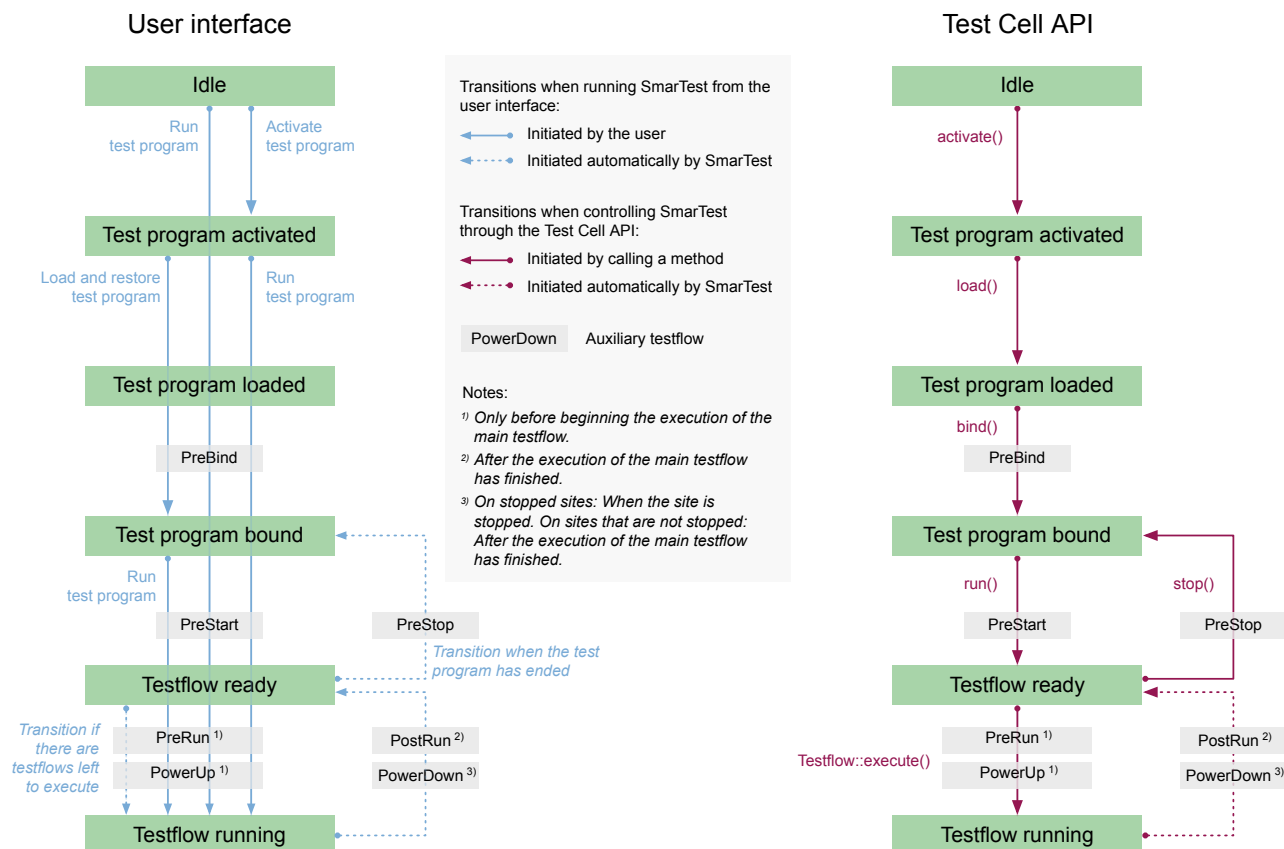
- **Automatic transition**: In certain cases, SmarTest automatically transitions to a different state, without requiring an external trigger. For example, when the execution of a test program is complete (and no looping is specified), it automatically transitions to the "Test program bound" state.

Automatic transitions can happen both in engineering and production applications. However, some of these transitions are automatic only in engineering applications and must be specifically triggered through the Test Cell API in production applications.

The diagrams below provide a graphic overview of the states and transitions in engineering and production settings. It also includes the auxiliary testflows that are automatically executed during these transitions. For more information, see *Auxiliary testflows* on page 96.

Note that a failed attempt to perform a transition usually returns SmarTest to the previous state. For example, when trying to load a test program fails, SmarTest returns to the "Test program activated" state.

Overview of states and transitions



1. Idle state
2. Test program activated state
3. Test program loaded state
4. Test program bound state
5. Testflow ready state
6. Testflow running state
7. Auxiliary testflows
8. Auxiliary testflows
9. Auxiliary testflows
10. Auxiliary testflows
11. Auxiliary testflows
12. Auxiliary testflows
13. Auxiliary testflows
14. Auxiliary testflows
15. Auxiliary testflows
16. Auxiliary testflows
17. Auxiliary testflows
18. Auxiliary testflows
19. Idle state
20. Test program activated state
21. Test program loaded state
22. Test program bound state

23. Testflow ready state**24. Testflow running state**

When a test program is bound, any changes to the setup files on disk (either by editing the setup files or through the Device Setup API) will cause the program to be bound again before its next execution. This creates a new setup.

If a setup is modified in memory in test method code, these changes persist for subsequent testflow executions unless the test program is manually bound again.

Related concepts

Test program execution on page 141

Executing a test program with SmarTest is the process of preparing all corresponding test data, converting them into an executable test, and executing the test.

Idle execution state

SmarTest is in the idle state when you have started it but not yet activated a test program. You can edit test program files in this state, but any changes you make take effect only once you activate the test program to which the files belong.

Available transitions:

User interface**Activate test program**

Leads to "*test program activated*".

SmarTest loads the selected test program, the DUT board file and the license requirement file. It collects the test data specified in the test program file.

Note that activating a test program is also possible in other execution states and always leads to the "*test program activated*" state.

Load and restore test program

Leads to "*test program bound*".

This is a convenience function that automatically performs all intermediate transitions and passes through the *test program activated* and *test program loaded* states.

The result is the same as invoking the intermediate transitions separately.

Run test program

Leads to "*testflow running*".

This is a convenience function that automatically performs all intermediate transitions and passes through the *test program activated*, *test program loaded*, *test program bound*, and *testflow ready* states.

The result is the same as invoking the intermediate transitions separately.

Test Cell API**activate()**

This is the same as the user interface transition "activate test program" described above.

See also the description of controlling the execution of a test program from the Test Cell API in "Test program state transitions (Topic 250546)".

Test program activated execution state

The active test program is the one you have selected to work with. Only one test program can be active at any one time.

SmarTest has collected the test data belonging to the active test program and read the license requirement files. It has read the setup files into memory. All test program variables, licenses,

and DUT board parameters can be read and modified in the SmarTest editors. No data logging events are generated. No licenses are checked out.

Available transitions:

User interface

Run test program

Leads to "*testflow running*".

SmarTest first loads, binds, and runs the test program automatically in the background. This function automatically performs all intermediate transitions and passes through the *test program loaded*, *test program bound*, and *testflow ready* states.

The result is the same as invoking the intermediate transitions separately.

Note that running individual testflows separately before running the entire test program can increase tester memory consumption, as described in *Memory consumption* on page 143.

Load and restore test program

Leads to "*test program bound*".

SmarTest first loads and binds the test program automatically in the background. This function automatically performs all intermediate transitions and passes through the *test program loaded* state.

Test Cell API

load()

Leads to "*test program loaded*".

SmarTest reads all testflow files into memory and determines the order in which the testflows call each other.

See also the description of controlling the execution of a test program from the Test Cell API in "Test program state transitions (Topic 250546)".

Automatic

When the "test program activated" state has been entered from the "*idle*" *execution state* using the "run test program" function, SmarTest automatically continues with the "load test program" transition to the *test program loaded* state.

Related concepts

SmarTest execution states and states transitions on page 149

During the preparation and execution of a test program, SmarTest has several distinct execution states.

Test program loaded execution state on page 153

In this state, all testflows that are specified in the test program file have been read from disk, representations of them created in memory, and the order in which they call each other established.

Test program bound execution state on page 153

In this state, the setup data of all testflows and test methods have been read from disk, their dependencies have been resolved, and they have been downloaded to the tester. All required licenses are checked out. The test program is ready to be executed.

Test program loaded execution state

In this state, all testflows that are specified in the test program file have been read from disk, representations of them created in memory, and the order in which they call each other established.

All test program variables and testflow variables can be read and modified. Test suite parameters, test descriptors and measurements can be read and modified. The test table can be loaded (which typically happens in an auxiliary testflow).

No data logging events have generated. No setup data has been loaded.

A test program can enter and stay in this state only when it is controlled from the Test Cell API. When a test program is run from the SmarTest user interface, SmarTest automatically passes through this state in the "run test program" and "load and restore test program" transitions.

Available transitions:

Test Cell API

bind()

Leads to *"test program bound"*.

SmarTest resolves all test data dependencies and downloads data to the tester. It checks out all required licenses.

See also the description of controlling the execution of a test program from the Test Cell API in "Test program state transitions (Topic 250546)".

Automatic

When the "test program loaded" state has been entered through the user interface from the *"test program activated" execution state* using the "run test program" or "load and restore" functions, SmarTest automatically continues with the "bind test program" transition to the *test program bound* state.

Test program bound execution state

In this state, the setup data of all testflows and test methods have been read from disk, their dependencies have been resolved, and they have been downloaded to the tester. All required licenses are checked out. The test program is ready to be executed.

No data logging events are generated.

When a test program is run from SmarTest, this state is automatically passed through when "run test program" is initiated from the "idle" or "test program activated" states. After a test program finishes running, SmarTest ends up in this state.

Available transitions:

User interface

Run test program

Leads to *"testflow running"*.

SmarTest activates the data logging and starts executing the test program.

Test Cell API

run()

Leads to *"testflow ready"*.

SmarTest activates the data logging and puts the test program into the "running" state. However, it does not yet begin the actual execution, which requires further method calls.

See also the description of controlling the execution of a test program from the Test Cell API in "Test program state transitions (Topic 250546)".

Automatic

When the "test program bound" state has been entered from the *"idle" execution state* or the *"test program activated" execution state* using the "run test program" function, SmarTest automatically continues with the transition to the *testflow ready* state.

Testflow ready execution state

In this state, data logging has started. Testflow parameters can still be read but no longer modified; test program variables and test suites can still be modified.

Available transitions:

Test Cell API

Testflow::execute()

Leads to *"testflow running"*.

SmarTest begins the execution of the specified testflow on the tester.

See also the description of controlling the execution of a test program from the Test Cell API in "Test program state transitions (Topic 250546)".

stop()

Leads to *"test program bound"*.

SmarTest terminates the execution of the currently running test program and stops data logging.

See also the description of controlling the execution of a test program from the Test Cell API in "Test program state transitions (Topic 250546)".

Automatic

As long as the execution of the main testflow has not yet finished, SmarTest automatically executes the next testflow in the control flow of the test program, which leads to the *testflow running* state.

If the execution of the main testflow has ended, SmarTest automatically transitions to the *test program bound* state. It terminates the execution of the currently running test program and stops data logging.

Testflow running execution state

In this state, a testflow is running. The test methods specified in its test suites are executed. Typically, this prevents other processes from accessing the required tester hardware used by the test suites.

Data logging events are generated if the testflow is running as part of a test program, but not if it is being executed in isolation.

The test program properties cannot be modified. A test method can only modify its own properties.

Available transitions:

Automatic

When a testflow completes executing, SmarTest automatically transitions to the *testflow ready* state. (It does not matter for this whether SmarTest is controlled from the user interface or through the Test Cell API.)

Multisite testing in SmarTest

SmarTest offers built-in multisite testing without special requirements for test programs and test methods. The sites that are actually tested by a given measurement can be flexibly configured.

Setting up multisite testing, built-in multisite

Setting up multisite testing requires no modification to the testing algorithms. Two principal steps are involved:

- Using multisite data types wherever values can apply to sites.

Multisite data types enable transparent handling of site-specific values. They require no dimensioning - as a normal array would - and so can be used without code modifications for any number of sites being tested.

- Specifying the arrangements of DUT pins and other elements into sites in the *DUT board description* on page 22.

A test program can be used with different DUT boards as long as the pins are arranged into sites in a similar manner. For example, a test program written for an engineering DUT board with a single site could be used without modification on a production DUT board with a large number of sites.

Note

SmarTest always uses the required number of instruments for your multisite testing from the same test head cards. This means, it cannot mix the required instruments, for example: dcVI instruments, from different test head cards such as: PS1600 and DPS128. It can only use the instruments from one or more PS1600 or one or more DPS128 cards.

Selecting sites to test

Of all theoretically available sites, often only a subset is tested. Only these so-called active sites produce test results and are included in data logs and are available for debugging.

This subset of active sites is determined by a mix of automatic and manual restrictions:

- All sites that the DUT board supports - the maximum set of sites that any test can access - are described in the *DUT board description* on page 22. These sites are known as the *configured sites*.
- A *test program* can be set up to run on all configured sites or to not run on some of them. The sites on which the test program will not run are set up in the "test program file (Topic 244389)" and are called *ignored sites*. In addition, the test program may not be able to test some of the configured sites because not sufficient hardware resources are available for them. In this case, SmarTest will issue an error upon binding.

Note

No electrical signals will be applied to the ignored sites unless they share resources with another site which is not ignored.

The remaining configured sites are called *available sites* and are the sites on which the test program runs by default.

- When the test program is run from the SmarTest GUI, you can set it to run on all available sites or just a subset of them. The sites on which the test program is run are called *enabled sites*. You can choose to disable some of the sites. These *disabled sites* are excluded from the current execution of the test program but could be included in a future run. You can disable sites from your test program or test methods.

Note

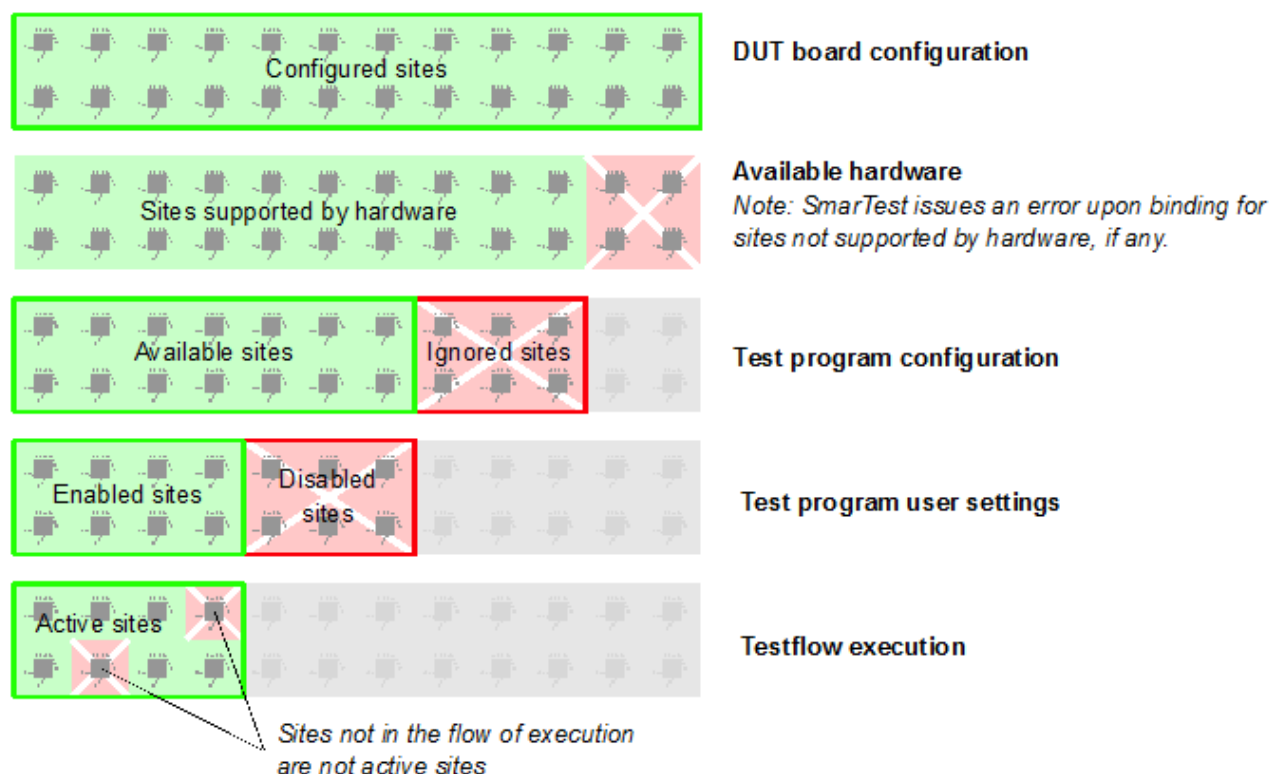
No electrical signals will be applied to the disabled sites unless they share resources with another site which is enabled.

- When the execution of the test program and its main *testflow* begins, all measurements are initially automatically applied to all enabled sites - all enabled sites are active sites that are being tested. However, as the execution progresses, some sites may no longer be included in it, for example because of conditional execution or branching in the testflow or the test method code. As long as a site is excluded from testing by the test logic, it is no longer an active site.

Note

- No electrical signals will be applied to the inactive sites unless they share resources with another site which is active.
- A site can be currently inactive but become active again later in the execution. For example, a testflow could execute test A on all sites, test B only on sites that have failed test A, and test C on all sites again. In this case, any site that passes test A would be inactive during the execution of test B but become active again for the execution of test C.

Tests are executed only on active sites. Inactive sites remain in the state they had when they became inactive.



As described above, all active sites are always tested automatically, their results are included in the data log, and they can be accessed with the "debugging tools (Topic 244556)". No configuration or modification to the testing code is required, even if the number of active sites changes. In fact, normally a test does not even need to be aware of the range of sites on which it is being executed.

Testflows in multisite testing

Each executed testflow in a test program begins to run on all enabled sites.

When the testflow execution reaches a branch ("if ... else (Topic 253646)" statement), the branching condition is evaluated for each site separately, and each site is assigned to a branch according to its individual evaluation result. Therefore, some sites may be assigned to the `if` branch and some to the `else` branch.

If both branches have sites assigned to them, both are executed. During the execution of a branch, sites that were assigned to the other branch are inactive and sites that were assigned to the executed branch are active.

This execution of both branches can increase testing time and reduce multisite efficiency (MSE). To mitigate this potential reduction in performance, you should put as little code as possible into a branch and run as much of the testflow as possible outside of branches.

Related concepts

DUT board description on page 22

The DUT board description describes how the DUT board connects pogo pins to DUT pins (or, DUT *signals*) and which pogo pins belong to which sites.

Site execution order in multisite testing on page 157

SmarTest automatically optimizes the measurement execution in multisite testing. It detects the instrument sharing status and depending on that it executes the measurement on one site at a time (sequentially), or on all sites at the same time (in parallel), or a mixture of these two (site interlacing). However, SmarTest allows you to optimize the measurement for aspects other than test time, for example, accuracy or signal power, which can lead to forcing a specific site execution order. This topic gives an overview on the site execution algorithm that SmarTest uses in multisite testing.

Site execution order in multisite testing

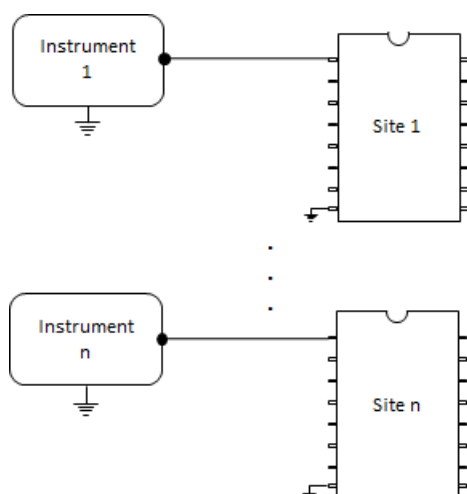
SmarTest automatically optimizes the measurement execution in multisite testing. It detects the instrument sharing status and depending on that it executes the measurement on one site at a time (sequentially), or on all sites at the same time (in parallel), or a mixture of these two (site interlacing). However, SmarTest allows you to optimize the measurement for aspects other than test time, for example, accuracy or signal power, which can lead to forcing a specific site execution order. This topic gives an overview on the site execution algorithm that SmarTest uses in multisite testing.

Instrument-sharing use cases

In multisite testing, there are three types of instrument use cases:

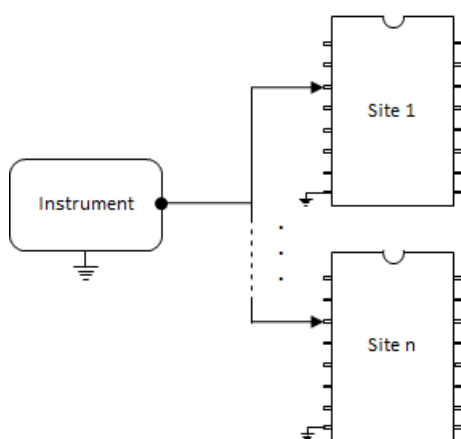
Non-shared instrument

The instrument is used by one individual site, such as digital instruments that drive a pattern.



Shared instrument

The instrument is shared by more than one site at the same time, such as an instrument that is used as a fan-out to provide the same voltage or current to all sites at the same time.



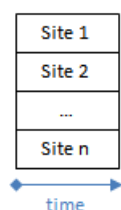
The elements in the operating sequence are executed on all the sites at the same time, that is, in parallel. This method provides the best throughput and test time compared to the other methods.

SmarTest uses site sharing by default for:

- Non-shared instruments
- Shared instruments

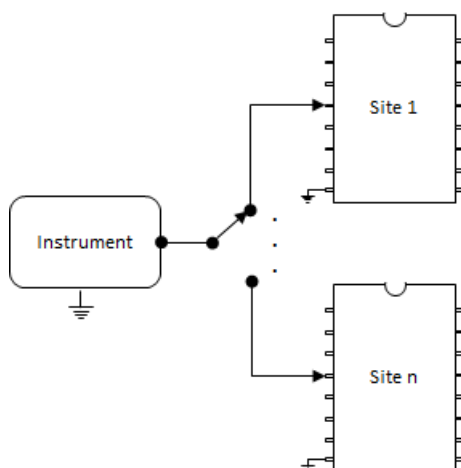
Note It is not allowed to set site-specific values in the case of shared instruments.

The actions or patterns for such instruments are applied in parallel on all sites.



Sequenced instrument

The instrument is shared by more than one site but needs to be switched from one to another, such as an RF instrument that is used to measure an RF signal.

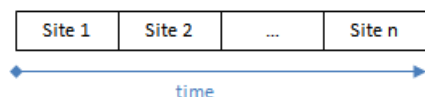


Site sequencing

The elements in the operating sequence are executed on one site or a group of shared sites until completion, after which the operating sequence is executed on the next site or the next group of shared sites.

SmarTest uses site sequencing by default for sequenced instruments.

The actions or patterns for such instruments are applied sequentially on the involved sites. Such instruments can be switched using SmarTest directly or through controlling the states of utility line signals.

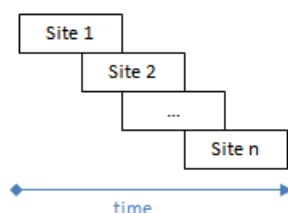


Site interlacing

Each element of the operating sequence is executed on all sites or groups of shared sites sequentially, after which the next element is executed on all sites or groups of shared sites sequentially. This provides better throughput and lower test time compared to site sequencing.

Site interlacing can be used only for instruments that support the `siteInterlacingON` option. Site interlacing cannot be used for instruments that are switched through utility line signals.

There are additional constraints and requirements for using site interlacing as described in *Site interlacing: Semiparallel execution in multisite testing* on page 160.



Controlling the site execution order in SmarTest

SmarTest allows you to disable specific sites during the measurement. Thus, these sites will be ignored. It also allows you to optimize your measurement for aspects other than test time, for example, higher signal accuracy or power, which can lead to forcing a specific site execution order. For instance:

- In specification files:
 - For `digInOut` and `dcVI` instruments, you can use the `highAccuracy` property to optimize the measurement for accuracy. This, depending on the DUT board layout, can lead to sequential site-execution.
 - For `rfStim` instrument, you can use the `multisiteSplitterOff` property to optimize the measurement for stimulus power and accuracy. This can lead, depending on the DUT board layout, to sequential site-execution.
- In test methods, you can specify which sites are disabled during the measurement. SmarTest will ignore these sites during the measurement execution.

Controlling site execution order with the `highAccuracy` property of the `digInOut` and `dcVI` instruments

The `highAccuracy` property can be specified for `digInOut` and `dcVI` instruments. It optimizes the measurement for accuracy. This, depending on the DUT board layout, can lead to sequential site execution:

```
setup digInOut|dcVI example {
    highAccuracy=true;
    ...
}
```

```
}
```

Controlling the site-execution order with the multisiteSplitterOff option of the rfStim instrument

The `multisiteSplitterOff` property can be specified for the `rfStim` instrument. It optimizes the measurement for stimulus power and accuracy. This can lead, depending on the DUT board layout, to executing the measurement using site sequencing:

```
setup rfStim example {  
    config.options=multisiteSplitterOff;  
    ...  
}
```

Controlling the site execution order with the siteInterlacingOn option of the rfStim and rfMeas instruments

The `SiteInterlacingOn` option can be specified for the `rfStim` and `rfMeas` instruments. It optimizes the measurement for test time. This option leads to executing the measurement using site interlacing (semiparallel). Note that this option must be specified for both instruments if they share the same resources. Otherwise, site-sequencing will be used (see, *Site interlacing: Semiparallel execution in multisite testing* on page 160 for details):

```
setup rfStim|rfMeas example {  
    config.options=siteInterlacingOn;  
    ...  
}
```

Site interlacing: Semiparallel execution in multisite testing

In multisite testing, the elements of your operating sequence (actions, patterns, sequential groups, and parallel groups) can be executed in several orders to optimize the measurement for higher throughput and lower test time. For instance, SmarTest can execute those elements on one site at a time (sequentially), on all sites at the same time (in parallel), or using site interlacing. This topic describes the specifics of site interlacing. For details on the site execution orders in SmarTest, refer to *Site execution order in multisite testing* on page 157.

Site interlacing

Site interlacing can be used to execute the elements of an operating sequence in a semiparallel manner. It can be used for all instruments that support the `siteInterlacingOn` option. To use site interlacing, specify the instrument option `siteInterlacingOn` in your instrument's `setup` in the specification file (*.spec):

```
config.options = siteInterlacingOn;
```

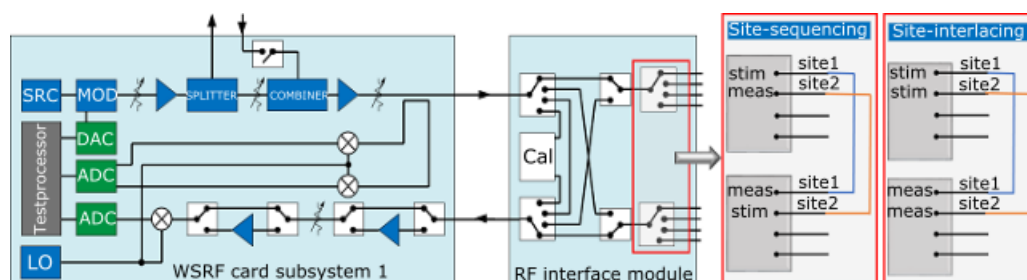
Note

This option must be specified for all instruments that share the same resources, such as the `rfStim` and `rfMeas` instruments used to stimulate and measure the DUT signals. Otherwise, site sequencing will be used when a resource conflict occurs.

There are several constraints for executing the elements of your operating sequence using site interlacing:

1. All shared-instrument patterns are not considered, only actions can be executed using site interlacing.
2. The shared-instrument actions can only be used in top-level sequential groups.

3. The actions must be directly called in the operating sequence. That is, actions that are anchored to a called pattern are not supported. For the latter case, SmarTest will use site sequencing.
4. All actions must be using either non-shared or equally shared instruments (that is, shared instruments that are shared between the same sequence of sites).
5. You cannot program any utility line signals that will be used during the execution to switch between the sites for site interlacing.
6. Site interlacing is not supported if switching between the same sequence of sites require switching of instruments. This will result in resource conflict and SmarTest will then use site sequencing. Refer to the figure below.

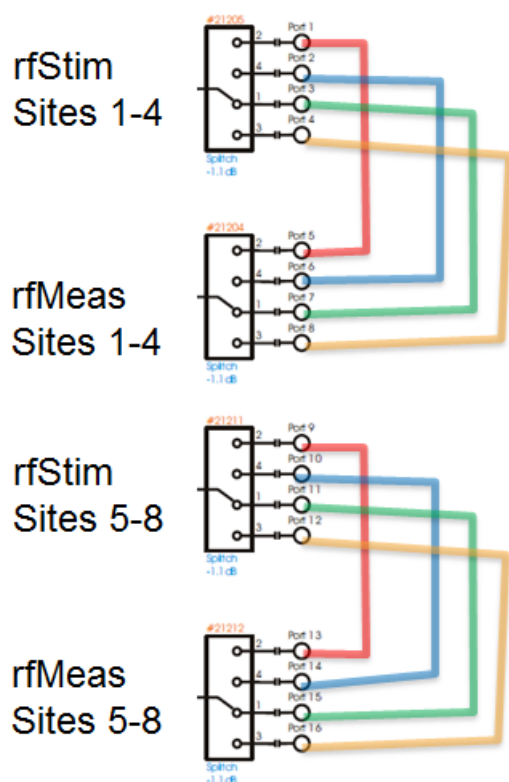


Site sequencing and site interlacing example

Measurement time with site sequencing and site interlacing

This example shows how to set up site interlacing for RF instruments in SmarTest. It also provides a comparison between the measurement times using site-sequencing and using site interlacing.

A simple loopback between `rfStim` and `rfMeas` instruments is used in eight sites configuration (see the following figure). The measurement can be executed on sets of two sites at a time: sites 1 and 5, sites 2 and 6, sites 3 and 7, and sites 4 and 8:



The following code snippet shows the specification file (*.spec) in which, the `rfStim` and `rfMeas` instruments are setup. To use the site-interlacing, you need to specify the instrument option `siteInterlacingOn`.

Note

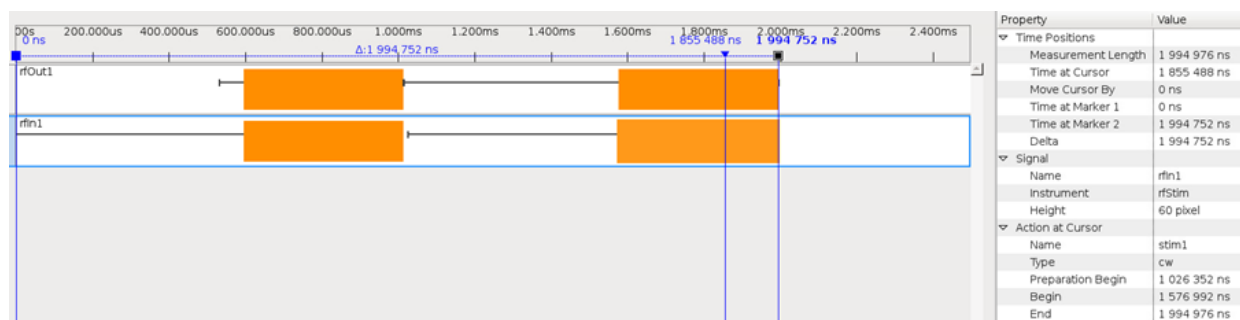
This option must be specified for all instruments that share the same test head resources. In this example, it needs to be specified for both the `rfStim` and `rfMeas` instruments. Otherwise site sequencing will be used when there is a resource conflict.

In this example, there are four actions defined: the stimulus actions `stim0` and `stim1` and the measure actions `modMeas0` and `modMeas1`:

The following code snippet shows the operating sequence (*.seq). First the actions `stim0` and `modMeas0` are executed in parallel. After that, the actions `stim1`, `modMeas1` are executed in parallel.

The following snapshots show the measurement in the "Timing Debug (Topic 246069)" view. The first snapshot shows the measurement performed using site sequencing. The second snapshot shows the same measurement performed using site interlacing. Note that the site count in both cases is eight:

- 1. Using site sequencing:** In this case, the measurement is executed on pairs of two sites at a time (1 and 5, 2 and 6, 3 and 7, and 4 and 8). Thus for the eight sites, four measurements are performed. The instruments must be reprogrammed after execution of each pair of sites which results in extra overhead to the overall measurement time. From the snapshot below, you can see that each measurement lasts for 2ms. Hence, the total measurement time is $2\text{ ms} * 4 = 8\text{ ms}$.



- 2. Using site interlacing:** in this case, unlike site sequencing, only one measurement is required. The instruments are programmed only once at the beginning of the measurement. The eight sites are sequenced during the measurement in the following order: sites 1 and 5, 2 and 6, 3 and 7, and 4 and 8. Only the switches and attenuators are reprogrammed as the execution is switched between those sites. In the snapshot below, you can see that the total measurement time is 5.4 ms. Thus in this example, the execution

time using site interleaving (5.4 ms) is shorter by around 45% compared to that for site sequencing (8 ms).



Flexible instrument sharing

Instruments can be shared between sites, between signals and also between a combination of both. Sharing of instruments allows better use of scarce and otherwise underused tester resources, but may require measurements using shared instruments to be serialized across sites, signals, or both.

Types of instrument sharing

Sharing instruments between sites

In the *DUT board description* on page 22, shared instruments are assigned to more than one site.

In order to apply site#specific values or to measure parameters of the DUT, relays are usually required on the DUT board and the routing information must be defined in such a way that SmarTest can automatically sequence the sites.

See also the *Shared instrument* on page 157 and *Sequenced instrument* on page 158 sections in *Site execution order in multisite testing* on page 157.

```
signal Signal_1_Resource_1_shared_by_sites {
    site 1 { @Shared pogo = 11701; }
    site 2 { @Shared pogo = 11701; }
    routing {
        ...
    }
}

signal Signal_2_Resource_2_shared_by_sites {
    site 1 { @Shared pogo = 11702; }
    site 2 { @Shared pogo = 11702; }
    routing {
        ...
    }
}
```

In the following example, the utility line "301" is shared between sites 1 and 2 for relay k01 and the utility line "302" is shared between sites 1 and 2 for relay K02 .

```
// Utility instrument
// K01
signal Utility_1_Resource_1_shared_by_sites {
    site 1 { settlingTime = 1 ms;    @Shared    pogo = 301; }
    site 2 { settlingTime = 1 ms;    @Shared    pogo = 301; }
}

// K02
signal Utility_2_Resource_2_shared_by_sites {
    site 1 { settlingTime = 1 ms;    @Shared    pogo = 302; }
    site 2 { settlingTime = 1 ms;    @Shared    pogo = 302; }
}
```

Sharing instruments between signals

Sharing instruments between signals means just that a new signal is defined with already used instruments while the existing assignment from instruments to sites is maintained.

Note that different signals using the same instruments cannot be used for the same measurement.

```
signal Signal_1_Resource_1_and_2_1st_measurement {
    site 1 { pogo = 11701; }
    site 2 { pogo = 11702; }
    routing {
        ...
    }
}

signal Signal_2_Resource_1_and_2_2nd_measurement {
    site 1 { pogo = 11701; }
    site 2 { pogo = 11702; }
    routing {
        ...
    }
}
```

Flexible instrument sharing

The available instruments can also be reused for other signals without maintaining the assignment to sites.

This type of sharing instruments requires relays on the DUT board and the sites using the same instrument must be tested sequentially.

For sequencing of the sites with flexible instrument sharing, you need to define different signal names for different measurement specifications that are used for different measurements. Site sequencing must be done manually and the appropriate measurement must be selected depending on the active sites. Note that the comment "Never used" in the code below means that the definition is required, but is never used: For the site specific#signal definition, the marked site must be disabled by manually set up site sequencing.

```
//
// Signals for measurement on site 1
//
signal Resource_1_flexible_sharing_direct_connection_site_1 {
    site 1 { @Shared pogo = 11701; }
    site 2 { @Shared pogo = 11701; } // Never used
    routing {
        ...
    }
}

signal Resource_2_flexible_sharing_swapped_connection_to_site_1 {
    site 1 { @Shared pogo = 11702; }
```

```

    site 2 { @Shared pogo = 11702; } // Never used
    routing {
        ...
    }
}

//
// Signals for measurement on site 2
//
signal Resource_1_flexible_sharing_direct_connection_site_2 {
    site 1 { @Shared pogo = 11702; } // Never used
    site 2 { @Shared pogo = 11702; }
    routing {
        ...
    }
}

signal Resource_2_flexible_sharing_swapped_connection_to_site_2 {
    site 1 { @Shared pogo = 11701; } // Never used
    site 2 { @Shared pogo = 11701; }
    routing {
        ...
    }
}

```

Flexible instrument sharing

The example shows how instruments can be flexibly shared between sites. In addition to the default configuration, in which measurements can be executed in parallel on both sites, two other configurations are available that temporarily borrow from the other site instruments for measurements with higher power requirements.

Please note that the diagrams are simplified. For borrowed instruments, the low sense pogo pins and (for floating instruments) the low force pogo pins may also have to be switched between the sites.

Parallel test

Two DPS and one digital pin are connected to each site. The tests performed for this configuration are executed in parallel on the sites.

Relays are all in the default state, meaning that relay 1 through relay 3, which connect the two sites, are open while the connections to the DUT, provided by relay 4 and relay 5 are closed.

```

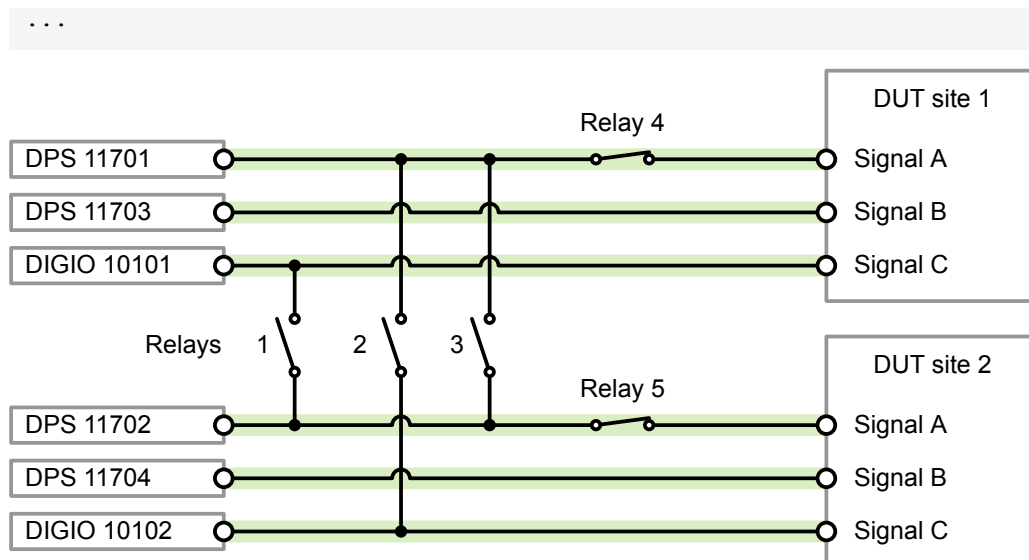
utilityLine relay_1 { pogo = 101; }
utilityLine relay_2 { pogo = 102; }
utilityLine relay_3 { pogo = 103; }
utilityLine relay_4 { pogo = 104; }
utilityLine relay_5 { pogo = 105; }

signal Signal_A_parallel {
    site 1 { pogo = 11701; }
    site 2 { pogo = 11702; }
    routing {
        site 1 { relay_1 = 0; relay_2 = 0; relay_3 = 0; relay_4 = 1; relay_5 = X; }
        site 2 { relay_1 = 0; relay_2 = 0; relay_3 = 0; relay_4 = X; relay_5 = 1; }
    }
}

signal Signal_B_parallel {
    site 1 { pogo = 11703; }
    site 2 { pogo = 11704; }
}

signal Signal_C_parallel {
    site 1 { pogo = 10101; }
    site 2 { pogo = 10102; }
}
...

```



DPS of the site and borrowed DPS of the other site are ganged

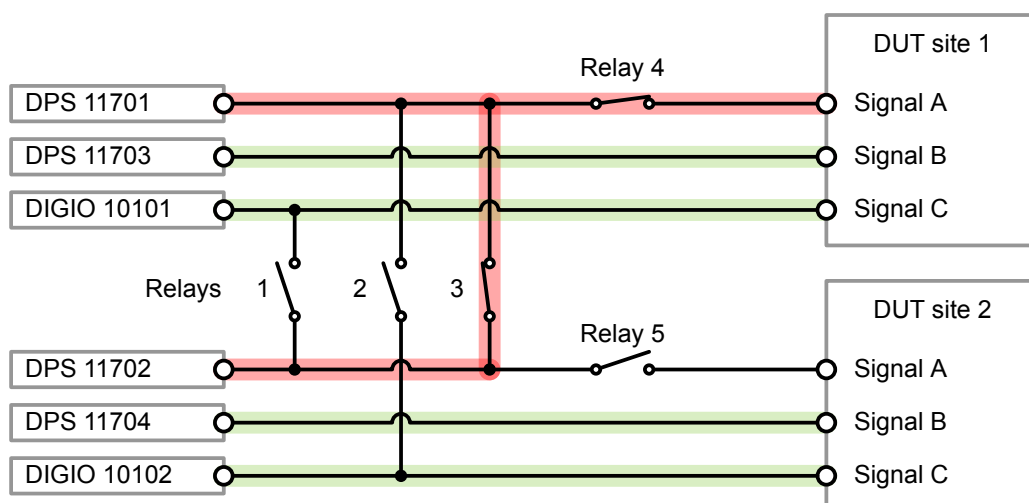
For this measurement, which also is sequenced over the sites, the DPS at the "Signal A" DUT pin and the borrowed DPS from the other site temporarily provide a ganging group at the "Signal A" DUT pin.

The ganging connection is made using relay 3. The borrowed DPS must be separated from its original site using relay 4 or relay 5. Note that compared to the next example, both channel 11701 and channel 11702 are used for both sites. Thus, a simpler set up can be used.

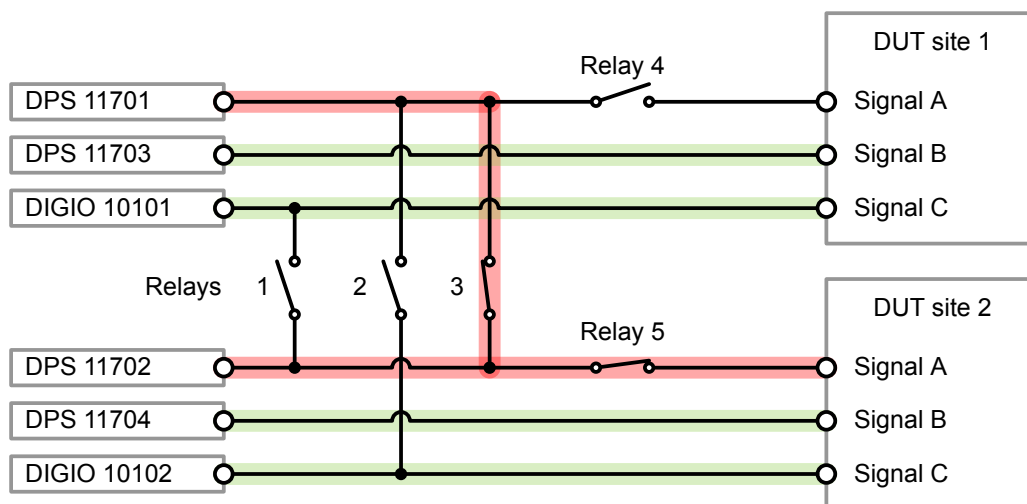
Based on the routing information, SmarTest recognizes that sequencing of the sites is required and applies it automatically.

```
...
...
signal Signal_A_borrowed_gang {
  site 1 { @Shared pogo = 11701 | 11702; }
  site 2 { @Shared pogo = 11702 | 11701; }
  routing {
    site 1 { relay_1 = 0; relay_2 = 0; relay_3 = 1; relay_4 = 1; relay_5 = 0; }
    site 2 { relay_1 = 0; relay_2 = 0; relay_3 = 1; relay_4 = 0; relay_5 = 1; }
  }
}
```

Site sequencing - Site 1:



Site sequencing - Site 2:



Borrowed DPS provides more load current for digital pin of the DUT

For this measurement, which is sequenced over the sites, a DPS is borrowed from the other site and connected in parallel to the digital instrument to provide temporarily more load on the "Signal C" DUT pin.

The parallel connection is made using relay 1 and relay 2. The borrowed DPS must be separated from its original site using relay 4 or relay 5.

For sequencing of the sites with flexible instrument sharing, you need to define different signal names for different measurement specifications that are used for different measurements. Site sequencing must be done manually and the appropriate measurement must be selected depending on the active sites. Note that the comment "Never used" in the code below means that the definition is required, but is never used: For the site specific#signal definition, the marked site must be disabled by manually set up site sequencing.

```
...
...
//
// Signals for measurement on site 1
//
signal Signal_A_serial_site_1 {
  site 1 { @Shared pogo = 11701; }
  site 2 { @Shared pogo = 11701; } // Never used
  routing {
    site 1 { relay_1 = 1; relay_2 = 0; relay_3 = 0; relay_4 = 1; relay_5 = 0; }
    site 2 { relay_1 = X; relay_2 = X; relay_3 = X; relay_4 = X; relay_5 = X; }
  }
}

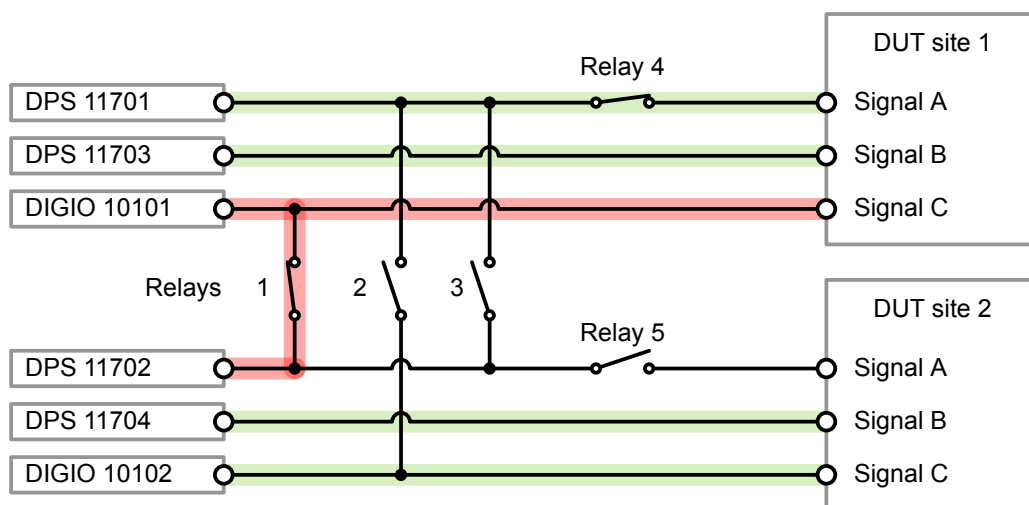
signal Signal_C_load_borrowed_from_site_2 {
  site 1 { @Shared pogo = 11702; }
  site 2 { @Shared pogo = 11702; } // Never used
}

//
// Signals for measurement on site 2
//
signal Signal_A_serial_site_2 {
  site 1 { @Shared pogo = 11702; } // Never used
  site 2 { @Shared pogo = 11702; }
  routing {
    site 1 { relay_1 = X; relay_2 = X; relay_3 = X; relay_4 = X; relay_5 = X; }
    site 2 { relay_1 = 0; relay_2 = 1; relay_3 = 0; relay_4 = 0; relay_5 = 1; }
  }
}

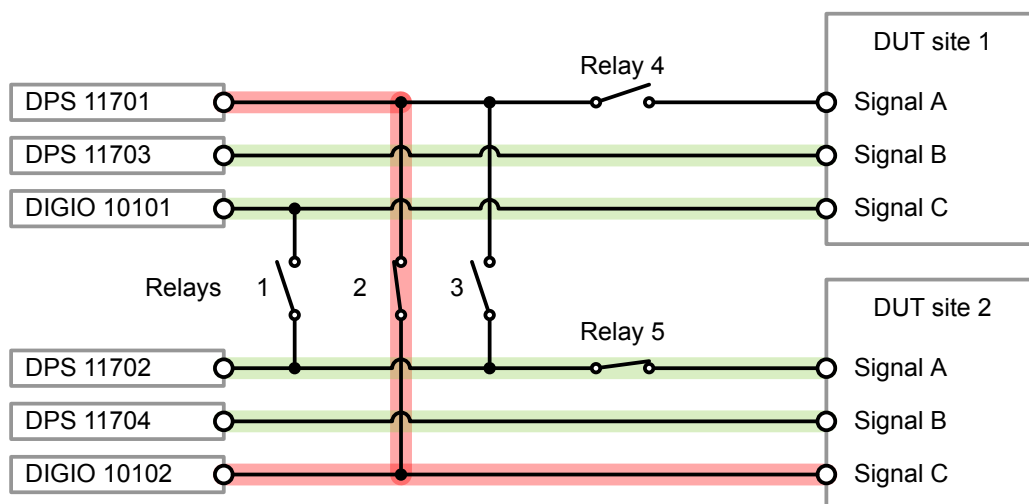
signal Signal_C_load_borrowed_from_site_1 {
  site 1 { @Shared pogo = 11701; } // Never used
```

```
site 2 { @Shared pogo = 11701; }  
}  
...  
...
```

Site sequencing - Site 1:



Site sequencing - Site 2:



Test data access

Test data is specified in variables, parameters, and properties of setups. Accessing test data is the process of reading, modifying or creating these variables, parameters, and properties either through a GUI or an API.

Test data is stored and used in several stages:

1. Initially, test data is stored on disk, either in setup files or in the `setup()` methods of test methods.

A `setup()` method can algorithmically create setup information using the Device Setup API. The method is executed once when the test program is bound.

2. Test data stored in files is loaded into memory during the *loading of a test program*.

All settings that are specified either in a setup file or a `setup()` method are loaded. If a particular setting is specified in both locations, the `setup()` method overwrites the setting from the file.

3. The binding process converts the loaded test data to executable data that is downloaded into the tester hardware.
4. When the test program is executed, all `setup()` methods of the test methods in the test program are executed.

If a `setup()` method specifies a setting that is already contained in a setup file, it overwrites the value from the file.

Note

When a test program is executed repeatedly (such as for the testing of a lot), the `setup()` methods are executed only if the setup files (whether created manually or through the Device Setup API) have changed since the last execution.

That means that, when setups have been changed in memory (see below) and you want the next execution to start without these modifications, you must restore the unmodified setups from the context object. Depending on the circumstances, this restoration must happen in the `update()` method (for example, to just restore the settings when a new lot is started) or the `execute()` method (to restore settings for each test iteration).

5. After the `setup()` methods have been executed, the `update()` methods from all test methods are executed, modifying the setups.

Such modifications are made using the Device Test API. They do not affect the original test data on the disk, but only affect the test data that is currently loaded in the memory (this loaded test data is used for all measurements and it exists in the memory until the test program finishes execution). The `update()` method is executed once the test program execution begins.

6. After the `update()` methods have been executed, the `execute()` methods of the test methods are executed, according to the position of the corresponding test suites in the testflows. (Depending on the control logic in the testflow, some test methods may not be executed.) The `execute()` methods can modify the data of the current measurement using the Device Test API.

These modifications affect the current measurement execution only and do not alter the test data for any other measurements in the current test program or the test data on disk. The modifications happen during each execution of the test method.

It is not possible to create test data using the `update()` or the `execute()` methods, only to modify existing data. For example, no new vectors can be created, but existing vectors could be modified.

All test data that is bound always applies to all tested sites. It is possible to specify values that apply only to one site or some of the tested sites. This must be done in the `execute()` method of a test method and can only affect the test data that has been loaded into the tester.

When a test program is debugged, test data can be interactively modified. Such modifications do not affect the test data stored on disk, unless you explicitly save them.

Data logging

Data logging is an integral part of testing with SmarTest and is always active. Standard data logging results are available as proprietary EDL2 or industry-standard STDF files. Custom events and formatters are supported.

Basic data logging process

When SmarTest executes a test, measured values are collected in specific data structures (so-called result accessors, implemented in the `xoc.dta.resultaccess` package from the Device Test API). In predefined test methods, this happens automatically; in custom test methods, you can control this process through methods.

System events such as the beginning or end of the execution of a test program or testflow are automatically logged.

Depending on the chosen log level, the data of interest from the measurement results are collected in another specific data structure (test descriptors, implemented in the `xoc.dta.testdescriptor` package, also from the Device Test API). Again, for predefined test methods this happens automatically, and for custom test methods, you can control the process.

Test descriptors then log the results and the evaluation to the data log stream. The evaluation can be based on test tables, for easy modification of pass/fail criteria and related test parameters.

The data log stream can be saved as an EDL2 or STDF file, or it can be accessed by custom data log formatters that produce output in the required format or react to events as they happen. Custom formatters access the stream using the EDL API.

Standard data logging

The standard data logging facilities of SmarTest do not require any programming or configuration. They log all relevant events and save them as EDL2 or STDF files which can then be evaluated with the tools of the Analysis perspective, or with third-party or custom tools.

Custom data logging

Customization of data logging is supported both for log creation - you can enter custom data in the log - and for log evaluation - you can create custom formatters that have full access to the data log stream and saved EDL files.

Inserting custom events in the data log stream requires programming a custom test method accordingly, while evaluating the data log stream or EDL files requires programming a "custom formatter (Topic 343573)" using the EDL API.

Alarms

Alarms are indications of errors or other unforeseen states during the test process. They are generated by SmarTest during the test execution and can trigger actions in the test code.

Responses to alarms can, among others, include silently ignoring them, generating data log entries, binning the DUT, or aborting the test. Not all responses are available for all alarms; in particular, some alarms always cause SmarTest to automatically abort the test.

For details, see these sections below:

- *Types of alarms* on page 172
- *Effects of alarms* on page 172
- *Enabling alarms* on page 173
- *Configuring alarms* on page 173
- *Alarms on nonparticipating instruments* on page 174
- *Handling alarms* on page 175
- *Logging alarms* on page 175

Types of alarms

There are different categories of alarms:

- Instrument alarms are triggered by a variety of irregularities during the measurement that an instrument detects.
- Software alarms are triggered by certain error conditions in the test execution software and are not directly related to the tester hardware.
- Test cell alarms are triggered by irregularities in the testing equipment (tester or prober/handler) that cannot be detected by the software execution engine and, therefore, cannot be reflected in the normal binning process.

For more information, see *Alarm types* on page 175.

Effects of alarms

Alarms can have a range of effects, depending on the type of alarm:

- In EDL data logs:
 - All alarms (unless disabled) are always logged in the TEST_SUITE_END_EVENT record.
 - When a result accessor is passed to the `evaluate()` method, all alarms from that measurement are logged with the result information.
 - When a value is passed to `evaluate()`, no automatic logging of alarms in the result information happens. Any desired alarm logging would have to be done by the test method.
- In STDF data logs, all alarms that are logged as part of the results are indicated by setting bit 0 of the TEST_FLG field. The alarm type is logged in the ALARM_ID field.

Alarms logged in the TEST_SUITE_END_EVENT record of the EDL log do not appear in the STDF log unless the `logAlarmToDTR` setting is enabled in the "STDF formatting settings (Topic 359358)".

- SmarTest visualizes alarms in its display of test results, for example, in the Analysis perspective.
- Alarms can influence the binning of DUTs: Unless a test descriptor is configured differently, it adds the "alarm bin (Topic 256361)" to the bin list of the current device when an alarm was raised.

Note that other bins can be assigned to a device even when an alarm occurs, for example, when the test results are evaluated. In that case, alarm bins do not automatically take

precedence over other bins in the final binning; this is only the case if they have the highest priority of all bins that have been assigned to a device.

Also note that alarm bins are assigned to devices after the test method execution ends. Therefore, bins assigned during the execution have precedence during final binning unless the alarm bin has a higher priority.

See "Assigning the final bin (Topic 256361)" for details.

- Alarms can force a measurement to evaluate to a `fail` result.

By default, no alarm forces a fail result, but you can configure it otherwise. (See *Configuring alarms* on page 173, below, for more information.)

- Alarms can terminate a test in progress.

This happens automatically for some types of alarms, for example, those that are caused by safety-relevant issues. For other alarm types, you can configure whether a test will be aborted when that kind of alarm is raised.

If an alarm affects only individual sites, the test is terminated only for the affected sites. Alarms that are not site-specific terminate the entire test.

- Test cell alarms are reported to the test cell controller, which may take actions of its own in response.

Enabling alarms

Software and test cell alarms are always enabled. Some instrument alarms are enabled by default, others are disabled, as noted in *Instrument alarms* on page 175. Alarms which are not enabled are never raised.

Enabling alarms is possible through the test table, in the test method code, and in measurement specifications:

- The `Alarm_Config` worksheet of a test table lets you enable and disable alarms without having to modify any code.

We recommend to configure alarms using this method, unless there is a specific reason not to use it.

- In test method code, alarms can be enabled or disabled using the `setEnabled()` method.
- In the measurement specifications, alarms can be enabled and disabled in the alarm property group.

If alarms are enabled or disabled in several places, the precedence described below in *Configuring alarms* on page 173 is used. If an alarm is never expressly enabled or disabled, its default state is used.

Configuring alarms

Several aspects of alarms can be configured through a test table or test method code:

- Whether the alarm is enabled (see above).
- Whether the alarm forces a fail result.

If an alarm forces a fail, any test suite in which the alarm occurs has a fail result. Also, every test descriptor that evaluates a measurement in which the alarm occurred, has a fail result.

By default, no alarm forces a fail result.

- Whether the alarm influences the binning of the device.

If an alarm influences binning, a specific bin (see below) is added to the bin list whenever the alarm occurs in a test suite. Note that the added bin is not necessarily the final bin for the device if the bin list contains other bins with higher priority.

- The bin which the alarm adds to the bin list.

If an alarm influences binning, the specified bin is added to the bin list. If no bin is specified, the default alarm bin is added.

To configure alarms in the test table, specify the desired settings in the `Alarm_Config` worksheet. .

For any alarm configurations you want to make in a test method, use the methods of *IAlarmConfiguration*. This kind of configuration is only possible in the `setup()` method of a test method or in test methods in the PreBind testflow.

For ease of use and increased robustness, we recommend to configure alarms through test tables, unless there is a specific reason for using test method code.

If you configure alarms in several locations, this is the precedence of the individual settings:

1. Settings in the `execute()` method of a test method (through the *IMeasurement* interface).
2. Settings in the `update()` method of a test method (through the *IMeasurement* interface).
3. Settings in the `setup()` method of a test method (through the context object).
4. Settings made in the PreBind testflow, most commonly by "reading a test table (Topic 249102)".
5. Settings made in the initializing code of a test method (that is, outside of any method within the test method).

Settings made in a location listed higher takes precedence over a location listed lower. For example, when an alarm is enabled in the test table (4) and disabled in the `update()` method of the test method (2), it will be disabled.

When you do not configure an alarm, its default settings are used.

Configuring alarms through a test method

This example shows a way to configure alarms in a test method code.

Alarms on nonparticipating instruments

Enabled instrument alarms are always reported for all active instruments. By default, instruments that have been used in a measurement and are then not included in the specifications of a subsequent measurement or disconnected altogether are no longer monitored for alarms..

When it is desired to also be notified of alarms on those instruments, the `defaults.alarm.continueReporting` property can be set to `true` for the respective instrument in the DUT board description file. (See *Default specification settings* on page 122 for more information.) That default can be changed dynamically from any test method using the `context.alarm.continueReporting` property. You can also change and query the setting for an individual instrument using the `setContinueReporting()` and `getContinueReporting()` methods of the *IAlarm* interface for the instrument (for example, `IDcVI.IAlarm`).

If `continueReporting` is `true` for an instrument, alarms from that instrument for all enabled sites are displayed in the **Result** and **Testflow** views and included in the data logs. They are also displayed in the separate **Not in spec** tab of the **Measurement** view. They are displayed in the **Instrument** view only if the measurement was executed from that view.

To avoid unintentional generation of alarms from nonparticipating instruments in subsequent measurements, we recommend to set `continueReporting` to `false` in the PowerDown testflow.

Handling alarms

Alarms are handled automatically by SmarTest and, where appropriate, the tester hardware and the test cell controller.

If you want your test methods to react to alarms as well, you can query for them using the `getAlarms()` method of the *IMeasurement* interface.

Logging alarms

To include alarm information in the data log, make sure to evaluate test descriptors with `evaluate()` methods that also take the *IMeasurement* object as an argument, for example:

```
testdescriptor.evaluate(myResults, myMeasurement)
```

Other `evaluate()` methods (that get passed just the results) do not have access to the alarm information and therefore cannot write them to the data log.

Related information

Alarm-related elements of the Device Test API

Alarm types

To distinguish the various sources of alarms, each alarm has a type that identifies the kind of issue it reports.

There are different categories of alarms:

- *Instrument alarms* are triggered by a variety of irregularities during the measurement that an instrument detects.
- *Software alarms* are triggered by certain error conditions in the test execution software and are not directly related to the tester hardware.
- *Test cell alarms* are triggered by irregularities in the testing equipment (tester or prober/handler) that cannot be detected by the software execution engine and, therefore, cannot be reflected in the normal binning process.

These categories and the individual alarm types are described in more detail below in *Instrument alarms* on page 175, *Software alarms* on page 178, and *Test cell alarms* on page 179.

Some alarm types are always triggered when their trigger conditions occurs and cannot be disabled. Most alarm types, however, can be enabled or disabled by the test code. Of these, most types are enabled by default (that is, if they are not configured in the test code, they are enabled), but some are disabled. This information is included below for each alarm type. For more information on enabling and disabling alarms, see *Instrument alarms* on page 175.

Instrument alarms

Instrument alarms are irregular conditions related to the measurement hardware. They are reported together with the measurement results. If they are safety-critical, the measurement hardware also automatically enters a safety mode; this requires no action by the test code.

Note that some alarm types can be configured for more cards than are listed below. These additional cards cannot actually issue the alarm. But enabling the alarm to be configured for them makes setups more portable between cards, because no bind errors are raised by the alarm configuration statements.

Alarm identifier	Trigger	Enabled by default	Can be disabled	Can be generated by
CURRENT_CLAMP	A source current exceeds the positive or negative clamp limit. The related signal is disconnected depending on the following <i>dcVI protection properties</i>: • <i>constCurrDisconnect</i> defines whether the signal is disconnected in the event of an alarm and • <i>constCurrTimeToDisconnect</i> defines the time after which the signal is disconnected. This property is supported only if <i>constCurrDisconnect</i> is set to <code>true</code> and is mandatory in this case. The valid range is 0 ... 1275 ms. For "XPS256 (Topic 350661)" signals, property <i>currentClampTimeToAlarm</i> of the <i>dcVI alarm properties</i> defines a dead time for this alarm: The alarm is triggered if the current clamp is still active after this time.	No	Yes	<i>List of instruments</i>
CURRENT_SURGE_T	A comparator detects a current outside the permitted range.	Yes	Yes	<i>List of instruments</i>
DISCONNECTED	An instrument has been disconnected while a measurement was in progress.	Yes	No	<i>List of instruments</i>
DUT_BOARD_UNDO	The DUT board has been removed during the execution of the test program.	No	Yes	<i>List of instruments</i>
ENERGY_LOW	The charge level of the on-board energy storage is too low.	Yes	No	<i>List of instruments</i>
HARDWARE_DEFECT	A defective instrument has been detected.	Yes	No	<i>List of instruments</i>
LIMIT_EXCEEDED	Voltage or current limit specified in the DUT board description was exceeded. The offending voltage or current is not applied.	Yes	Yes	<i>List of instruments</i>
MEMORY	Not enough memory is available to capture the results.	Yes	Yes	<i>List of instruments</i>
OVERRANGE	The voltage or current of a captured signal exceeds the measurement range of the instrument. Measurements that trigger this alarm may return NaN as the measured value.	Yes	Yes	<i>List of instruments</i>
OVERRUN	An instrument is not able to keep up with the fast rate of arriving events (for example, during a TMU measurement), which results in incomplete recording.	Yes	Yes	<i>List of instruments</i>

Alarm identifier	Trigger	Enabled by default	Can be disabled	Can be generated by
POWER_BUDGET	The power budget of an instrument is exceeded, for example because it supplies too many channels, the battery of an high-current unit is recharging, or too many power spikes exceed the short-term budget.	Yes	No	<i>List of instruments</i>
PROTECTION	An overvoltage or overcurrent event caused by a defective device has been detected. This may be caused, for example, by shortening a high power source to a digital pin or too many current spikes exceeding the short-term budget. The time after which the alarm is triggered can be set with the <i>safeDisconnectTimeout</i> property of the <i>dcVI protection properties</i>. Default timeout is 2.0 s.	Yes	No	<i>List of instruments</i>
PROTOCOL_VIOLATION	In protocol-aware testing, the specified protocol has been violated so that, for example, a register could not be read.	No	Yes	<i>List of instruments</i>
SAFETY	A PMU has shut down because it received an external safety signal (for example, because the cover was not closed).	Yes	No	<i>List of instruments</i>
SOURCE_NOT_READ	The regulation loop of a source is not in a stable state.	Yes	Yes	<i>List of instruments</i>
SPEC_LIMIT_EXCEEDED	A soft limit of the card's technical specification has been exceeded while overprogramming was enabled.	Yes	Yes	<i>List of instruments</i>
SPU_ERROR	The SPU has experienced an unrecoverable error (for example, underflow, overflow, timeout). Note that this alarm is called <i>SPUERROR</i> (without the underscore) in the test cell software.	Yes	No	<i>List of instruments</i>
TEMPERATURE	A temperature sensor on an instrument exceeds its alarm limit.	Yes	No	<i>List of instruments</i>
TIMEOUT	The sequencer does not find a match in a counted match loop or has a timeout, or a TMU measurement detects too few events.	Yes	Yes	<i>List of instruments</i>
UNSAFE_DISCONNECT	The hardware was disconnected before having reached 0 V.	Yes	Yes	<i>List of instruments</i>

Alarm identifier	Trigger	Enabled by default	Can be disabled	Can be generated by
VOLTAGE_CLAMP	A source voltage exceeds the positive or negative clamp limit. Note that this alarm is called VOLTAG_CLAMP (without the E) in the test cell software.	No	Yes	List of instruments
VOLTAGE_DROP	Voltage drop limit on force line or return line is exceeded. The instrument is disconnected. The thresholds for this alarm are defined by properties • <i>maxVoltageDropForce</i> and • <i>maxVoltageDropReturn</i> of the <i>dcVI protection properties</i>. You must set the <i>voltageDropDisconnect</i> property of the <i>dcVI protection properties</i> to <code>true</code> to allow the signal to be automatically disconnected. When set to the default value <code>false</code>, an alarm is triggered if a threshold is exceeded but the signal is not disconnected.	Yes	Yes	List of instruments
VOLTAGE_DROP_WA	The voltage drop level on force/return line is reached. This alarm indicates that a probe maintenance or socket cleanup has to be done soon. The warning threshold must be set by properties <i>warnVoltageDropForce</i> or <i>warnVoltageDropReturn</i> of the <i>dcVI protection properties</i> to enable this alarm.	Yes	Yes	List of instruments
VOLTAGE_SURGE_TF	A comparator detects a current outside the permitted range.	Yes	Yes	List of instruments

Software alarms

Software alarms are triggered by some error conditions in the test program. They are detected by the tester software and do not involve the tester hardware.

Software alarms always cause the result of the test suite to be fail, regardless of the setting of the alarm's *force fail flag*.

Identifier	Trigger	Enabled by default	Can be disabled
DYNAMIC_BIND	A test method programmatically modified test setups, and the modified setups could not be bound or required unavailable licenses.	Yes	No

Identifier	Trigger	Enabled by default	Can be disabled
DYNAMIC_SETUP_ERROR	This alarm is deprecated and not issued any more.	-	-
SOFTWARE_DEFECT	An internal SmarTest process that handles test execution has crashed.	Yes	No
TEST_METHOD	A test method caused an exception but did not handle it.	Yes	No

Test cell alarms

Test cell alarms are caused by issues outside that the test execution engine cannot detect. They usually must be handled by the test cell controller.

While these alarms are specific to test cell issues, all other enabled alarm are also available to the test cell controller, for example, for handling in the `OnAlarm` element. Instrument alarms do not abort the lot execution. The `SOFTWARE_DEFECT` alarm aborts the lot execution if the test process throws an exception.

Identifier	Trigger	Enabled by default	Can be disabled	Aborts lot execution
CALIBRATION_EXPIR	The hardware has not been calibrated recently enough.	Yes	No	No
DISK_SPACE_EXCEE	The minimum required free disk space (as specified by the <code>system.datalog.size_</code> setting) is not available. (This condition may additionally raise a <code>FORMATTER_DEFECT</code> alarm.)	Yes	No	No
DOCK_BOARD_NOT_	Testing of a lot has started before the DUT board was fully docked. This alarm is supported only by EXA Scale testers.	Yes	Yes	Yes
DOCK_BOARD_NOT_	Testing of a lot has started before the DUT board was fully contacted. This alarm is supported only by EXA Scale testers.	Yes	Yes	Yes

Identifier	Trigger	Enabled by default	Can be disabled	Aborts lot execution
FORMATTER_DEFECT	The data log formatter crashed or could not write the log to disk (for example, because the disk is full), or a user-specified check for sufficient disk space failed.	Yes	No	Yes
INVALID_DATALOG	Data that the data log formatter requires to generate valid output is missing or invalid.	Yes	No	Yes
LICENSE_SERVER	The license server cannot be reached and the grace period has expired.	Yes	No	Yes
OPERATION_ABORTED	A test program operation could not be completed due to an internal error.	Yes	No	Yes
PH_COMMUNICATION	There are problems communicating with the prober/handler.	Yes	No	Yes
PH_CUSTOM_ALARM	User-defined.	Yes	No	No
PH_MATERIAL_HANDLER	The prober/handler does not work as expected.	Yes	No	Yes
RAN_OUT_OF_DISK	The disk space of the test system is full. (This condition may additionally raise a FORMATTER_DEFECT alarm.)	Yes	No	No
SOFTWARE_DEFECT	SmarTest shut down unexpectedly.	Yes	No	Yes
STATIC_BIND	The initial binding of the test program was unsuccessful, for example, because of a missing license.	Yes	No	Yes (if the reported issue has the "error" type)
UNKNOWN	An alarm of an unknown type has been raised.	Yes	No	No

Debugging and characterization

SmarTest provides device-test oriented tools which makes it easier for you to set up your measurement, debug it, and characterize your devices. For example, the Pattern Editor shows the vectors that are applied to your DUT signals per device cycle.

Unified setup, debugging, and characterization tools and views

SmarTest provides unified tools and views that are device-oriented and allow you to setup, debug, and characterize, your device consistently regardless of which test domain you are working on (DC, digital, analog, mixed-signal, or RF).

Some of the tools and views that SmarTest provides include:

- "Testflow view (Topic 255242)"
- "Pattern editor reference (Topic 249072)"
- "Timing Debug view (Topic 246069)"
- "Operating Sequence view (Topic 253305)"
- "Signal Analyzer Tool (Topic 246089)"
- "Measurement view (Topic 348998)"
- "Instrument view (Topic 250415)"
- "Tester view (Topic 250416)"
- "Shmoo view (Topic 246335)"

- SmarTest provides device-oriented workflows and tools. This example shows the **Pattern Editor**. It displays your pattern and measurement results per *device cycles*:

Functional00_03x1.pat

Signal
X-Mode: x1
Site Memory Sharing: OFF

Actions

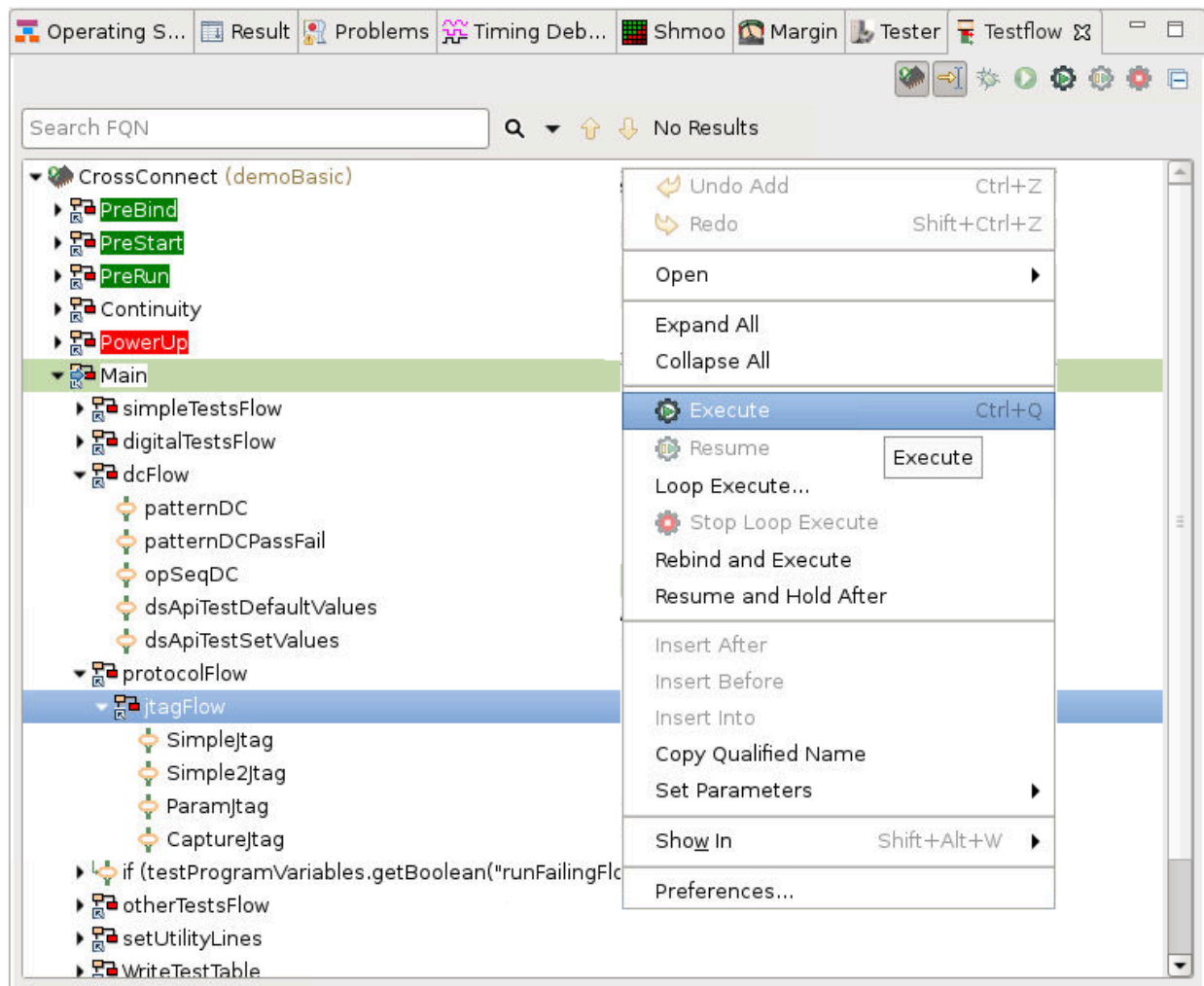
Vector#	Instruction	Comment	RPTV 1, 1	D03	D02	D01	D00	R03	R02	R01	R00
0	RPTV 1, 1		0	0	0	0	L	L	L	L	
1			0	0	0	1	L	L	L	H	
2			0	0	1	0	L	L	H	L	
3			0	0	1	1	L	L	H	H	
4			0	1	0	0	L	H	L	L	
5			0	1	0	1	L	H	L	H	
6			0	1	1	0	L	H	H	L	
7			0	1	1	1	L	H	H	H	
8			1	0	0	0	H	L	L	L	
9			1	0	0	1	H	L	L	H	
10			1	0	1	0	H	L	H	L	
11			1	0	1	1	H	L	H	H	
12			1	1	0	0	H	H	L	L	
13			1	1	0	1	H	H	L	H	
14			1	1	1	0	H	H	H	L	
15			1	1	1	1	H	H	H	H	
16			0	0	0	0	L	L	L	L	
17			0	0	0	1	L	L	L	H	
18			0	0	1	0	L	L	H	L	
19			0	0	1	1	L	L	H	H	

Pattern Global Comments

The Pattern Editor shows the sequencer instructions, vectors, and comments applied to your DUT signals per device cycles.

- Interactive debugging is predominantly driven from the Testflow view, from which you can select the complete test program to execute, or a flow or a test suite. Executing a flow

or a test suite minimizes the initial overheads, freeing more time for you to work on the debugging tasks at hand.

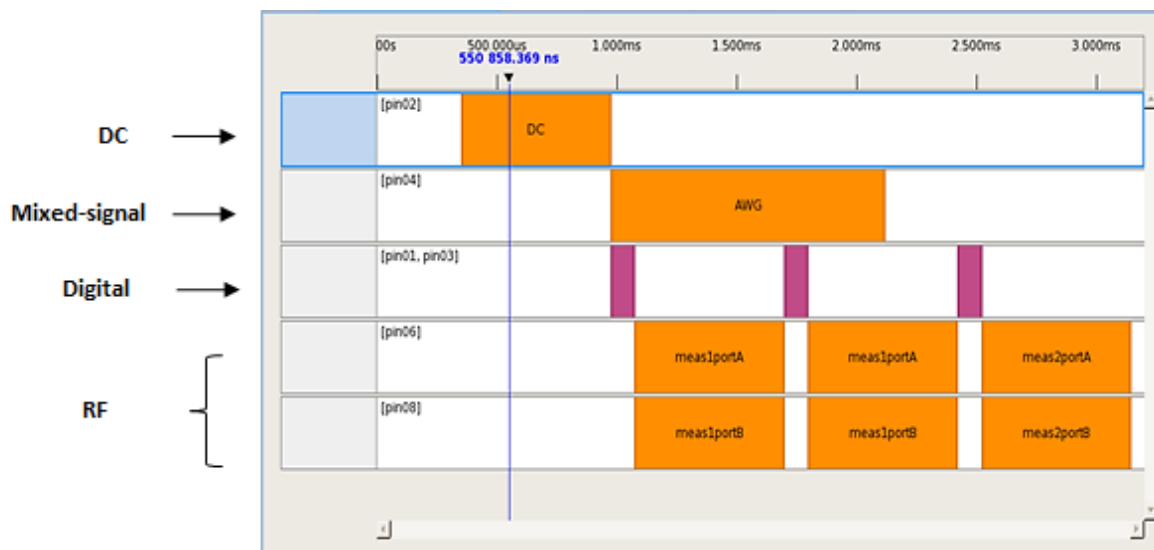


For further details see "Starting debugging (Topic 346111)".

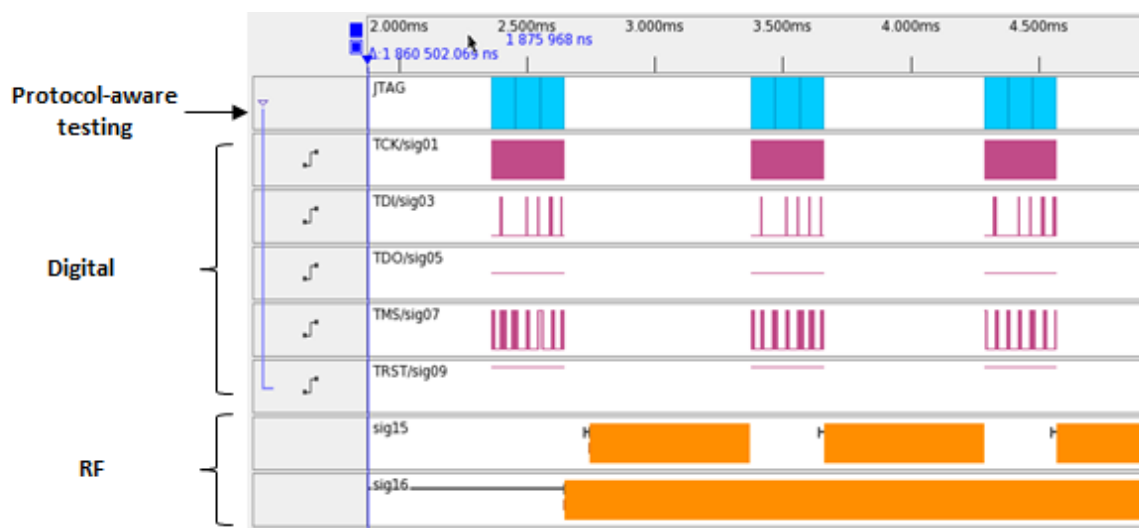
- The figures below show the **Operating Sequence Debugger** and the **Timing Debug View**. These are debugging tools that you can use to easily and effectively debug (and

characterize) your measurement in all test domains. In addition, they show the waveforms, actions, or transaction sequences applied to your DUT signals per device cycle:

1. The **Operating sequence Debugger** can be used universally across all test domains (DC, digital, analog, mixed-signal, or RF).



2. The **Operating sequence Debugger** can be used universally across all test domains (DC, digital, analog, mixed-signal, or RF).



Debugging

SmarTest offers tools both for the debugging of test setups and test programs as well as for debugging tested devices. These tools are unified, you can use them for any test domain, digital, analog, or RF.

Debugging test programs

SmarTest supports both interactive and static debugging of test programs:

- The "Device Debug perspective (Topic 244556)" includes the Testflow view, which, in conjunction with the perspective toolbar contain the main controls for running interactive debugging. The interactive device debugging tools, for example, the Measurement view and the Timing Debug view enable you to dynamically examine the electrical signals being measured and see their reaction to changes in the code or settings. This helps, for example, in verifying that the test program tests the DUT as it is intended to.

For more details, see "Debugging test programs, testflows and test suites (Topic 346093)".

- Static analysis involves analyzing and evaluating the data logs of completed tests. The numerical and graphical tools SmarTest offers for this purpose are grouped in the Analysis perspective.

This helps, for example, in ensuring that the test program provides the expected results with a golden device.

Debugging test setups

SmarTest checks the test setup data and reports errors that prevent you from executing the test program or testflow. These errors are reported in the "Problems view (Topic 244529)" after the SmarTest project is loaded or after saving a new or modified test setup file, for example, a measurement specification or pattern.

Debugging your test setups with the help of the messages reported to the Problems view, includes understanding the problem messages, navigating to the source of the reported error and certain run-time errors that may occur, for example, because a calculated value exceeds an instrument's limit.

Setup data you create in SmarTest does not need to be syntactically correct, as long as it is not bound. This allows you, for example, to save incomplete setups. When a test program is *bound*, all setups used by it are bound as well. This includes verifying their syntactical correctness (and sometimes their semantic correctness, to a limited degree). Any errors that are found are displayed in the Problems view, with links to location of the incorrect setup where possible.

Setups and the test programs that use them cannot be bound if they contain errors, to ensure that you have formally correct setups when running a test program.

For debugging logical errors, such as incorrect equations, in bound setups, SmarTest provides tools such as the "Tester view (Topic 250416)" and log files.

Debugging test methods

Test methods are normal program code and are debugged in the same way as other code. SmarTest offers interactive debugging of individual testflows or entire test programs using breakpoints and watchpoints:

- Breakpoints are locations in the code where the execution is interrupted so that you can examine the state of variables and other data structures.
- Watchpoints are similar to breakpoints, but specify a condition instead of a location. The execution is interrupted whenever the specified condition occurs, again allowing you to inspect the state of the test program.

Debugging is handled in the "Device Debug perspective (Topic 244556)", which allows you to manage breakpoints and watchpoints, view the code being executed, and examine and modify variables. When modifying variables, you can set site-specific values.

Characterization

SmarTest provides powerful characterization workflows to facilitate your characterization tasks.

In general, the aim of device characterization is to make sure that your device conforms with the specifications required for its intended application before the device goes into production. SmarTest provides multiple tools that enable you to obtain your device characterization. With shmoo tests being the most prominent way to characterize devices, SmarTest provides the "Shmoo view (Topic 246335)" and the Shmoo Analysis view that you can use to perform shmoo tests effectively and with ease.

Using these tools, you can obtain your device characterization in two ways:

- Interactively: while the measurement is being executed
- Automatically: after your test program has finished executing

For a detailed overview on the shmoo use model, see "Shmoo capability overview" (topic 254302).

See the tutorial: "Characterization (Topic 253799)".

Message logging

Your code can output messages to the console as a feedback mechanism, for example, for debugging and tracing purposes. By setting the message log level, you can dynamically control the level of detail of these messages without having to touch your code. The message log level also controls certain console output by the system.

Note

Despite the similar name, message logging has nothing to do with data logging and is completely separate from it. For information on data logging, see "Logging test results (Topic 247442)".

Message log level

The message log level is a runtime setting that controls which messages are output to the console. It affects both user messages and system messages:

- User messages are printed to the console only if the current log level is equal to or greater than the log level set in the message.

For example, if a message is specified by `message(7, "Hello world.")`, it is output only if the message log level is at least 7 at the time when the `message()` method is executed.

- SmarTest outputs a trace of the testflow and test suite execution to the console when the message log level is at least 10. It otherwise ignores the message log level for system messages.

The message log level is set for a testflow or test suite as the `messageLogLevel` parameter. (See "Specifying testflow parameters in a test table (Topic 256274)" and "Specifying test suite parameters in a test table (Topic 256154)" for information on setting these parameters through test tables.) The higher the message log level, the more messages generally appear in the console.

If a testflow or test suite does not specify a message log level, it inherits the level from the calling testflow. Therefore, the message log level set for the main testflow applies to the entire test program when the other testflows and test suites do not override it.

If you do not set a message log level for a test program, the default level is 0 - meaning that none of the `message()` methods in your code actually output messages to the console, and that SmarTest does not provide an execution trace in the console.

Specifying messages to log

You can include in your test method code statements that cause a message to be output to the console by including the "message()" (Topic 253646)" method of the *TestMethod* class.

The method takes two parameters:

```
message(messageLogLevel, message);
```

messageLogLevel

The message log level is an integer that specifies the minimum log level that is needed to output this message.

For details, see *Message log level* on page 188, above.

message

The message itself that is output to the console. A string surrounded by quotes.

Setting the message log level as a testflow parameter

For increased flexibility and ease of use, we recommend setting message log levels in test tables, but you can also set them for a testflow in a testflow file or in the test program file. Because the log level is specified as a testflow parameter, you must set it in the calling file,

that is, in the program file for the main testflow and in a testflow file for subflows called from this testflow.

If you set the message log level for the main testflow as shown below, it applies to all other testflows and test suites as well, except where it is expressly overridden.

```
testflow Main {  
    flow = common.ExampleMain;  
    flow.messageLogLevel = 10;  
}
```

Setting the message log level for a test suite

For increased flexibility and ease of use, we recommend setting message log levels in test tables, but you can also set them in the testflow code:

```
flow MyFlow {  
    setup {  
        suite mySuite calls MyTestMethod {  
            messageLogLevel = 5;  
        }  
    }  
}
```

Specifying a message

When this test method is executed, it prints "MyTestMethod executing" to the console if the message log level is set to 12 or higher.

```
public MyTestMethod extends TestMethod {  
    ...  
    public void execute {  
        ...  
        message (12, "MyTestMethod executing");  
        ...  
    }  
}
```